

ETAS RTA-BSW v12.7.0



RTA-Eth Stack Reference Guide

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-Eth Stack Reference Guide R01 EN – 05.2025

Contents

1	Diagnostic Over IP	10
1.1	Introduction	10
1.2	Supported Features	11
1.3	Extensions	11
1.4	Limitations	18
1.5	Exclusions	18
1.6	Deviations	18
1.7	API Definition	24
1.8	Configuration Advice	25
1.9	Integration Advice	30
1.10	Hardware Requirements	35
2	Ethernet Interface	36
2.1	Introduction	36
2.2	Supported Features	37
2.3	Extensions	45
2.4	Limitations	63
2.5	Exclusions	64
2.6	Deviations	67
2.7	API Definition	69
2.8	Configuration Advice	75
2.9	Integration Advice	83
2.10	Hardware Requirements	89
3	Ethernet State Manager	90
3.1	Introduction	90
3.2	Supported Features	91
3.3	Extensions	93
3.4	Limitations	96
3.5	Exclusions	96
3.6	Deviations	96
3.7	API Definition	97
3.8	Configuration Advice	97
3.9	Integration Advice	99
3.10	Hardware Requirements	100

4	Time Synchronization over Ethernet	101
4.1	Introduction.....	101
4.2	Supported Features	102
4.3	Extensions	103
4.4	Limitations	105
4.5	Exclusions.....	105
4.6	Deviations.....	106
4.7	API Definition.....	112
4.8	Configuration Advice.....	113
4.9	Integration Advice.....	113
4.10	Hardware Requirements	116
5	rba_EthArp.....	118
5.1	Introduction.....	118
5.2	Supported Features	119
5.3	Extensions.....	119
5.4	Limitations.....	119
5.5	Exclusions	119
5.6	Deviations	119
5.7	API Definition	119
5.8	Configuration Advice	120
5.9	Integration Advice	121
5.10	Hardware Requirements.....	122
6	rba_EthAutolp	123
6.1	Introduction	123
6.2	Supported Features	124
6.3	Extensions	124
6.4	Limitations	124
6.5	Exclusions.....	124
6.6	Deviations	124
6.7	API Definition	124
6.8	Configuration Advice	125
6.9	Integration Advice	126
6.10	Hardware Requirements.....	127
7	rba_EthDHCP.....	128

7.1	Introduction	128
7.2	Supported Features	129
7.3	Extensions	129
7.4	Limitations	129
7.5	Exclusions	130
7.6	Deviations	130
7.7	API Definition	130
7.8	Configuration Advice	131
7.9	Integration Advice	131
7.10	Hardware Requirements	132
8	rba_EthDHCPv6	133
8.1	Introduction	133
8.2	Supported Features	134
8.3	Extensions	134
8.4	Limitations	135
8.5	Exclusions	135
8.6	Deviations	135
8.7	API Definition	135
8.8	Configuration Advice	136
8.9	Integration Advice	137
8.10	Hardware Requirements	137
9	rba_EthICMP	138
9.1	Introduction	138
9.2	Supported Features	139
9.3	Extensions	139
9.4	Limitations	139
9.5	Exclusions	139
9.6	Deviations	139
9.7	API Definition	139
9.8	Configuration Advice	140
9.9	Integration Advice	141
9.10	Hardware Requirements	142
10	rba_EthICMPV6	143
10.1	Introduction	143

10.2	Supported Features	144
10.3	Extensions	145
10.4	Limitations	145
10.5	Exclusions	145
10.6	Deviations	145
10.7	API Definition	145
10.8	Configuration Advice	146
10.9	Integration Advice	147
10.10	Hardware Requirements	147
11	rba_EthIPv4	148
11.1	Introduction	148
11.2	Supported Features	149
11.3	Extensions	152
11.4	Limitations	154
11.5	Exclusions	154
11.6	Deviations	154
11.7	API Definition	154
11.8	Configuration Advice	155
11.9	Integration Advice	157
11.10	Hardware Requirements	157
12	rba_EthIPv6	158
12.1	Introduction	158
12.2	Supported Features	159
12.3	Extensions	160
12.4	Limitations	160
12.5	Exclusions	161
12.6	Deviations	161
12.7	API Definition	161
12.8	Configuration Advice	162
12.9	Integration Advice	163
12.10	Hardware Requirements	163
13	rba_EthNdp	164
13.1	Introduction	164
13.2	Supported Features	165

13.3	Extensions	166
13.4	Limitations	168
13.5	Exclusions	169
13.6	Deviations	169
13.7	API Definition	169
13.8	Configuration Advice	169
13.9	Integration Advice	171
13.10	Hardware Requirements	172
14	rba_EthTcp	173
14.1	Introduction	173
14.2	Supported Features	174
14.3	Extensions	174
14.4	Limitations	174
14.5	Exclusions	174
14.6	Deviations	174
14.7	API Definition	174
14.8	Configuration Advice	175
14.9	Integration Advice	179
14.10	Hardware Requirements	180
15	rba_EthUdp	181
15.1	Introduction	181
15.2	Supported Features	182
15.3	Extensions	186
15.4	Limitations	187
15.5	Exclusions	188
15.6	Deviations	188
15.7	API Definition	188
15.8	Configuration Advice	188
15.9	Integration Advice	190
15.10	Hardware Requirements	190
16	Service Discovery	192
16.1	Introduction	192
16.2	Supported Features	196
16.3	Extensions	196

16.4	Limitations	209
16.5	Exclusions	210
16.6	Deviations	211
16.7	API Definition	216
16.8	Configuration Advice	217
16.9	Integration Advice	218
16.10	Hardware Requirements	222
17	Socket Adaptor	223
17.1	Introduction	223
17.2	Supported Features	224
17.3	Extensions	224
17.4	Limitations	238
17.5	Exclusions	239
17.6	Deviations	240
17.7	API Definition	244
17.8	Configuration Advice	246
17.9	Integration Advice	247
17.10	Hardware Requirements	249
18	SOME/IP Transport Protocol	250
18.1	Introduction	250
18.2	Supported Features	250
18.3	Extensions	250
18.4	Limitations	251
18.5	Exclusions	251
18.6	Deviations	251
18.7	API Definition	252
18.8	Configuration Advice	253
18.9	Integration Advice	254
18.10	Hardware Requirements	257
19	SOME/IP Transformer	258
19.1	Introduction	258
19.2	Supported Features	260
19.3	Extensions	261
19.4	Limitations	261

19.5	Exclusions	261
19.6	Deviations	261
19.7	API Definition	262
19.8	Configuration Advice	262
19.9	Integration Advice	264
19.10	Hardware Requirements	265
20	TCP/IP Stack	266
20.1	Introduction	266
20.2	Supported Features	267
20.3	Introduction	271
20.4	Advantages of using SACK	271
20.5	Disadvantages of using SACK	271
20.6	Extensions	271
20.7	Limitations	330
20.8	Exclusions	332
20.9	Deviations	336
20.10	API Definition	347
20.11	Configuration Advice	348
20.12	Integration Advice	352
20.13	Hardware Requirements	353
21	UDP Network Management	354
21.1	Introduction	354
21.2	Supported Features	355
21.3	Extensions	355
21.4	Limitations	358
21.5	Exclusions	359
21.6	Deviations	359
21.7	API Definition	363
21.8	Configuration Advice	365
21.9	Integration Advice	365
21.10	Hardware Requirements	366
22	Glossary	367

1 Diagnostic Over IP

Module name	DoIP
Package	RTA-ETH
AUTOSAR Module ID	173
AUTOSAR Version	22-11

1.1 Introduction

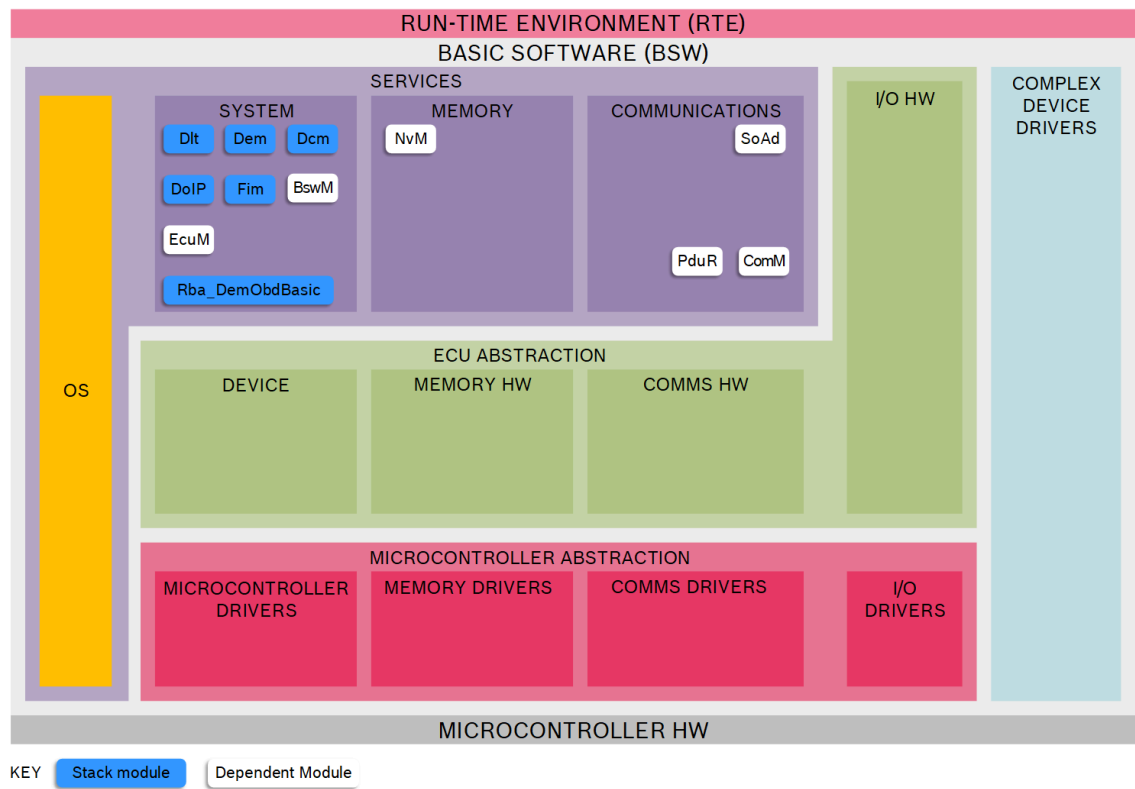


Figure 11.RTA-DIAG Architecture

The Diagnostic Over IP (DoIP) module is located in the system services abstraction layer, forming part of the Diagnostic (RTA-DIAG) stack. The diagram above shows the position of DoIP in the AUTOSAR layered architecture.

The Socket Adapter (SoAd) module is considered as the lower layer with respect to DoIP, as SoAd provides access to the TCP/IP network protocols which DoIP requires. DoIP communicates with other modules through the PDU Router (PduR) for the transmission and reception of diagnostic messages.

The primary purpose of DoIP is to control the transfer of diagnostic messages between the SoAd and PduR modules.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator

(ConfGen) and must be manually configured.

1.2 Supported Features

- Vehicle identification and announcement.
- Routing line activation.
- Node information.
- IP socket and address change notification.
- Post-Build (Selectable and Loadable).
- Protocol Versions are configurable with respect to ISO 13400-2:2023 Amd 1, ISO 13400-2:2021 and ISO 13400-2:2019, including vehicle identification.
- Enabling or disable communication over unsecured port(13400) during runtime.
- Support of DoIP Header with Version 0x04, according to ISO 13400-2:2023 Amd 1
- Configurable option for DoIP Lite to Enable or Disable Acknowledgments and Negative Acknowledgments 0x8001, 0x8002, 0x8003 and 0x0000 payload type.
- Support for DoIP Lite payload type 0x8001 only.
- Support for Periodic Diagnostic Message payload 0x8004 for DoIP Protocol Version 4 according to ISO 13400-2:2023 Amd 1.
- Support for (transmission protocol) TP Status Indication payload 0x9001 for DoIP Protocol Version 4 according to ISO 13400-2:2023 Amd 1.

1.2.1 UDP WildCard Remote Address

If Wildcard Remote Addressing is used for UDP connections, upon setting the remote address for wildcard, the Socket connection goes in to RECONNECT mode. However, this is only considered as OFFLINE mode by DoIP, and thus the delay to bring the connection back ONLINE must be considered before the next UDP request on the same connection.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.1.1 DoIP/DoIPConfigSet/DoIPInterface/DoIPChannel/DoIPPduRTxPdu/DoIPRbChannelBufferSize

Short Name	DoIPRbChannelBufferSize
------------	-------------------------

Description	DoIP Internal Channel buffer for Periodic diagnostic Message payload- (0x8004) this buffer will store Timestamp + length + Periodic DID, everytime when DoIP_IfTransmit is called. Only used when PduType is DOIP_IFPDU
Multiplicity	0..1
Value Range	1..65536
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.2 DoIP/DoIPConfigSet/DoIPInterface/DoIPChannel/DoIPPduRTxPdu/DoIPRbPeriodicDataTxTimeout

Short Name	DoIPRbPeriodicDataTxTimeout
Description	Timeout in [s] for maximum time DoIP shall wait before transmitting the stored data inside of DoIP Channel Buffer for Periodic Diagnostic Message Payload(0x8004). Once the timer expires DoIP will Transmit the data to SoAd The value shall be multiple of DoIPMainFunctionPeriod and shall be configured when tx PduType is DOIP_IFPDU
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

1.3.1.3 DoIP/DoIPConfigSet/DoIPInterface/DoIPChannel/DoIPRbApplicationServiceIDSupported

Short Name	DoIPRbApplicationServiceIDSupported
------------	-------------------------------------

Description	Specifies the functionality of DoIP whether to support Application Service Identifier in Transport Protocol Payload 0x9001. TRUE : This indicates that the DoIP will support Application Service Identifier in Transport Protocol Payload 0x9001. Note here DoIP will extract the Service ID from the api TP Status indication. FALSE : This indicates that the DoIP does not support Application Service Identifier in Transport Protocol Payload 0x9001. Note here DoIP will fill value 0x00 according to ISO, which indicates Application Service Identifier not Applicable. Note This paramter shall be configured only when DoIPRbTPPayloadTypeSupported is set to TRUE.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.4 DoIP/DoIPConfigSet/DoIPInterface/DoIPProtocolVersion

Short Name	DoIPProtocolVersion
Description	This container defines the protocol version of the DoIP entity.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

1.3.1.5 DoIP/DoIPConfigSet/DoIPInterface/DoIPProtocolVersion/DoIPAnnouncedProtocolVersion

Short Name	DoIPAnnouncedProtocolVersion
Description	This parameter defines the used protocol version for vehicle announcements and vehicle identification responses if the vehicle identification request contains protocol version 0xFF.
Multiplicity	1..1
Default Value	-
Enumeration Literals	DOIP_ISO13400_2_2012, DOIP_ISO13400_2_2019, DOIP_Rb_ISO13400_2_2023Amd1
Type	ECUC-ENUMERATION-PARAM-DEF

1.3.1.6 DoIP/DoIPConfigSet/DoIPInterface/DoIPProtocolVersion/ DoIP_ISO13400_2_2012

Short Name	DoIP_ISO13400_2_2012
Description	If this parameter is enabled, the DoIP entity supports communication with ISO 13400-2:2012 compliant tester using protocol version 0x02.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.7 DoIP/DoIPConfigSet/DoIPInterface/DoIPProtocolVersion/ DoIP_ISO13400_2_2019

Short Name	DoIP_ISO13400_2_2019
Description	If this parameter is enabled, the DoIP entity supports communication with ISO 13400-2:2019 compliant tester using protocol version 0x03.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.8 DoIP/DoIPConfigSet/DoIPInterface/DoIPProtocolVersion/DoIP_R- b_ISO13400_2_2023Amd1

Short Name	DoIP_Rb_ISO13400_2_2023Amd1
Description	If this parameter is enabled, the DoIP entity supports communication with ISO 13400-2:2023 Amd1 compliant tester using protocol version 0x04.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.9 DoIP/DoIPConfigSet/DoIPInterface/DoIPRbTPPayloadTypeSupported

Short Name	DoIPRbTPPayloadTypeSupported
------------	------------------------------

Description	Specifies the functionality of DoIP whether to support Transport Protocol Payload 0x9001. TRUE : This indicates that the DoIP support Transport Protocol Payload 0x9001 (Tp Status Indication). FALSE : This indicates that the DoIP does not support Transport Protocol Payload 0x9001 (Tp Status Indication). Note: This paramter has dependency on DoIP_Rb_ISO13400_2_2023Amd1
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.10 DoIP/DoIPGeneral/DoIPRbGeneral

Short Name	DoIPRbGeneral
Description	This container contains RB specific configuration parameters.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

1.3.1.11 DoIP/DoIPGeneral/DoIPRbGeneral/DoIPRbFurtherActionByteIndependentFromRA

Short Name	DoIPRbFurtherActionByteIndependentFromRA
Description	This parameter determines the value of further action byte of vehicle announcement / identification message: TRUE Further action byte in the vehicle announcement message will be set to 0x10, if routing activation with central security(0xE0) is configured. FALSE If routing activation with central security(0xE0) is configured, further action byte in the vehicle announcement message will be set to: 0x00, if routing activation with central security(0xE0) is referred by any of the doip testers and this routing activation is successful for at least one tester. Otherwise it will be set to 0x10. 0x10, if routing activation with central security(0xE0) is configured and the corresponding routing activation is not successful for any of the testers. If routing activation with central security(0xE0) is not configured, further action byte will always be set to 0x00 in vehicle announcement message
Multiplicity	0..1

Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.12 DoIP/DoIPGeneral/DoIPRbGeneral/DoIPRbLite

Short Name	DoIPRbLite
Description	Specifies the functionality of DoIP to support payload types. TRUE This indicates that the functionality of DoIP is restricted to support only TCP payload type 0x8001. Other payload types are not supported. FALSE This indicates that the DoIP supports all payload types as per ISO 13400.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.13 DoIP/DoIPGeneral/DoIPRbGeneral/DoIPRbLiteAckNackEnable

Short Name	DoIPRbLiteAckNackEnable
Description	Specifies the functionality of DoIP to support payload types. TRUE This indicates that 0x0000 (Generic NACK), 0x8002 (Diag ACK) and 0x8003 (Diag NACK) payload also supported along with payload type 0x8001. FALSE This indicates that the DoIP supports all payload types as per ISO 13400. Note :- This parameter must be configured only when DoIPRbLite is configured or enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.14 DoIP/DoIPGeneral/DoIPRbGeneral/DoIPRbStbmTimeBaseReference

Short Name	DoIPRbStbmTimeBaseReference
Description	This parameter references the StbmSynchronizedTimeBase for Timestamp in Periodic Diagnostic Message 0x8004.
Multiplicity	0..1
Default Value	-

Type	ECUC-REFERENCE-DEF
------	--------------------

1.3.1.15 DoIP/DoIPGeneral/DoIPRbTlsActivationCallout

Short Name	DoIPRbTlsActivationCallout
Description	This parameter provides the function name for the callout DoIP_TlsActivationCallout. It is called by DoIP after an unsecured DoIP connection is established on a connection that the parameter DoIPTcpConnectionSecurityRequired set to False. If DoIP_TlsActivationCallout returns true, DoIP will close the unsecured connection with NRC 0x07 and require the tester to use TLS. If DoIPTlsActivationCallout is not configured or DoIP_TlsActivationCallout returns false, DoIP will accept the unsecured connection.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

1.3.1.16 DoIP/DoIPGeneral/DoIPRbTlsActivationCallout/DoIPTlsActivationCallout

Short Name	DoIPTlsActivationCallout
Description	If the DoIPTlsActivationCallout parameter exist the DoIP module shall call the configured callback function (<User>_DoIPTlsActivationCallout) direct. (It is not needed to specify a service port to the DoIP service component.)
Introduction	If the DoIPTlsActivationCallout parameter does NOT exist the DoIP module shall use a RPort with a CallbackTlsActivationCallout type of client-server port interface.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

1.3.1.17 DoIP/DoIPGeneral/DoIPUseEidAsGidRef

Short Name	DoIPUseEidAsGidRef
Description	Reference to the interface containing the EID which shall be used as GID. Depending on the configuration, the GID is derived either from DoIPEid or from DoIPUseMacAddressForIdentification.
Multiplicity	0..1
Default Value	-

Type	ECUC-REFERENCE-DEF
------	--------------------

1.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.2.1 DoIP_TpStatusIndication

Syntax	DoIP_TpStatusIndication(PduIdType PduId, const PduInfoType* PduInfoPtr, TpStatusType_ten TpStatusType_en, TpStatusCode_ten TpStatusCode_en)
Parameters	(IN) <i>PduId</i> - DoIP unique identifier of the PDU to be transmitted by the PduR.
	(IN) <i>PduInfoPtr</i> - Pointer to the Application Layer SID data.
	(IN) <i>StatusType</i> - Transport layer status type.
	(IN) <i>StatusCode</i> - Transport layer status code related to error info.
Return Value	NONE
Description	This service is called to transmit the TP status indication from the PduRouter to the SoAd.

1.4 Limitations

Version v12.7.0 has following limitations.

- The DoIPNumByteDiagAckNack parameter does not support Post-Build.
- DoIP doesn't support the internal buffer concept of Autosar, and instead uses the Zero Copy Concept to handle TCP messages.

1.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- DoIP/DoIPGeneral/DoIPUseEIDasGID

1.6 Deviations

1.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

1.6.1.1 DoIP/DoIPConfigSet/DoIPInterface/DoIPAliveCheckResponseTimeout

[RTA-BSW] Description	Timeout in [s] for waiting for a response to an Alive Check request before the connection is considered to be disconnected. Represents parameter T_TCP_Alive_Check of ISO 13400-2:2019.
[AUTOSAR] Description	Timeout in [s] for waiting for a response to an Alive Check request before the connection is considered to be disconnected. Represents parameter T_TCP_AliveCheck of ISO 13400-2:2012.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.2 DoIP/DoIPConfigSet/DoIPInterface/DoIPChannel/DoIPPduRTxPdu/DoIPPduType

[ADDED] Introduction	The functionality associated with this parameter is not supported yet. The PDU type is assumed to be DOIP_TPPDU by default.
----------------------	---

1.6.1.3 DoIP/DoIPConfigSet/DoIPInterface/DoIPConnections/DoIPTcpConnection

[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	2

1.6.1.4 DoIP/DoIPConfigSet/DoIPInterface/DoIPConnections/DoIPUdpConnection

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.5 DoIP/DoIPConfigSet/DoIPInterface/DoIPConnections/DoIPUdpConnection/DoIPSoAdUdpTxPdu

[ADDED] Introduction	If this DoIPUdpConnection is mapped to an IPv6 multicast connection in the lower layer, then the PDU referenced in this container is not used for Tx. Instead, the response to a request received over such a multicast connection will be sent via a unicast UDP connection (Hint: Only vehicle identification requests are supported over muticast).
----------------------	--

1.6.1.6 DoIP/DoIPConfigSet/DoIPInterface/DoIPEid

[RTA-BSW] Parent Container	DoIP/DoIPConfigSet/DoIPInterface/DoIPEid
----------------------------	--

[AUTOSAR] Parent Container	DoIP/DoIPConfigSet/DoIPEid
----------------------------	----------------------------

1.6.1.7 DoIP/DoIPConfigSet/DoIPInterface/DoIPGeneralInactivityTime

[RTA-BSW] Description	Timeout in [s] for maximum inactivity of a TCP socket connection before the DoIP module will close the according socket connection. Represents parameter T_TCP_General_Inactivity of ISO 13400-2:2019
[AUTOSAR] Description	Timeout in [s] for maximum inactivity of a TCP socket connection before the DoIP module will close the according socket connection. Represents parameter T_TCP_General_Inactivity of ISO 13400-2:2012
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.8 DoIP/DoIPConfigSet/DoIPInterface/DoIPInitialInactivityTime

[RTA-BSW] Description	Timeout in [s] used for initial inactivity of a connected TCP socket connection directly after socket connection. Represents parameter T_TCP_Initial_Inactivity of ISO 13400-2:2019
[AUTOSAR] Description	Timeout in [s] used for initial inactivity of a connected TCP socket connection directly after socket connection. Represents parameter T_TCP_Initial_Inactivity of ISO 13400-2:2012
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.9 DoIP/DoIPConfigSet/DoIPInterface/DoIPInitialVehicleAnnouncementTime

[RTA-BSW] Description	Time to wait in [s] for sending first vehicle announcement message after IP address assignment. Represents parameter A_DoIP_Announce_Wait of ISO 13400-2:2019
[AUTOSAR] Description	Time to wait in [s] for sending first vehicle announcement message after IP address assignment. Represents parameter A_DoIP_Announce_Wait of ISO 13400-2:2012
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.10 DoIP/DoIPConfigSet/DoIPInterface/DoIPInterfaceActLineCtrl

[RTA-BSW] Description	This attribute defines whether the network interface Tags: atp.Status=draft
[AUTOSAR] Description	This attribute defines whether the network interface
[RTA-BSW] Introduction	- is started "on-demand" when an activation line is sensed (TRUE) or - is always available (FALSE).
[AUTOSAR] Introduction	* is started "on-demand" when an activation line is sensed (TRUE) or * is always available (FALSE).

1.6.1.11 DoIP/DoIPConfigSet/DoIPInterface/DoIPInterfaceAnnouncementStart

[RTA-BSW] Description	This attribute defines, when vehicle announcement is started on a DoIPInterface Tags: atp.Status=draft
[AUTOSAR] Description	This attribute defines, when vehicle announcement is started on a DoIPInterface
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[REMOVED] Default Value	DOIP_AUTOMATIC_ANNOUNCE

1.6.1.12 DoIP/DoIPConfigSet/DoIPInterface/DoIPInterfaceId

[RTA-BSW] Description	This parameter is an identifier of the DoIPInterface. The value of this parameter will be assigned to the symbolic name derived from the container short name. Tags: atp.Status=draft
[AUTOSAR] Description	This parameter is an identifier of the DoIPInterface. The value of this parameter will be assigned to the symbolic name derived from the container short name.

1.6.1.13 DoIP/DoIPConfigSet/DoIPInterface/DoIPMaxTesterConnections

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.14 DoIP/DoIPConfigSet/DoIPInterface/DoIPTester/DoIPNumByteDiagAckNack

[RTA-BSW] Lower Multiplicity	0
------------------------------	---

[AUTOSAR] Lower Multiplicity	1
------------------------------	---

1.6.1.15 DoIP/DoIPConfigSet/DoIPInterface/DoIPTester/DoIPRoutingActivationRef

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.16 DoIP/DoIPConfigSet/DoIPInterface/DoIPUseMacAddressForIdentification

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.17 DoIP/DoIPConfigSet/DoIPInterface/DoIPUseVehicleIdentification-SyncStatus

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.18 DoIP/DoIPConfigSet/DoIPInterface/DoIPVehicleAnnouncement-Count

[RTA-BSW] Description	Number of vehicle announcement messages on IP address assignment. Represents parameter A_DoIP_Announce_Num of ISO 13400-2:2019.
[AUTOSAR] Description	Number of vehicle announcement messages on IP address assignment. Represents parameter A_DoIP_Announce_Num of ISO 13400-2:2012.

1.6.1.19 DoIP/DoIPConfigSet/DoIPInterface/DoIPVehicleAnnouncementInterval

[RTA-BSW] Description	Time to wait in [s] for sending subsequent vehicle announcement messages. Represents parameter A_DoIP_Announce_Interval of ISO 13400-2:2019
[AUTOSAR] Description	Time to wait in [s] for sending subsequent vehicle announcement messages. Represents parameter A_DoIP_Announce_Interval of ISO 13400-2:2012

1.6.1.20 DoIP/DoIPConfigSet/DoIPLogicalAddress

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.21 DoIP/DoIPGeneral/DoIPDhcpOptionVinUse

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.22 DoIP/DoIPGeneral/DoIPEntityStatusMaxByteFieldUse

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.23 DoIP/DoIPGeneral/DoIPGIDInvalidityPattern

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.24 DoIP/DoIPGeneral/DoIPHostNameSizeMax

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.25 DoIP/DoIPGeneral/DoIPMaxRequestBytes

[RTA-BSW] Description	Specifies the maximum allowed bytes of a DoIP message request without the DoIP header(8bytes) . Note: DoIP Payload length is sum of Source Address + Target Address (2 + 2 = 4Bytes) and Diagnostic message. For Eexample classic CAN handle max 4095 bytes, So DoIPMaxRequestBytes shall be configured to 4095 + 4 Bytes of SA and TA = 4099. for more details check DoIP-072 in ISO 13400-2
[AUTOSAR] Description	Specifies the maximum allowed bytes of a DoIP message request without the DoIP header.

1.6.1.26 DoIP/DoIPGeneral/DoIPMaxUDPRequestPerConnection

[RTA-BSW] Description	This parameter captures the maximum amount of UDP Requests which can be handled parallel within a single UDP connection. (These requests will be received in separate UDP messages. In one UDP message, only one request will be accepted)
[AUTOSAR] Description	This parameter captures the maximum amount of UDP Requests necessary to handle parallel within a single UDP connection.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.27 DoIP/DoIPGeneral/DoIPNodeType

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.28 DoIP/DoIPGeneral/DoIPPowerModeCallback

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.29 DoIP/DoIPGeneral/DoIPVinGidMaster

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.30 DoIP/DoIPGeneral/DoIPVinInvalidityPattern

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.7 API Definition

List of supported API functions

Table 1.50. DoIP Supported API functions

DoIP_Init	Initialises the module.
DoIP_MainFunction	Schedules the DoIP module.
DoIP_ActivationLineSwitch	Informs the DoIP implementation about the status of the activation line of a DoIP interface.
DoIP_GetVersionInfo	Returns version information for the module.
DoIP_IfCancelTransmit	Requests cancellation of an ongoing transmission of a PDU in a lower layer communication module.
DoIP_IfTransmit	Requests transmission of a PDU.
DoIP_LocalIpAddrAssignmentChg	Callback function which is called by lower layer to notify an IP address assignment change.
DoIP_SoAdTpCopyTxData	Callback function which acquires the transmit data of an I-PDU segment from the lower layer.
DoIP_SoAdTpTxConfirmation	Callback function which indicates success or failure of I-PDU transmission.
DoIP_SoAdTpCopyRxData	Callback function which provides the received data of an I-PDU segment to the upper layer.

DoIP_SoAdTpStartOfReception	Callback function called at the beginning of an N-SDU transmission.
DoIP_SoAdTpRxIndication	Callback function which indicates success or failure of an I-PDU reception.
DoIP_SoAdIfRxIndication	Callback function which indicates a received I-PDU from the lower layer.
DoIP_SoAdIfTxConfirmation	Callback function which is used by lower layer to confirm I-PDU transmission.
DoIP_SoConModeChg	Callback function which is called by lower layer to notify a socket connection state change.
DoIP_TpCancelReceive	Requests cancellation of an ongoing reception of a PDU in a lower layer transport protocol module.
DoIP_TpCancelTransmit	Requests cancellation of an ongoing transmission of a PDU in a lower layer communication module.
DoIP_TpTransmit	Requests the transfer of data from the PduR module to the SoAd module.
DoIP_TriggerVehicleAnnouncement	Notifies DoIP to start vehicle announcement for specified DoIP interface(s).

For further information on these functions, please refer to AUTOSAR_SWS_DiagnosticOverIP.pdf

1.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

1.8.1 General

Det development errors are only reported by DoIP if the configuration parameter DoIPDevelopmentErrorDetect is set to TRUE.

1.8.2 Interface Configuration

- If DoIPInterfaceActLineCtrl is set to TRUE for an interface, a PPort will be generated that will be used by all Interfaces on the server side. The Client/Appl must create multiple RPorts (depending on the number of Interfaces with DoIPInterfaceActLineCtrl set to TRUE) to connect to the same PPort.
- If there are no DoIP Interfaces with DoIPInterfaceActLineCtrl = TRUE, then no p-port of the type DoIPActivationLineSwitch will be generated.
- If DoIPInterfaceAnnouncementStart is set to DOIP_ONTRIGGER_ANNOUNCE

for an interface, a PPort will be generated that will be used by all Interfaces on the server side. The Client/Apl must create multiple RPorts (depending on the number of Interfaces with DoIPInterfaceAnnouncementStart = DOIP_ONTRIGGER_ANNOUNCE) to connect to the same PPort.

- If there are no DoIP Interfaces with DoIPInterfaceAnnouncementStart = DOIP_ONTRIGGER_ANNOUNCE, then no p-port of the type DoIPTriggerVehicleAnnouncement will be generated.
- For an interface with DoIPInterfaceActLineCtrl set to TRUE, the DoIPInterfaceAnnouncementStart parameter cannot be configured as DOIP_ONTRIGGER_ANNOUNCE because the Interface will be an External Interface.
- For an interface that does not have the DoIPUdpVehicleAnnouncementConnections parameter configured, DoIP will not send any Vehicle Announcements messages to the Tester.

1.8.3 DoIPRequestAddressAssignment

DoIP requests the IP address only for connections where DoIPRequestAddressAssignment is set to TRUE. If there are several connections on the same Socket Connection Group, DoIPRequestAddressAssignment must be set to TRUE for only one of these connections.

1.8.4 Code Generation based on configuration class

DoIP module supports 3 kinds of configuration class:

- Pre-compile
- Post-build Loadable
- Post-build Selectable

If either of the two Post-Build options is to be used for DoIP, the following two configuration steps must first be completed.

Step 1 Select the Configuration Class

Go to the DoIP module configuration, and then the “properties” tab.

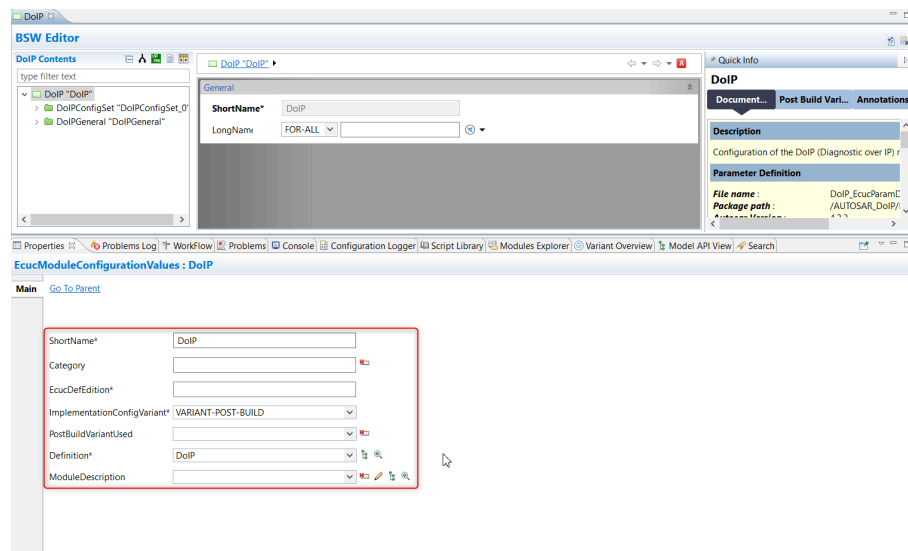


Figure 1.2. Select which configuration class of DoIP

ImplementationConfigVariant will be set to “variant-pre-compile” by default, but if either of the two Post-Build options will be used for DoIP, you must select the generic “variant-post-build” option (and not the specific “variant-post-build-loadable” or “variant-post-build-selectable” options).

Step 2 EcuM Configuration

If ImplementationConfigVariant is set to “variant-post-build”, the EcuM configuration for DoIP and all its dependent modules must be configured with POST-BUILD_PTR.

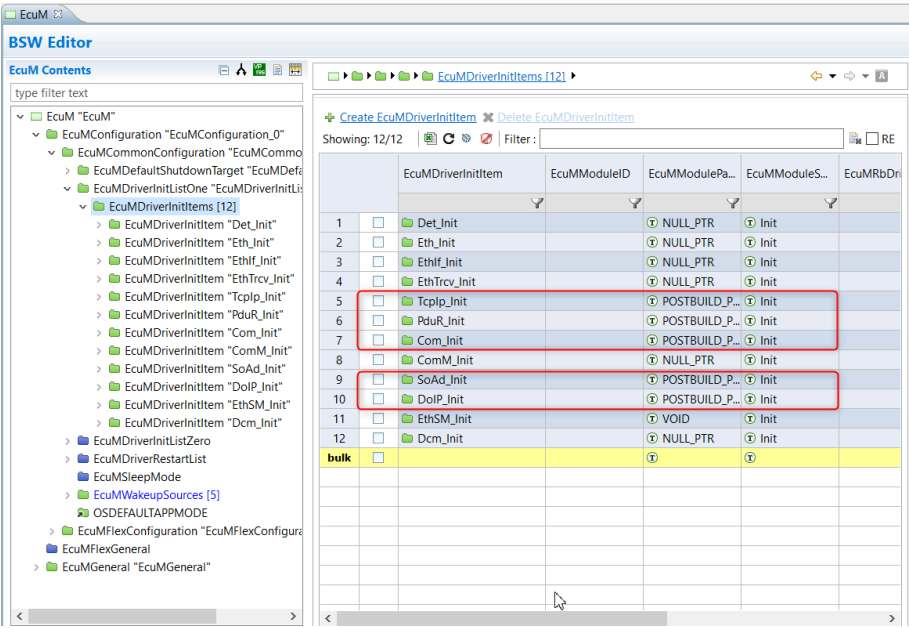


Figure 1.3.EcuM configuration

1.8.5 Code Generation Options

Depending on the above setting of ImplementationConfigVariant, the following options can be deployed at Code Generation.

Pre-Compile

If this option is chosen, all ECUs using the generated BSW will have the same configuration and there is no way to change this configuration without re-generating and re-compiling the whole of the ECU's software.

As shown below, select the "DoIP_PC" option for the DoIP module. The other options on the right can be set at your discretion.

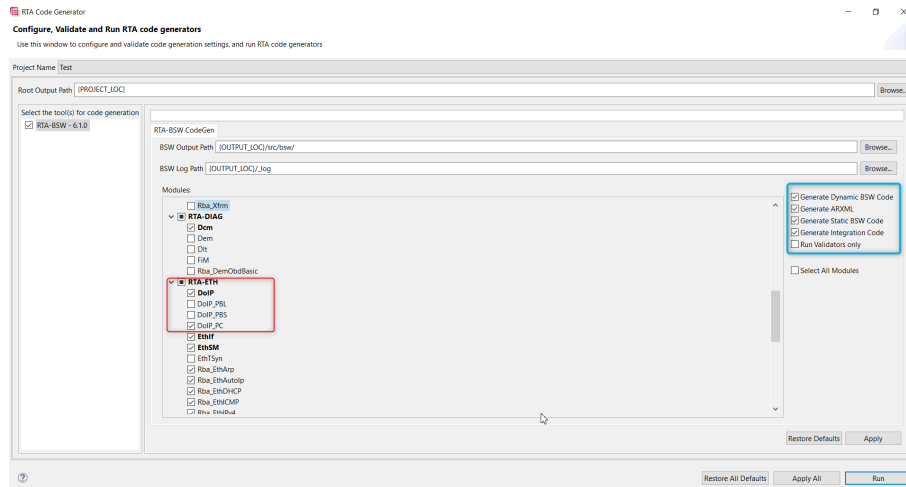


Figure 1.4. Generate DoIP with Configuration Class as Pre-compile

Post-Build Loadable

This option results in the generation of only the configuration variant for the DoIP module and each configuration variant can be individually reloaded, no runtime selection.

As shown below, the DoIP_PBL option is ticked.

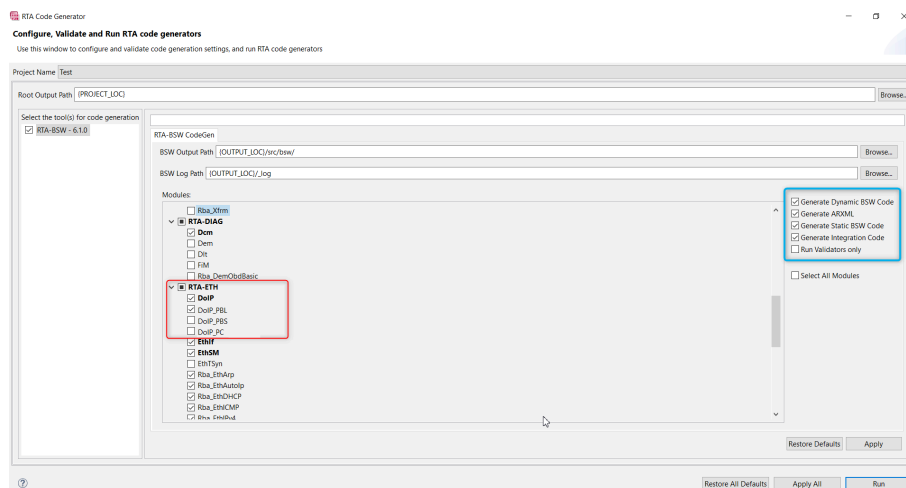


Figure 1.5. Code Generation with Loadable Post-Build

Post-Build Selectable

This option is used when multiple configurations for DoIP will be created, and the required configuration can be selected at run time.

As shown below, the DoIP_PBS option is ticked.

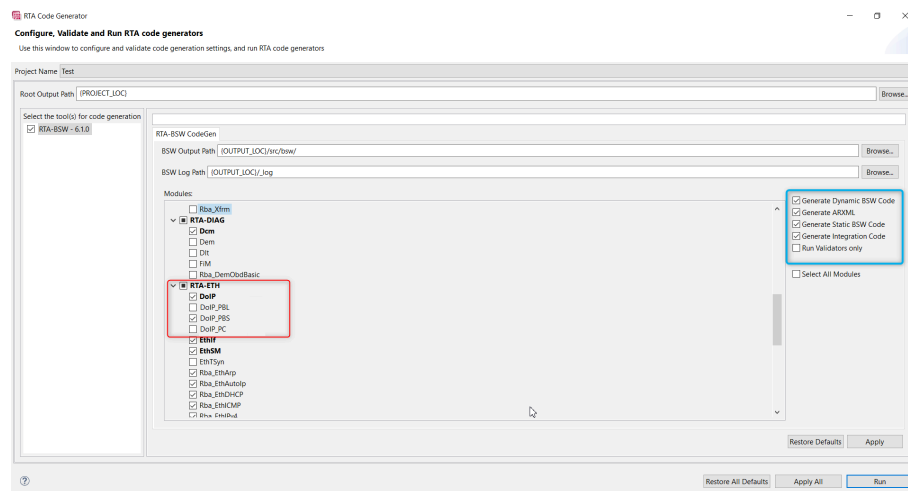


Figure 1.6.Code Generation with Selectable Post-Build

1.9 Integration Advice

1.9.1 Init Functions

DolP is initialized by calling DolP_Init. It should be initialized before PduR and after SoAd.

1.9.2 Delnit Functions

No Delnit is required.

1.9.3 Scheduled Functions

The DoIP main function should be scheduled for cyclic operation, and it should only be called after the PduR and Dcm modules are running. If the DoIP main function were to be started first, incoming messages from SoAd would be ignored and discarded until PduR is active.

1.9.4 Interaction with other modules

For successful integration of the Dolp module, the Socket Adapter (SoAd), PDU Router (PduR) and Diagnostic Communication Manager (Dcm) modules must also be present in the target project. The following diagram shows the interaction between these modules.

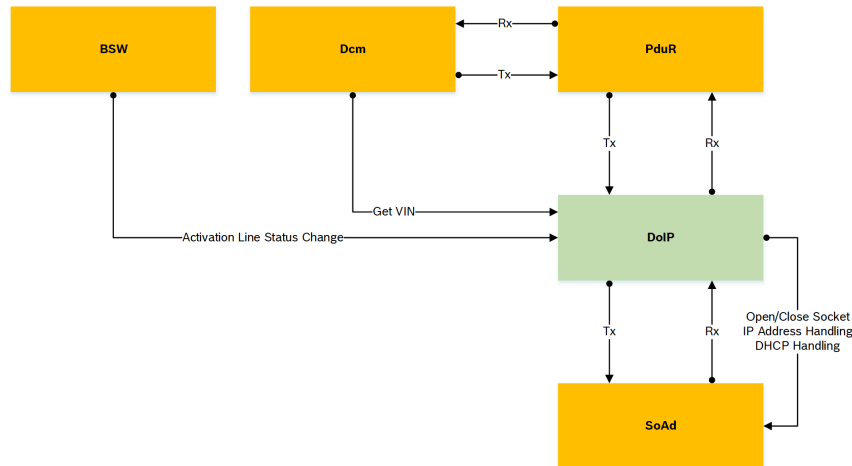


Figure 1.7. DoIP Interaction

1.9.5 Socket Adapter (SoAd) Configuration

The functionality of DoIP is dependent on the configuration of the Socket Adapter (SoAd) module. Configure one `SocketConnectionGroup` for TCP connections and one for UDP connections (including UDP vehicle announcement connections, if applicable) of each `DoIPInterface`.

The following SoAd parameter settings are required for DoIP TCP connections.

- `SoAdPduHeaderEnable` = FALSE
- `SoAdSocketAutomaticSoConSetup` = FALSE
- `SoAdSocketIpAddrAssignmentChgNotification` = FALSE
- `SoAdSocketLocalPort` = valid local port number or 0 for an ephemeral port
- `SoAdSocketSoConModeChgNotification` = TRUE
- `SoAdSocketLocalAddressRef` = reference to some `/Tcplp/TcplpConfig/TcplpLocalAddr`
- `SoAdSocketProtocol` = `SoAdSocketTcp`
- `UL_SoConModeChg` = `DoIP_SoConModeChg`
- `SoAdSocketRemoteIpAddress` - wildcard addressing is allowed, if `SoAdSocketRemotePort` is also specified as a wildcard.
- `SoAdSocketRemotePort` - wildcard addressing is allowed, if `SoAdSocketRemoteIpAddress` is also specified as a wildcard.
- `SoAdRxUpperLayerType` = TP
- `SoAdUpperLayerRxPduId` = <To be filled in here manually that DoIP Rx Pdu Id corresponding to `SoAdRxPduRef`>
- `SoAdRxUser` = DoIP
- `SoAdUpperLayerTxPduId` = <To be filled in here manually that DoIP Tx Conf Pdu Id corresponding to `SoAdTxPduRef`>

- SoAdRxSocketConnOrSocketConnBundleRef = < Should be referred only to SoAdSocketConnection >.
- SoAdTxSocketConnOrSocketConnBundleRef = < Should be referred only to SoAdSocketConnection >.
- SoAdSocketRouteDest = < For DoIP specific connection groups, only the first SoAdSocketRouteDest will be considered, even though it has upper multiplicity * >

The following SoAd parameter settings are required for DoIP UDP connections.

- SoAdPduHeaderEnable = FALSE
- SoAdSocketAutomaticSoConSetup = FALSE
- SoAdSocketIpAddrAssignmentChgNotification = TRUE
- SoAdSocketLocalPort = valid local port number or 0 for an ephemeral port
- SoAdSocketSoConModeChgNotification = TRUE
- SoAdSocketLocalAddressRef = reference to some /Tcplp/TcplpConfig/TcplpLocalAddr
- SoAdSocketProtocol = SoAdSocketUdp
- SoAdSocketUdpListenOnly = FALSE
- SoAdSocketUdpRetryEnabled = FALSE
- UL_LocalIpAddrAssignmentChg = DoIP_LocalIpAddrAssignmentChg
- UL_SoConModeChg = DoIP_SoConModeChg
- SoAdSocketRemotelpAddress - wildcard addressing is allowed, if SoAdSocketRemotePort is also specified as a wildcard.
- SoAdSocketRemotePort - wildcard addressing is allowed, if SoAdSocketRemotelpAddress is also specified as a wildcard.
- SoAdUpperLayerRxPduId = < To be filled in here manually that DoIP Rx Pdu Id corresponding to SoAdRxPduRef >
- SoAdRxUser = DoIP
- SoAdRxUpperLayerType = IF
- SoAdUpperLayerTxPduId = < To be filled in here manually that DoIP Tx Conf Pdu Id corresponding to SoAdTxPduRef >
- SoAdRxSocketConnOrSocketConnBundleRef = < Should be referred only to SoAdSocketConnection >.
- SoAdTxSocketConnOrSocketConnBundleRef = < Should be referred only to SoAdSocketConnection >.
- SoAdSocketRouteDest = < For DoIP specific connection groups, only the first SoAdSocketRouteDest will be considered, even though it has upper multiplicity * >

1.9.6 TcplpLocalAddr

For all TCP connections relating to DoIP:

- The TcplpAssignmentTrigger parameter in the Tcplp configuration (in TcplpAddrAssignment) must be set to TCPIP_MANUAL.
- The parameter TcplpAddressType shall be set to TCPIP_UNICAST.

For all UDP connections relating to DoIP:

- The parameter TcplpAddressType shall be set to TCPIP_UNICAST or TCPIP_MULTICAST (but only if TcplpDomainType is set to TCPIP_AF_INET_6 and is not a UDP Vehicle Announcement connection).

1.9.7 Doip - DoIPTcpConnection had lower multiplicity as 2

As per ISO each DoIP entity shall support $\langle n+1 \rangle$ TCP data sockets (where $\langle n \rangle$ is the number of concurrent TCP data connections supported by the respective DoIP entity). Therefore when configuring, one extra DoIP connection is always required. Each DoIPTcpConnection is linked to each SoAdSocketConnection, and each SoAdSocketConnection has a different remote Port and IpAddress. As shown in the image below, in the case of $\langle n+1 \rangle$ TCP data sockets, for each socket group, you should configure SoAdSocketRemoteIpAddress as "ANY" and SoAdSocketRemotePort as "0".

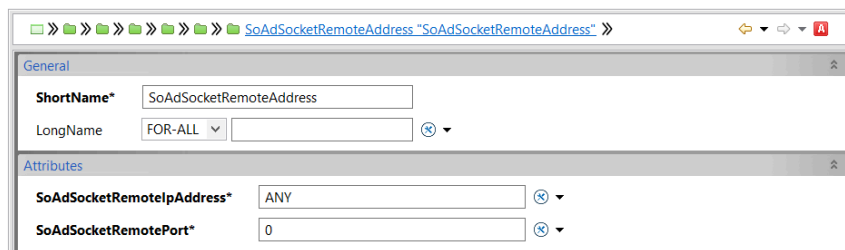


Figure 1.8. SoAdSocketConnection

1.9.8 Post Build Selectable

When Post-build Selectable is used with multiple variants, the target variant can be selected by using the EcuM_DeterminePbConfiguration user-defined API from the EcuM integration. Please refer to the relevant integration advice for EcuM for more detailed information.

1.9.9 PduR

A communication interface is supported for the DoIP module in PduR, but the Rx path and trigger transmission is not supported for DoIPIf.

For TP communication, DoIP should be configured, and DoIPTp should be used as the shortname under PduRBswModules.

For IF communication with DoIP, DoIPIf should be used as the shortname in PduRBSwModules.

1.9.10 DoIP Lite Configuration

When DoIP Lite is enabled many Autosar/RB specific parameters are unavailable and therefore blocked for editing by the policy file, and include:

- No UDP connections and UDP Vehicle Announcement connections shall be configured when DoIP Lite is enabled.
- Only one TCP connection per Interface shall be configured when DoIP Lite is enabled.
- The *DoIPInterfaceActLineCtrl* parameter shall be configured as FALSE when DoIP Lite is enabled.
- For parameter *DoIPTcpConnection*, with DoIP Lite enabled the lower multiplicity is 1, and when disabled the lower multiplicity is 2.

1.9.11 Diagnostic Periodic Message Payload 0x8004 Configuration

- The ISO 13400:2 2023 Amd 1 protocol version should be enabled to support the 0x8004 communication.
- At least one DoIP Channel only with IF Rx Pdu should be configured.
- The following parameters should be configured *DoIPRbChannelBufferSize*, *DoIPRbPeriodicDataTxTimeout* and *DoIPRbStbmTimeBaseReference*.
- The Diagnostic Periodic Message Payload 0x8004 has a dependency on STBM module.

1.9.12 Transport Protocol Payload 0x9001 Configuration

- The ISO 13400:2 2023 Amd 1 protocol version should be enabled to support 0x9001 communication.
- DoIPNodeType should be configured as DOIP_GATEWAY.
- The *DoIPRbTPPayloadTypeSupported* and *DoIPRbApplicationServiceIDSupported* should be configured.

1.9.13 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual imple-

mentationof the corresponding BSW API calls to ensure exclusivity.

The DoIP module uses the following OS resources:

- DoIP_StateMonitoring

1.10 **Hardware Requirements**

Table 1.51.DoIP Hardware Requirements

Support	Usage
FLASH	12884 bytes
RAM	1579 bytes

Usage calculated based on a sample configuration.

2 Ethernet Interface

Module name	EthIf
Package	RTA-ETH
AUTOSAR Module ID	65
AUTOSAR Version	22-11

2.1 Introduction

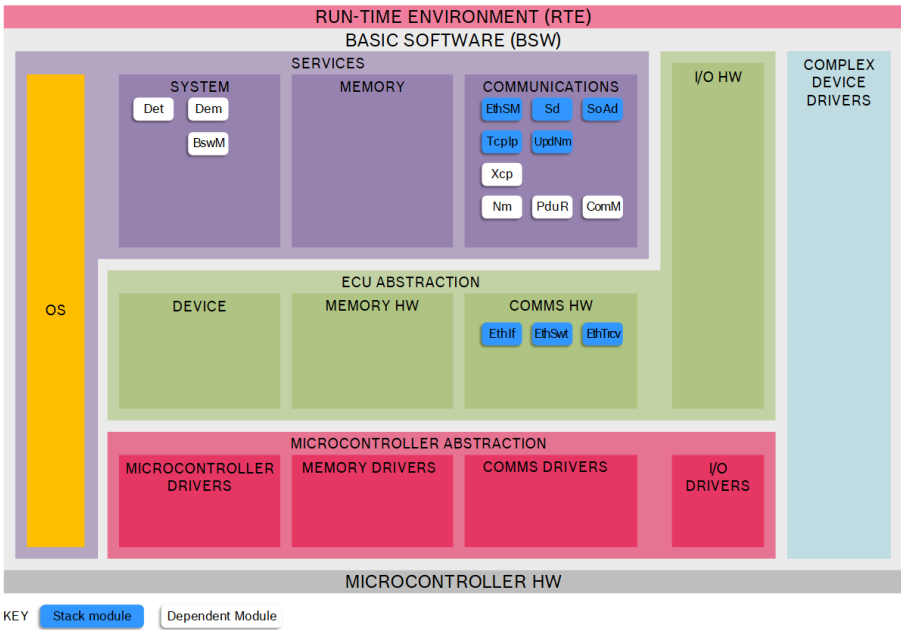


Figure 2.1.RTA-ETH Architecture

The Ethernet Interface module (EthIf) is located in the communication services HW abstraction layer, acting as the lower layer for the communication protocol modules in the TCP/IP stack (encapsulated by the TcpIp module), the Ethernet State Manager (EthSM) module and any CDD communication modules. The diagram above shows the position of EthIf in the AUTOSAR layered architecture.

EthIf provides a hardware-independent interface to the low-level communication system, which contains the Ethernet controller, transceiver and switch drivers that (via the Eth, EthTrcv and EthSwt drivers in the lower layer) provide hardware-specific access to ECUs which utilise an Ethernet-based network to send and receive data and control messages.

This allows the upper layer modules to access the communications bus system in a uniform manner without any knowledge of the underlying hardware.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually

configured by the user.

2.2 Supported Features

2.2.1 General

The EthIf module provides the following functionality:

- Message Transmission.
- Message Reception.
- VLAN Support.
- EthIfCtrl and PNC port group state handling.
- Support for different AR versions of the Ethernet Driver (e.g. AR v4.0.3, v4.2.2, v22-11).
- Wakeup and Sleep on Dataline.
- EthIf supports the calling to the Firewall module for inspecting every received packet.
- EthIf provides an interface to manage PTP hardware clocks operated by the Ethernet controller driver.

2.2.2 Transmission (via the AR Interface)

EthIf performs the following actions for the successful transmission of an Ethernet frame using the AUTOSAR interface.

2.2.2.1 Buffer Request

The upper layer of EthIf requests access to a transmit buffer of the Ethernet controller (EthIf_ProvideTxBuffer), specifying the indexed EthIf controller, the Ethernet Type and the desired number of bytes to be transmitted. Depending on the virtual controller configuration, 4 bytes are additionally added to the desired length if VLAN is used.

The physical Ethernet controller index mapped to the virtual controller index is fetched.

If the desired length is less than or equal to the maximum transmission unit configured, the request is forwarded to Eth_ProvideTxBuffer.

On successful allocation of a transmit buffer, if VLAN is used, the VLAN Id, Ethernet frame type and the priority (PCP) field is filled in the buffer, and the buffer pointer is shifted to the beginning of the IP payload. If VLAN is not configured, the buffer pointer is already at the beginning of the IP payload.

2.2.2.2 Override source MAC Address

The upper layer of EthIf requests the overwriting of the source MAC address of the frame to be transmitted (via the EthIf_OverrideTxBufSrcAddr API). This feature is optional.

2.2.2.3 Frame Transmission and Confirmation

The upper layer fills the transmit buffer and triggers Ethlf_Transmit. Successful transmission is handled in two modes:

- Polling mode: Ethlf_MainfunctionTx triggers Eth_TxConfirmation for any pending confirmations. All pending confirmations are issued through Ethlf_TxConfirmation. The confirmation is forwarded to the configured callback functions User_TxConfirmation.
- Interrupt mode: After transmission is completed, an interrupt triggers Ethlf_TxConfirmation through the Eth module. The call is forwarded to the configured callback functions.

2.2.2.4 TxConfirmation handling

The input parameters of the Ethlf_TxConfirmation API are not enough to convert the physical Ethernet controller index provided by the Ethernet driver (Ethlf-PhysControllerIdx) into the virtual controller index (EthlfCtrlIdx) that is passed to the upper layer of Ethlf. There are two ways to make this conversion possible:

1. By using the AUTOSAR conform API and storing the required data during TX in an array. The data is then obtained when a TxConfirmation is triggered.
2. Extending the API so additional parameters are passed from the Ethernet driver to obtain the required data from there. This implementation is more efficient, as no additional data needs to be stored and searched for. The data can be extracted directly from the two additional parameters (DataPtr and its length).

2.2.3 Transmission (via the ZeroCopy Interface)

Ethlf performs the following actions for the successful transmission of an Ethernet frame using the ZeroCopy interface.

2.2.3.1 Header Filling

The upper layer of Ethlf requests the filling of the ZeroCopy header buffer (Ethlf_ZeroCopyFillHeader), specifying the indexed Ethlf controller, the Ethernet Type, the Frame Priority and the Physical target address (MAC address). The upper layer also provides a ZeroCopy argument containing at least one header buffer of 18 bytes.

Depending on the virtual controller configuration, Ethlf writes the Ethernet header fields from index 4 if VLAN is not used, or from index 0 if VLAN is used.

The destination MAC address, source MAC address and Ethernet frame type are filled into the header buffer given in the Zerocopy argument.

If VLAN is used, the TPID, priority (PCP) and VLAN Id fields are filled in the header buffer given in the Zerocopy argument.

2.2.3.2 ZeroCopy Transmission

The upper layer calls `EthIf_ZeroCopyTransmit` to request the transmission of a ZeroCopy Ethernet frame, reusing the same ZeroCopy argument as the one provided previously in the call of `EthIf_ZeroCopyFillHeader`. It also fills the ZeroCopy argument with a data buffer (containing the payload of the Ethernet frame) and the desired number of bytes to be transmitted.

The `TxConf` and the `TriggerTx` flags are also set by the upper layer in the ZeroCopy argument. These flags will be used by the Eth module to know if TxConfirmation callback needs to be invoked, and if the transmission needs to be placed directly on the bus.

`EthIf_ZeroCopyTransmit` performs the following actions:

- It checks if the desired length is less than or equal to the maximum transmission unit configured.
- It fetches the physical Ethernet controller mapped to the virtual controller.
- If the ZeroCopy argument contains `TxConf` set to `TRUE`, it returns a unique `TxId` and the driver layer confirms a successful transmission (`EthIf_ZeroCopyTxConfirmation`).
- After transmission is completed, the Eth module's `MainFunction` triggers `EthIf_TxConfirmation` with the same `TxId` as the one returned by `EthIf_ZeroCopyTransmit`. The call is forwarded to the configured callback functions.

`ZeroCopyTxConfirmation` must be configured so that the upper layer of `EthIf` will know when to release the allocated header and data buffers.

2.2.4 Reception

Reception of Ethernet frames is handled in two modes, Polling Mode and Interrupt Mode.

2.2.4.1 Polling mode

Every scheduled cycle, `EthIf_MainFunctionRx_{IngressQueueProcessing ShortName}()` triggers the `Eth_Receive` API with corresponding FIFO index. If more data is available in the driver, `Eth_Receive` is triggered multiple times.

Polling can be optionally limited by configuring the maximum number of frames that can be polled in one main function and/or by the maximum run time of the main function, after which polling is stopped. `Eth_Receive` calls `EthIf_RxIndication` if a frame is received.

The Ethernet type is checked for VLAN tagging in `EthIf_RxIndication`. If the type is VLAN, then the VLAN ID and the Ethernet Type is checked. If it's not VLAN, the `Frametype` input parameter is used. The `EthIf` controller index is extracted using the physical controller index and the VLAN ID.

The configured upper layer is identified using the Ethernet frame type and the `EthIf` controller index. The buffer is pointed to the IP payload, and is forwarded to the registered callback function's `User_RxIndication`.

2.2.4.2 Interrupt mode

A frame reception interrupt triggers the `EthIf_RxIndication` API through the `Eth` module. The handling in `EthIf_RxIndication` is the same as described above for Polling Mode.

2.2.5 Security Events

`EthIf` supports calling the `IdsM` module for reporting security events (double tagging detection, invalid EtherType, invalid `CtrIdx/Vlan ID`, MAC address collision).

2.2.6 Firewall

If `EthIf`, `EthIfGeneral` or `EthIfFwSupport` parameter is configured with `FIREWALL_WITHOUT_PERSTREAMFILTERING` or `FIREWALL_WITH_PERSTREAMFILTERING`, then `EthIf` will execute packet inspection during reception. With the inspection achieved by calling the Firewall API `Fw_InspectPacket()`.

When the received frame arrives through a switch supporting `EthSwtStreamIdentificationTable` configuration and if `EthIf`, `EthIfGeneral` or `EthIfFwSupport` parameter is configured with `FIREWALL_WITH_PERSTREAMFILTERING`, then `EthIf` will additionally invoke `EthSwt_ExtractStreamHandleIdx()` API during reception. The extracted `StreamHandleIdx` from the switch vendor specific part of the network packet header will be given to the Firewall module during the call of `Fw_InspectPacket()`.

If `Fw_InspectPacket()` returns `E_OK` with `FW_ALLOW_NETWORK_PACKET` as `InspectionResultPtr`, then `EthIf` will execute further processing of the packet and forward it to UL if remaining acceptance criteria has passed, otherwise, the packet will be silently dropped.

The `EthIf` APIs `EthIf_GetStreamHandleIdxStatistics()` and `EthIf_SetStreamHandleIdxConfiguration()` are supported and are used by `Fw` component to interact with Ethernet Switch Driver. These APIs are needed to get the bucket counter statistics from the switch and to control the activity status of a `StreamHandleIdx` in the switch. Once the actions are performed by `EthSwt` component, the `EthIf` callbacks `EthIf_StreamHandleIdxStatistics()` and `EthIf_StreamHandleIdxConfiguration()` will be invoked which will forward the call to respective `Fw` callbacks.

2.2.7 EthIf Controller State Machine

`EthIf` implements a state machine per `EthIfCtrl`, and it processes the `EthIfCtrl` mode change requests of `EthSM` according to the configuration of `EthIfCtrl`.

`EthIf` brings all HW mapped to the `EthIfCtrl` to the requested mode. `EthIfCtrl` must always reference `EthIfPhysController`, and optionally to `EthIfTransceiver` or `EthIfSwitch` or `EthIfSwitchPortGroup` (or non of them). This results in following configuration possibilities:

- `EthIfCtrl` has reference to `EthIfTransceiver`.
- `EthIfCtrl` has reference to `EthIfSwitch`.

- EthIfCtrl has reference to EthIfSwitchPortGroup of type "control". The mode is changed by EthIf by request from EthSM.
- EthIfCtrl has reference to EthIfSwitchPortGroup of type "link-information". The mode is never changed. EthIfSwitchPortGroup is only used for link status accumulation towards EthSM. The mode of switch ports referenced by "link-information" is changed either by BswM via EthIfSwitchPortGroup not referenced by any EthIfCtrl (PNC EthIfSwitchPortGroup) or by EthIf via EthIfSwitchPortGroup of type "control" referencing the same ports.
- EthIfCtrl has no reference to EthIfTransceiver nor to EthIfSwitch nor to EthIfSwitchPortGroup.

2.2.7.1 PNC Switch Port Group State Machine

EthIfSwitchPortGroup can be configured so that is not referenced by any EthIfCtrl (PNC switch port group). For this type of port group semantic is not relevant. PNC switch port group mode is controlled by BswM using the EthIf_SwitchPortGroupRequestMode or EthIf_StartAllPorts APIs. EthIf implements a state machine per PNC switch port group, and it triggers actions required to bring the port group into the requested state.

2.2.8 Global Time Synchronization

EthIf provides interface APIs for the upper layer (EthTSyn) for the distribution of time information over Ethernet.

The following APIs are as per AR v19-11 specification and based on the Precision Time Protocol (PTP) standard. The function calls are forwarded to the Eth module.

- EthIf_GetCurrentTime
- EthIf_EnableEgressTimeStamp
- EthIf_GetEgressTimeStamp
- EthIf_GetIngressTimeStamp

The following APIs are based on RFC77618. The function calls are forwarded to the Switch module.

- EthIf_GetRxMgmtObject
- EthIf_GetTxMgmtObject

2.2.9 Transceiver diagnostic APIs

EthIf provides transceiver diagnostic APIs for the upper layer to get the status of a transceiver. The following APIs are as per AR v22-11 specification:

- EthIf_SetPhyTestMode
- EthIf_SetPhyTxMode
- EthIf_SetPhyLoopbackMode

- EthIf_GetCableDiagnosticsResult
- EthIf_GetPhyIdentifier
- EthIf_GetTransceiverMode
- EthIf_TransceiverGetLinkState
- EthIf_TransceiverGetBaudRate
- EthIf_TransceiverGetDuplexMode
- EthIf_RunCableDiagnostic
- EthIf_TransceiverGetMacMethod

The function calls are forwarded to the EthTrcv module if the below driver supports the API.

The EthIf Transceiver Index, in the context of the Ethernet Interface, is translated to the Transceiver Index in the context of the driver component before forwarding the function calls to the EthTrcv module.

2.2.10 Switch diagnostic APIs

EthIf provides switch diagnostic APIs for the upper layer to get the status of a switch.

The following APIs are as per AR v20-11 specification. The function calls are forwarded to EthSwt module.

- EthIf_DeletePortMirrorConfiguration
- EthIf_GetArlTable
- EthIf_GetPortCableDiagnosticsResult
- EthIf_GetPortMacAddr
- EthIf_GetSwitchIdentifier
- EthIf_GetSwitchPortIdentifier
- EthIf_GetSwitchPortMode
- EthIf_ReadPortMirrorConfiguration
- EthIf_RunPortCableDiagnostic
- EthIf_SetPortLoopbackMode
- EthIf_SetPortTestMode
- EthIf_SetPortTxMode
- EthIf_SwitchGetCfgDataRaw
- EthIf_SwitchGetCfgDataInfo
- EthIf_SwitchPortGetBaudRate
- EthIf_SwitchPortGetCounterValues
- EthIf_SwitchPortGetDuplexMode
- EthIf_SwitchPortGetLinkState

- EthIf_SwitchPortGetMacLearningMode
- EthIf_SwitchPortGetMaxFIFOBufferFillLevel
- EthIf_SwitchPortGetRxStats
- EthIf_SwitchPortGetTxErrorCounterValues
- EthIf_SwitchPortGetTxStats
- EthIf_WritePortMirrorConfiguration

The EthSwt_GetSwitchPortMode API is implemented as per the AR v19-11 specification.

The EthSwt_SetPortMirrorState and EthIf_GetPortMirrorState are implemented as per AR v20-11 specification.

The Switch index in the context of EthIf is translated to Switch index in the context of EthSwt before any call is forwarded to EthSwt module.

2.2.11 Signal Quality

If they are enabled, EthIf periodically polls the signal quality results from EthTrcv/EthSwt in the EthIf_MainFunctionState API. The polled results are stored in EthIf global variable.

The Upper layer can read the polled signal quality results via the EthIf_GetTrcvSignalQuality and EthIf_GetSwitchPortSignalQuality APIs, and it can clear the polled signal quality results via the EthIf_ClearTrcvSignalQuality and EthIf_ClearSwitchPortSignalQuality APIs.

2.2.11.1 MacSec Support in EthIf

EthIf only supports MacSec for hardware with transceiver functionality.

MacSec Wrapper APIs

EthIf provides MacSec related wrapper APIs for transceivers which support MacSec functionality.

The following APIs are as per AR v22-11 specification. The function calls are forwarded to EthTrcv module.

- EthIf_MacSecAddRxSa
- EthIf_MacSecAddTxSa
- EthIf_MacSecDeleteRxSa
- EthIf_MacSecDeleteTxSa
- EthIf_MacSecGetMacSecStats
- EthIf_MacSecGetTxSaNextPn
- EthIf_MacSecInitRxSc
- EthIf_MacSecResetRxSc
- EthIf_MacSecUpdateRxSa
- EthIf_MacSecUpdateSecY

- EthIf_MacSecUpdateTxSa
- EthIf_MacSecSetControlledPortEnabled

EthIf additionally provides specific EthIf_MacSecResetTxSc wrapper API for transceivers which support MacSec functionality.

EthIfTransceiver index in the context of Ethernet Interface is translated to transceiver index in the context of the driver component before any call is forwarded to EthTrcv module.

MacSec MKA Sequence

EthIf triggers MKA_LinkStateChange() for every link state transition for the referenced transceiver which has MacSec capability. The ethernet transceiver shall have a 1:1 mapping to an MkaPaeInstance, and is ensured by validation script. When MKA_LinkStateChange() is called with an active link state then Mka will start MacSec establishment.

When Mka process to secure the related EthTrcv is completed, the Mka will call EthIf API EthIf_MacSecOperational() with MacSecOperational argument set to True. In case MacSec channel is lost, then EthIf_MacSecOperational() will be called with a False value.

The EthIf will not inform link state active to EthSM until EthIf_MacSecOperational() is called with a True value. This means EthSM will remain in ETHSM_STATE_WAIT_TRCVLINK during MacSec establishment.

If EthIf_MacSecOperational() is called with a False value then EthIf will inform the link state down to EthSM using EthSM_TrvcLinkStateChg() API.

EthIf blocks all the traffic except EAPOL frames, these are required for MKA sequence when EthIf controller is MacSec protected but not yet operational.

2.2.12 Wakeup And Sleep On Dataline

EthIf facilitates the use of Wakeup and Sleep on Dataline, an OA TC10-compliant AUTOSAR feature which enables the use of a dataline, rather than a wakeup line, in a network to send and receive such requests.

The advantage lies in reducing hardware costs and simplifying the wiring between ECUs. This approach is arguably more efficient and with less latency.

An ECU, connected to a dedicated port and acting as a communication Master, can force via an Ethernet transceiver or switch, the Slave ECU to either wake up or sleep.

Additionally, if more than one communication Master is connected to the Ethernet transceiver/switch, then the Slave ECU has to maintain the communication request of the Masters.

EthIf interacts with the EthSM, EthTrcv, EthSwt and BswM modules when Wakeup and Sleep on Dataline is enabled.

For more details, please see Section 2.8.22.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 EthIf/EthIfConfigSet/EthIfFrameConfig

Short Name	EthIfFrameConfig
Description	Configuration of an Ethernet frame. Tags: atp.Status=draft
Introduction	The EthIfFrameConfig container is used to map Ethernet frames with a particular EtherType to an UpperLayer.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.2 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameRxPool

Short Name	EthIfFrameRxPool
Description	Pool of Rx PDUs for the upper layer modules. Tags: atp.Status=draft
Introduction	Several container instances can be defined for different EthIfControllers.
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.3 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameRxPool/ EthIfFrameRxControllerRef

Short Name	EthIfFrameRxControllerRef
Description	Optional reference to an EthIfController. If this reference is defined then only messages to/from that EthIfController are assigned to the Pdu. Tags: atp.Status=draft
Introduction	With this setup it is possible to define VLAN specific PDUs for dedicated EtherTypes.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.4 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameRxPool/ EthIfFrameRxPdu

Short Name	EthIfFrameRxPdu
Description	PDU in the EthIfFrameRxPool to be used for the transport of a the corresponding Ethernet frame. Tags: atp.Status=draft
Introduction	Supported MetaDataItemTypes: * TIMETUPLE_TYPE_PTR * ETHERNET_MAC_64 * BROADCAST_8
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.5 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameRxPool/ EthIfFrameRxPdu/EthIfFrameRxPduId

Short Name	EthIfFrameRxPduId
Description	PDU ID used by the upper layer module to indicated completed asynchronous reception of the corresponding Ethernet frame. Tags: atp.Status=draft
Multiplicity	1..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.6 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameRxPool/ EthIfFrameRxPdu/EthIfFrameRxPduRef

Short Name	EthIfFrameRxPduRef
Description	Reference to the global PDU representing the Ethernet frame to the upper layer module. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.7 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameTxPool

Short Name	EthIfFrameTxPool
Description	Pool of Tx PDUs for the upper layer modules. Tags: atp.Status=draft
Introduction	Several container instances can be defined for different EthIfControllers or with different EthIfFrameTxPriority.
Multiplicity	0..*

Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.8 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameTxPool/ EthIfFrameTxControllerRef

Short Name	EthIfFrameTxControllerRef
Description	Optional reference to an EthIfController. If this reference is defined then only messages to/from that EthIfController are assigned to the Pdu. Tags: atp.Status=draft
Introduction	With this setup it is possible to define VLAN specific PDUs for dedicated EtherTypes.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.9 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameTxPool/ EthIfFrameTxPdu

Short Name	EthIfFrameTxPdu
Description	PDU in the EthIfFrameTxPool to be used for the transport of the corresponding Ethernet frame. Tags: atp.Status=draft
Introduction	Supported MetaDataItemTypes: * ETHERNET_MAC_64 * TIMETUPLE_TYPE_PTR * LISTELEM_PTR * PRIORITY_8
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.10 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameTxPool/ EthIfFrameTxPdu/EthIfFrameTxPduld

Short Name	EthIfFrameTxPduld
Description	PDU ID used by the upper layer module to transmit the corresponding Ethernet frame. Tags: atp.Status=draft
Multiplicity	1..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.11 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameTxPool/ EthIfFrameTxPdu/EthIfFrameTxPduRef

Short Name	EthIfFrameTxPduRef
Description	Reference to the global PDU representing the Ethernet frame to the upper layer module. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.12 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameTxPool/ EthIfFrameTxPriority

Short Name	EthIfFrameTxPriority
Description	Definition of a fixed VLAN priority of the pool. Tags: atp.Status=draft
Introduction	This priority shall only be configured if the EthIfController referenced by this EthIfFrameTxPool has a EthIfVlanId. If this parameter is configured, the TxPdus of this pool shall not use MetaDataItems of type PRIORITY_8.
Multiplicity	0..1
Value Range	0..7
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.13 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfRbRxBypass

Short Name	EthIfRbRxBypass
Description	ETAS (vendor 11) specific parameter: Enables / Disables RxIndication bypass for the FrameOwner.
Introduction	When the bypass is enabled, EthIf_RxIndication() will forward to UL_RxIndication() all the received Ethernet frames without identifying EthIfCtrl and without checking EthIfCtrl state. This bypass is intended to be used for RMU EtherType only (Marvell Switch control frames over Ethernet).
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.14 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfRxIndicationRef

Short Name	EthIfRxIndicationRef
------------	----------------------

Description	ETAS (vendor 11) specific parameter: Reference to EthIfRxIndicationConfig container where the RxIndication callback function is configured.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.15 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfTxConfirmationRef

Short Name	EthIfTxConfirmationRef
Description	ETAS (vendor 11) specific parameter: Reference to EthIfTxConfirmationConfig container where the TxConfirmation callback function is configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.16 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfZeroCopyTxConfirmationRef

Short Name	EthIfZeroCopyTxConfirmationRef
Description	ETAS (vendor 11) specific parameter: Reference to EthIfZeroCopyTxConfirmationConfig container where the ZeroCopyTxConfirmation callback function is configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.17 EthIf/EthIfConfigSet/EthIfPhysController/EthIfClkUnit

Short Name	EthIfClkUnit
Description	This container contains the configuration of a HW clock unit in an Ethernet Controller.
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.18 EthIf/EthIfConfigSet/EthIfPhysController/EthIfClkUnit/EthIfClkUnitIdx

Short Name	EthIfClkUnitIdx
Description	Zero-based consecutive index of the HW clock units in the Ethernet Controller. Upper layer BSW modules and the Eth itself use this index to identify a clock in the Ethernet Controller.

Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.19 EthIf/EthIfConfigSet/EthIfPhysController/EthIfClkUnit/EthIfClkUnitRef

Short Name	EthIfClkUnitRef
Description	Reference to a HW clock unit which is provided by the Ethernet controller for ingress/egress timestamping of frames and optionally to follow PTP time.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.20 EthIf/EthIfConfigSet/EthIfPhysController/EthIfEthDriverPrefix

Short Name	EthIfEthDriverPrefix
Long Name	EthIfEthDriverPrefix (automatically generated for RTA-BSW driver)
Description	ETAS (vendor 11) specific parameter: Defines the related Eth Driver API prefix. In case EthIfEthCtrlRef refers to a RTA-BSW driver, the parameter EthIfEthDriverPrefix is automatically forwarded and do not need to be configured by integrator.
Introduction	The prefix need to be constructed in the following way: "vi_ai" where vi is the vendorId (11 for ETAS) and ai is the vendorApilnfix. For example, if the prefix is defined as "11_SynopsisQos", then the transmission API called by EthIf will be Eth_11_SynopsisQos_Transmit(). The EthIfEthDriverPrefix is optional. If it is disabled then no prefix will be used in API names (for example transmission API called by EthIf will be Eth_Transmit()). EthIfEthDriverPrefix is also used to determine the name of the Eth Driver header file to be included. Range of characters is 3..32.
Multiplicity	0..1
String Length Range	3..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.21 EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMain-

FunctionIngressQueueProcessingEthIfPhysCtrlRxMainFunction-IngressQueueProcessing

Short Name	EthIfPhysCtrlRxMainFunctionIngressQueueProcessing
Description	The function checks for new received Ethernet frames at the related Ethernet controller and the related ingress queue referenced via EthIfPhysCtrlRxIngressQueueRef, or at the related CanXL controller and the related ingress FIFO referenced via EthIfCanXLCtrlRxIngressFifoRef.
Introduction	In case of Ethernet controller calling Eth_Receive() with the respective QueueIdx. In case of CanXL controller calling CanXL_Receive() with the respective FifoIdx. An EthIf_MainFunctionRx_{IngressQueueProcessing ShortName} API is generated for each EthIfPhysCtrlRxMainFunction-IngressQueueProcessing container based on the short name of this container. In case Rx interrupt is enabled, this container shall not be configured. In polling mode, EthIf-PhysCtrlRxMainFunctionIngressQueueProcessing shall be configured at least one time.
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.22 EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunctionIngressQueueProcessing/EthIfPhysCtrlRxMainFunction-RuntimeLimit

Short Name	EthIfPhysCtrlRxMainFunctionRuntimeLimit
Description	ETAS (vendor 11) specific parameter: Specifies the runtime limit of the EthIf-PhysCtrlRxMainFunction in seconds.
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

2.3.1.23 EthIf/EthIfConfigSet/EthIfPhysController/EthIfRbContinuousGetCurrentTime

Short Name	EthIfRbContinuousGetCurrentTime
Description	ETAS (vendor 11) specific parameter: This parameter indicates whether the physical controller supports continuous availability of GetCurrentTime.

Introduction	Continuous availability of Eth_GetCurrentTime() is ensured for Cadence driver while for some other Ethernet drivers, there can be a dependency of Eth_GetCurrentTime() with mode ACTIVE. This parameter will be autoforwarded with value TRUE when physical controller supports continuous GetCurrentTime otherwise it will be forwarded with value FALSE. If it is not possible to detect whether continuous GetCurrentTime is available or if EthGlobalTimeSupport is disabled, then the parameter will remain unconfigured.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.24 EthIf/EthIfConfigSet/EthIfPhysController/EthIfRbDisableEthControllerInit

Short Name	EthIfRbDisableEthControllerInit
Description	ETAS (vendor 11) specific parameter: This parameter disables(true) or enables(false) the Eth_ControllerInit() function for the respective EthIfPhysController. Eth_ControllerInit() is enabled by default in case the parameter is not configured.
Introduction	Eth_ControllerInit() is removed from Autosar but required for RTA-BSW drivers. If 3rd party EthDriver is referred by EthIfPhysController, which implements latest available Autosar version and the call to API Eth_ControllerInit() is not existent, then this parameter has to be configured to true.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.25 EthIf/EthIfConfigSet/EthIfPhysController/EthIfRbPendingTxConfirmationMax

Short Name	EthIfRbPendingTxConfirmationMax
Description	ETAS (vendor 11) specific parameter: Enables and defines the size of the TxConfirmation storage which is used to look up the pending TxConfirmations

Introduction	The TxConfirmation storage is only needed for 3rd party driver and shall be unconfigured for RTA-BSW driver. When configured, the parameters DataPtr and DataLen of the EthIf_TxConfirmation function shall be set to NULL_PTR respectively 0 as they will not be used. Hints for selecting the right value: The pending TxConfirmation list should have enough elements to handle concurrency situations, where sending is done on multiple cores in parallel and the Tx confirmations are received in arbitrary order (size should be bigger than number of cores). Additionally the number of frames that can be send before a Tx confirmation is received has to be respected.
Multiplicity	0..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.26 EthIf/EthIfConfigSet/EthIfSwitch/EthIfEthSwtDriverPrefix

Short Name	EthIfEthSwtDriverPrefix
Long Name	EthIfEthSwtDriverPrefix (automatically generated for RTA-BSW driver)
Description	ETAS (vendor 11) specific parameter: Defines the related Eth Switch Driver API prefix. In case EthIfEthCtrlRef refers to a RTA-BSW switch driver, the parameter EthIfEthSwtDriverPrefix is automatically forwarded and do not need to be configured by integrator.
Introduction	The prefix need to be constructed in the following way: "vi_ai" where vi is the vendorId (11 for ETAS) and ai is the vendorApilnfix. For example, if the prefix is defined as "11_S32Z", then the switch port mode control API called by EthIf will be EthSwt_11_S32Z_SetSwitchPortMode(). The EthIfEthSwtDriverPrefix is optional. If it is disabled then no prefix will be used in API names (for example switch port mode control API called by EthIf will be EthSwt_SetSwitchPortMode()). EthIfEthSwtDriverPrefix is also used to determine the name of the Eth Switch Driver header file to be included. Range of characters is 3..32.
Multiplicity	0..1
String Length Range	3..32

Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.27 EthIf/EthIfConfigSet/EthIfSwitch/EthIfSwitchEnableLinkStateInterrupt

Short Name	EthIfSwitchEnableLinkStateInterrupt
Description	ETAS (vendor 11) specific parameter: Enables / Disables link state change interrupt.
Introduction	When link state change interrupt is enabled, EthIf expects that the switch calls EthIf_SwitchLinkUpIndication and EthIf_SwitchLinkDownIndication to inform about link state change for a particular port. When link state change interrupt is disabled, EthIf will poll the link state by calling directly the EthSwt_GetLinkState API from EthIf_MainFunctionState. When EthIfSwitchEnableLinkStateInterrupt is not configured, the default Autosar functioning is used : link state change is polled via EthSwt_GetLinkState API from EthIf_MainFunctionState.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.28 EthIf/EthIfConfigSet/EthIfTransceiver/EthIfEthTrcvDriverPrefix

Short Name	EthIfEthTrcvDriverPrefix
Long Name	EthIfEthTrcvDriverPrefix (automatically generated for RTA-BSW driver)
Description	ETAS (vendor 11) specific parameter: Defines the related Trcv Driver API prefix. In case EthIfEthTrcvRef refers to a RTA-BSW driver, the parameter EthIfEthTrcvDriverPrefix is automatically forwarded and do not need to be configured by integrator.

Introduction	The prefix need to be constructed in the following way: "vi_ai" where vi is the vendorId (11 for ETAS) and ai is the vendorApilnfix. For example, if the prefix is defined as "11_MultiPartn", then the link state polling API called by EthIf will be EthTrcv_11_MultiPartn_GetLinkState(). The EthIfEthTrcvDriverPrefix is optional. If it is disabled then no prefix will be used in API names (for example link state polling API called by EthIf will be EthTrcv_GetLinkState()). EthIfEthTrcvDriverPrefix is also used to determine the name of the Trcv Driver header file to be included. Range of characters is 3..32.
Multiplicity	0..1
String Length Range	3..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.29 EthIf/EthIfConfigSet/EthIfTransceiver/EthIfRbDisableEth-TransceiverInit

Short Name	EthIfRbDisableEthTransceiverInit
Description	ETAS (vendor 11) specific parameter: This parameter disables(true) or enables(false) the EthTrcv_TransceiverInit() function for the respective EthIf-Transceiver. EthTrcv_TransceiverInit() is enabled by default in case the parameter is not configured.
Introduction	EthTrcv_TransceiverInit() is removed from Autosar but required for RTA-BSW drivers. If 3rd party Trcv-Driver is referred by EthIfTransceiver, which implements latest available Autosar version and the call to API EthTrcv_TransceiverInit() is not existent, then this parameter has to be configured to true.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.30 EthIf/EthIfConfigSet/EthIfZeroCopyTxConfirmationConfig

Short Name	EthIfZeroCopyTxConfirmationConfig
Description	ETAS (vendor 11) specific container: Configuration of ZeroCopy transmit indication callback functions.
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.31 EthIf/EthIfConfigSet/EthIfZeroCopyTxConfirmationConfig/ EthIfZeroCopyTxConfirmationFunction

Short Name	EthIfZeroCopyTxConfirmationFunction
Description	ETAS (vendor 11) specific parameter: Specifies ZeroCopy transmit indication callback function
Introduction	The transmit callback function (ZeroCopyTxConfirmation) is called by EthIf to confirm to the Upper Layer the transmission of an ZeroCopy Ethernet frame.
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

2.3.1.32 EthIf/EthIfGeneral/EthIfFwSupport

Short Name	EthIfFwSupport
Description	Enables / Disables the Firewall usage.
Multiplicity	1..1
Default Value	-
Enumeration Literals	FIREWALL_WITHOUT_PERSTREAMFILTERING, FIREWALL_WITH_PERSTREAMFILTERING, NO_FIREWALL
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.33 EthIf/EthIfGeneral/EthIfOverrideSrcMACSupport

Short Name	EthIfOverrideSrcMACSupport
Description	ETAS (vendor 11) specific parameter: Functionality provided to upper layer of EthIf to override the source MAC address.
Introduction	Enables/Disables the API EthIf_OverrideTxBufSrcAddr based on the configuration.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.34 EthIf/EthIfGeneral/EthIfPhcSupport

Short Name	EthIfPhcSupport
Description	Enables/Disables the PTP HW Clock (PHC).
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.35 EthIf/EthIfGeneral/EthIfRbForeignDrvSupport

Short Name	EthIfRbForeignDrvSupport
Description	ETAS (vendor 11) specific parameter: Enable third party driver support.
Introduction	When enabled, all the callback APIs of EthIf will be renamed EthIf_Integn_{CallbackAPIName}. This allows integrator to add required integration code between 3rd party driver and EthIf callbacks for resolving the difference in term of API signatures (allow integration of 3rd party driver based on different Autosar versions). When this configuration parameter is enabled, the template files EthIf_Integn.c_tpl and EthIf_Integn.h_tpl shall be used for implementing the call adaptations between EthIf and drivers. For the forward call remapping through integration code (EthIf calls to driver modules), the parameters EthIfEthDriverPrefix, EthIfEthTrcvDriverPrefix and EthIfEthSwtDriverPrefix can be used to provide a prefix that will point to the integration code (for example "11_Integn"). Additionally, when EthIfRbForeignDrvSupport is enabled, EthIf hook named EthIf_Hook_SetControllerMode() present in EthIf_Integn.c will be called everytime the requested mode of an EthIfController changes.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.36 EthIf/EthIfGeneral/EthIfRbTrcvModeChgMainReload

Short Name	EthIfRbTrcvModeChgMainReload
Description	ETAS (vendor 11) specific parameter: Specifies the frequency of transceiver mode state change checks in each period of main function EthIf_MainFunction-State. Parameter needs to be configured only when EthTrcv is not calling EthIf_TrvcModeIndication() callback upon mode change. In that case, EthIf polls the mode change of the Trcv via EthTrcv_GetTransceiverMode() API
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.37 EthIf/EthIfGeneral/EthIfRbTrcvModeChgMainReloadOffset

Short Name	EthIfRbTrcvModeChgMainReloadOffset
Description	ETAS (vendor 11) specific parameter: Specifies the offset between EthIfRbTrcvModeChgMainReload and EthIfTrcvLinkStateChgMainReload. This parameter influences the initial value of the reload counter for Trcv mode change polling. If this parameter is configured, the first polling cycle will be shorter than the subsequent one. This is useful to desynchronize the polling of the Trcv mode and the polling of the Trcv link state in order to spread the runtime consumption across different MainFunctions.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.38 EthIf/EthIfGeneral/EthIfRbWakeupSleepOnDatelineEnabled

Short Name	EthIfRbWakeupSleepOnDatelineEnabled
Description	ETAS (vendor 11) specific parameter: Configures if wakeup and sleep over dataline is supported or not.
Introduction	When enabled and if EthIf_SetControllerMode() or EthIf_SwitchPortGroupRequestMode() is called with ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST, then EthIf will propagate the wakeup request to the underlying switch port or transceiver. EthIf_SleepIndication() processing of EthIf will be enabled by this configuration parameter.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.39 EthIf/EthIfGeneral/EthIfSecurityEventRefs/SEV_ETH_DROP_INV_VLAN

Short Name	SEV_ETH_DROP_INV_VLAN
Description	An Ethernet datagram was dropped due to an invalid CrtlIdx/VLAN.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

2.3.1.40 EthIf/EthIfGeneral/EthIfSecurityEventRefs/SEV_ETH_DROP_-MAC_COLLISION

Short Name	SEV_ETH_DROP_MAC_COLLISION
Description	An Ethernet datagram was dropped because local MAC was same as source MAC in an incoming frame.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

2.3.1.41 EthIf/EthIfGeneral/EthIfSecurityEventRefs/SEV_ETH_DROP_UNKNOWN_ETHERTYPE

Short Name	SEV_ETH_DROP_UNKNOWN_ETHERTYPE
Description	An Ethernet datagram was dropped due to an unknown Ethertype.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

2.3.1.42 EthIf/EthIfGeneral/EthIfSecurityEventRefs/SEV_ETH_DROP_VLAN_DOUBLE_TAG

Short Name	SEV_ETH_DROP_VLAN_DOUBLE_TAG
Description	An Ethernet datagram was dropped due to double VLAN tag.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

2.3.1.43 EthIf/EthIfGeneral/EthIfSwitchGlobalTimeSupport

Short Name	EthIfSwitchGlobalTimeSupport
Description	ETAS (vendor 11) specific parameter: Enables/Disables the Global Time APIs related to switch used amongst others by Global Time Synchronization over Ethernet.
Introduction	The following APIs are enabled / disabled by this configuration parameter : EthIf_GetTxMgmtObject and EthIf_GetRxMgmtObject.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.44 EthIf/EthIfGeneral/EthIfVirtualAddressFilterSize

Short Name	EthIfVirtualAddressFilterSize
Description	ETAS (vendor 11) specific parameter: Specifies the size of virtual address filter over all configured EthCtrls. It should be set to the overall number of simultaneously used multicast IP addresses and the size MUST NOT be greater than the summed capacity of physical address filters of all configured EthCtrls. Dependency: If multicast communication is not used, then EthIfVirtualAddressFilterSize should not be configured.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.45 EthIf/EthIfGeneral/EthIfZeroCopyTxSupport

Short Name	EthIfZeroCopyTxSupport
Description	ETAS (vendor 11) specific parameter: Enables/Disables the ZeroCopy transmission APIs.
Introduction	The following APIs are enabled / disabled by this configuration parameter : EthIf_ZeroCopyFillHeader and EthIf_ZeroCopyTransmit.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.2.1 EthIf_SwitchLinkUpIndication

Syntax	EthIf_SwitchLinkUpIndication(uint8 SwitchIdx, uint8 PortIdx)
Parameters	(IN) <i>SwitchIdx</i> Index of the switch within the context of the Ethernet switch driver.
	(IN) <i>PortIdx</i> Index of the port at the addressed switch.
Return Value	NONE
Description	Called by EthSwt, if link state becomes ETHIF_LINKSTATE_ACTIVE for a specified port of the addressed switch.

2.3.2.2 EthIf_SwitchLinkDownIndication

Syntax	EthIf_SwitchLinkDownIndication(uint8 SwitchIdx,uint8 PortIdx)
Parameters	(IN) <i>SwitchIdx</i> Index of the switch within the context of the Ethernet switch driver.
	(IN) <i>PortIdx</i> Index of the port at the addressed switch.
Return Value	NONE
Description	Called by EthSwt, if link state becomes ETHIF_LINKSTATE_DOWN for a specified port of the addressed switch.

2.3.2.3 EthIf_OverrideTxBufSrcAddr

Syntax	EthIf_OverrideTxBufSrcAddr(uint8 CtrlIdx,uint8 BufIdx,uint8 PhysAddrPtr)
Parameters	(IN) <i>CtrlIdx</i> Index of the Ethernet controller within the context of the Ethernet interface.
	(IN) <i>BufIdx</i> Index of the buffer resource.
	(IN) <i>PhysAddrPtr</i> Physical target address (MAC address) in network byte order.
Return Value	E_OK Success, E_NOT_OK Failure.
Description	Calls the driver function to override the source MAC address.

2.3.2.4 EthIf_ZeroCopyFillHeader

Syntax	EthIf_ZeroCopyFillHeader(uint8 CtrlIdx_u8, Eth_FrameType FrameType_u16, uint8 Priority_u8, const uint8 * PhysAddrPtr_pcu8, const Eth_ZeroCopyArg_tst * ZeroCopyArg_pcst)
Parameter In	CtrlIdx_u8 Index of the Ethernet controller within the context of the Ethernet Interface. FrameType_u16 Ethernet Frame Type (EtherType). Priority_u8 Priority value which shall be used for the 3-bit PCP field of the VLAN tag. PhysAddrPtr_pcu8 Physical target address (MAC address) in network byte order. ZeroCopyArg_pcst ZeroCopy argument containing the allocated headers and the pointer to the UL data to be transmitted.
Description	Prefills the ZeroCopy header buffer with the Ethernet header fields.

2.3.2.5 EthIf_ZeroCopyTransmit

Syntax	EthIf_ZeroCopyTransmit(uint8 CtrlIdx_u8, Eth_ZeroCopyArg_tst * ZeroCopyArg_pst, uint32 * TxId_pu32)
--------	---

Parameter In	CtrlIdx_u8 Index of the Ethernet controller within the context of the Ethernet Interface. ZeroCopyArg_pcst ZeroCopy argument containing the allocated headers and the pointer to the UL data to be transmitted. TxId_pu32 Unique identifier in the context of EthDriver representing the confirmed ZeroCopy transmission.
Description	Execute the ZeroCopy transmission.

2.3.2.6 EthIf_ZeroCopyTxConfirmation

Syntax	<code>EthIf_ZeroCopyTxConfirmation(uint8 CtrlIdx_u8, uint16 TxId_u16, const Eth_DataType * HeaderPtr_pcu8, uint16 HeaderLen_u16)</code>
Parameter In	CtrlIdx Index of the Ethernet controller within the context of the Ethernet Interface. TxId_u16 Unique identifier in the context of EthDriver representing the confirmed ZeroCopy transmission. HeaderPtr_pu8 Pointer to the transmitted Ethernet header buffer related to the ZeroCopy transmission with TxId given in argument. HeaderLen_u16 Length of the Ethernet header buffer.
Description	Confirms ZeroCopy frame transmission by the indexed controller.

2.3.2.7 EthIf_PhysCtrlInit

Syntax	<code>Std_ReturnType EthIf_PhysCtrlInit (uint8 PhysCtrlIdx)</code>
Description	Initializes the indexed Eth controller (EthIfPhysController).

2.3.2.8 EthIf_SetPhysCtrlMode

Syntax	<code>Std_ReturnType EthIf_SetPhysCtrlMode (uint8 PhysCtrlIdx, Eth_ModeType CtrlMode)</code>
Description	Enables/disables the indexed Eth controller (EthIfPhysController).

2.3.2.9 EthIf_ZeroCopyMarkLastTx

Syntax	<code>void EthIf_ZeroCopyMarkLastTx(uint8 CtrlIdx_u8, uint32 * TxId_pu32)</code>
Description	Marks the last executed ZeroCopy transmission with TxConf=TRUE.

2.3.2.10 EthIf_OverrideTxBufSrcAddr

Syntax	<code>Std_ReturnType EthIf_OverrideTxBufSrcAddr (uint8 CtrlIdx, Eth_BufIdxType BufIdx, const uint8 * PhysAddrPtr)</code>
Description	Calls the driver function to override the source MAC address.

2.3.2.11 EthIf_MacSecResetTxSc

Syntax	Std_ReturnType EthIf_MacSecResetTxSc (uint8 CtrlIdx, uint64 Sci)
Description	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to reset to default the MACsec values of the Transmission Secure Channel for the given Secure Channel Identifier.

2.4 Limitations

The following features are not supported for version v12.7.0:

- PostBuild configuration and Link-time configuration are not supported.
- Multiple EthSwt configurations are not supported.
- The EthIf_GetTxMgmtObject and EthIf_GetRxMgmtObject APIs do not support a cascaded switch (i.e. if the APIs are called with EthIfCtrlIdx referring to a SwtPG containing ports of multiple switches). The API prototype is currently not precise enough to know to which EthSwt the call needs to be routed.
- The diagnostics function EthIf_GetAndResetMeasurementData only supports retrieval of one measurement index at a time, i.e getting data for ALL measurement index values is not supported. Diagnostic data for switch and transceiver devices is not supported.
- The EthIf_GetArITable function will copy all or numberOfElements into the output list. If the input value of numberOfElements is 0, no data will be copied and will return the number of valid entries in the cache. arItableList-Pointer may be NULL_PTR in this case.
- The EthIf_CheckWakeup function supports the verification of the signaled wakeup for all configured EthIfTransceivers and EthIfSwitches (argument WakeupSource is not used).

2.4.1 General

- Multiple switches, controllers and transceiver drivers are not supported.
- Persistent storage of MAC/port tables for switches are not supported.
- Wireless controllers and transceivers are not supported.
- When Tcplp is used, the interrupt mode of EthIf must not be used
- When Tcplp is used, all the EthIf_MainFunctionRx_{IngressQueueProcessing ShortName} must be scheduled in one core in case of multicore context and must be scheduled in one task or in multiple tasks with preemption disabled

2.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

2.5.1 API Functions

The following API functions are not supported in this release of the module.

- EthIf_SetTransceiverWakeupMode
- EthIf_GetTransceiverWakeupMode
- EthIf_StoreConfiguration
- EthIf_ResetConfiguration
- EthIf_GetCtrlIdxList
- EthIf_SetSwitchMgmtInfo
- EthIf_SwitchEnableTimeStamping
- EthIf_VerifyConfig
- EthIf_SetForwardingMode
- EthIf_GetBufWRxParams
- EthIf_GetBufWTxParams
- EthIf_SetBufWTxParams
- EthIf_SetRadioParams
- EthIf_SetChanRxParams
- EthIf_SetChanTxParams
- EthIf_GetChanRxParams
- EthIf_GetBufCV2xPC5RxParams
- EthIf_GetBufCV2xPC5TxParams
- EthIf_GetChanCV2xPC5TxParams
- EthIf_SwitchMacSecUpdateSecY
- EthIf_SwitchMacSecUpdateSecYNotification
- EthIf_SwitchMacSecInitRxSc
- EthIf_SwitchMacSecResetRxSc
- EthIf_SwitchMacSecAddTxSa
- EthIf_SwitchMacSecAddTxSaNotification
- EthIf_SwitchMacSecUpdateTxSa
- EthIf_SwitchMacSecDeleteTxSa
- EthIf_SwitchMacSecAddRxSa
- EthIf_SwitchMacSecAddRxSaNotification
- EthIf_SwitchMacSecUpdateRxSa

- EthIf_SwitchMacSecDeleteRxSa
- EthIf_SwitchMacSecGetTxSaNextPn
- EthIf_SwitchMacSecGetMacSecStats
- EthIf_SwitchMacSecGetMacSecStatsNotification
- EthIf_SwitchMacSecOperational
- EthIf_SwitchMacSecSetControlledPortEnabled
- EthIf_ImmediateTransmit
- EthIf_ReleaseRxBuffer

2.5.2 Configuration Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- EthIf/EthIfConfigSet/EthIfController/EthIfMacSecSupport
- EthIf/EthIfConfigSet/EthIfController/EthIfMaxTxBufsTotal
- EthIf/EthIfConfigSet/EthIfController/EthIfPaeInstanceRef
- EthIf/EthIfConfigSet/EthIfController/EthIfSwitchRefOrPortGroupRef
- EthIf/EthIfConfigSet/EthIfFrameOwnerConfig
- EthIf/EthIfConfigSet/EthIfFrameOwnerConfig/EthIfOwner
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfCV2xCtrlRef
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunction-IngressQueueProcessing/EthIfCanXLCtrlRxIngressFifoRef
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunction-IngressQueueProcessing/EthIfPhysCtrlRxIndicationIterations
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunction-IngressQueueProcessing/EthIfPhysCtrlRxIngressFifoRef
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunction-IngressQueueProcessing/EthIfPhysCtrlRxIngressQueueRef
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunction-IngressQueueProcessing/EthIfPhysCtrlRxMainFunctionPeriod
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfPhysCtrlRxMainFunctionPriorityProcessing
- EthIf/EthIfConfigSet/EthIfRxIndicationConfig/EthIfRxIndicationFunction
- EthIf/EthIfConfigSet/EthIfSwitch/EthIfSwitchIdx
- EthIf/EthIfConfigSet/EthIfSwitch/EthIfSwitchRef
- EthIf/EthIfConfigSet/EthIfSwitchPortGroup/EthIfPortRef
- EthIf/EthIfConfigSet/EthIfSwitchPortGroup/EthIfSwitchPortGroupIdx
- EthIf/EthIfConfigSet/EthIfSwitchPortGroup/EthIfSwitchPortGroupRefSemantics

- EthIf/EthIfConfigSet/EthIfTransceiver/EthIfCanXLTrcvRef
- EthIf/EthIfConfigSet/EthIfTransceiver/EthIfQualifiedUnexpectedLinkDown-Time
- EthIf/EthIfConfigSet/EthIfTransceiver/EthIfTransceiverIdx
- EthIf/EthIfConfigSet/EthIfTransceiver/EthIfWEthTrcvRef
- EthIf/EthIfConfigSet/EthIfTrcvLinkStateChgConfig
- EthIf/EthIfConfigSet/EthIfTrcvLinkStateChgConfig/EthIfTrcvLinkStateChg-Function
- EthIf/EthIfConfigSet/EthIfTxConfirmationConfig/EthIfTxConfirmationFunc-tion
- EthIf/EthIfGeneral/EthIfDevErrorDetect
- EthIf/EthIfGeneral/EthIfEnableCV2xApi
- EthIf/EthIfGeneral/EthIfEnableRxInterrupt
- EthIf/EthIfGeneral/EthIfEnableSecurityEventReporting
- EthIf/EthIfGeneral/EthIfEnableSignalQualityApi
- EthIf/EthIfGeneral/EthIfEnableTxInterrupt
- EthIf/EthIfGeneral/EthIfEnableWEthApi
- EthIf/EthIfGeneral/EthIfGetAndResetMeasurementDataApi
- EthIf/EthIfGeneral/EthIfGetBaudRate
- EthIf/EthIfGeneral/EthIfGetCounterState
- EthIf/EthIfGeneral/EthIfGetCtrlIdxList
- EthIf/EthIfGeneral/EthIfGetVlanIdSupport
- EthIf/EthIfGeneral/EthIfMainFunctionPeriod
- EthIf/EthIfGeneral/EthIfMainFunctionStatePeriod
- EthIf/EthIfGeneral/EthIfMaxTrcvsTotal
- EthIf/EthIfGeneral/EthIfPortStartupActiveTime
- EthIf/EthIfGeneral/EthIfPublicCddHeaderFile
- EthIf/EthIfGeneral/EthIfRxIndicationIterations
- EthIf/EthIfGeneral/EthIfSecurityEventRefs
- EthIf/EthIfGeneral/EthIfSecurityEventRefs/ETHIF_SEV_DROP_ETH_MAC_-COLLISION
- EthIf/EthIfGeneral/EthIfSecurityEventRefs/ETHIF_SEV_DROP_INV_VLAN
- EthIf/EthIfGeneral/EthIfSecurityEventRefs/ETHIF_SEV_DROP_UN-KNOWN_ETHERTYPE
- EthIf/EthIfGeneral/EthIfSecurityEventRefs/ETHIF_SEV_DROP_VLAN_DOU-BLE_TAG
- EthIf/EthIfGeneral/EthIfSetForwardingModeApi

- EthIf/EthIfGeneral/EthIfSignalQualityCheckPeriod
- EthIf/EthIfGeneral/EthIfStartAutoNegotiation
- EthIf/EthIfGeneral/EthIfSwitchManagementSupport
- EthIf/EthIfGeneral/EthIfSwitchOffPortTimeDelay
- EthIf/EthIfGeneral/EthIfTrcvLinkStateChgMainReload
- EthIf/EthIfGeneral/EthIfVerifyConfigApi
- EthIf/EthIfGeneral/EthIfVersionInfoApi
- EthIf/EthIfGeneral/EthIfVersionInfoApiMacro
- EthIf/EthIfGeneral/EthIfWakeUpSupport

2.6 Deviations

2.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.1.1 EthIf/EthIfConfigSet/EthIfController

[ADDED] Introduction	An EthIfController is used to abstract the access to VLANs from the underlying communication system composed of physical Ethernet controller and Ethernet Transceiver (optional) or Ethernet Switch (optional) or Ethernet Switch Port Group (optional).
----------------------	--

2.6.1.2 EthIf/EthIfConfigSet/EthIfController/EthIfCtrlIdx

[RTA-BSW] Description	This parameter provides a zero-based consecutive index of the Ethernet Communication Controllers. Upper layer BSW modules and the EthIf itself use this index to identify an Ethernet Communication Controller.
[AUTOSAR] Description	This parameter provides a zero-based consecutive index of the Ethernet Communication Controllers. Upper layer BSW modules and the EthIf itself use this index to identify a Ethernet CC.

2.6.1.3 EthIf/EthIfConfigSet/EthIfController/EthIfCtrlMtu

[RTA-BSW] Introduction	Note: In case a VLAN tag is used for the EthIfCtrl, the frame length of the Ethernet frame will increase by 4 bytes. When Ethernet driver buffer concatenation feature is disabled, it should be ensured that the Ethernet header length (14 byte without VLAN or 18 byte with VLAN) + configuration parameter EthIfCtrlMtu is greater than physical controller Egress buffer size. Hint: physical controller Egress buffer size is configured in EthCtrlConfigEgressFifoBufLenByte or EthCtrlConfigEgressQueueBufLenByte. When the buffer concatenation feature is enabled, the Ethernet driver will chain multiple Egress buffers together to store a longer frame. Therefore EthIfCtrlMtu can be configured bigger than the physical controller Egress buffer size in that case. See Eth driver documentation to get more information about buffer concatenation feature (not all RTA-BSW drivers support this feature).
[AUTOSAR] Introduction	Note: In case a VLAN tag is used for the EthIfCtrl, the frame length of the Ethernet frame will increase by 4 bytes.

2.6.1.4 EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameType

[RTA-BSW] Parent Container	EthIf/EthIfConfigSet/EthIfFrameConfig/EthIfFrameType
[AUTOSAR] Parent Container	EthIf/EthIfConfigSet/EthIfFrameOwnerConfig/EthIfFrameType
[RTA-BSW] Description	Selects the Ethernet frame type. Tags: atp.Status=draft
[AUTOSAR] Description	Selects the Ethernet frame type.
[ADDED] Introduction	A list of EtherTypes specified for IETF protocols can be found in RFC7042, Appendix B.

2.6.1.5 EthIf/EthIfConfigSet/EthIfPhysController/EthIfWEthCtrlRef

[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

2.6.1.6 EthIf/EthIfGeneral/EthIfGlobalTimeSupport

[ADDED] Introduction	The following APIs are enabled / disabled by this configuration parameter : EthIf_GetCurrentTime, EthIf_EnableEgressTimeStamp, EthIf_GetEgressTimeStamp and EthIf_GetIngressTimeStamp.
----------------------	--

2.7 API Definition

Table 2.64.EthIf Supported API functions

EthIf_Init	Initialises the module.
EthIf_MainFunctionRx_<IngressQueueProcessing ShortName>	Checks for new received frames and issues reception indications in polling mode.
EthIf_MainFunctionTx	Issues transmission confirmations in polling mode.
EthIf_MainFunctionState	Polls communication hardware (Ethernet transceiver and Ethernet switch ports) related information such as link state and signal quality.
EthIf_SetControllerMode	Enables / disables the specified controller.
EthIf_GetControllerMode	Obtains the current state of the specified controller.
EthIf_GetTransceiverMode	Retrieves the Mode of the indexed transceiver.
EthIf_GetPhysAddr	Obtains the physical source address used by the specified controller.
EthIf_SetPhysAddr	Sets the physical source address used by the indexed controller.
EthIf_UpdatePhysAddrFilter	Updates the physical source address to/from the specified controller filter.
EthIf_GetPortMacAddr	Obtains the port over which this MAC-address can be reached.
EthIf_GetAr1Table	Obtains the address resolution table of a switch and copies the list into a user provided buffer.
EthIf_GetCurrentTime	Reads the current time stamp information from the hardware register of the specified controller.
EthIf_GetCurrentTimeTuple	Reads the current time of the time-stamp clock and the current time of the PHC in an atomic operation.
EthIf_SetPhcTime	Sets the absolute time of the PHC.

EthIf_SetPhcCorrection	Sets PHC parameters to adapt rate and offset of the PHC.
EthIf_GetPhcTime	Returns the current time value out of the HW registers of the PHC.
EthIf_SetPpsSignalMode	Enables/disables the generation of a PPS signal.
EthIf_EnableEgressTimeStamp	Activates egress time stamping on a message object for the specified controller.
EthIf_GetEgressTimeStamp	Reads the egress time stamp on a message object for the specified controller.
EthIf_GetIngressTimeStamp	Reads the ingress time stamp on a message object for the specified controller.
EthIf_ProvideTxBuffer	Requests provision of a transmit buffer for the specified controller.
EthIf_Transmit	Triggers transmission of the contents of the specified transmit buffer for the specified controller.
EthIf_GetVersionInfo	Returns version information for the module.
EthIf_RxIndication	Callback function which signals reception of an Ethernet frame by the specified controller.
EthIf_TxConfirmation	Callback function which confirms an Ethernet frame transmission by the specified controller.
EthIf_TrcvModeIndication	Call back function triggered by a change to the operating mode of the specified transceiver to notify EthSM of mode change.
EthIf_GetVlanId	Returns the VLAN identifier of the specified Ethernet controller.
EthIf_GetAndResetMeasurementData	Allows reading and resetting of measurement data for diagnostic purposes.
EthIf_SwitchPortGroupRequestMode	Forwarded to EthSwt to request a mode for the EthIfSwtPortGroup.
EthIf_StartAllPorts	Request starting of all configured ports.
EthIf_GetRxMgmtObject	Requests the Management object for the corresponding switch port and is called from Rxconfirmation.

EthIf_GetTxMgmtObject	Requests the Management object for the corresponding switch port and is called from Txconfirmation.
EthIf_GetTrcvSignalQuality	Retrieves the signal quality of the link of the given Ethernet transceiver.
EthIf_ClearTrcvSignalQuality	Clears the stored signal quality of the link of the given Ethernet transceiver.
EthIf_GetSwitchPortSignalQuality	Retrieves the signal quality of the link of the given Ethernet switch port.
EthIf_ClearSwitchPortSignalQuality	Clears the stored signal quality of the link of the given Ethernet switch port.
EthIf_SetPhyTestMode	Activates a given test mode.
EthIf_SetPhyTxMode	Activates a given transmission mode.
EthIf_SetPhyLoopbackMode	Activates a given loopback mode.
EthIf_GetCableDiagnosticsResult	Retrieves the cable diagnostics result of a given transceiver.
EthIf_GetPhyIdentifier	Obtains the PHY identifier of the Ethernet Interface according to IEEE 802.3-2015.
EthIf_GetSwitchPortMode	Obtains the mode of the indexed switch port.
EthIf_SwitchPortGetLinkState	Obtains the link state of the indexed switch port.
EthIf_TransceiverGetLinkState	Retrieves the Link State of the indexed transceiver.
EthIf_SwitchPortGetBaudRate	Obtains the baud rate of the indexed switch port.
EthIf_TransceiverGetBaudRate	Retrieves the Baud Rate of the indexed transceiver.
EthIf_SwitchPortGetDuplexMode	Obtains the duplex mode of the indexed switch port.
EthIf_TransceiverGetDuplexMode	Retrieves the duplex mode of the indexed transceiver.
EthIf_SwitchPortGetCounterValues	Reads a list with drop counter values of the corresponding port of the switch.
EthIf_SwitchPortGetRxStats	Returns a list of statistic counters defined with Eth_RxTatsType.
EthIf_SwitchPortGetTxStats	Returns the list of Transmission Statistics out of IETF RFC1213 defined with Eth_TxStatsType.

EthIf_SwitchPortGetTxErrorCounterValues	Returns the list of Transmission Error Counters out of IETF RFC1213 and RFC1643 defined with Eth_TxError-CounterValuesType.
EthIf_SwitchPortGetMacLearningMode	Returns the MAC learning mode.
EthIf_GetSwitchPortIdentifier	Retrieves the OUI (24 bit) of the indexed Ethernet switch port.
EthIf_GetSwitchIdentifier	Obtains the Organizationally Unique Identifier that is given by the IEEE of the indexed Ethernet switch.
EthIf_WritePortMirrorConfiguration	Stores the given port mirror configuration in a shadow buffer in the Ethernet switch driver for the given MirroredSwitchIdx.
EthIf_ReadPortMirrorConfiguration	Obtains the port mirror configuration of the given Ethernet switch.
EthIf_DeletePortMirrorConfiguration	Deletes the stored port mirror configuration of the given MirroredSwitchIdx.
EthIf_GetPortMirrorState	Obtains the current status of the port mirroring for the indexed Ethernet switch port.
EthIf_SetPortMirrorState	Sets the given port mirroring state of the port mirror configuration for the given Ethernet switch.
EthIf_SetPortTestMode	Activates a given test mode of the indexed Ethernet switch port.
EthIf_SetPortLoopbackMode	Activates a given test loop-back mode of the indexed Ethernet switch port.
EthIf_SetPortTxMode	Activates a given transmission mode of the indexed Ethernet switch port.
EthIf_GetPortCableDiagnosticsResult	Retrieves the cable diagnostics result of the indexed Ethernet switch port respectively the referenced Ethernet Transceiver Driver.
EthIf_RunPortCableDiagnostic	Triggers the cable diagnostics of the given Ethernet Switch port (PortIdx) by calling EthTrcv_RunCableDiagnostic of the referenced Ethernet transceiver.
EthIf_RunCableDiagnostic	Triggers the cable diagnostics for the given Ethernet transceiver.
EthIf_SwitchGetCfgDataRaw	Retrieves the data in memory of the indexed Ethernet switch in variable length.

EthIf_SwitchGetCfgDataInfo	Retrieves the total size of data and the memory start address of the indexed Ethernet Switch.
EthIf_SwitchPortGetMaxFIFOBufferFillLevel	Retrieves the maximum amount of allocated FIFO buffer of the indexed Ethernet switch egress port.
EthIf_SwitchPortModeIndication	Called by EthSwt if the port mode changes for a particular port of the specified switch.
EthIf_SleepIndication	Callback by the corresponding EthTrcv if a sleep indication is detected on the network.
EthIf_CtrlModeIndication	Called asynchronously when mode has been read out.
EthIf_CheckWakeup	Requests the affected Ethernet hardware to check for a signaled wake-up.
EthIf_EthGetSpiStatus	When MACPHY controller used obtains the SPI interface status.
EthIf_MacSecUpdateSecY	Requests the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver to update the SecY/PAC of the PHY with the provided parameters.
EthIf_MacSecUpdateSecYNotification	Callback to notify that EthIf_Switch-MacSecUpdateSecY finished.
EthIf_MacSecInitRxSc	Requests the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to configure a Reception Secure Channel for the given Secure Channel Identifier.
EthIf_MacSecResetRxSc	Requests the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to reset to default the MACsec values of the Reception Secure Channel for the given Secure Channel Identifier.
EthIf_MacSecAddTxSa	Requests the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to create a Transmission Secure Association in the Transceiver.
EthIf_MacSecAddTxSaNotification	Callback to notify that EthIf_Switch-MacSecAddTxSa finished.

EthIf_MacSecUpdateTxSa	Requests the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to update the Transmission Secure Association with the given Packet Number.
EthIf_MacSecDeleteTxSa	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to remove the Transmission Secure Association identified by the provided Association Number.
EthIf_MacSecAddRxSa	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to create a Reception Secure Association in the Transceiver.
EthIf_MacSecAddRxSaNotification	Callback to notify that EthIf_MacSecAddRxSa finished.
EthIf_MacSecUpdateRxSa	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to update the Reception Secure Association with the given Packet Number.
EthIf_MacSecDeleteRxSa	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to remove the Reception Secure Association identified by the provided Association Number.
EthIf_MacSecGetTxSaNextPn	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to return the Packet Number that is used for the next packet in the given Transmission Secure Association.
EthIf_MacSecGetMacSecStats	Request the Ethernet Interface (MACsec per SW) or the Ethernet Transceiver Driver to provide MACsec statistics.
EthIf_MacSecGetMacSecStatsNotification	Callback to notify that EthIf_MacSecGetMacSecStats finished and provide the requested statistics.
EthIf_MacSecOperational	To inform EthIf that MacSec is operational and that EthSM can be informed.
EthIf_MacSecSetControlledPortEnabled	Requests to set the Controlled Port enabled parameter of a PAE.
EthIf_GetPhyWakeupReason	Obtains the wake up reasons of the indexed Ethernet Transceiver (PHY).

EthIf_TransceiverGetMacMethod	Obtains the media access mode of the transceiver.
EthIf_GetSwitchPortWakeupReason	Obtains the wake up reasons of the indexed Ethernet switch port.
EthIf_GetStreamHandleIdxStatistics	Function called by the Firewall module to get bucket counter statistics from the switch.
EthIf_SetStreamHandleIdxConfiguration	Function is called by the Firewall module to control the activity status of a StreamHandleIdx in the switch.
EthIf_StreamHandleIdxConfiguration	Called by the EthSwtDrv once it has successfully set the StreamHandleIdx activity status in the switch.
EthIf_StreamHandleIdxStatistics	Called by the EthSwtDrv once it has successfully retrieved the bucket counter values from the switch.

For Further information please refer to AUTOSAR_SWS_EthernetInterface.pdf

2.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

2.8.1 Configurable Rx Callback Interface

- EthIfRxIndicationFunction is configured as the upper layer module function name which will be called by EthIf_RxIndication. Therefore, the file which prepares the declaration of <upper layer module>_RxIndication needs to be referenced in the EthIfPublicCddHeaderFile parameter.

For example, if EthIfRxIndicationFunction is set to TcpIp_RxIndication, and if TcpIp_RxIndication is declared in header file *TcpIp_Cbk.h*, then EthIfPublicCddHeaderFile should be set to “TcpIp_Cbk.h”

2.8.2 Reception/Transmission Modes

The EthIfEnableRxInterrupt and EthIfEnableTxInterrupt (in EthIf/EthIfGeneral) are not supported. Instead the values are read from EthCtrlEnableRxInterrupt and EthCtrlEnableTxInterrupt (in EthConfigSet/EthCtrlConfig) for each physical controller.

2.8.3 EthIf_TxConfirmation

The EthIf_TxConfirmation API deviates from the AUTOSAR specification to add DataPtr and LenByte as additional parameters.

The AUTOSAR specified input parameters are not enough to convert the physical Ethernet controller index provided by the Ethernet driver (EthCtrlIdx) into virtual controller index (EthIfCtrlIdx) that shall be passed to the upper layer of EthIf. For this reason, 2 extra input parameters are provided.

DataPtr is a pointer to the payload of the transmitted frame, and LenByte is the length of the transmitted frame in bytes. These additional parameters are passed from Ethernet driver so that conversion is possible.

Adaptations for AUTOSAR-compliant Ethernet drivers must be made as described in the "Integration Advice" section.

2.8.4 TxConfirmation handling

When the AUTOSAR conform interface is used by a third-party driver, it is mandatory to set the configuration parameter EthIf/EthIfConfigSet/EthIfPhysController/EthIfRbPendingTxConfirmationMax to a meaningful value. The parameter needs to be set per EthIfPhysController, so one controller can use the AUTOSAR API while another can use the API with the additional parameters. This parameter defines the size of the array where the pending TxConfirmations will be stored. If there is no more space in that array (i.e. the next entry to write to is already filled) the TX is not done.

When using the AUTOSAR conform interface, the two parameters (DataPtr, DatsLen) of the TxConfirmation API are unused (they should be set to NULL_PTR, resp. 0). To achieve this, the integration template for third-party drivers has to be filled.

2.8.5 Buffer Overflow

In order to avoid a buffer overflow in the Ethernet driver module, the value of EthIfCtrlMtu must be less than or equal to the configured value of EthTxBufLenByte (frame header length + frame checksum length).

2.8.6 Frame Type

- The parameter EthIfFrameType should be set to a number following the Ethernet type definition as listed in the IEEE 802.3 standard.
- The EthIfFrameType is a number that follows the IEEE 802.3 standard for EtherType.
- The value of 0x8100 is not supported in Ethernet stack of RTA-BSW v12.7.0.

2.8.7 EthIf_MainFunctionRx

EthIf_MainFunctionRx is not supported because there is no known use case for queues that are not assigned for processing via EthIfPhysCtrlRxMainFunction-IngressQueueProcessing. Therefore, the EthIfRxIndicationIterations parameter is

also not supported, and the `EthIfMainFunctionPeriod` parameter is not used to specify the period of the `EthIf_MainFunctionRx` (it specifies only the period of the `EthIf_MainFunctionTx`).

2.8.8 `EthIf_MainFunctionRx_<IngressQueueProcessing ShortName>`

The Priority Processing feature, which was introduced in EthIf in AUTOSAR 4.3.1, supports EthIf's priority processing of incoming RX messages through Fifoldx. These priority processes can be allocated into different cyclic tasks, higher than the one that would normally be processed by `EthIf_MainFunctionRx`.

Priority Processing feature is supported as follows:

- RX polling mode: when EthIf is configured to work in polling mode, EthIf does not support the management of this job by `EthIf_MainFunctionRx`, but instead you need to configure one `EthIf_MainFunctionRx_<IngressQueueProcessing ShortName>` and use it as `EthIf_MainFunctionRx`.
- `EthIfPhysCtrlRxIngressFifoRef`: Full support for Priority Processing has a dependence on the Ethernet Driver for the target FiFo index, which requires EthIf to be configured for a `Fifoldx` reference. You must configure one `RxIngressFifo` from the Ethernet Driver for full EthIf code generation.
- `EthIfPhysCtrlRxIngressQueueRef`:

Note that this parameter or `EthIfPhysCtrlRxIngressQueueRef` is required if `EthIfEthCtrlRef` is configured.

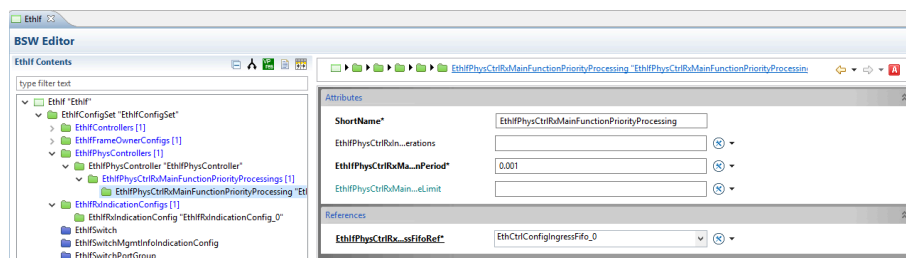


Figure 2.2. `EthIf_MainFunctionRx_<IngressQueueProcessing ShortName>` sample configuration

All ingress queues configured in the physical controller must be referenced by a `EthIfPhysCtrlRxMainFunctionIngressQueueProcessing` container in EthIf. It is not possible to have standalone que that are not referenced by any `EthIfPhysCtrlRxMainFunctionIngressQueueProcessing` container.

2.8.9 `EthIfSwitchOffPortTimeDelay`

This parameter denotes the time delay in seconds after the mode "ETHTRCV_MODE_DOWN" of a `EthIfSwitchPortGroup` will be executed. This is only used for `EthIfSwitPortGroups` which are not referenced by a `EthIfController`. This parameter is used by the `EthIf_SwitchPortGroupRequestMode` API that is called by BswM. The time delay must be greater than the `UdpNm` timings, because `UdpNm` shall finish its shutdown handling (Repeat Message State, Prepare Bus-Sleep

state, Bus-Sleep state). $\text{EthIfSwitchOffPortTimeDelay} > (\text{UdpNmTimeoutTime} + \text{UdpNmWaitBusSleepTime})$.

2.8.10 EthIfPortStartupActiveTime

This parameter denotes the time delay in seconds. This is only used for ports in EthIfSwtPortGroups which are not referenced by a EthIfController. EthIf starts the timer with EthIfPortStartupActiveTime for all these ports activated by the EthIf_StartAllPorts API. After the timer elapses, EthIf switches down all EthSwtPorts which are not requested with ETHTRCV_MODE_ACTIVE via the EthIf_SwitchPortGroupRequestMode API.

2.8.11 EthIfVirtualAddressFilterSize

This parameter specifies the size of the virtual address filter over all configured Eth controllers. It should be set to the overall number of simultaneously-used multicast IP addresses and the size MUST NOT be greater than the total capacity of physical address filters of all configured Eth controllers. This parameter is only valid if EthUpdatePhysAddrFilter is enabled. If multicast communication is not used, then EthUpdatePhysAddrFilter should be disabled in Eth and EthIfVirtualAddressFilterSize should not be configured.

2.8.12 EthIfSignalQualityCheckPeriod

This parameter specifies the period in seconds in which the signal quality will be polled in the context of EthIf_MainfunctionState. The value must be an integral multiple of EthIfMainFunctionStatePeriod. If this parameter is defined, EthIfEnableSignalQualityApi must be set to TRUE.

2.8.13 EthIfCtrlIdx

This parameter provides a zero-based consecutive index of the Ethernet Communication Controllers. Upper layer BSW modules and the EthIf itself use this index to identify an Ethernet Communication Controllers. EthIfCtrlIdx is automatically generated via GenerateId script and must not be manually filled. Any manually entered value will not be overwritten and will lead to an error during Id generation.

2.8.14 EthIfSwitchPortGroup

This parameter provides a zero-based consecutive index of the Ethernet Switch Port Groups. Upper layer BSW modules and the EthIf itself use this index to identify an Ethernet Switch Port Group. EthIfSwitchPortGroupIdx is automatically generated via GenerateId script and shall not be manually filled. Any manually entered value will not be overwritten and will lead to an error during Id generation.

2.8.15 EthIfSwitchGlobalTimeSupport

If EthIfSwitchGlobalTimeSupport is configured to TRUE, the EthSwtGlobalTimeSupportApi configuration parameter in EthSwt must also be configured as TRUE.

2.8.16 EthIfRbTrcvModeChgMainReload

If transceiver mode change is polled (EthIfRbTrcvModeChgMainReload is configured), then EthIf is not able to detect an unexpected transceiver mode DOWN indication from WAIT_ACTIVE. It is the responsibility of the EthTrcv driver to call EthIf_TransceiverModeIndication(DOWN) when an unexpected HW error occurs.

2.8.17 EthIfRbDisableEthControllerInit

This parameter should be set to TRUE if the Ethernet driver does not provide an Eth_ControllerInit function. This function was specified in AR 4.0.3 but was removed in later versions.

2.8.18 EthIfRbDisableEthTransceiverInit

This parameter should be set to TRUE if the Ethernet driver does not provide an EthTrcv_TransceiverInit function. This function was specified in AR 4.0.3 but was removed in later versions.

2.8.19 EthIfRbForeignDrvSupport

RTA-BSW recommends that this parameter should be set to TRUE.

In RTA-BSW v12.7.0, all Ethernet drivers are considered to be "third party".

This parameter redirects all communication between EthIf and the Ethernet driver via the integration files *EthIf_Integrn.c* and *EthIf_Integrn.h*. These files must be modified as described in the Integration Advice section.

2.8.20 EthIfEnableSecurityEventReporting

Enables/disables the reporting of security events to the IdsM.

2.8.21 EthIfSecurityEventRefs

This container contains the configuration of IdsM Event elements representing the security events that the EthIf module shall report to the IdsM (via IdsM_SetSecurityEventWithContextData) in case the corresponding security-related event occurs.

EthIfSecurityEventRefs includes following parameters:

- ETHIF_SEV_DROP_UNKNOWN_ETHERTYPE: Reference to the security event id which will be reported when an ethernet datagram is dropped due to unknown EtherType.
- ETHIF_SEV_DROP_VLAN_DOUBLE_TAG: Reference to the security event id which will be reported when an ethernet datagram is dropped due to double VLAN tag.
- ETHIF_SEV_DROP_INV_VLAN: Reference to the security event id which will be reported when an ethernet datagram is dropped due to an invalid CrtIdx/VLAN.
- ETHIF_SEV_DROP_ETH_MAC_COLLISION: Reference to the security event id which will be reported when an Ethernet datagram is dropped because

the local MAC is same as the source MAC in the incoming frame.

2.8.22 Wakeup And Sleep On Dataline

This feature is enabled in EthIf by setting the configuration parameter EthIfGeneral/EthIfRbWakeupSleepOnDatalineEnabled to *TRUE*.



NOTE

EthIf also has the configuration parameter EthIfGeneral/EthIfWakeUpSupport, which enables/disables the EthIf_CheckWakeup API that complements this feature.

It must be ensured that the underlying Switch/Transceiver also supports Wakeup and Sleep on Dataline before enabling EthIf functionality.

EthIf is involved in the following actions:

Send Wakeup Request

For a Master ECU, if EthIf_SetControllerMode(ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST) is called by EthSM then, depending on the reference type of the EthIfCtrl, either with a call to EthTrcv_SetTransceiverMode(ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST) or EthSwt_SetSwitchPortMode(ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST).



NOTE

This call is possible even if the Switch/Transceiver module is already in *ACTIVE* mode.

Send Wakeup Request - Standalone Switch Port Groups

It is possible to send Wakeup Requests via Standalone Switch Port Groups (i.e. port groups that are not referenced by an EthIfCtrl).

In this case, for a Master ECU, EthIf_SwitchPortGroupRequestMode(ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST) will be called by the BswM module.

Send Sleep Request

For a Master ECU, if EthIf_SetControllerMode(ETH_MODE_DOWN) is called by EthSM, EthIf will call either EthTrcv_SetTransceiverMode(ETH_MODE_DOWN) or EthSwt_SetSwitchPortMode(ETH_MODE_DOWN) to forward the *Sleep.Request* to the dataline. Any pending Wakeup Request will be cleared.

Receive Sleep Request

For a Slave ECU, the referenced EthTrcv of the affected EthIfTransceiver will have EthTrcvConfig/EthTrcvActAsSlavePassiveEnabled set to *TRUE*. It will detect a *Sleep.Indication* (triggered by a *Sleep.Request* from the connected Master ECU) and call EthIf_SleepIndication, which forwards the request to EthSM_SleepIndication.

**NOTE**

EthIf_SleepIndication is *only* supported for transceivers (a limitation of *CONC_632_EthernetWakeOnDataLine*). In other words, communication of a *Sleep.Request* to a Slave ECU is not handled by an Ethernet switch. Such a topology is theoretically possible, however. A Master ECU connected via a dedicated port could force the Ethernet switch and all of its Slave ECUs (including other connected Ethernet switches) to shut down communication. Additionally, if more than one Master was connected to the Ethernet Switch, then the host ECU could maintain the *Sleep.Request* of the Masters. However, these are rated as use cases by AUTOSAR i.e. they are *not* used for an in-vehicle network.

Receive Wakeup Request

For a Slave ECU, EcuM_CheckWakeup calls EthIf_CheckWakeup, which forwards the request to either EthTrcv_CheckWakeup or EthSwt_SwitchCheckWakeup, which then calls EcuM_SetWakeupEvent.

Unexpected Link Down

For a Slave ECU, if the referenced EthTrcv of the affected EthIfTransceiver has EthTrcvConfig/EthTrcvActAsSlavePassiveEnabled set to *TRUE*, then EthIf will check for an unexpected link down.

If the requested link state to EthTrcv is *ETHTRCV_LINK_STATE_ACTIVE*, but the current link state is *ETHTRCV_LINK_STATE_DOWN* (for a time longer than the configured EthIfTransceiver/EthIfQualifiedUnexpectedLinkDownTime for the EthIfTransceiver), then EthIf will call EthSM_SleepIndication to release the affected communication channel.

2.8.23 General

- EthIfCtrlMtu - this parameter supports a maximum MTU size of 1500 bytes.
- EthIf/EthIfGeneral/EthIfGlobalTimeSupport will be configured to *TRUE* only if Eth/EthGeneral/EthGlobalTimeSupport or Can/CanGeneral is *TRUE*.
- EthIfVlanId - is an optional parameter. EthIfVlanId 0 is a valid VLAN id that indicates priority-tagged frames. For untagged VLANs no EthIfVlanId parameter is necessary.
- EthIfSwitchRefOrPortGroupRef - this parameter allows configuration of a reference to either a switch or a switch port group. If port groups are controlled by partial networking, EthIfSwitchPortGroupRefSemantics should be set to *ETHIF_SWITCH_PORT_GROUP_LINK_INFO*, otherwise set the value to *ETHIF_SWITCH_PORT_GROUP_CONTROL*.
- EthIfEthCtrlRef - a reference to a physical Ethernet controller, which is handled by a specific Ethernet controller driver. This parameter is mandatory.
- EthIfEthTrcvRef - a reference to an Ethernet transceiver, which is handled by a specific Ethernet transceiver driver. Note that this parameter is manda-

tory.mandatory.

- EthIfSwitchEnableLinkStateInterrupt - enables/disables link state change interrupt. When set to TRUE, EthSwt calls EthIf_SwitchLinkUpIndication and EthIf_SwitchLinkDownIndication to inform EthIf about link state changes for a particular port. When set to FALSE, EthIf polls for link state changes to a port via EthSwt_GetLinkState. EthIfSwitchEnableLinkStateInterrupt must be set to TRUE or FALSE depending on the capability of the underlying switch.
- EthIfPortRef - a reference to an Ethernet switch port. An EthIfSwitchPortGroup must reference a host port and at least one standard port.
- EthIfSwitchPortGroupRefSemantics - defines how the EthIfSwitchRefOrPortGroupRef referencing a EthIfSwitchPortGroup will be interpreted. This value must be configured if a port group is referenced by EthIfCtrl, otherwise it need not be configured.
- EthIfTxConfirmationFunction - If the Tcplp module is used as the upper layer with respect to EthIf, no TxConfirmation function needs to be configured.
- EthIfMaxTxBufsTotal - this parameter is not supported as the number of TX buffers is handled in the Ethernet Driver, and EthIf does not have its own buffers.
- EthIfMaxTrcvsTotal - this parameter is not supported. Instead, the number of EthTrcvs or EthIfTrcvs can be derived from the current configuration.
- EthIfVersionInfoApiMacro - this parameter is not supported. Only the GetVersionInfo Function is provided by EthIf.
- EthIfOwner - this parameter is not supported. Instead, references to the EthIfRxIndicationRef and EthIfTxConfirmationRef functions have been introduced to simplify the configuration.
- EthIfTrcvLinkStateChgConfig - this parameter is not supported. Configuration of the link state change callback function has moved from EthIf to EthSM to allow upper layer specific callback configuration and prevent unwanted link state change indications to the upper layer for EthIfCtrls that are not used by that upper layer.
- EthIfPhysControllerIdx - this parameter provides a zero-based consecutive index of the physical controllers. The upper layer BSW modules and the EthIf itself use this index to identify a physical controller. It is automatically generated via the GenerateId script.
- EthIfRxIndicationIterations - this parameter is not used because only the EthIfPhysCtrlRxIndicationIterations parameter for each EthIfPhysCtrlRxMainFunctionIngressQueueProcessing container needs to be configured. (See EthIf_MainFunctionRx above).
- EthIfEthDriverPrefix - see the Integration Advice for more details.
- EthIf_EthGetSpiStatus - Obtains the SPI interface status when MACPHY controller is used.
- EthIfRbPendingTxConfirmationMax - the maximum number of pending TxConfirmations that can be stored for this controller. When set, the pending TxConfirmations are stored in a list which is then searched when a

TxConfirmation occurs. When not set, the TxConfirmation function has to be called with valid DataPtr and LenByte and the required information is obtained from there.

2.9 Integration Advice

2.9.1 Init Functions

EthIf is initialized via EthIf_Init, which must be called by EcuM or BswM during the system initialization.

- In AR 4.0.3, the controller state should be set to ETH_MODE_ACTIVE before Ethernet Transceiver Init is called. Eth_ControllerInit is called from EthIf and it should be remapped to the function Eth_<EthIfEthDriverPrefix>_ControllerInit. You'll need to refine the implementation (in *EthIf_Integn.c*) and the corresponding declaration in EthIf_Integn.h.
- In AR 4.2.2 and later, the functionality implemented in EthTrcv_TransceiverInit and Eth_ControllerInit have been merged into Eth_Init.

2.9.2 Delnit Functions

No Delnit is required.

2.9.3 Scheduled Functions

Each EthIf_MainFunctionRx, EthIf_MainFunctionTx and EthIf_MainFunctionState must be scheduled cyclically as a OS task with a fixed cycle time.

- EthIf_MainFunctionTx must be scheduled for transmit (Tx) processing, and it should be scheduled after Tcplp_MainFunction.
- EthIf_MainFunctionState must be scheduled for the processing of the link state of the communication hardware (transceiver, switch ports), and it should be scheduled after EthSM_MainFunction.
- EthIf_MainFunctionRx must be scheduled for triggering frame reception in polling mode. Note that one of these functions will be generated for each EthIfPhysCtrlRxMainFunctionIngressQueueProcessing container, based on the short name of this container. The EthIf_MainFunctionRx used to pool the physical controller ingress queue in which TCP frames are received should be scheduled before Tcplp_MainFunction to avoid unnecessary TCP retransmissions.

If faster reception is required, EthIf_MainFunctionRx can be scheduled with a smaller cycle time raster than EthIf_MainFunctionTx and Tcplp_MainFunction.

Each EthIf_MainFunctionRx can be scheduled independently. One EthIf_MainFunctionRx polls one physical controller ingress queue at the Ethernet Driver layer.

2.9.4 Dependencies

EthIf is a primary component of the EthStack package, and it requires at least the following modules to be integrated along with it.

- Tcplp
- EthSM
- Eth Driver
- EthTrcv and/or EthSwt (optional)

2.9.5 Interrupt Mode

The Tcplp module does not support re-entrancy. Therefore, if Tcplp is configured as the upper layer module, the interrupt mode of EthIf must not be used. The interrupt mode can only be used in combination with a CDD configured as the upper layer.

2.9.6 Integration with Third Party Ethernet Drivers

In RTA-BSW v12.7.0, all Ethernet drivers are considered to be "third party".

EthIf supports third party Ethernet Drivers via the integration files *EthIf_Integrn.c* and *EthIf_Integrn.h*.

These files must be modified to implement an interface between the Ethernet drivers in the MCAL and EthIf.

Modifications are necessary regardless of the AUTOSAR version of the Ethernet driver. AUTOSAR versions before v22-11 can be supported by adapting the interface appropriately, but modifications are needed for v22-11 too. In most cases this is a matter of uncommenting sample code, but some functions require other changes as described below.

The following parameters must be configured in order to enable the *EthIf_Integrn.c* functionality:

- EthIf/EthIfGeneral/EthIfRbForeignDrvSupport = TRUE
- EthIf/EthIfConfigSet/EthIfPhysController/EthIfEthDriverPrefix = 6_Integrn

If ConfGen is used, these may be set in algo.properties as follows:

```
manprop_EthIfGeneral=\
EthIfRbForeignDrvSupport:true

manprop_EthernetCommunicationController=\
EthIfEthDriverPrefix:6_Integrn
```

2.9.6.1 EthIf_TxConfirmation

EthIf_TxConfirmation is only called by the Ethernet Driver if Eth_Transmit is called with the TxConfirmation flag set to TRUE.

The purpose of EthIf_TxConfirmation is to confirm back to upper layers when the Tx message is ready to be processed by the lower layer. EthIf will resolve the

correct upper layer index (by VlanId and Control Index), but the VlanId is calculated based on the Ethernet Header VLAN tag, which requires the Ethernet Driver to inform EthIf for processing.

The implementation of EthIf_TxConfirmation must be compatible with the target driver module. The name and parameters must be adapted as follows:

- If you are using an AR 4.0.3 MCAL, the definition of *EthIf_TxConfirmation* must change to *EthIf_Cbk_TxConfirmation(uint8 CtrlIdx, uint8 BufIdx)* to ensure compatibility.
- If you are using an AR 4.2.2 MCAL, the definition of *EthIf_TxConfirmation* must change to *EthIf_TxConfirmation(uint8 CtrlIdx, Eth_BufIdxType BufIdx)* to ensure compatibility.
- If you are using an AR 4.3.1 or later MCAL, the definition of *EthIf_TxConfirmation* is already correct.

Within the function body, you must provide code to adapt the AUTOSAR standard interface of *EthIf_TxConfirmation* to the internal interface used by EthIf, named *EthIf_Integn_TxConfirmation*. Additional parameters are required:

```
void EthIf_Integn_TxConfirmation ( uint8 CtrlIdx,
                                  Eth_BufIdxType BufIdx,
                                  const Eth_DataType * DataPtr,
                                  uint16 LenByte,
                                  Std_ReturnType Result );
```

CtrlIdx, *BufIdx* and *Result* are the same as in AR 4.3.1 or later.

LenByte is not currently used.

DataPtr is used to check the VLAN tag. If this is required, then *DataPtr* should point to the first byte of the destination MAC address in the transmitted data, so that the Ethernet type appears at *DataPtr*[12] and *DataPtr*[13], and the VLAN tag (if present) appears at *DataPtr*[14] and *DataPtr*[15]. MCAL-specific code is needed to obtain this information.

If the VLAN tag is not required, then *DataPtr* can point to a dummy buffer. Note that the buffer must be large enough to include the VLAN tag, and must be zeroed:

```
uint16 LenByte = 0;
Eth_DataType EthIf_TxBuffer_Stub[ETHIF_ETHTHDR_OFFSET + ETHIF_VLANTAG_LENGTH] = {0};
DataPtr = EthIf_TxBuffer_Stub;
```

2.9.6.2 EthTrcv_TransceiverLinkStateRequest

The *EthTrcv_TransceiverLinkStateRequest* is required by BSW EthIf but not defined in older AR versions.

If you are using a non-BSW *EthTrcv* module which does not implement *EthTrcv_TransceiverLinkStateRequest*, provide a stub implementation such as:

```
Std_ReturnType EthTrcv_TransceiverLinkStateRequest
( uint8 TrcvIdx, EthTrcv_LinkStateType LinkState )
{
    Std_ReturnType RetVal_o = E_OK;
    return RetVal_o;
}
```

2.9.6.3 EthIf_RxIndication

The implementation of `EthIf_RxIndication` must be compatible with the target driver module, so in some cases it may be necessary to modify this function.

- If you are using an AR 4.2.2 or later MCAL, the definition of *EthIf_RxIndication* is already correct. The call to *EthIf_Integn_RxIndication* should be uncom-mented.
- If you are using an AR 4.0.3 MCAL, the definition of *EthIf_RxIndication* must change to *EthIf_Cbk_RxIndication(uint8 CtrlIdx, Eth_DataType* DataPtr, uint16 LenByte)* to ensure compatibility.

For AR 4.0.3, the function body must be adapted to generate the *FrameType*, *IsBroadcast* and *PhysAddr* parameters for *EthIf_Integn_RxIndication*. Additionally, the *DataPtr* parameter should be offset by the Ethernet header size. For example:

```
void EthIf_Cbk_RxIndication(uint8 CtrlIdx, Eth_DataType* DataPtr, uint16 LenByte) {
    /* Local variable declaration */
    uint8 PhysAddr[6];
    Eth_FrameType FrameType;
    boolean IsBroadcast;
    uint16 lLenByte_u16;

    /* Local variable initialization */
    FrameType = ((DataPtr[ETHIF_ETHTYPE_BYTE1] << 8) & 0xFF00) | DataPtr[ETHIF_ETHTYPE_BYTE2];
    IsBroadcast = FALSE;
    /* Copy 6 bytes Source MAC address */
    rba_BswSrv_MemCopy(PhysAddr, DataPtr + ETHIF_OFFSET_SOURCEMACADDR, 6U);

    if (LenByte >= ETHIF_ETHTHDR_OFFSET) { // Check if Ethernet header length enough
        lLenByte_u16 = LenByte - ETHIF_ETHTHDR_OFFSET;
    } else {
        lLenByte_u16 = 0;
    }

    if ((0xFF == DataPtr[0]) // Check if the destination address is a broadcast address.
        && (0xFF == DataPtr[1]) && (0xFF == DataPtr[2])
        && (0xFF == DataPtr[3]) && (0xFF == DataPtr[4])
        && (0xFF == DataPtr[5])) {
        IsBroadcast = TRUE;
    }
    EthIf_Integn_RxIndication(CtrlIdx, FrameType, IsBroadcast, PhysAddr,
        (Eth_DataType*) &DataPtr[ETHIF_ETHTHDR_OFFSET], lLenByte_u16);
}
```

2.9.6.4 Eth_UpdatePhysAddrFilter and Eth_SetPhysAddr

The implementation of `Eth_6_Integn_UpdatePhysAddrFilter` and `Eth_6_Integn_SetPhysAddr` must be compatible with the target driver module:

- If you are using an AR 4.2.2 or later MCAL, the definitions of these functions are already correct. The call to *Eth_UpdatePhysAddrFilter* or *Eth_SetPhysAddr* can be uncommented.
- If you are using an AR 4.0.3 MCAL, the definitions of these functions should be replaced by empty dummies (returning `E_NOT_OK` in the case of `Eth_6_Integn_UpdatePhysAddrFilter`).

For AR 4.0.3, the `Eth_UpdatePhysAddrFilter` API is not supported. Users are responsible for ensuring the following:

- Any `EthIf` features that are dependent on the `Eth_UpdatePhysAddrFilter` API

(e.g. `EthIfVirtualAddressFilterSize`) are not used.

- `Tcplp` is not configured with `TcplpAddressType` as `TCPIP_MULTICAST`.
- `Tcplp` is not configured with `TcplpDomainType` as `TCPIP_AF_INET6`.

For AR 4.2.2, the `EthUpdatePhysAddrFilter` parameter can be used to enable/disable the `Eth_UpdatePhysAddrFilter` API based on project needs.

For AR 4.3.1 or later, the `Eth_UpdatePhysAddrFilter` API is mandatory, but because the `EthIf` validators are common for AR 4.2.2 and AR 4.3.1, the `EthUpdatePhysAddrFilter` parameter should be set to `TRUE` (enabled) if any dependent features (e.g. `EthIfVirtualAddressFilterSize`) are used.

2.9.6.5 Eth_ProvideTxBuffer

The implementation of `Eth_ProvideTxBuffer` must be compatible with the target driver module.

For an AR 4.0.3 MCAL, a suggested implementation is:

```
BufReq_ReturnType Eth_6_Integn_ProvideTxBuffer( uint8 CtrlIdx,
                                                uint8 Priority,
                                                Eth_BufIdxType * BufIdxPtr,
                                                Eth_DataType ** BufPtr,
                                                uint16 * LenBytePtr )
{
    BufReq_ReturnType lRetVal_en = BUFREQ_E_NOT_OK;
    uint8 BufIdx = 0;
    lRetVal_en = Eth_ProvideTxBuffer(CtrlIdx, &BufIdx, BufPtr, LenBytePtr);
    *BufIdxPtr = (BufIdxPtr) BufIdx;
    return lRetVal_en;
}
```

For an AR 4.2.2 MCAL, a suggested implementation is:

```
BufReq_ReturnType Eth_6_Integn_ProvideTxBuffer( uint8 CtrlIdx,
                                                uint8 Priority,
                                                Eth_BufIdxType * BufIdxPtr,
                                                Eth_DataType ** BufPtr,
                                                uint16 * LenBytePtr )
{
    BufReq_ReturnType lRetVal_en = BUFREQ_E_NOT_OK;
    lRetVal_en = Eth_ProvideTxBuffer(CtrlIdx, BufIdxPtr, BufPtr, LenBytePtr);
    return lRetVal_en;
}
```

For an AR 4.3.1 or later MCAL, a suggested implementation is:

```
BufReq_ReturnType Eth_6_Integn_ProvideTxBuffer( uint8 CtrlIdx,
                                                uint8 Priority,
                                                Eth_BufIdxType * BufIdxPtr,
                                                Eth_DataType ** BufPtr,
                                                uint16 * LenBytePtr )
{
    BufReq_ReturnType lRetVal_en = BUFREQ_E_NOT_OK;
    lRetVal_en = Eth_ProvideTxBuffer(CtrlIdx, Priority, BufIdxPtr, BufPtr, LenBytePtr);
    return lRetVal_en;
}
```

2.9.6.6 Eth_Transmit and Eth_Receive

The implementation of these functions must be compatible with the target driver module.

The `Eth_6_Integn_Transmit` and `Eth_6_Integn_Receive` functions should adapt the parameter types and number of parameters appropriately:

- For an AR 4.0.3 MCAL, *Eth_6_Integn_Transmit* should cast *BuIdx* to `uint8`, and *Eth_6_Integn_Receive* should ignore *FifIdx* and assign `ETH_RECEIVED` to *RxStatusPtr[0]*.
- For an AR 4.2.2 MCAL, *Eth_6_Integn_Receive* should ignore *FifIdx*.
- For all other cases, parameters should be passed as-is to *Eth_Transmit* and *Eth_Receive*.

2.9.7 Using the EthIfEthDriverPrefix parameter

For some specific MCAL providers, the APIs are developed with customized names. Infineon MCALs, for example, have APIs that have a prefix of `<_17_EthMac_>` inserted into the name (e.g. *Eth_17_EthMac_Transmit*).

EthIf allows you to reference the appropriate prefix for your MCAL with the *EthIfEthDriverPrefix* configuration parameter.

The recommended value "6_Integn" matches the sample *EthIf_Integn.c* provided with RTA-BSW. If another *EthIfEthDriverPrefix* value is used, changes must be made to the integration file *EthIf_Integn.c* to reflect the prefix in Macro names and API names (Extern and Definition).

2.9.8 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The EthIf module uses the following OS resources:

- *EthIf_EthCtrlState*
- *EthIf_EthCtrlOperationOngoing*
- *EthIf_SetTrcvState*
- *EthIf_UpdatePhysAddrFilter*
- *EthIf_UpdateMeasurementData*
- *EthIf_TrvcSignalQuality*
- *EthIf_SwtPortSignalQuality*
- *EthIf_SetSwtPortState*
- *EthIf_PendingTxConfirmationList*

2.9.9 Execution Order Constraints

Execution Order Constraint (EOC) defines the execution order of the mainfunctions/executable objects/runnable entities between different modules to avoid problems at execution time. If a module is selected in Code Generation, the execution order for that module is stored in <ModuleName>_EOC.xml.

EthIf has dependences on EthTrcv_EOC.xml to fulfil the constraints of order between itself and EthTrcv, so if your project uses an EthTrcv module from a third party, which does not support EOC, you should note the following:

- To avoid errors when doing the Rte generation, you should not select EthIf_EOC.xml. This means you should not utilize this feature from ISOLAR A/B.
- If you want to use this feature on ISOLAR A/B, then you should select EthIf_EOC.xml, but prior to doing the Rte generation you should manually configure the EthTrcv execution order constraints.

2.9.10 EthIf_CtrlModelIndication

EthIf_CtrlModelIndication is required to be called by Ethernet Driver, in order to make EthIf state machine works as expected.

If a third party MCAL Ethernet Driver is used that does not call EthIf_CtrlModelIndication when the Ethernet controller has successfully switched to request mode with a Eth_SetControllerMode call, then the user must add integration code or change the MCAL implementation to call the EthIf_CtrlModelIndication in order for EthIf to work successfully.

2.10 Hardware Requirements

Table 2.65.EthIf Hardware Requirements

Support	Usage
FLASH	5744 bytes
RAM	442 bytes

Usage calculated based on a sample configuration.

3 Ethernet State Manager

Module name	EthSM
Package	RTA-ETH
AUTOSAR Module ID	143
AUTOSAR Version	22-11

3.1 Introduction

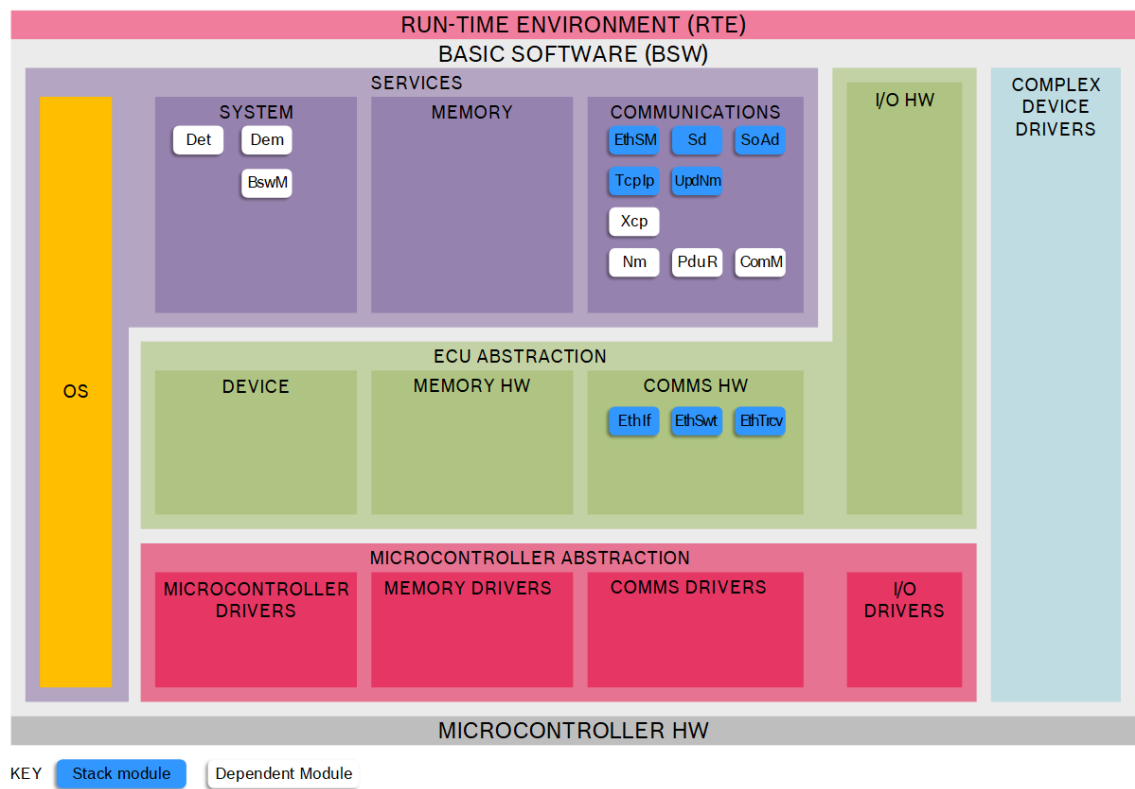


Figure 3.1.RTA-ETH Architecture

The Ethernet State Manager module (EthSM) is located in the communication services abstraction layer, acting as part of the upper layer for the RTA-ETH stack, and resides at the same level as the communication protocol modules in the TCP/IP stack (encapsulated by the TcpIp module) and any CDD communication modules (e.g. EthTSyn). The primary interaction with the lower layer is through the EthIf module. The diagram above shows the position of EthSM in the AUTOSAR layered architecture.

EthSM provides a hardware-independent interface to the low-level communication system, and is primarily used for controlling startup/shutdown and other mode changes for communication channels (requested from the ComM module) on Ethernet clusters which may contain one or more virtual controllers. It also performs similar operations at the controller level for requests from the TcpIp

module, as TCP/IP network requests require an IP address for each configured node on the network.

The primary functionality provided by the EthSM can be summarised as follows.

- Maintains a state machine for each configured Ethernet network, which processes ComM state change requests.
- Provides information about the current communication mode.
- Controls the mode and state (with respect to TcpIp) of each configured Ethernet controller on the network.
- Provides transceiver link state change notification to network user modules such as EthTSyn.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

3.2 Supported Features

3.2.1 General

- Asynchronous API operation.
- TCP/IP and CDD network user types.
- Separate state machine handling for each configured Ethernet network.
- Facilitates the use of Wakeup and Sleep on Dataline.

3.2.2 Offline to Online Transition

If the current network state is ETHSM_STATE_OFFLINE and FULL_COMMUNICATION is requested by ComM, EthSM triggers the transition to ETHSM_STATE_ONLINE. This operation includes the following steps.

1. A request is made to EthIf to set the specified controller mode to ETH_MODE_ACTIVE. EthIf activates the hardware that is mapped to the specified EthIfCtrl index. The underlying hardware mode change may be asynchronous, e.g. in case EthIfCtrl has a reference to a configured transceiver. EthSM waits for the callback from EthIf indicating that the EthIfCtrl mode change was successfully finished. If for any reason no indication is received, EthSM can not proceed the state change and will remain in the state ETHSM_SUBSTATE_OFFLINE_WAIT_CTRLMODE_ACTIVE.
2. EthSM waits for the callback from EthIf indicating that the link is up (ETHIF_LINKSTATE_ACTIVE). If for any reason the link will not go up, EthSM cannot proceed with the state change and will remain in state ETHSM_STATE_WAIT_TRCVLINK.
3. If EthSMNetworkUserType is set to CDD, then EthSM calls the configured {User}_TrcvLinkStateChg function to indicate a link state change and

informs ComM of a state change to mode FULL_COMMUNICATION, when EthSM changes state from ETHSM_STATE_WAIT_TRCVLINK to ETHSM_STATE_WAIT_ONLINE.

4. If EthSMNetworkUserType is set to TCPIP, a request is made to Tcplp to switch to TCPIP_STATE_ONLINE and wait for the callback from Tcplp indicating that the state was reached, i.e. the IP address was successfully assigned. In this case EthSM will switch the network to the ETHSM_STATE_ONLINE state. Communication over TCP/IP is now possible, and ComM is notified that the FULL_COMMUNICATION state is now reached. Depending on the configuration of Tcplp, the IP address may be assigned automatically, or manually by the upper layer. If for any reason Tcplp cannot assign an IP address, EthSM can not proceed with the state change and the network will remain in the state ETHSM_STATE_WAIT_ONLINE.



NOTE

An invalid configuration of Ethernet controller driver (ETH), the Ethernet Transceiver driver (EthTrcv), the Ethernet switch (EthSwt), the Ethernet Interface (EthIf) or Tcplp modules can block the EthSM state machine transition to ETHSM_STATE_ONLINE.

3.2.3 Online to Offline Transition

If the current network state is ETHSM_STATE_ONLINE and NO_COMMUNICATION is requested by ComM, EthSM triggers the transition to ETHSM_STATE_OFFLINE. This operation includes the following steps.

1. If EthSMNetworkUserType is set to TCPIP, a request is made to Tcplp to switch to TCPIP_STATE_OFFLINE, and EthSM waits for the callback from Tcplp indicating that on the specified controller, all sockets were closed and all IP addresses released. A request is made to EthIf to set the controller mode to ETH_MODE_DOWN. EthIf then deactivates the hardware mapped to the specified controller.
2. If EthSMNetworkUserType is set to CDD, EthSM notifies the CDD module that the link state is ETHIF_LINKSTATE_DOWN (via the configured {User}_TrcvLinkStateChg function) as soon as EthIf indicates the state change was successful and sets the network state to ETHSM_STATE_OFFLINE.
3. EthSM informs ComM of a state change to mode NO_COMMUNICATION.

**NOTE**

{User}_TrcvLinkStateChg does NOT change the physical link state of the hardware, only the virtual link state of the controller. For example, if the controller state is ETH_MODE_DOWN, its link state is considered to be ETHIF_LINKSTATE_DOWN, although the physical link state may still be ETHTRCV_LINK_STATE_ACTIVE if it is used by other controllers.

3.2.4 Onhold State Transition

If the network state is ETHSM_STATE_ONLINE, and the link state changes to ETHIF_LINKSTATE_DOWN, the communication mode will be set to ETHSM_STATE_ONHOLD till the link state changes to ETHIF_LINKSTATE_ACTIVE, ComM requests NO_COMMUNICATION, or the assigned IP address is released by Tcplp.

EthSM notifies any configured network users of type CDD of a link state change to ETHIF_LINKSTATE_DOWN, if a transition occurs from ETHSM_STATE_ONLINE to ETHSM_STATE_ONHOLD, and similarly notifies the network user of a link state change to ETHIF_LINKSTATE_ACTIVE, if a transition occurs from ETHSM_STATE_ONHOLD to ETHSM_STATE_ONLINE.

In the ETHSM_STATE_ONHOLD state, the state machine implements an intermediate transition to ETHSM_STATE_WAIT_OFFLINE before EthSM switching to ETHSM_STATE_OFFLINE, as EthSM waits for Tcplp to signal the state of the controller. During this intermediate transition, EthSM indicates state ETHSM_STATE_OFFLINE to BswM and the communication mode COMM_NO_COMMUNICATION to the ComM module, after the request to set the controller mode to ETH_MODE_DOWN was accepted. This is so that the application can be informed as early as possible when communication is disabled. If the application was not informed, it may try still to send frames, which can lead to timeouts or other errors in the application.

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 EthSM/EthSMGeneral/EthSMDDataLinkInfoApi

Short Name	EthSMDDataLinkInfoApi
Long Name	EthSMDDataLinkInfoApi (currently unused)
Description	Enables and disables the API: ChrgM_DataLinkIndication()
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.2 EthSM/EthSMNetwork/EthSMDemEventParameterRefs/EthSMRb-DynCreateEvent

Short Name	EthSMRbDynCreateEvent
Description	ETAS (vendor 11) Selects the way how the EthSM DemEvents are created. If set to "Disabled" no DemEvent is automatically created. If configured to "Standard", the reference to a DemEvent is also automatically created and shall therefore not to be set manually.
Multiplicity	0..1
Default Value	Disabled
Enumeration Literals	Standard, Disabled
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.3 EthSM/EthSMNetwork/EthSMNetworkUserRef

Short Name	EthSMNetworkUserRef
Description	ETAS (vendor 11) specific parameter: Reference to EthSM Network user container. The referenced container has the user specific TrcvLinkStateChg function and header file.
Multiplicity	1..*
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.4 EthSM/EthSMNetwork/EthSMTcplpUsed

Short Name	EthSMTcplpUsed
Long Name	EthSMTcplpUsed (currently unused)
Description	Specifies if the interaction with a Tcplp module is enabled or disabled for the EthSMNetwork
Multiplicity	0..0
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.5 EthSM/EthSMNetworkUserConfig

Short Name	EthSMNetworkUserConfig
Description	ETAS (vendor 11) specific container: This container contains configuration of EthSM Network Users.
Multiplicity	1..*

Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.6 EthSM/EthSMNetworkUserConfig/EthSMNetworkUserHeaderFileName

Short Name	EthSMNetworkUserHeaderFileName
Description	ETAS (vendor 11) specific parameter: This parameter specifies the name of the header file containing the definition of the User_TrvcLinkStateChg function. The range of characters is 1 to 32. This parameter shall only be configurable if NetworkUserType is set to CDD.
Multiplicity	0..1
String Length Range	1..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

3.3.1.7 EthSM/EthSMNetworkUserConfig/EthSMNetworkUserType

Short Name	EthSMNetworkUserType
Description	ETAS (vendor 11) specific parameter: This parameter specifies the EthSM network user module.
Introduction	* TCPIP : Transmission Control Protocol / Internet Protocol (Tcplp is currently a mandatory module for EthSM and thus one and only one EthSMNetworkUser of type TCPIP shall be configured in EthSMNetworkUserConfig). * CDD : Complex Device Driver (for example, EthTSyn is a CDD, it is possible to configure 0..* Complex Device Drivers in EthSMNetworkUserConfig).
Multiplicity	1..1
Default Value	-
Enumeration Literals	CDD, TCPIP
Type	ECUC-ENUMERATION-PARAM-DEF

3.3.1.8 EthSM/EthSMNetworkUserConfig/EthSMUserTrcvLinkStateChgFunction

Short Name	EthSMUserTrcvLinkStateChgFunction
Description	ETAS (vendor 11) specific parameter: This parameter defines the name of the User_TrvcLinkStateChg function of the Network User module. This parameter shall only be configurable if NetworkUserType is set to CDD.

Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

3.4 Limitations

The following features are not supported for version v12.7.0

- There are no known limitations.

3.5 Exclusions

3.5.1 Configuration Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- EthSM/EthSMGeneral
- EthSM/EthSMGeneral/EthSMDevErrorDetect
- EthSM/EthSMGeneral/EthSMDummyMode
- EthSM/EthSMGeneral/EthSMMainFunctionPeriod
- EthSM/EthSMGeneral/EthSMVersionInfoApi
- EthSM/EthSMNetwork/EthSMComMNetworkHandleRef
- EthSM/EthSMNetwork/EthSMDemEventParameterRefs
- EthSM/EthSMNetwork/EthSMDemEventParameterRefs/ETHSM_E_LINK_-DOWN
- EthSM/EthSMNetwork/EthSMForwardSilentCommunicationEnabled
- EthSM/EthSMNetwork/EthSMWakeupSleepOnDatalineEnabled

3.6 Deviations

3.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1.1 EthSM/EthSMNetwork/EthSMEthIfControllerRef

[RTA-BSW] Description	Reference to EthIfCtrl container where a physical Ethernet controller and Ethernet Transceiver (optional) or Ethernet Switch (optional) or Ethernet Switch Port Group (optional) combination is configured.
[AUTOSAR] Description	Reference to EthIfCtrl container where a ETH controller and
[REMOVED] Introduction	transceiver (optional) combination is configured.

3.7 API Definition

List of supported API functions

Table 3.11.EthSM supported API functions

EthSM_Init	Initialises the module.
EthSM_GetVersionInfo	Provides version information for the module.
EthSM_RequestComMode	Requests a communication mode for the specified network handle.
EthSM_GetCurrentComMode	Returns the current communication mode for the specified network handle.
EthSM_TcpIpModeIndication	Callback function used by TcpIp to report the state of the specified controller.
EthSM_CtrlModeIndication	Callback function triggered by EthIf which notifies EthSM of a specified controller's mode change.
EthSM_TrcvLinkStateChg	Callback function called by EthIf to report a transceiver link state change.
EthSM_SleepIndication	Callback function called by EthIf when a sleep indication is detected on the network.
EthSM_MainFunction	Cyclic Main Function which is called from the Scheduler.

For Further information please refer to AUTOSAR_SWS_EthernetStateManager.pdf

3.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

3.8.1 General

The EthSM/EthSMNetworkUserConfig container, and the EthSMNetworkUserType, EthSMUserTrcvLinkStateChgFunction and EthSMNetworkUserHeaderFileName parameters, have been added to support multiple network users. See the Supported Features section for more details.

If EthSMNetworkUserType is set to CDD, the EthSMUserTrcvLinkStateChgFunction and EthSMNetworkUserHeaderFileName must also be configured.

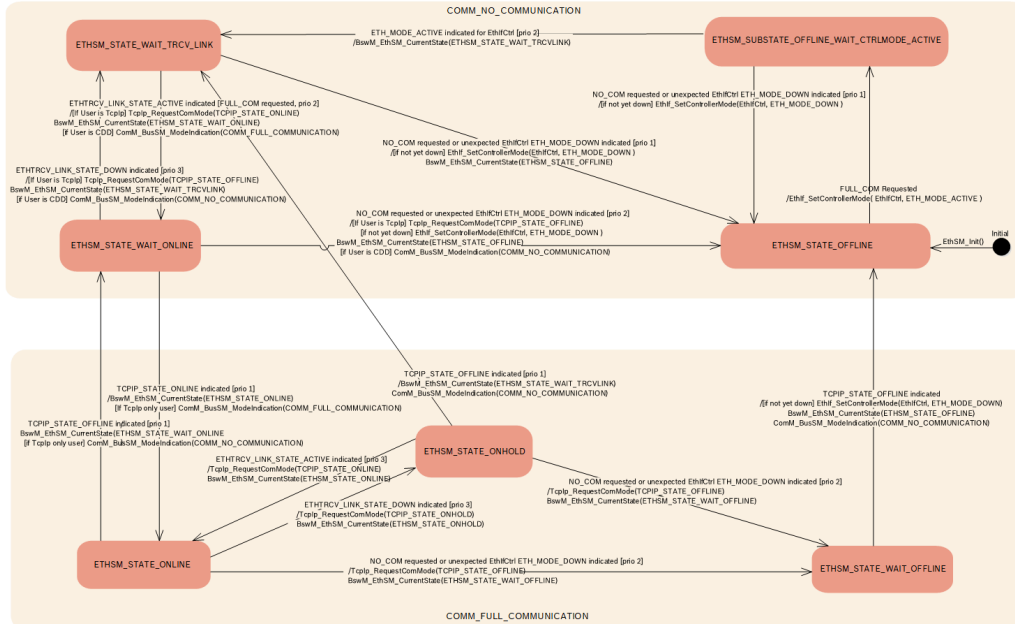
The EthSMNetworkUserRef parameter has been added per EthSM Network as a reference to EthSM/EthSMNetworkUserConfig.

3.8.2 ETHSM_STATE_WAIT_ONLINE

The state name ETHSM_STATE_WAIT_ONLINE is only meaningful when EthSMNetworkUserType is set to TCP/IP. If EthSMNetworkUserType is set to CDD (i.e. non-IP protocols) the name is not valid, but is retained since it is specified by AUTOSAR and used to inform upper layers (e.g. BswM).

3.8.3 ETHSM_SUBSTATE_OFFLINE_WAIT_CTRLMODE_ACTIVE

An additional state ETHSM_SUBSTATE_OFFLINE_WAIT_CTRLMODE_ACTIVE (not specified by AUTOSAR) has been implemented in the EthSM network mode state machine.



3.8.4 Wakeup And Sleep On Dataline

This feature is enabled in EthSM if the parameter EthSMNetwork/EthSMWakeupSleepOnDatalineEnabled is set to *TRUE*.

Send Wakeup Request

For a Master ECU, if ComM calls `EthSM_RequestComMode(COMM_FULL_COMMUNICATION_WITH_WAKEUP_REQUEST)`, EthSM will send the Wakeup Request to EthIf with a call to `EthIf_SetControllerMode(ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST)`. EthIf will then forward the request to either EthTrcv or EthSwt.

If `EthSMWakeupSleepOnDataLineEnabled` is set to *FALSE*, then EthSM will interpret the ComM call `EthSM_RequestComMode(COMM_FULL_COMMUNICATION_WITH_WAKEUP_REQUEST)` as a normal `EthSM_RequestComMode(COMM_FULL_COMMUNICATION)` request.



NOTE

`EthSM_RequestComMode(COMM_FULL_COMMUNICATION_WITH_WAKEUP_REQUEST)` can be called by ComM even after a previous `EthSM_RequestComMode(COMM_FULL_COMMUNICATION)` call.

Sending the Wakeup Request is possible in the following EthSM states: *ETHSM_STATE_OFFLINE*; *ETHSM_SUBSTATE_OFFLINE_WAIT_CTRLMODE_ACTIVE*; *ETHSM_STATE_WAIT_TRCVLINK*; *ETHSM_STATE_WAIT_ONLINE*; *ETHSM_STATE_ONLINE*; *ETHSM_STATE_ONHOLD*.



NOTE

EthSM is not involved when a Wakeup Request is received by a Slave ECU.

Send Sleep Request

For a Master ECU, if ComM calls `EthSM_RequestComMode(COMM_NO_COMMUNICATION)`, EthSM will send a Sleep Request to EthIf via a call to `EthIf_SetControllerMode(ETH_MODE_DOWN)`. EthIf will then forward the request to either the transceiver or switch, which places a *Sleep.Request* on the dataline.

Receive Sleep Request

For a Slave ECU, EthSM allows the forwarding of a Sleep Indication to ComM after EthIf calls `EthSM_SleepIndication`, the Ethernet transceiver having detected a *Sleep.Indication*. The latter is triggered by a *Sleep.Request* from the Master ECU.

3.9 Integration Advice

3.9.1 Init Functions

`EthSM_Init` is called by `EcuM` during the system initialization.

3.9.2 Delnit Functions

No `Delnit` is required.

3.9.3 Scheduled Functions

EthSM_MainFunction operates the effects of the EthSM state machine, which EthSM implements for each configured network. EthSM_MainFunction should be called cyclically with a fixed cycle time defined via the EthSMMainFunctionPeriod parameter, and it should be scheduled after ComM_MainFunction_Eth_Channelx because ComM communication mode requests are processed in EthSM_MainFunction.

3.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The EthSM module does not use any exclusive OS resources.

3.10 Hardware Requirements

Table 3.12. EthSM Hardware Requirements

Support	Usage
FLASH	2262 bytes
RAM	244 bytes

Usage calculated based on a sample configuration.

4 Time Synchronization over Ethernet

Module name	EthTSyn
Package	RTA-ETH
AUTOSAR Module ID	164
AUTOSAR Version	22-11

4.1 Introduction

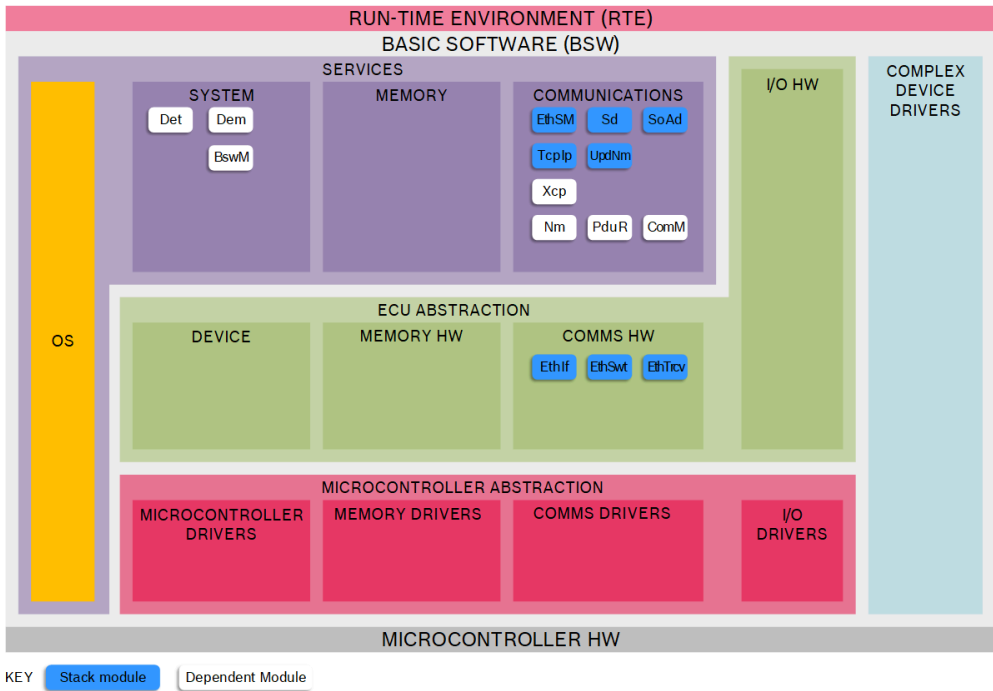


Figure 4.1.Ethernet Stack Architecture

The Global Time Synchronization over Ethernet module (EthTSyn) handles the collection and distribution of synchronized time information over an Ethernet network. EthTSyn is a member of the Communication Service Layer, and the diagram above shows its position in the Autosar layered architecture.

EthTSyn is based on PTP (Precision Time Protocol), a protocol used to synchronize clocks throughout a computer network. PTP is described in the IEEE1588 standard, and in IEEE802.1AS (a profile or subset for using IEEE1588).

On a local area network, PTP can achieve clock accuracy in the sub-microsecond range, which makes it suitable for measurement and control systems. However, because PTP is not developed specifically for the automotive industry, the Autosar specification for EthTSyn has some deviations from the IEEE specification on which PTP is based. Those deviations are as follows:

- The role and functions of an ECU are known at the configuration and compile time.

- The network is static, which means the role of Master and Slave ECU will not change at runtime.
- The Latency Calculation feature is optional.
- PTP works over VLAN.
- There is a Debounce Time between two PTP messages.
- PTP messages can be transmitted asynchronously.

EthTSyn interacts with StbM to get/set the current time value. It also has interactions with EthIf (for receiving and transmitting messages) and BswM (for the coordination of network access). The diagram below shows EthTSyn's place in the overall Communications architecture.

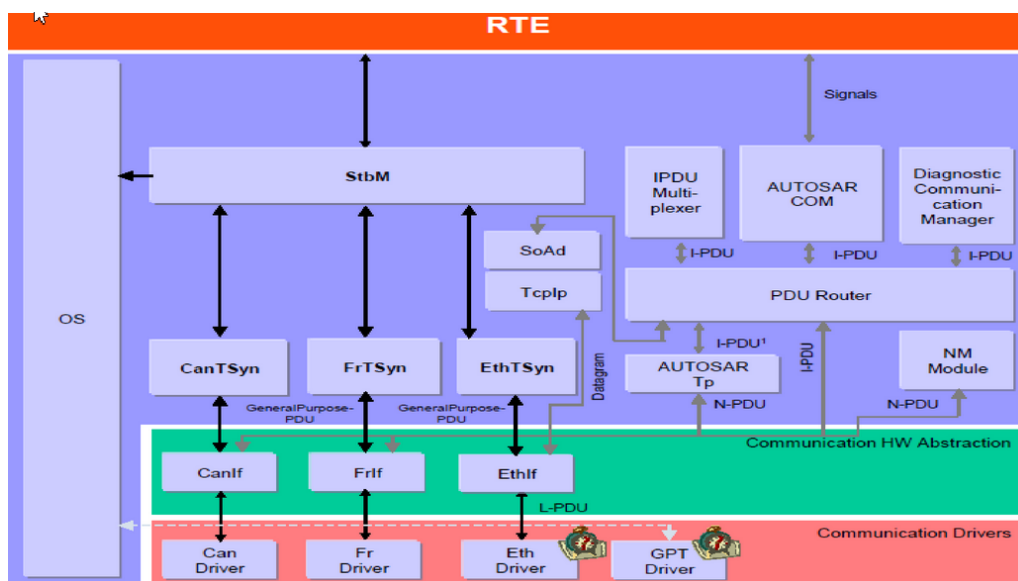


Figure 4.2.EthTSyn Architecture

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

4.2 Supported Features

- Acting as Time Master
- Acting as Time Slave
- Latency Calculation using Pdelay Logic
- Hardware Time Stamp
- TLV for the Slave
- Debounce Time

- PTP over VLAN
- Support for IEEE 802.1AS message format with AUTOSAR extension.

4.3 Extensions

4.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.1.1 EthTSyn/EthTSynGeneral/EthTSynPdelayErrorCounterEnable

Short Name	EthTSynPdelayErrorCounterEnable
Description	ETAS (Vendor11) Specific Parameter: This parameter enables or disables error logging when Pdelay_Resp/Pdelay_Resp_Follow_Up is not received.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.2 EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/SEV_T-SYN_ETH_FRESHNESS_NOT_AVAILABLE

Short Name	SEV_TSYN_ETH_FRESHNESS_NOT_AVAILABLE
Description	FV not available from FVM. Context data provides the respective domain ID.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.3 EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/SEV_T-SYN_ETH_ICV_GENERATION_FAILED

Short Name	SEV_TSYN_ETH_ICV_GENERATION_FAILED
Description	ICV generation for Follow_Up message failed. Context data provides the respective domain ID.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.4 EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/SEV_T-SYN_ETH_ICV_VERIFICATION_FAILED

Short Name	SEV_TSYN_ETH_ICV_VERIFICATION_FAILED
------------	--------------------------------------

Description	ICV verification for Follow_Up message failed. Context data provides the respective domain ID.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.5 EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/SEV_T-SYN_ETH_MSG_SEQUENCE_ERROR

Short Name	SEV_TSYN_ETH_MSG_SEQUENCE_ERROR
Description	Failed to receive correct sequence of SYNC and Follow_Up from the TimeMaster within (EthTSynGlobalTimeFollowUpTimeout).
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.6 EthTSyn/EthTSynGeneral/EthTSynTimeBridge

Short Name	EthTSynTimeBridge
Description	ETAS (Vendor11) Specific Parameter: Parameter that is used to configure Time Bridge.
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.7 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxIcvVerification/EthTSynRxAuthenticationBuildAttempts

Short Name	EthTSynRxAuthenticationBuildAttempts
Description	This parameter specifies the number of authentication build attempts that are to be carried out when the verification of the ICV failed for a given Follow_Up message. If zero is set, then only one ICV verification attempt is done.
Multiplicity	1..1
Value Range	0..65535
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.8 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynTxAuthenticationBuildAt-

temptsEthTSynTxAuthenticationBuildAttempts

Short Name	EthTSynTxAuthenticationBuildAttempts
Description	This parameter specifies the number of authentication build attempts that are to be carried out when the generation of the ICV failed for a given Follow-Up message. If zero is set, then only one ICV generation attempt is done.
Multiplicity	1..1
Value Range	0..65535
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

4.3.2 Additional Services

The following API functions are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

4.4 Limitations

The following features are not supported for version v12.7.0:

- The supported max value of EthTSynPdelayLatencyThreshold is 4.29 Second.
- Time Bridge

4.4.1 Switch Latency

If a switch is present in the Ethernet network it will have a switch latency. The current implementation of EthTSyn does not provide any mechanism to compensate for these switch delays for Global Time Synchronization.

4.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/ETHTSYN_SEV_FRESHNESS_NOT_AVAILABLE
- EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/ETHTSYN_SEV_ICV_GENERATION_FAILED
- EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs/ETHTSYN_SEV_ICV_VERIFICATION_FAILED
- EthTSyn/EthTSynGeneral/EthTSynSwitchMgmtRxMessageBufferCount
- EthTSyn/EthTSynGeneral/EthTSynTimeValidationSupport

- EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynEnableTimeValidation
- EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeMinMsgGap

4.6 Deviations

4.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

4.6.1.1 EthTSyn/EthTSynGeneral/EthTSynEnableSecurityEventReporting

[RTA-BSW] Description	Switches the reporting of security events to the IdsM. True: reporting is enabled. False: reporting is disabled.
[AUTOSAR] Description	Switches the reporting of security events to the IdsM: Tags: atp.Status=draft
[REMOVED] Introduction	- true: reporting is enabled. - false: reporting is disabled.

4.6.1.2 EthTSyn/EthTSynGeneral/EthTSynSecurityEventRefs

[RTA-BSW] Description	Container for the references to IdsMEvent elements representing the security events that the EthTSyn module shall report to the IdsM in case the corresponding security related event occurs (and if EthTSynEnableSecurityEventReportings set to "true"). The standardized security events in this container can be extended by vendor-specific security events.
[AUTOSAR] Description	Container for the references to IdsMEvent elements representing the security events that the EthTSyn Tags: atp.Status=draft
[REMOVED] Introduction	module shall report to the IdsM in case the corresponding security related event occurs (and if EthTSynEnableSecurityEventReportings set to "true"). The standardized security events in this container can be extended by vendor-specific security events.

4.6.1.3 EthTSyn/EthTSynGlobalTimeDomain/EthTSynGlobalTimeRxDebounceTime

[RTA-BSW] Description	This represents the configuration of a RX debounce time for Sync and Follow_Up. Unit:seconds
[AUTOSAR] Description	This represents the configuration of a RX debounce time for Sync and Follow_Up. Unit: seconds Tags: atp.Status=draft

4.6.1.4 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSyn-GlobalTimeRxlcvVerification

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSyn-GlobalTimeRxlcvVerification
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSyn-GlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification

4.6.1.5 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSyn-GlobalTimeRxlcvVerification/EthTSynIcVrxLength

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSyn-GlobalTimeRxlcvVerification/EthTSynIcVrxLength
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification/EthTSynIcVrxLength
[RTA-BSW] Description	Length of ICV to be transmitted within Follow_Up Message on the bus (in bytes).
[AUTOSAR] Description	Length of ICV to be transmitted within Follow_Up Message on the bus (in bytes). Tags: atp.Status=draft

4.6.1.6 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSyn-GlobalTimeRxlcvVerification/EthTSynIcVVerificationAttempts

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynIcVVerificationAttempts
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification/EthTSynIcVVerificationAttempts
[RTA-BSW] Description	This parameter specifies the number of ICV verification attempts that are to be carried out when the verification of the ICV failed for a given secured Follow_Up message. If zero is set, then only one ICV verification attempt is done.
[AUTOSAR] Description	This parameter specifies the number of ICV verification attempts that are to be carried out when the verification of the ICV failed for a given secured Follow_Up message. If zero is set, then only one ICV verification attempt is done. Tags: atp.Status=draft

4.6.1.7 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationBase

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationBase
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationBase
[RTA-BSW] Description	Symmetric or asymmetric cryptography selection for the ICV generation
[AUTOSAR] Description	Symmetric or asymmetric cryptography selection for the ICV generation Tags: atp.Status=draft
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.8 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationFvldRef

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationFvldRef
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationFvldRef
[RTA-BSW] Description	This represents the reference to the FV taken to verify the ICV.
[AUTOSAR] Description	This represents the reference to the FV taken to verify the ICV. Tags: atp.Status=draft

4.6.1.9 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationJobRef

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationJobRef
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationJobRef
[RTA-BSW] Description	This represents the reference to the CSM job to fetch the CSM job ID.

[AUTOSAR] Description	This represents the reference to the CSM job to fetch the CSM job ID. Tags: atp.Status=draft
--------------------------	---

4.6.1.10 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationTimeout

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationTimeout
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeRxlcvVerification/EthTSynlcvVerificationTimeout
[RTA-BSW] Description	Timeout of ICV verification (respective CSM job completion in asynchronous behavior). A value of 0 disables the ICV timeout monitoring. Unit: Seconds
[AUTOSAR] Description	Timeout of ICV generation (respective CSM job completion in asynchronous behaviour). Unit: Seconds Tags: atp.Status=draft

4.6.1.11 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxlcvGeneration

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxlcvGeneration
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynGlobalTimeTxlcvGeneration
[RTA-BSW] Description	This container collects configuration that shall be used for ICV generation.
[AUTOSAR] Description	This container collects configuration that shall be used for ICV generation. Tags: atp.Status=draft

4.6.1.12 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxlcvGeneration/EthTSynlcvGenerationBase

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxlcvGeneration/EthTSynlcvGenerationBase
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynGlobalTimeTxlcvGeneration/EthTSynlcvGenerationBase
[RTA-BSW] Description	Symmetric or asymmetric cryptography selection for the ICV generation

[AUTOSAR] Description	Symmetric or asymmetric cryptography selection for the ICV generation Tags: atp.Status=draft
--------------------------	---

4.6.1.13 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationFvIdRef

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationFvIdRef
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationFvIdRef
[RTA-BSW] Description	This represents the reference to the FV taken to generate the ICV generation.
[AUTOSAR] Description	This represents the reference to the FV taken to generate the ICV generation. Tags: atp.Status=draft

4.6.1.14 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationJobRef

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationJobRef
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationJobRef
[RTA-BSW] Description	This represents the reference to the CSM job to fetch the CSM job ID.
[AUTOSAR] Description	This represents the reference to the CSM job to fetch the CSM job ID. Tags: atp.Status=draft

4.6.1.15 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationTimeout

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationTimeout
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvGenerationTimeout
[RTA-BSW] Description	Timeout of ICV generation (respective CSM job completion in asynchronous behaviour). A value of 0 disables the ICV timeout monitoring. Unit: Seconds

[AUTOSAR] Description	Timeout of ICV generation (respective CSM job completion in asynchronous behaviour). Unit: Seconds Tags: atp.Status=draft
--------------------------	--

4.6.1.16 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvTxLength

[RTA-BSW] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortConfig/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvTxLength
[AUTOSAR] Parent Container	EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynGlobalTimeTxIcvGeneration/EthTSynIcvTxLength
[RTA-BSW] Description	Length of ICV to be transmitted within Follow_Up Message on the bus (in bytes).
[AUTOSAR] Description	Length of ICV to be transmitted within Follow_Up Message on the bus (in bytes). Tags: atp.Status=draft

4.6.1.17 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeMaster/EthTSynTLVFollowUpIcvSubTLV

[RTA-BSW] Description	This represents the configuration of whether an AUTOSAR Follow_Up ICV Sub-TLV is used or not.
[AUTOSAR] Description	This represents the configuration of whether an AUTOSAR Follow_Up TLV ICV Sub-TLV is used or not. Tags: atp.Status=draft
[RTA-BSW] Introduction	- true: This represents a configuration where an AUTOSAR Follow_Up ICV Sub-TLV is used. - false: This represents a configuration where an AUTOSAR Follow_Up ICV Sub-TLV is not used.
[AUTOSAR] Introduction	- true: This represents a configuration where an AUTOSAR Follow_Up TLV ICV Sub-TLV is used. - false: This represents a configuration where an AUTOSAR Follow_Up TLV ICV Sub-TLV is not used.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.18 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeSequenceCounterHysteresis

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.19 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynGlobalTimeSequenceCounterJump-Width

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.20 EthTSyn/EthTSynGlobalTimeDomain/EthTSynPortRole/EthTSynGlobalTimeSlave/EthTSynRxIcvVerificationType

[RTA-BSW] Description	This parameter controls whether or not ICV verification shall be supported.
[AUTOSAR] Description	This parameter controls whether or not ICV verification shall be supported. Tags: atp.Status=draft
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.7 API Definition

List of supported API functions

Table 4.30.EthTSyn Supported API Functions

EthTSyn_Init	Initialises the Time Synchronization over Ethernet
EthTSyn_GetVersionInfo	Returns the version information of this EthTSyn module
EthTSyn_SetTransmissionMode	Turns on and off the TX capabilities of the EthTSyn module
EthTSyn_RxIndication	Handles a frame received by the indexed controller
EthTSyn_TxConfirmation	Confirms frame transmission by the indexed controller
EthTSyn_TrcvLinkStateChg	Allows resetting state machine in case of unexpected Link loss
EthTSyn_MainFunction	Main function for cyclic call/response SYNC and Follow_Up and Pdelay_Req transmission
EthTSyn_SetProtocolParam	Sets FollowUp information TLV parameters for a Follow_Up message prior to transmission.

EthTSyn_GetProtocolParam

Reads FollowUp information TLV parameters from a received Follow_Up message.

For Further information please refer to AUTOSAR_SWS_TimeSyncOverEthernet.pdf

4.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

4.8.1 Hardware Timestamp

The optional Hardware Timestamp functionality can be enabled/disabled via the EthTSynHardwareTimestampSupport configuration parameter.

4.8.2 EthTSynPdelayLatencyThreshold

The maximum supported value for the EthTSynPdelayLatencyThreshold parameter is 4.29 seconds.

4.8.3 Message Format

EthTSyn's message format can be switched between the "standard" message format defined by IEEE 802.1AS and the "extended" format based on IEEE 802.1AS with an Autosar extension. This is controlled via the EthTSynMessageCompliance configuration parameter.

4.8.3.1 Debounce

The following points should be noted with regard to the Debounce Time feature -

- Debounce Time takes highest priority compared all other counters present.
- Because of Debounce Time, the time between two Sync messages during runtime can be different from the value configured in EthTSynGlobalTimeTxPeriod.

4.9 Integration Advice

4.9.1 Init Functions

EthTSyn is initialized via EthTSyn_Init, which must be called during the startup phase of the ECU.

Except for EthTSyn_GetVersionInfo, the API functions of EthTSyn can only be called once the EthTSyn module has been fully initialized.

4.9.2 Delnit Functions

No specific handling necessary.

4.9.3 Scheduled Functions

EthTSyn_MainFunction should be scheduled cyclically. This can be configured (from a minimum of 1ms) via the EthTSynMainFunctionPeriod configuration parameter. The maximum value for this parameter will depend upon the module timer configuration.

4.9.4 Integration Workflow Diagrams

The following diagrams illustrate the state changes of EthTSyn and may be of use during debugging.

4.9.4.1 EthTSyn Master State Machine

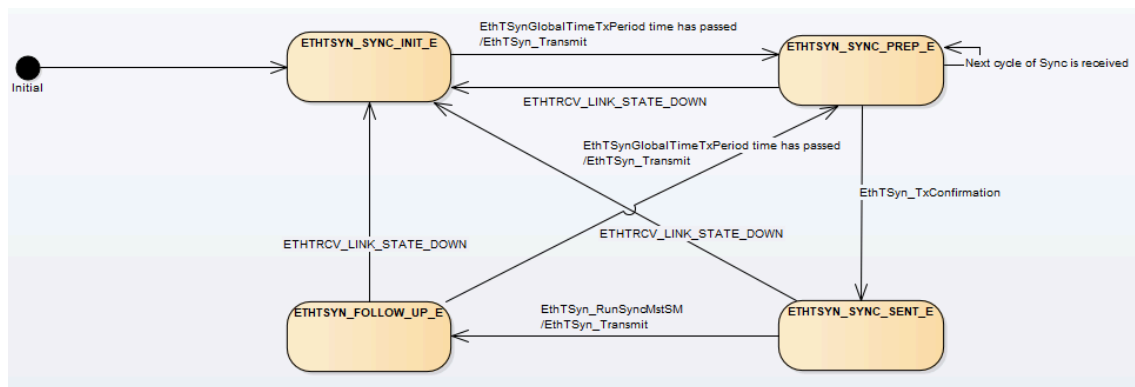


Figure 4.3.EthTSyn Master State Machine

4.9.4.2 EthTSyn Slave State Machine

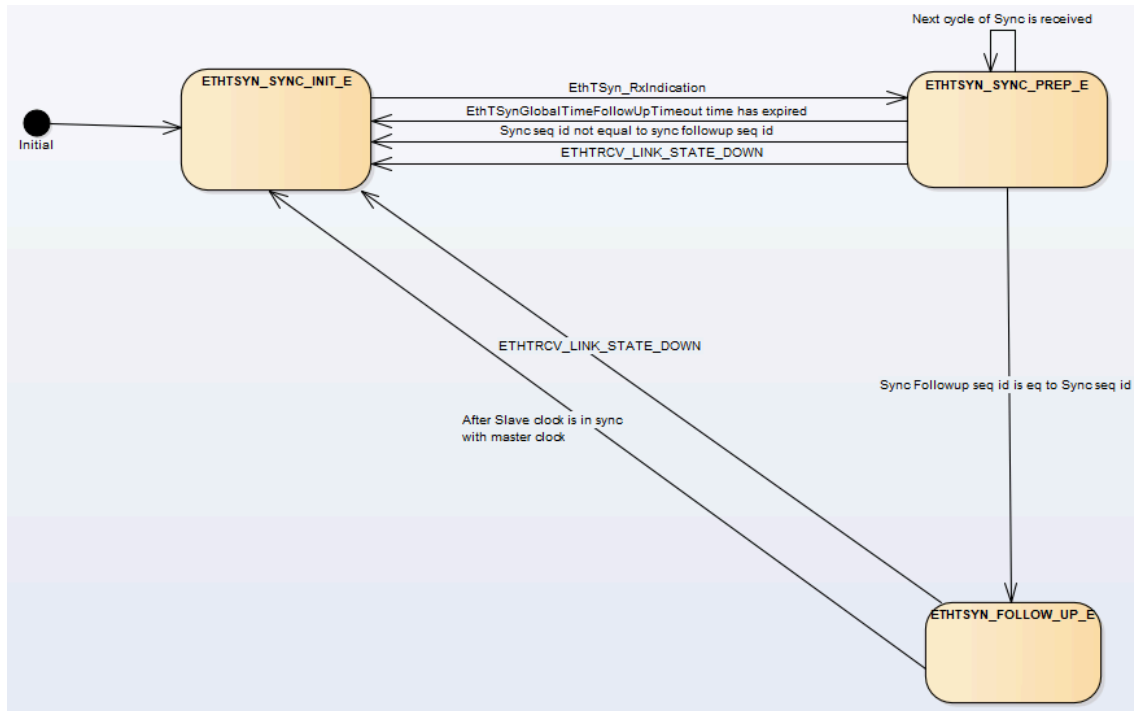


Figure 4.4.EthTSyn Slave State Machine

4.9.4.3 EthTSyn Pdelay Initiator State Machine

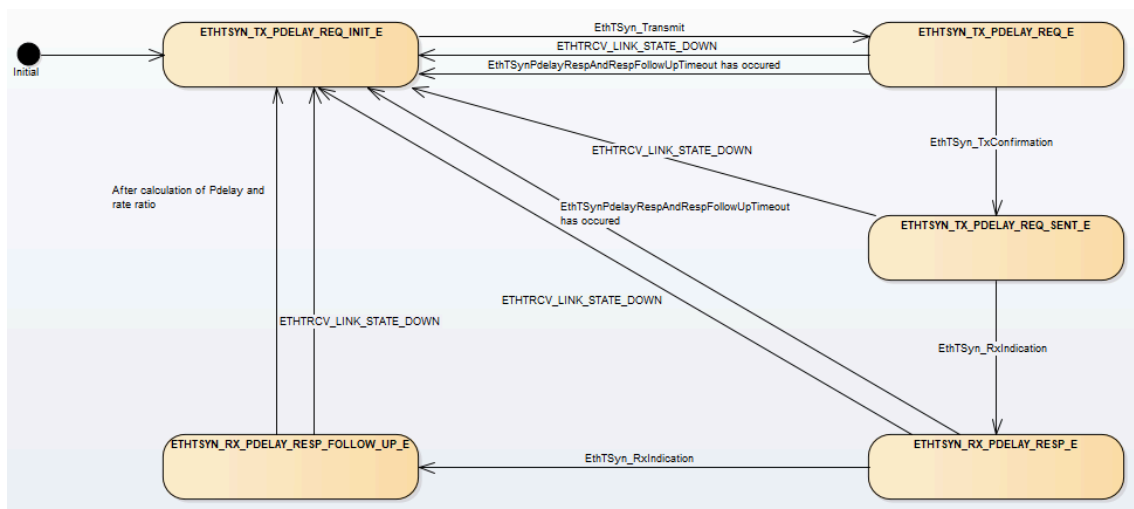


Figure 4.5.Pdelay Initiator State Machine

4.9.4.4 EthTSyn Pdelay Responder State Machine

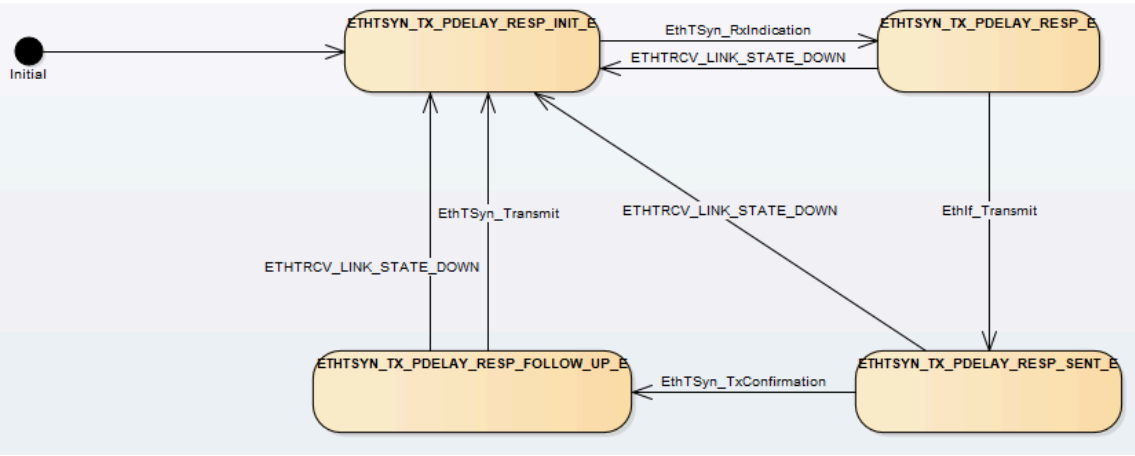


Figure 4.6.Pdelay Responder State Machine

4.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user’s responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The EthTSyn module uses the following OS resources:

- EthTSyn_VirtualLocalTime
- EthTSyn_TimeMasterSM
- EthTSyn_TimeSlaveSM
- EthTSyn_PdelayInitSM
- EthTSyn_PdelayRespSM
- EthTSyn_RamDataTypSt

4.10 Hardware Requirements

Table 4.31.EthTSyn Hardware Requirements

Support	Usage
FLASH	5614 bytes

RAM	708 bytes
-----	-----------

Usage calculated based on a sample configuration.

5 rba_EthArp

Module name	rba_EthArp
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

5.1 Introduction

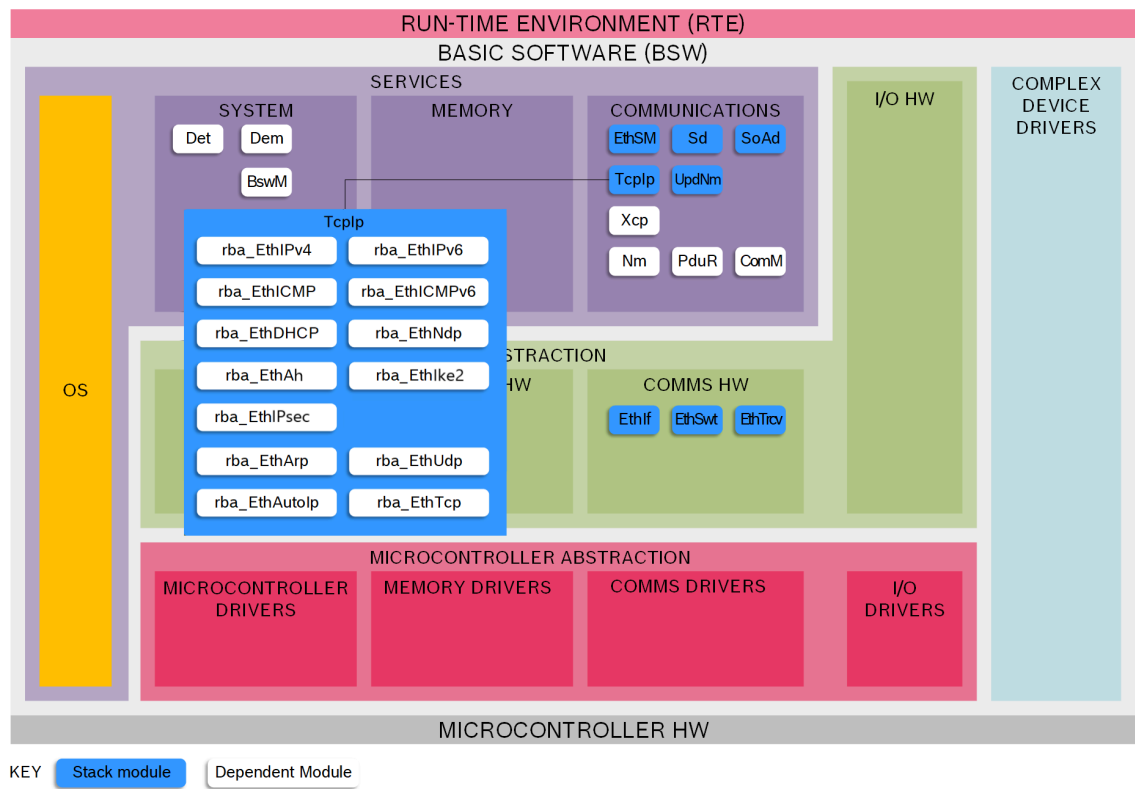


Figure 5.1.RTA-ETH Architecture

The rba_EthArp module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthArp in the AUTOSAR layered architecture. The modules rba_EthIPv4/rba_EthIPv6, rba_EthAutoIp and rba_EthDHCP are designated as the upper layer modules with respect to rba_EthArp, while EthIf is designated as the lower layer module.

This RTA-BSW specific module provides address resolution services which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules. It is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data transfer.

The specific functionality of the rba_EthArp module is to resolve IP addresses into Ethernet addresses. It finds the hardware address, also known as the Media Access Control (MAC) address of a host from its known IP address, and maintains a table in which MAC addresses are mapped to equivalent IP addresses. One such table is maintained for each configured EthIf controller on the network.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

5.2 Supported Features

- Translation of IP address to Ethernet address.
- ARP frame transmission.
- ARP reply mechanism (including gratuitous replies).
- IP conflict detection.
- ARP table management (per Ethernet controller).

5.3 Extensions

- No known extension parameters are defined in this version.

5.4 Limitations

- Only one gratuitous ARP reply may be sent after each IP address assignment. Multiple replies are not supported.

5.5 Exclusions

- There are no known exclusions in this version.

5.6 Deviations

- There are no known deviations in this version.

5.7 API Definition

List of supported API functions

Table 5.2.rba_EthArp Supported API functions

rba_EthArp_Init	Initialises the module.
rba_EthArp_MainFunction	rba_EthArp Main Function.
rba_EthArp_GetDestEthAddr	Returns the destination MAC address associated with the specified IP address.

rba_EthArp_UpdateArpTable	Update the internal ARP table with a new IP address from an EthIf controller.
rba_EthArp_SendArpFrame	Sends an ARP frame to the specified destination.
rba_EthArp_SendArpGratuitousFrame	Sends a gratuitous ARP frame to the specified EthIf controller with a specified IP header.
rba_EthArp_ReadArpTable	Reads the ARP table for the designated EthIf controller.
rba_EthArp_IpConflictDetection	Checks the IP address received in a frame and detects any conflict with another assigned IP address.
rba_EthArp_CleanArpTable	Cleans the ARP table for the designated EthIf controller.
rba_EthArp_GetArpTableSize	Returns the maximum possible size for an ARP table.
rba_EthArp_RxIndication	Callback function to notify module if an ARP frame has been received.
rba_EthArp_Enable	Used to enable or disable ARP functionality for an EthIf controller.
rba_EthArp_GetEnableStatus	Used to get the ARP functionality status for the EthIf controller.
rba_EthArp_Proc_TransmitGratuitous	Used to manage gratuitous frames transmissions.

5.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration Mapping can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

5.8.1 General

Configuration of the rba_EthArp module is determined by the following AUTOSAR parameters from the Tcplp configuration (TcplpArpConfig container).

- TcplpArpMaxRequests - Maximum number of ARP requests that can be sent for a particular destination IP address. This RTA-BSW-specific parameter is present in the TcplpArpConfig container and has no assigned default value, therefore it must always be configured by the user.
- TcplpArpNumGratuitousARPOnStartup - Specifies the number of gratuitous

ARP replies sent on assignment of a new IP address. Currently this parameter is not used. One gratuitous ARP message is always sent on successful IP address assignment.

- TcplpArpReplyBroadcastEnabled - Enables (TRUE) or disables (FALSE) if the ARP reply shall be sent as broadcast for a IPv4 Link- Local address range.
- TcplpArpRequestTxTimeout - configures a timeout after which ARP requests to a particular IP address are blocked. Using this configuration results in the blocking of ARP table entries if an ARP reply is not received for the entry, hence this configuration must be judiciously used.
- TcplpArpTableEntryTimeout - Timeout in seconds after which an unused ARP entry is removed.
- TcplpArpTableSizeMax - Maximum number of entries in the ARP table.
- TcplpArpRequestTimeout - Timeout in seconds during which an ARP table pending entry waits for an ARP reply. If no reply was received during the configured timeout period, an ARP request will be resent for this entry. The number of attempts is defined by configuration parameter TcplpArpMaxRequests.
- TcplpIpConflictDetectionArpFrames - Enables (TRUE) or disables (FALSE) IP conflict detection based on ARP frames.
- TcplpIpConflictDetectionIPv4Frames - Enables (TRUE) or disables (FALSE) IP conflict detection based on IPv4 frames.



NOTE

ARP requests are sent based on only one of the two timeout mechanisms described above (parameters TcplpArpMaxRequests and TcplpArpRequestTxTimeout). These parameters are mutually exclusive and only one of them may be configured.

5.8.2 TcplpArpEnabled

This parameter is not currently used because Arp is enabled only if any IPv4 unicast communication has been configured. For multicast and broadcast communication, Arp is disabled.

5.9 Integration Advice

5.9.1 Init Functions

rba_EthArp is initialised implicitly by the Tcplp module when Tcplp_Init is called, so rba_EthArp does not require separate initialisation.

5.9.2 Delnit Functions

No Delnit is required.

5.9.3 Scheduled Functions

rba_EthArp_MainFunction is triggered by Tcplp_MainFunction, so only the scheduling of Tcplp_Mainfunction is required, which should be scheduled cyclically.

5.9.4 General

rba_EthArp does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthArp code generation.

5.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthArp module uses the following OS resources:

- rba_EthArp_UpdateTable

5.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

6 rba_EthAutolp

Module name	rba_EthAutolp
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

6.1 Introduction

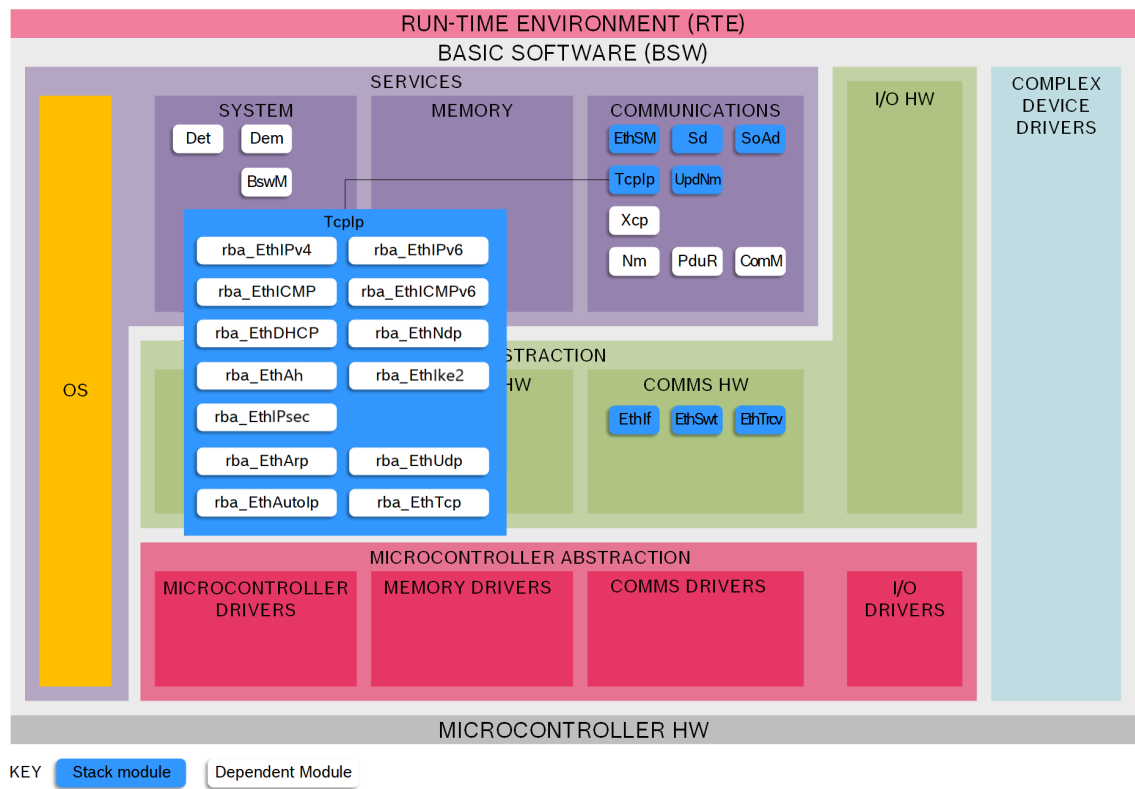


Figure 6.1.RTA-ETH Architecture

The rba_EthAutolp module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthAutolp in the AUTOSAR layered architecture. The Tcplp module is designated as the upper layer modules with respect to rba_EthAutolp, while rba_EthArp is designated as the lower layer module.

rba_EthAutolp is a RTA-BSW specific module that provides auto-configuration of local IP addresses which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules. It is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data transfer.

The specific functionality of the rba_EthAutolp module is to provide automatic IP address configuration for devices on a local network (physical or logical). rba_EthAutolp will assign IP addresses in the range 169.254.0.0/16, which is the reserved link-local address range defined for the IPv4 protocol. This allows communication between link-local devices in the absence of any other available address configuration service (such as DHCP).

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

6.2 Supported Features

- Local-link IP address assignment (IPv4 only).
- IP Conflict detection.
- Base IP address allocation with persistent NVM storage (see below).

6.3 Extensions

- No known extension parameters are defined in this version.

6.4 Limitations

- rba_EthAutolp does not currently support the provision of measurement or calibration data.
- Only one Autolp instance can be configured per EthIf controller.
- rba_EthAutolp does not support re-entrancy protection through the use of exclusive areas or critical sections.
- rba_EthAutolp does not support the storing of the already-generated IP address in NVM. This feature is dependent on Tcplp, specifically the TcplpN-vmBlockDescriptorRef parameter, which is not currently supported. Tcplp does not support the storing of the assigned IP to reassign it back to rba_EthAutolp after start up.

6.5 Exclusions

- There are no known exclusions in this version.

6.6 Deviations

- There are no known deviations in this version.

6.7 API Definition

List of supported API functions

Table 6.2.rba_EthAutoIp Supported API functions

rba_EthAutoIp_Init	Initialises the module.
rba_EthAutoIp_Start	Begins the sequence of link-local IP address assignment.
rba_EthAutoIp_Stop	Stops the sequence of link-local IP address assignment.
rba_EthAutoIp_IpConflictDetected	Informs the module that an IP address conflict occurred on the specified controller (called by rba_EthArp).
rba_EthAutoIp_GetIpAddress	Sends the currently generated IP address for the specified controller to rba_EthArp for conflict checking.
rba_EthAutoIp_AllocBaseIpAddr	Generates the base IP address (called by Tcplp).
rba_EthAutoIp_MainFunction	Called by an OS_Schedul_Task to handle the AutoIP Process and send ARP announce and probe frame.

6.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

Configuration of the module is determined by the relevant parameters of the Tcplp configuration. The following Tcplp parameters are relevant for rba_EthAutoIp configuration.

- TcplpAutoIpInitTimeout - The time in seconds that the module waits after startup, before any ARP probe requests are begun. This delay is used to give DHCP time to acquire a lease in case a DHCP server is present.
- TcplpAssignmentMethod - Setting this parameter to TCPIP_LINKLOCAL or TCPIPLINKLOCAL_DOIP selects an AutoIp configuration as described below.
- TcplpAssignmentPriority - Priority of assignment (1 is highest).
- TcplpAssignmentTrigger - Trigger of address assignment.

Table 6.3.rba_EthAutoIp Configurations

Parameter Description	TCPIP_LINKLOCAL	TCPIPLINKLOCAL_DOIP
Delay before ARP probe	1s	1s
Number of probe packets	3	1

Minimum delay till ARP probe repeats	1s	1s
Maximum delay till ARP probe repeats	2s	1s
Delay before announcement (gratuitous ARP message)	2s	1s
Number of announcement packets	2	1
Time between announcement packets	2s	1s

6.8.1 TcplpAutolpEnabled

This parameter is not currently used because Autolp is enabled only if there is at least one local IP address assignment method configured as Autolp (link-local or link-local Dolp).

6.9 Integration Advice

6.9.1 Init Functions

rba_EthAutolp is initialized implicitly by the Tcplp module when Tcplp_Init is called during the startup phase of the ECU, so rba_EthAutolp does not require separate initialization.

6.9.2 Delnit Functions

No Delnit is required.

6.9.3 Scheduled Functions

rba_EthAutolp_MainFunction is triggered by Tcplp_MainFunction, so only the scheduling of Tcplp_Mainfunction is required, which is to be done cyclically.

6.9.4 General

rba_EthAutolp does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthAutolp code generation.

6.9.5 Storing IP address in NVM

rba_EthAutolp can optionally store the already generated IP address in NVM, and then retrieve it during the next boot of the ECU to start the assignment process..

In order to enable this feature, the rba_EthAutolp_AllocBaseIpAddr function must be called by the Tcplp module to set the base IP address stored in NVM. If the function is not called after initialisation of rba_EthAutolp, then Autolp will start the normal assignment process. If a conflict is detected for the specified base IP address, then Autolp will restart the assignment process by incrementing the base IP address value by one.

6.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user

configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthAutolp module uses the following OS resources:

- rba_EthAutolp_UpdateLinkLocalAddr
- rba_EthAutolp_UpdateEntryState

6.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

7 rba_EthDHCP

Module name	rba_EthDHCP
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

7.1 Introduction

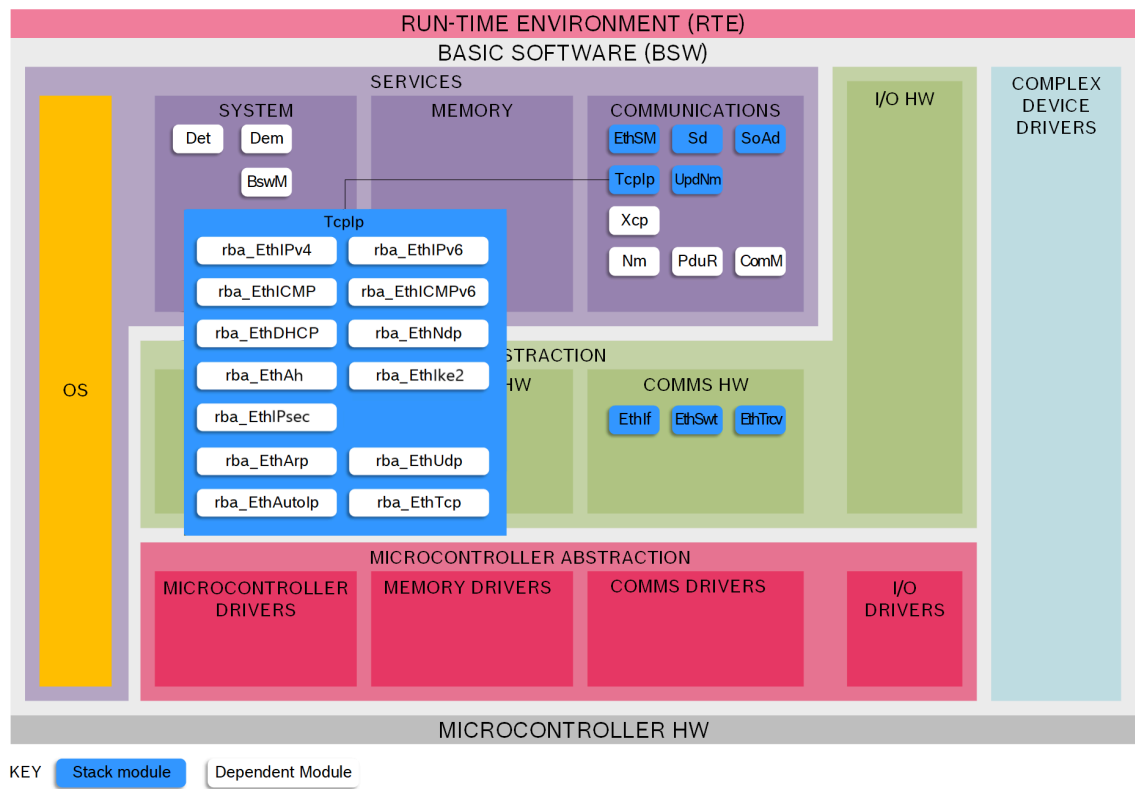


Figure 7.1.RTA-ETH Architecture

The rba_EthDHCP module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthDHCP in the AUTOSAR layered architecture.

The rba_EthArp and rba_EthUdp modules are designated as the lower layer modules with respect to rba_EthDHCP, while Tcplp is designated as the upper layer module.

rba_EthDHCP is a RTA-BSW specific module that provides DHCP host configuration services which are used by the Tcplp module to assign host-specific configuration parameters and network addresses to host network devices. It is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data transfer.

The specific functionality of the rba_EthDHCP module is to provide a mechanism for supporting dynamically configured hosts on a network. The module implements a client-server interface, with a DHCP server responsible for assigning an IP address and other dynamic configuration details to hosts (clients) over a UDP socket connection.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

7.2 Supported Features

- Client/server DHCP process implementation (see below).
- Automatic allocation of reusable network addresses and host configuration data.
- IP conflict detection.
- Read/write DHCP option data.

7.2.1 DHCP Client

rba_EthDHCP interacts mainly with the Tcplp module. The request to start the DHCP process for the client comes from the Tcplp module when:

- An IP address assignment request is called from the SoAd module via the Tcplp_RequestIpAddrAssignment API (AssignmentMethod=Manual).

Or...

- When the EthSM module requests TCPIP_STATE_ONLINE state from TCPIP_STATE_OFFLINE state for an EthIf controller configured with DHCP IP address assignment method (TcplpAssignmentMethod). Tcplp is informed once an IP address is successfully assigned via DHCP (AssignmentMethod=Automatic)

7.2.2 DHCP Server

The DHCP server process starts when it receives a DHCP frame from the UDP module. DHCP server/client use UDP sockets to communicate with its peers.

7.3 Extensions

- No known extension parameters are defined in this version.

7.4 Limitations

- Exclusive areas and critical sections are not supported, therefore nested calls are not supported. Thus the Ethernet Stack (PAC EthStd) can only be used at a single core.

- The module does not currently support the provision of measurement or calibration data.

7.5 Exclusions

- There are no known exclusions in this version.

7.6 Deviations

- There are no known deviations in this version.

7.7 API Definition

Table 7.2.rba_EthDHCP Supported API functions

rba_EthDHCP_Init	Initialises the DHCP table (client & server) and global variables.
rba_EthDHCP_MainFunction	rba_EthDHCP Main Function.
rba_EthDHCP_ClientMainFunction	rba_EthDHCP Main Function.
rba_EthDHCP_Start	Starts the sequence to get an IP address from the DHCP server.
rba_EthDHCP_Stop	Stops the IP address assignment sequence.
rba_EthDHCP_RxIndication	Callback function (called by rba_EthUdp) to notify module of a received DHCP frame.
rba_EthDHCP_IpConflictDetected	Called by rba_EthArp to check if there's an IP address conflict.
rba_EthDHCP_GetIpAddress	Provides IP address to rba_EthArp to begin conflict detection.
rba_EthDHCP_ReadOption	Allows Tcplp module to retrieve DHCP option data for the specified EthIf controller and address identifier.
rba_EthDHCP_WriteOption	Allows Tcplp module to write DHCP option data for the specified EthIf controller and address identifier.
rba_EthDHCP_ClientInit	Initialises the DHCP client process.
rba_EthDHCP_ClientStart	Starts the sequence to get an IP address from the DHCP server.
rba_EthDHCP_ClientStop	Stops the IP address assignment sequence.
rba_EthDHCP_ClientRxIndication	Callback function (called by rba_EthUdp) to notify module of a received DHCP frame.

rba_EthDHCP_ClientIpConflictDetected	Called by rba_EthArp to check if there's an IP address conflict.
rba_EthDHCP_ClientGetIpAddress	Provides IP address to rba_EthArp to begin conflict detection.
rba_EthDHCP_ClientReadOption	Allows Tcplp module to retrieve DHCP option data for the specified EthIf controller and address identifier.
rba_EthDHCP_ClientWriteOption	Allows Tcplp module to write DHCP option data for the specified EthIf controller and address identifier.

7.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

7.8.1 General

Configuration of the module is determined by the relevant parameters of the Tcplp configuration. The following Tcplp parameters are relevant for rba_EthDHCP configuration.

- TcplpDHCPTimeToWaitBeforeReTx - time to wait before the retransmission if no response is received from a Server (ms).

7.9 Integration Advice

7.9.1 Init Functions

rba_EthDHCP is initialised implicitly by the Tcplp module when Tcplp_Init is called during the startup phase of the ECU, so rba_EthDHCP does not require separate initialisation.

7.9.2 Delnit Functions

No Delnit is required.

7.9.3 Scheduled Functions

rba_EthDHCP_MainFunction and rba_EthDHCP_ClientMainFunction are triggered by Tcplp_MainFunction, so only the scheduling of Tcplp_Mainfunction is required, which should be scheduled cyclically.

7.9.4 General

rba_EthDHCP does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthDHCP code generation.

7.9.5 Message Retransmission

When message retransmission is underway, it may get interrupted for transmission of a new DHCP request during the lease time renewal or rebind processes. Such an event would cause the retransmission process to be aborted. If the application still requires to retransmit a message, it must be triggered again.

7.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthDHCP module uses the following OS resources:

- rba_EthDHCP_StateCheck

7.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

8 rba_EthDHCPv6

Module name	Rba_EthDHCPv6
Package	RTA-ETH
Module ID	N/A
AUTOSAR Version	22-11

8.1 Introduction

The *Dynamic Host Configuration Protocol* for IPv6 (DHCP) provides a framework for passing configuration information to hosts on a TCP/IP network. DHCP facilitates the automatic allocation of re-usable network addresses and additional configuration options.

DHCP is built on a client-server model, where designated DHCP server hosts allocate network addresses and deliver configuration parameters to dynamically-configured hosts. The term “server” will therefore refer to a host providing initialization parameters through DHCP, and the term “client” refers to a host requesting initialization parameters from a DHCP server.

8.1.1 Interaction

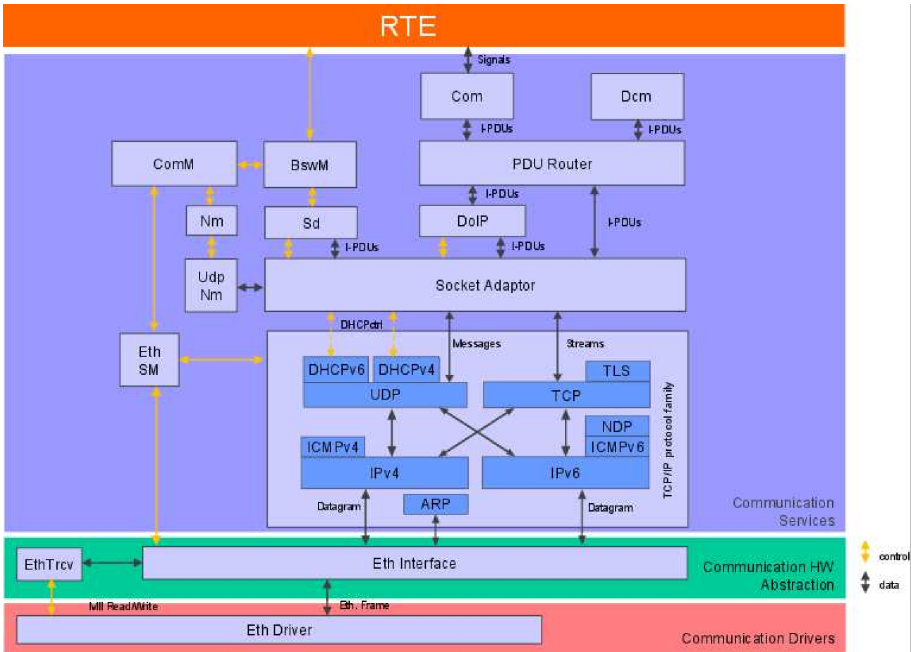


Figure 8.1. Architecture overview

The *TcpIp* interface is the lower-layer module of the DHCPv6 module. It is used to control the DHCPv6 process and to get/provide information from/to the process.

The DHCPv6 client is implemented in the *rba_EthDHCPv6* module, which is therefore part of the Communication Stack and an RB-specific sub-module of *TcpIp*.

The *NDP* interface is used by *rba_EthDHCPv6* to verify an IPv6 address's uniqueness.

8.1.1.1 DHCPv6 Client

The *rba_EthDHCPv6* module mainly interacts with the *TcpIp* module. The request to start the DHCPv6 process for the client comes from *TcpIp* in one of two ways:

1. *AssignmentMethod - Manual* i.e. an IP address assignment request is called from the *SoAd* module via the API *TcpIp_RequestIpAddrAssignment*.
2. *AssignmentMethod - Automatic* i.e. when the *EthSM* module requests a change from the *TCPIP_STATE_OFFLINE* state to the *TCPIP_STATE_ONLINE* state for an *EthIf* controller configured with the DHCP IPv6 address assignment method (*tcpIpAssignmentMethod*).

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

8.2 Supported Features

According to section 7.5.5.2 of AUTOSAR's "Specification of TCP/IP" SWS document, the *DHCPv6* sub-module supports the following client functionality (as defined in IETF RFC 4704):

- The *Fully Qualified Domain Name (FQDN)* Option
- No DNS

The following, therefore, are *not* supported:

- Relay Agent Behavior
- Server Behavior
- Management of Temporary Addresses
- Authentication of DHCP Messages
- Identity Association for Temporary Addresses Option
- Authentication Option
- Rapid Commit Option

Please see section 7.5.5.2 of *AUTOSAR_SWS_TcpIp.pdf* for more information.

8.3 Extensions

8.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

- No known extension parameters are defined in this version.

8.4 Limitations

Version v12.7.0 has following limitations.

- This implementation supports only one IPv6 address instance with DHCP assignment per Tcplp controller.
- No re-entrancy is protected via the Critical Sections/Exclusive Areas mechanism.
- Post-build variants are not supported.

8.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

8.6 Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

- There are no known deviations in this version.

8.7 API Definition

List of supported API functions

Table 8.2.rba_EthDHCPv6 Supported API functions

rba_EthDHCPv6_CopyTxData	Creates the DHCPv6 frame. Called when Tcplp_UdpTransmit is called with buffPtr as NULL_PTR.
rba_EthDHCPv6_Init	Used to initialize each instance of the client.
rba_EthDHCPv6_LocalIpAddrAssignmentCb	Callback used to detect if an IPv6 address has bounded or has been detected as duplicated.
rba_EthDHCPv6_MainFunction	Called periodically by the Tcplp main function.
rba_EthDHCPv6_ReadOption	TCP/IP retrieves the DHCPv6 option data identified by the parameter option for already received DHCPv6 options.
rba_EthDHCPv6_RunModeRequest	Used to start/stop/pause the assignment of IPv6 addresses using DHCPv6 method.
rba_EthDHCPv6_RxIndication	Called when a DHCPv6 frame is received.
rba_EthDHCPv6_WriteOption	TCP/IP stack writes the DHCPv6 option data identified by the parameter option.
rba_EthDHCPv6_Ndp_Init	

rba_EthNDP_LocalIpAddrUpdate	
rba_EthNDP_LocalAddrStatusUpdate	

8.8 Configuration Advice

8.8.1 Configuration interface

This section describes the DHCPv6-relevant parameters of the *TcpIp* configuration. They are present in the *TcpIpIPv6Ctrl* and *TcpIpIPv6Config* containers.



NOTE

The *rba_EthDHCPv6* component is fully dependent upon the *TcpIp* component, since it does not have its own *EcucParamDef* file. *TcpIp_EcucParamDef.xml* is therefore required for the *rba_EthDHCPv6* code generation.

8.8.1.1 TcpIpIPv6Ctrl

This container specifies an Internet Protocol version 6 (IPv6) address instance and contains a reference to a DHCPv6 configuration, *TcpIpIPv6DhcpConfigRef*. (Multiple IPv6 address instances may use the same configuration container but will operate independently.)

8.8.1.2 TcpIpIPv6Config

This container specifies the configuration parameters of the IPv6 address instance and so also contains the configuration parameters for the *rba_EthDHCPv6* sub-module.

The latter are specified in the container *TcpIpDhcpV6Config*, one of which may be referenced by multiple IPv6 address instances if they all have the same configuration. However, *TcpIpDhcpV6Config* will have multiple instances if there are multiple IPv6 address instances with different configurations.

Parameters

TcpIpDhcpV6CnfDelayMax: The maximum delay(s) before the first Confirm message is sent. If this value is > the previous minimum delay value, a random delay will be chosen from the interval.

TcpIpDhcpV6CnfDelayMin: The minimum delay(s) before the first Confirm message is sent.

TcpIpDhcpV6InfDelayMax: The maximum delay(s) before the first Information Request message is sent. If this value is > the previous minimum delay value, a random delay will be chosen from the interval.

TcpIpDhcpV6InfDelayMin: The minimum delay(s) before the first Information Request message is sent.

TcpIpDhcpV6SolDelayMax: The maximum delay(s) before the first Solicit message is sent. If this value is > the previous minimum delay value, a random delay will be chosen from the interval.

TcpIpDhcpV6SolDelayMin: The minimum delay(s) before the first Solicit message is sent.



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

8.9 Integration Advice

8.9.1 How To Integrate

Include the component in the IDE and build, configure the component, run generate id, validation, code generation, then build.

For the *rba_EthDHCPv6* component to work properly, it is mandatory to include the *Tcplp* and *rba_EthNDP* components for compilation.

8.9.2 Init Functions

rba_EthDHCPv6_Init will only be called by *TcpIp_Init* during the start-up phase of the ECU. *rba_EthDHCPv6_Init* is also called to re-initialize the module.

8.9.3 Delnit Functions

A separate shutdown API is not supported.

8.9.4 Scheduled Functions

The *TcpIp_MainFunction* is to be scheduled cyclically.

8.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The *Rba_EthDHCPv6* module does not use any exclusive OS resources.

8.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

9 rba_EthICMP

Module name	rba_EthICMP
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

9.1 Introduction

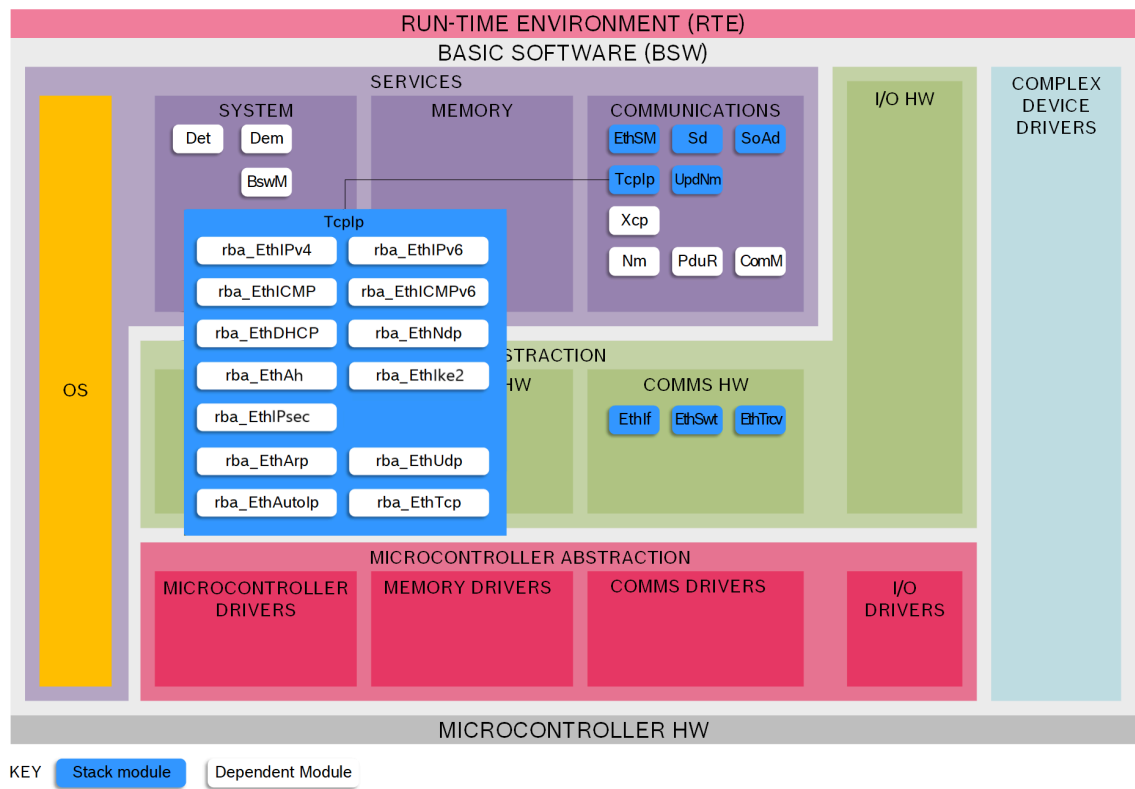


Figure 9.1.RTA-ETH Architecture

The rba_EthICMP module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthICMP in the AUTOSAR layered architecture.

The rba_EthIPv4 module is designated as the lower layer module with respect to rba_EthICMP, while Tcplp and rba_EthUdp are designated as the upper layer modules.

This RTA-BSW specific module provides ICMP services which are used by the Tcplp module to provide error message and query communications between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules. It is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data transfer.

The specific functionality of the rba_EthICMP module is to provide a mechanism for reporting error states (e.g. an unreachable network host or unavailable service), and to signal errors in the processing of datagrams. Except in very specific circumstances (e.g. low-level network diagnostic tools), ICMP is not typically used to exchange data messages between systems.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

9.2 Supported Features

- ICMP message transfer over IPv4.

9.3 Extensions

- No known extension parameters are defined in this version.

9.4 Limitations

- Exclusive areas and critical sections are not supported, therefore there is no re-entrancy protection for this module.

9.5 Exclusions

- There are no known exclusions in this version.

9.6 Deviations

- There are no known deviations in this version.

9.7 API Definition

Table 9.2.rba_EthICMP Supported API functions

rba_EthIcmp_Init	Initialises the module.
rba_EthIcmp_Transmit	Transmits an ICMP message according to the specified parameters.
rba_EthIcmp_RxIndication	Callback function which signals receipt of an ICMP message.
rba_EthIcmp_Enable	Allows the enabling or disabling of the ICMP frame.

9.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

9.8.1 General

rba_EthICMP does not have a separate configuration parameter file. The Tcplp configuration parameters in the following containers are used to configure the rba_EthICMP module.

- TcplpGeneral/TcplpV4General
- TcplpConfig/TcplpV4Config/TcplplcmpConfig

The following Tcplp parameters are relevant to rba_EthICMP.

- TcplplcmpEnabled - Enables (TRUE) or disabled (FALSE) support of the ICMP protocol.
- TcplplcmpTtl - Default time-to-live value of outgoing ICMP packets.
- TcplplcmpMsgHandler - This container is an optional sub-container of TcplplcmpConfig and specifies the configuration parameters for the ICMP message handler.
- TcplplcmpMsgHandlerHeaderFileName - This parameter specifies the name of the header file containing the declaration of the ICMP message handler function.
- TcplplcmpMsgHandlerName - This parameter defines the name of the ICMP message handler function.
- TcplplcmpEchoBroadcastEnabled - Enables (TRUE) or disables (FALSE) replies to ICMP Broadcast echo requests per TcplpCtrl.
- TcplplcmpMessageFragmentationAllowed - Allows fragmentation of ICMP messages exceeding MTU size.
- TcplpRblcmpProtocolDisabledByDefault - Disables (TRUE) or enables (FALSE) the support of ICMP protocol by default. Disabling ICMP protocol means that all ICMP message types will be deactivated. When configuring this parameter, the API Tcplp_IcmpEnable, and the rba_EthIcmp_Enable will be available to enable or disable ICMP protocol during runtime.

9.8.2 Checksum Calculation

Hardware checksum calculation is supported from AR v4.2.2 via the EthCtrlEn-

ableOffloadChecksumICMPconfiguration parameter (in Eth/EthGeneral/EthCtrlOffloading). This parameter is set to FALSE by default, which means that checksum calculation is handled by rba_EthICMP rather than the Ethernet Driver. If Ethernet Driver v4.0.3 is used, it does not support protocol checksum calculation, so EthCtrlEnableOffloadChecksumICMP should be set as FALSE.

9.9 Integration Advice

9.9.1 Init Functions

rba_EthICMP is initialised implicitly by the Tcplp module when Tcplp_Init is called, so rba_EthICMP does not require separate initialisation.

9.9.2 Delnit Functions

No Delnit is required.

9.9.3 Scheduled Functions

rba_EthICMP has no cyclic main function, so there are no scheduling requirements for this module.

9.9.4 Re-entrancy Support

To support re-entrancy of the rba_EthIcmp module without the need for a locking mechanism, ICMP echo reply messages exceeding MTU size are truncated to MTU size. If the transmission of echo reply messages exceeding the MTU size is required (e.g. if the testability module is configured), then TcplplcmpMessageFragmentationAllowed should be configured as TRUE. The critical sections provided in rba_EthIcmp_Cfg_Schm.h should be implemented to make the module re-entrant.

9.9.5 General

rba_EthICMP does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthICMP code generation.

When configuring TcplplcmpConfig/TcplplcmpMsgHandler, the parameter 'TcplplcmpMsgHandlerName' shall be the same as the name of the function inside the template file that was generated inside the Rba_EthICMP/integration folder. Example: CDD_ULIcmpRxIndication

9.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can

be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthICMP module uses the following OS resources:

- rba_EthICMP_ExclusiveAreacmpFragmentedMsg

9.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

10 rba_EthICMPV6

Module name	rba_EthICMPv6
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

10.1 Introduction

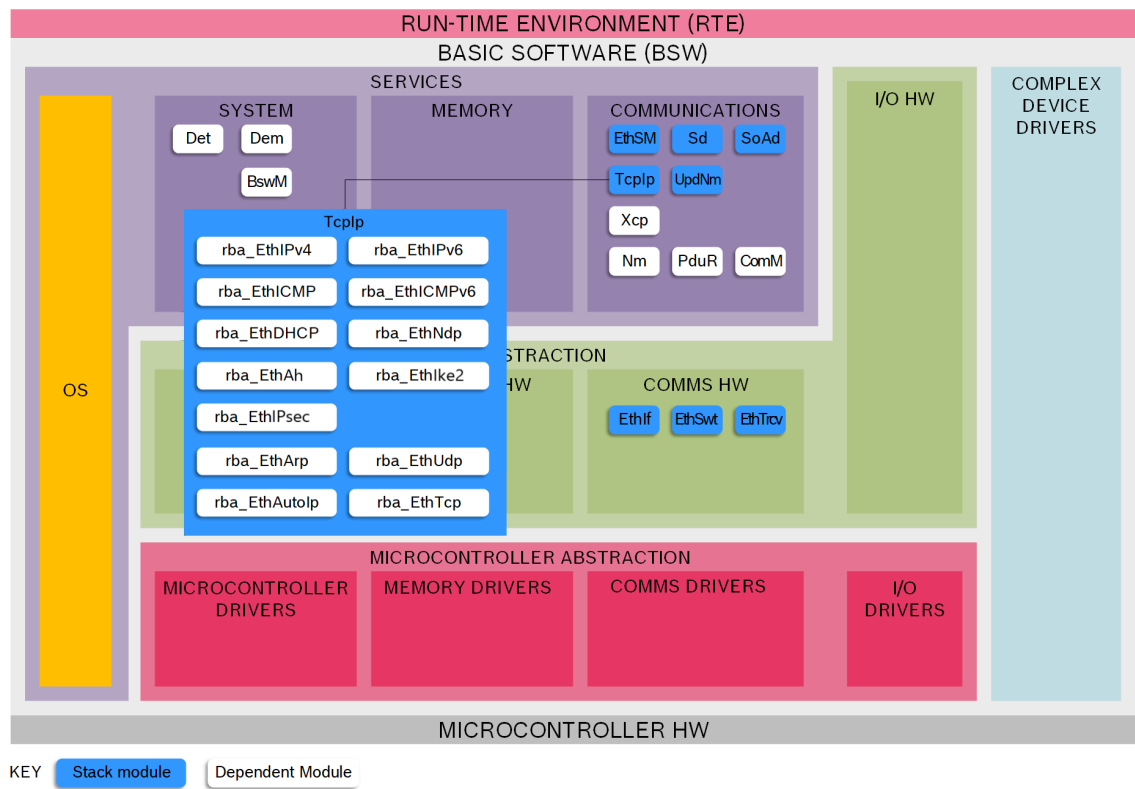


Figure 10.1.RTA-ETH Architecture

The Internet Control Message Protocol V6 (ICMPv6) is one of the core protocols of the Internet Protocol Suite. It is used to send error messages indicating, for example, that a requested service is not available or that a host or router could not be reached. ICMPv6 can also be used to send information messages and to relay query messages.

Gateways or destination hosts will use ICMPv6 when communicating with a source host to report errors in datagram processing.

rba_EthIcmpV6 is RTA-BSW's implementation of ICMPv6. It is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthIcmpV6 in the AUTOSAR layered architecture.

rba_EthIcmpV6 interacts with EthIPv6 (lower level), and with rba_EthNdp (upper

level). The TcpIp module acts as a wrapper around the EthIcmpV6.

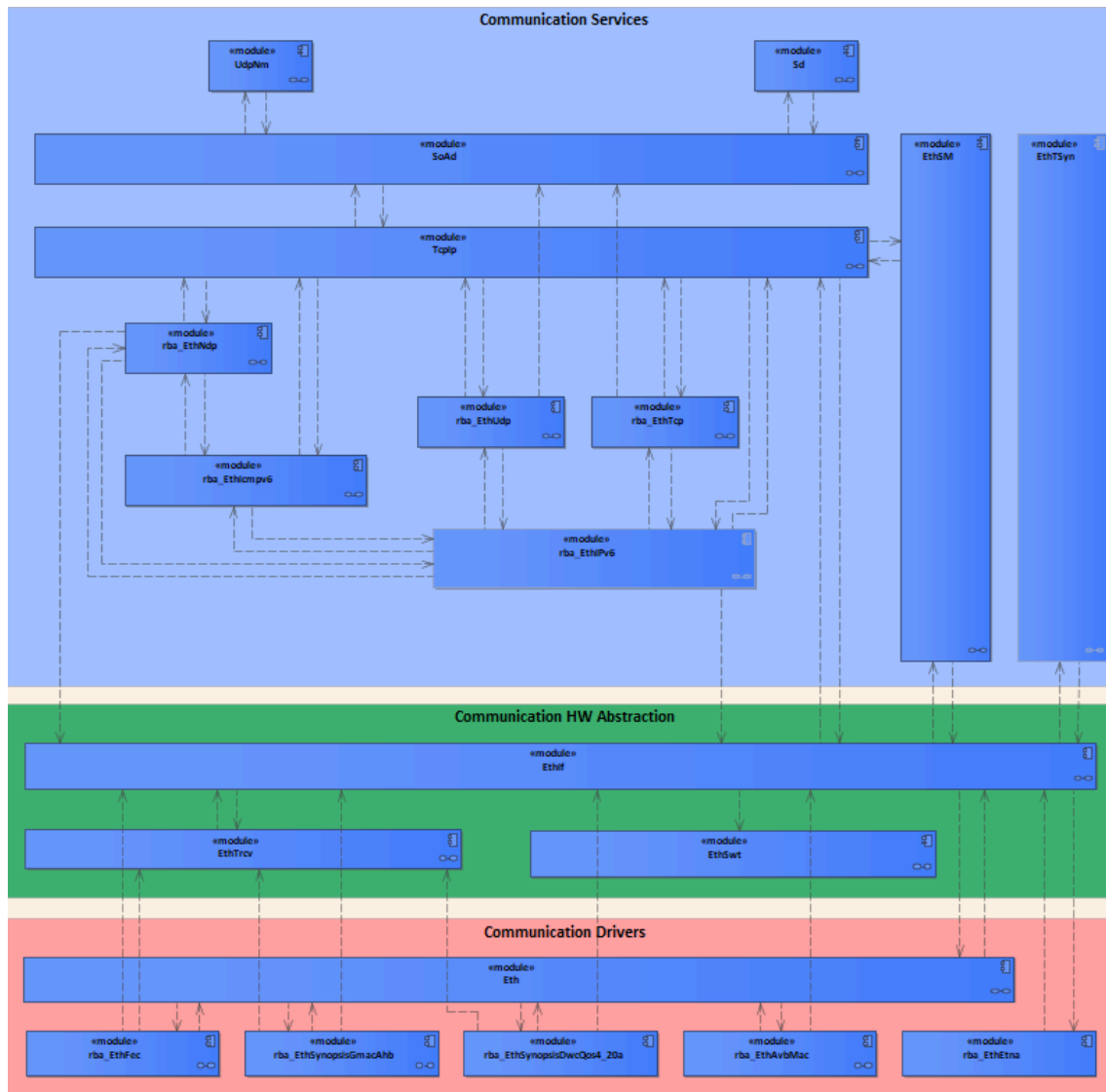


Figure 10.2. IPv6 Architecture

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

10.2 Supported Features

- ICMP message transfer over IPv6.
- Support for Destination Unreachable (indicated by ICMP "Port Unreachable" error message).
- Support for Parameter Problem (indicated by ICMP "Parameter Unreachable" error message).

- Support for PING and Neighbor Discovery Protocol (NDP) information messages.

10.2.1 Error Messages

Reporting of errors in the processing of datagrams (Type < 128).

- Port Unreachable (Type 1)
- Parameter Problem (Type 2)

10.2.2 Information Messages

Facilitating the transfer of NDP information messages (Type > 128) through the rba_EthNdp module. Currently four information message types are supported:

- Router Solicitation (Type 133). IPv6 hosts multicast (to a destination All router multicast IPv6 address FF02::2)
- Router Advertisement (Type 134). IPv6 routers reply back to a Router Solicitation with a Router Advertisement (RA) message (at a destination IPv6 all nodes multicast address FF02::1).
- Neighbor Solicitation (Type 135). When an IPv6 device needs to discover the MAC address of the corresponding IPv6 address of another interface in the Local Link.
- Neighbor Advertisement (Type 136). Response to the Node that has sent the Neighbor Solicitation message (returns with data Mac address).

10.3 Extensions

- No known extension parameters are defined in this version.

10.4 Limitations

- Not all errors are currently supported (e.g. Packet Too Big are not supported).

10.5 Exclusions

- There are no known exclusions in this version.

10.6 Deviations

- There are no known deviations in this version.

10.7 API Definition

Table 10.2.rba_EthICMPv6 Supported API functions

rba_EthIcmpV6_Init	Initialises the module.
--------------------	-------------------------

rba_EthIcmpV6_Transmit	Transmits an ICMPv6 message according to the specified parameters.
rba_EthIcmpV6_RxIndication	Callback function which signals receipt of an ICMPv6 message.

10.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

10.8.1 General

rba_EthICMPv6 does not have a separate configuration parameter file. Its configuration is determined by rba_EthIcmpv6 specific parameters within the configuration of Tcplp, which are located in the TcplpV6General and TcplpV6Config containers. The following Tcplp parameters are relevant for the configuration of rba_EthIcmpv6.

- TcplpIcmpV6MsgHandlerName: Specifies the name of the ICMP message handler function <User_IcmpMsgHandler>.
- TcplpIcmpV6EchoReplyEnabled: If enabled, the stack will respond to incoming ICMPv6 Echo Requests (Pings).
- TcplpIcmpV6EchoReplyAvoidFragmentation: If enabled, the stack will respond only to incoming ICMPv6 Echo Requests (Pings) that fit the MTU of the respective interface (i.e. can be transmitted without IPv6 fragmentation). Only relevant if TcplpIcmpV6EchoReplyEnabled is enabled.
- TcplpIcmpV6HopLimit: The default Hop-Limit value of outgoing ICMPv6 packets.
- TcplpIcmpV6MsgDestinationUnreachableEnabled: Disables/Enables the transmission of Destination Unreachable Messages.
- TcplpIcmpV6MsgParameterProblemEnabled: If enabled, an ICMPv6 parameter problem message will be sent if a received packet has been dropped due to unknown options or headers that are found in the packet. [RFC2460 4. IPv6 Extension Headers]

10.8.2 Checksum Calculation

Hardware checksum calculation is supported from AR v4.2.2 via the EthCtrlEnableOffloadChecksumICMPv6 configuration parameter (in Eth/EthGeneral/EthCtrlOffloading). This parameter is set to FALSE by default, which means that checksum calculation is handled by rba_EthICMPv6 rather than the Ethernet

Driver. If Ethernet Driver v4.0.3 is used, it does not support protocol checksum calculation, so `EthCtrlEnableOffloadChecksumICMPv6` should be set as `FALSE`.

10.9 Integration Advice

10.9.1 General

This module is fully dependent on the `Tcplp` module since it does not have its own `EcucParamDef` file. `Tcplp_EcucParamDef.arxml` is required for `rba_EthICMPv6` code generation.

10.9.2 Initialisation

The `rba_EthICMPv6` module is initialised by the `Tcplp` module implicitly, when `Tcplp_Init` is called, and so does not require separate initialisation.

10.9.3 Scheduled Functions

There are no scheduled functions.

10.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The `Rba_EthICMPv6` module uses the following OS resources:

- `rba_EthIcmpV6_ExclusiveAreaCurrAssignIPAddrAccess`

10.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

11 rba_EthIPv4

Module name	rba_EthIPv4
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

11.1 Introduction

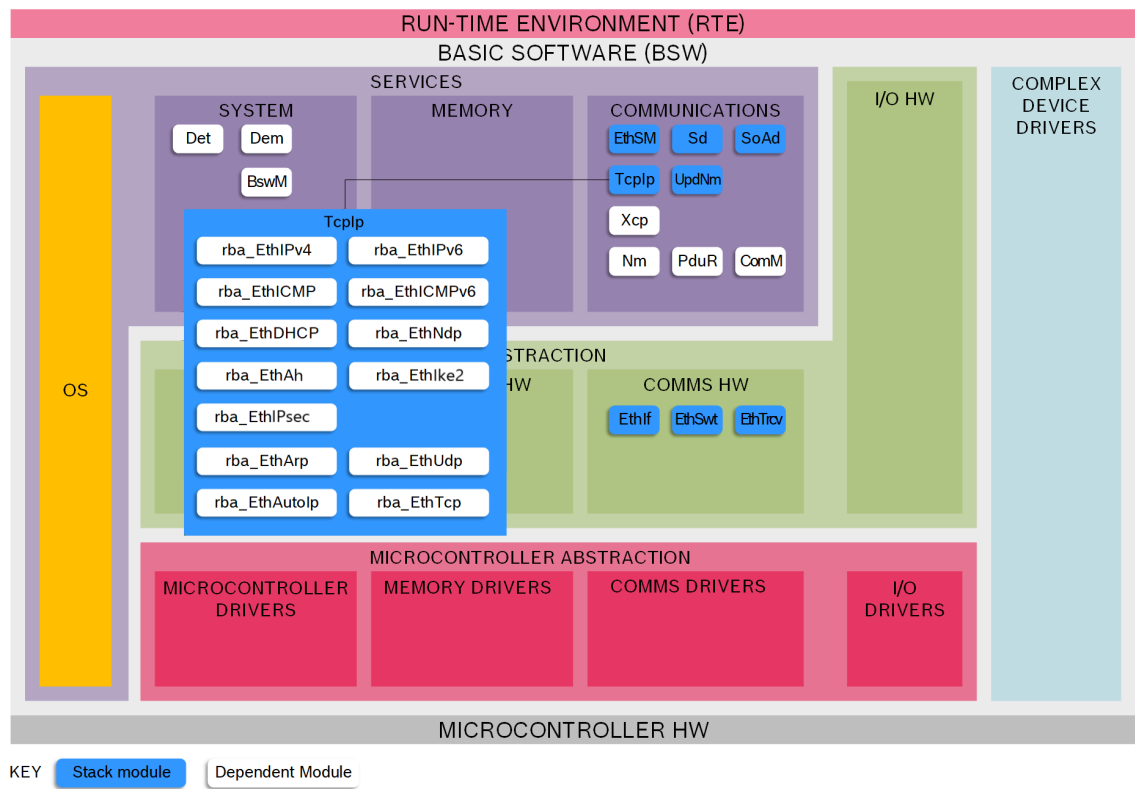


Figure 11.1.RTA-ETH Architecture

The rba_EthIPv4 module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of the module in the AUTOSAR layered architecture. The modules rba_EthArp and EthIf are designated as the lower layer modules with respect to rba_EthIPv4, while rba_EthIcmp, rba_EthUdp and rba_EthTcp are designated as the upper layer modules.

This RTA-BSW specific module provides IPv4 protocol services which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules. It is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data transfer. The specific functionality of the rba_EthIPv4 module is to deliver datagrams from a

source host (identified by the source address) to one or more destination hosts (identified by the destination address).

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

11.2 Supported Features

11.2.1 Transmission of IPv4 frames

rba_EthIPv4 provides a transmission buffer to the upper layer protocol. After the upper layer fills out the IPv4 payload into the provided buffer, rba_EthIPv4 fills out the IPv4 header and triggers transmission of IPv4 frame. There are two stages in the transmission of IPv4 frames: Buffer request and Frame transmission.

11.2.1.1 Buffer Request

Through the rba_EthIPv4_ProvideTxBuffer API, the rba_EthUdp, rba_EthTcp and rba_EthIcmp modules request rba_EthIPv4 for access to a transmit buffer for a LocalAddrId.

The following actions are performed:

- The source IP address is checked to see whether it is unicast or multicast. If it is not unicast, the request is not processed.
- The TcpIp controller must be in a state where communication is allowed which is when the controller is in ONLINE state, or is in STARTUP or SHUT-DOWN state and the address assignment method is DHCP.
- rba_EthIPv4 then checks if the destination IP address is valid and, if it is, will carry out further actions depending on the type (e.g. UNICAST, MULTICAST, BROADCAST).
- For UNICAST, if the destination node is on-link, then the MAC address of the remote node is fetched from the ARP table. If it is off-link, the MAC address of the default router is fetched.
- For MULTICAST, rba_EthIPv4 constructs the multicast MAC address of the multicast destination IP address.
- For BROADCAST, the MAC address is set to FF:FF:FF:FF:FF:FF. Routers block all MAC broadcasts, so the broadcasted IP frames are sent only to the on-link destination hosts.
- The derived destination MAC address is later forwarded later to EthIf.
- If the IP address is invalid or the corresponding MAC address is not available in the ARP table, rba_EthIPv4 returns BUFREQ_E_NOT_OK. Note that ARP starts address resolution process automatically if the requested IP/MAC combination is not yet available in ARP table.
- If the buffer length requested plus IPv4 header length is less than or equal to MTU, the buffer request will be forwarded to EthIf to get the buffer from the driver. If it is greater than MTU, the buffer will be provided from the IPv4 fragmentation buffer pool (if fragmentation is enabled and the requested

controller has sufficient free buffers available). If fragmentation is disabled, or if there are no sufficient free buffers available, BUFREQ_E_NOT_OK is returned.

11.2.1.2 Frame Transmission

Through the `rba_EthIPv4_Transmit` API, the `rba_EthUdp`, `rba_EthTcp` and `rba_EthIcmp` modules request `rba_EthIPv4` to transmit an IPv4 frame.

- The IPv4 frame transmission is done only if the total length to be transmitted is greater than zero, otherwise the buffer allocated is released and no frame transmission occurs.
- If the frame to be transmitted is a DHCP frame with destination IP broadcast and DHCP IP not yet assigned to the `TcplpCtrl`, then the source IP address in the IPv4 header is set to zero.
- `rba_EthIPv4` calculates the checksum for UDP and ICMP protocols if the hardware checksum offload is disabled, or if the frame undergoes fragmentation.
- If the transmission is requested over an IPv4 fragmentation buffer, the number of fragments is calculated and the driver API is called through `EthIf` to get the driver buffer for each fragment. These driver buffers will be filled with IPv4 header and payload according to the fragmentation algorithm, followed by the transmit call to `EthIf`. The allocated IPv4 fragmentation buffer is always released independently of the transmission status, including when transmission fails. The IPv4 specification does not provision for any retransmission mechanism for IP fragments.
- If the transmission request is for a driver-allocated buffer, IPv4 headers are filled with fragmentation fields set zero, followed by a transmit call to `EthIf`.

11.2.1.3 ZeroFrame Transmission

`rba_EthIPv4` supports zero copy frame transmission if the feature is enabled in `Tcplp`.

With zero copy frame transmission, the transmission occurs in two steps:

- Resuable Header filling: A reusable header is a buffer in which the static header fields of IPv4, which do not change for different fragments of the frame or different frame transmissions for the same socket connection, are filled. It is created by `Tcplp` and forwarded by `rba_EthUdp`. After filling the static IPv4 header fields, the same reusable header is forwarded to `EthIf` to fill the rest of the resuse buffer. In the dynamic mode of zero copy transmission (where fragmentation is allowed), the header fields which remain constant over different frame transmissions are only filled. The size of the reuse header is 36 bytes. In the static mode of zero copy transmission (where fragmentation is not allowed), all the IPv4 header fields are filled. The size of the resuse header is 46 bytes.
- One time header filling and frame transmission: Through the `rba_EthIPv4_ZeroCopyTransmit` API, `rba_EthUdp` requests zero copy frame

transmission to IPv4. In dynamic mode, the remaining fields that change for different frames are filled. If the frame being transmitted is greater than MTU, then the fragmentation algorithm is used. The header and data length is updated depending on which fragment is being transmitted accordingly. The zero copy continuous header buffers are created and passed to EthIf. In static mode, the trigger transmit flag is set to true and the header and data buffer pointers are passed to EthIf.

11.2.2 Fragmentation of oversized IPv4 datagrams

If the transmission is requested for a frame greater than MTU, and if fragmentation is enabled for the corresponding Tcplp controller, and if sufficient fragmentation buffer is available, UDP and ICMP frames undergo fragmentation. Note that a TCP frame will never undergo fragmentation because TCP never requests a transmission greater than MSS, which is derived from MTU.

11.2.3 Reception of IPv4 frames

Frame reception is notified to rba_EthIPv4 by Tcplp via the rba_EthIPv4_RxIndication API.

After the reception of an IPv4 frame, rba_EthIPv4 validates the header fields of the received frame. If the frame is considered invalid, it is dropped.

If the remote node is on-link, and if IP address conflict detection for IPv4 frames has been enabled (via TcplpConflictDetectionIPv4Frames), rba_EthIPv4 requests rba_EthARP to check whether there is a conflict between the local IP address and the remote IP address. If a conflict is detected, the frame is dropped.

If the frame is fragmented, and if fragmentation is enabled for the Tcplp controller on which frame was received, and if reassembly is enabled for the corresponding Tcplp controller, and if there is sufficient buffer available, rba_EthIPv4 reassembles the received fragments. Otherwise the frame is dropped. If the reassembly isn't completed within configured timeout (i.e. due to missing fragments), rba_EthIPv4 will discard the frame and trigger an ICMP time exceeded message.

If the remote node is on-link, rba_EthIPv4 requests rba_EthARP to update the remote node IP/MAC combination in the ARP table.

Then rba_EthIPv4 checks whether the upper layer protocol is not supported or has been disabled via configuration. In these cases, the frame is dropped. If the conditions for sending ICMP errors are met, rba_EthIPv4 will also request ICMP to send ICMP protocol unreachable frame.

If an ICMP or UDP frame is completely received, and if software checksum validation is enabled, or if the received frame is fragmented, rba_EthIPv4 checks the validity of the ICMP or UDP checksum. If it is invalid, the frame is dropped.

Even though hardware is capable of calculating checksum, UDP and ICMP checksum calculation can only be done by software for incoming and outgoing fragmented frames. Because only the IP layer has the information about frame undergoing fragmentation or reassembly, and to keep the design simple, checksum calculation for all fragmented and non-fragmented outgoing and incoming UDP and ICMP frames is done within rba_EthIPv4.

If all checks are passed and the complete frame is received, rba_EthIPv4 forwards the frame to the corresponding upper layer protocol:

- TCP frames are forwarded to rba_EthTcp via rba_EthTcp_RxIndication.
- UDP frames are forwarded to rba_EthUdp via rba_EthUdp_RxIndication.
- ICMP frames are forwarded to rba_EthIcmp via rba_EthIcmp_RxIndication.

After the completely reassembled frame is forwarded to the upper layer, reassembly resources are released.

11.2.4 Get and Reset Measurement data

rba_EthIPv4 provides a count of all the received datagrams dropped due to invalid source or destination IP address. There is also provision to clear the measurement data count.

11.2.5 Security Events

rba_EthIPv4 can call the IdsM module to report security events (e.g. dropped datagrams due to an invalid source or destination IP address).

11.3 Extensions

- No known extension parameters are defined in this version.

11.3.1 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

11.3.1.1 rba_EthIPv4_CheckFragmentation

Syntax	rba_EthIPv4_CheckFragmentation (uint8 LocalAddrId_u8, uint16 DataLength_u16, uint8 * NumOfIPFragments_pu8)
Parameters In	LocalAddrId_u8 LocalAddrId on which data is to be transmitted. DataLength_u16 Data length to be transmitted.
Parameters Out	NumOfIPFragments_pu8 Number of fragments required to transmit the frame.
Description	Returns the number of IPv4 fragments required to transmit the data with length provided in input parameter.

11.3.1.2 rba_EthIPv4_ZeroCopyTransmit

Syntax	rba_EthIPv4_ZeroCopyTransmit (uint8 LocalAddrId_u8, uint16 UdpDataLength_u16, const Eth_ZeroCopyArg_tst * ZeroCopyArg_pcst, uint32 * TxId_pu32)
--------	---

Parameters In	LocalAddrId_u8 LocalAddrId on which data is to be transmitted. UdpDataLength_u16 Data length to be transmitted. ZeroCopyArg_pcst ZeroCopy argument containing the allocated headers and the pointer to the UL data to be transmitted.
Parameters Out	TxId_pu32 Unique identifier of the ZeroCopy transmission, valid only if ZeroCopyArg_pst → TxConfFlag_b is TRUE.
Description	Executes the ZeroCopy transmission.

11.3.1.3 rba_EthIPv4_ZeroCopyFillHeader

Syntax	<code>rba_EthIPv4_ZeroCopyFillHeader(uint8 LocalAddrId_u8, uint8 Priority_u8, uint32 DestIpAddr_u32, const TcpIp_IPHeader_tst * IpHeader_pcst, uint16 UdpDataLength_u16, const Eth_ZeroCopyArg_tst * ZeroCopyArg_pcst)</code>
Parameters In	LocalAddrId_u8 LocalAddrId on which data is to be transmitted. Priority_u8 Priority value which shall be used for the 3-bit PCP field of the VLAN tag. DestIpAddr_u32 Destination IP address to which the frame has to be transmitted. IpHeader_pcst Data structure containing the fields of the IP header. UdpDataLength_u16 Data length to be transmitted.
Parameters Out	ZeroCopyArg_pcst ZeroCopy argument containing the allocated headers and the pointer to the UL data to be transmitted.
Description	Prefills the reusable part of the IPv4 header.

11.3.1.4 rba_EthIPv4_GetPseudoHeader

Syntax	<code>rba_EthIPv4_GetPseudoHeader(uint8 LocalAddrId_u8, uint8 ULSocketOwner_u8, uint16 BufLength_u16, TcpIp_PseudoHdr_tst * IPseudoHdr_pst)</code>
Parameters In	LocalAddrId_u8 Index of local address requesting configuration information. ULSocketOwner_u8 Upper layer socket owner. BufLength_u16 Buffer length.
Parameters Out	IPseudoHdr_pst This structure contains IPv4 header field values to be used in transmitted frame.
Description	Provides the structure containing pseudo header information required for Udp checksum calculation.

11.3.1.5 rba_EthIPv4_GetDestEthAddr

Syntax	<code>rba_EthIPv4_GetDestEthAddr(const TcpIp_LocalAddrConfig_to * const LocalAddrConfig_cpcst, uint32 DestIpAddress_u32, uint8 * DestEthAddr_pu8)</code>
Parameters In	LocalAddrConfig_cpcst Pointer to the localAddress configuration structure present in TcpIp. DestIpAddress_u32 Destination IP address to which the data is to be transmitted.
Parameters Out	DestEthAddr_pu8 Destination MAC for which frame will be transmitted to.
Returns	TcpIp_ReturnType TCPIP_OK: If the destination MAC is successfully updated. TCPIP_E_ARP_CACHE_MISS: If the operation failed because of an ARP cache miss. TCPIP_E_NOT_OK: If the operation failed due to failure of internal function calls.
Description	Provides the destination MAC address for requested destination IP.

11.4 Limitations

There are no known limitations.

11.5 Exclusions

- There are no known exclusions in this version.

11.6 Deviations

- There are no known deviations in this version.

11.7 API Definition

Table 11.7.rba_EthIPv4 Supported API functions

<code>rba_EthIPv4_Init</code>	Initialises the module.
<code>rba_EthIPv4_MainFunction</code>	<code>rba_EthIPv4</code> Main Function.
<code>rba_EthIPv4_ChkIPv4AddrType</code>	Returns the destination IP address type (Unicast/Broadcast/Multicast).
<code>rba_EthIPv4_GetAndResetMeasurementData</code>	Reads and resets detailed measurement data.
<code>rba_EthIPv4_GetIPMulticastDestEthAddr</code>	Returns multicast Ethernet address for multicast IPv4 destination.
<code>rba_EthIPv4_ProvideTxBuffer</code>	Provides a transmission buffer for an upper layer module.
<code>rba_EthIPv4_Transmit</code>	Transmits an IPv4 packet.

rba_EthIPv4_RxIndication	Callback function used by lower layer module to signal reception of IPv4 packet.
rba_EthIPv4_CheckIsFrameSecured	Returns TRUE if the transport protocol is updated.
rba_EthIPv4_CheckRemoteAddressValidity	Returns if the Src Ip and Dst IP are valid and in same subnet.

11.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration Mapping can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

11.8.1 General

Configuration of rba_EthIPv4 is determined by the IPv4 relevant parameters of the Tcplp configuration. They are present in the TcplpV4General, TcplpV4-Config and TcplpV4Ctrl containers. The following Tcplp parameters are relevant for rba_EthIPv4 configuration.

- TcplpTxFragmentBufferSize - specifies the fragmentation buffer size in terms of bytes for the controller. This parameter is added because there's no provision in the AUTOSAR specification for a parameter to configure fragmentation in transmit path. If the fragmentation needs to be disabled for the controller, this parameter need not be configured.
- TcplpReassemblyBufferCount - configures the number of frames that can be reassembled in parallel. A value of 0 disables IPv4 reassembly. This extension parameter replaces the AUTOSAR parameters TcplpFragmentation-RxEnabled and TcplpNumReassDgrams to simplify configuration and harmonize IPv4 and IPv6 configurations.
- TcplpNumFragments - this AR parameter is not supported as it is not required for the RFC 815-recommended algorithm used in rba_EthIPv4. Out of order reception of IPv4 fragments is implicitly supported by this algorithm.
- TcplpReassTimeout - specifies the timeout in seconds after which an incomplete datagram gets discarded. This parameter is only relevant if TcplpReassemblyBufferCount is not zero. Note that the reassembly timeout min/max values defined by AUTOSAR are insufficient because timeout zero would actually disable reassembly and INF would block the buffer forever. For this reason, the value range of TcplpReassTimeout has been changed from 0 .. INF to 0.001 .. 120. The minimum value of 0.001 was chosen as it will be converted to milliseconds in code, and the maximum

value of 120 seconds is recommended by IPv4 RFC 1122.

- `TcplpReassemblyBufferSize` - specifies the buffer size required to reassemble all the fragments of an original datagram. The expected value corresponds to the IPv4 payload of the original datagram before fragmentation. This parameter is only relevant if `TcplpReassemblyBufferCount` is not zero.
- `TcplpFragmentationConfigRef` - reference to fragmentation configuration specified in `TcplpFragmentationConfig` container. If this optional reference is not configured for the particular Tcplp controller, fragmentation and reassembly are considered disabled for that controller.



NOTE

IPv4 fragmentation and reassembly can be enabled or disabled separately for each specific Tcplp controller. All fragmentation and reassembly related parameters are Tcplp controller-specific. Multiple Tcplp controllers may reference the same `TcplpFragmentationConfig` container.

11.8.2 Conflict Detection

The IPv4 frames received by `rba_EthIPv4` can be validated and checked for IP conflict. This conflict detection can be enabled/disabled via `TcplpConflictDetectionIPv4Frames`. It is disabled by default because the conflict detection in ARP is sufficient in most cases.

11.8.3 Checksum Calculation

Hardware checksum calculation is supported from AR v4.2.2 via the `EthCtrlEnableOffloadChecksumIPv4` configuration parameter (in `Eth/EthGeneral/EthCtrlOffloading`). This parameter is set to `FALSE` by default, which means that checksum calculation is handled by `rba_EthIPv4` rather than the Ethernet Driver. If Ethernet Driver v4.0.3 is used, it does not support protocol checksum calculation, so `EthCtrlEnableOffloadChecksumIPv4` should be set as `FALSE`.

11.8.4 Security Events

During reception, `rba_EthIPv4` can report security events to the `IdsM` module, if the corresponding event is configured in the configuration container `TcplpSecurityEventRefs` and if `Tcplp/TcplpGeneral/TcplpEnableSecurityEventReporting` is set to `TRUE`.

If a frame is received with an invalid source or destination IP address, then IPv4 supports the reporting of a security event to the `IdsM` module via the API `IdsM_SetSecurityEventWithContextData`. In AUTOSAR v22-11, context data is not specified, therefore the `contextData` argument is set to `NULL_PTR`.

11.9 Integration Advice

11.9.1 Init Functions

rba_EthIPv4 is initialised implicitly by the Tcplp module when Tcplp_Init is called, so rba_EthIPv4 does not require separate initialisation.

11.9.2 Delnit Functions

No Delnit is required.

11.9.3 Scheduled Functions

rba_EthIPv4_MainFunction is triggered by Tcplp_MainFunction, so only the scheduling of Tcplp_Mainfunction is required.

11.9.4 General

rba_EthIPv4 does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthIPv4 code generation.

11.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthIPv4 module uses the following OS resources:

- rba_EthIPv4_ExclusiveAreaCurrAssignIPAddrAccess

11.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

12 rba_EthIPv6

Module name	rba_EthIPv6
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

12.1 Introduction

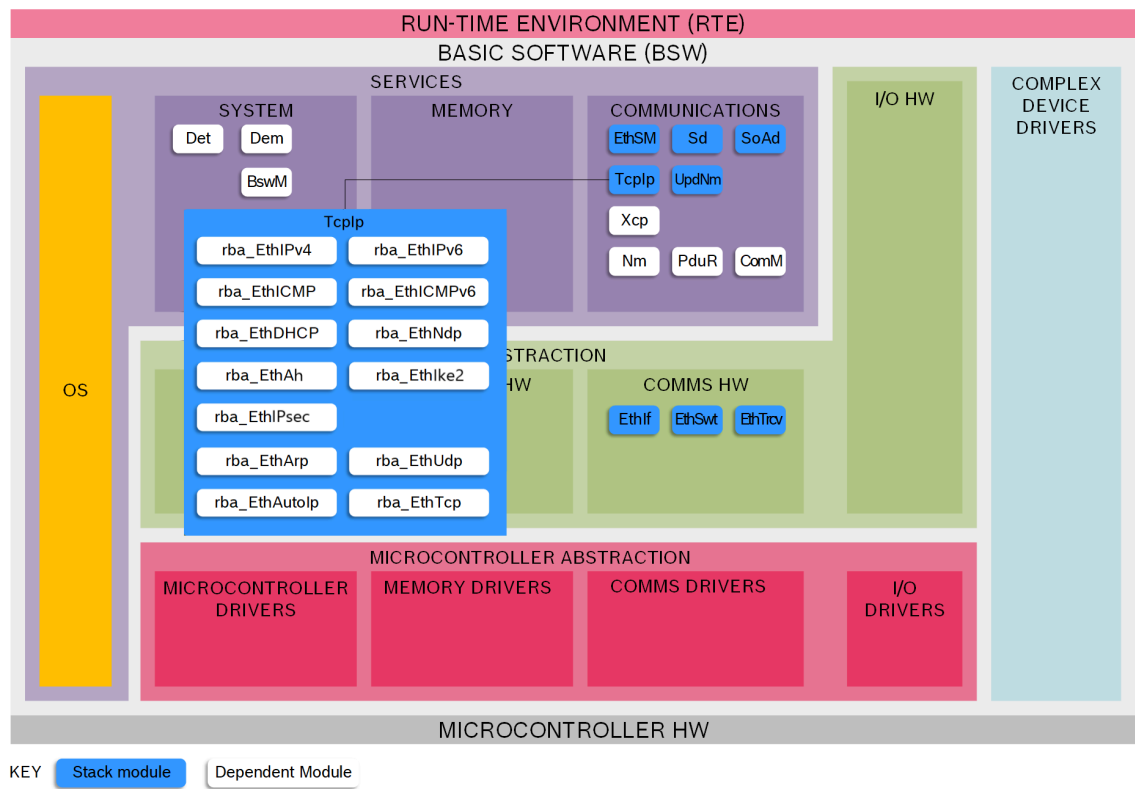


Figure 12.1.RTA-ETH Architecture

Internet Protocol version 6 (IPv6) is the most recent version of the Internet Protocol layer. It provides an identification and location system on Ethernet networks, and it routes traffic across the Internet. The rba_EthIPv6 module is the RTA-BSW implementation of the Autosar IPv6 module.

rba_EthIPv6 is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthIPv6 in the AUTOSAR layered architecture.

rba_EthIPv6 is a member of the group of sub-modules encapsulated by Tcplp which, together, form the 'protocol stack' for TCP/IP messaging and data transfer. The specific functionality of the rba_EthIPv6 module is to deliver datagrams from a source host (identified by the source address) to one or more destination hosts

(identified by the destination address).

rba_EthIPv6 interacts with EthIf (lower level), and with rba_EthlcmpV6, rba_EthNdp, rba_EthUdp and rba_EthTcp (upper level). The Tcplp module acts as a wrapper around the EthIPv6 module.

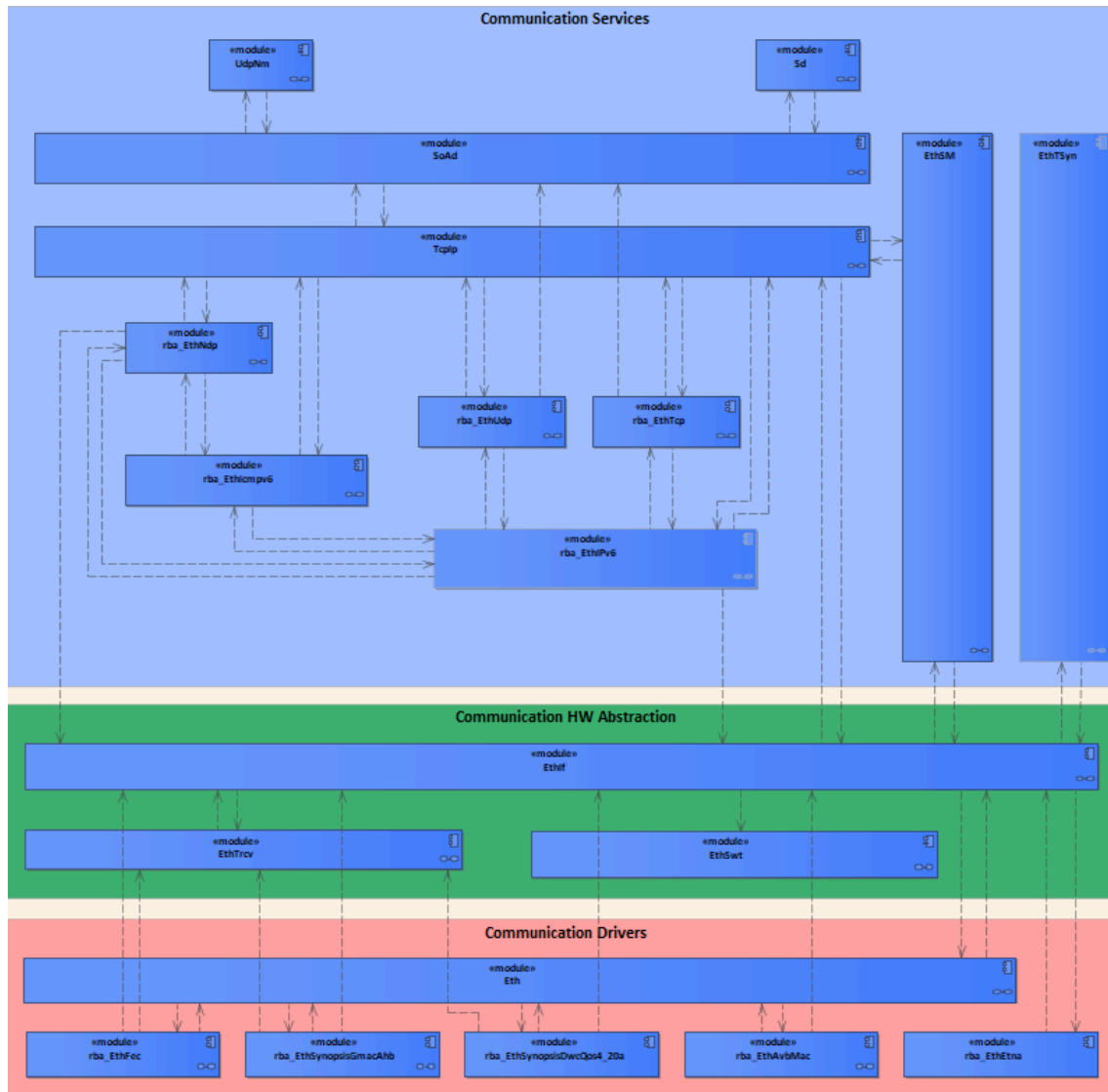


Figure 12.2. IPv6 Architecture

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

12.2 Supported Features

- Transmission/reception of IPv6 frames.
- Duplicate Address Detection.
- Checksum calculation and validation for the UDP protocol.

- Get and Reset Measurement data.
- Provide a count of all the received datagrams dropped due to invalid destination IP address.
- Information/Error reporting in the processing of a datagram (provided by rba_EthIcmpV6 messages).
- Support for Multiple global IP address configuration.
- Multiple TcpIpAddrAssignment methods for a single TcpIpLocalAddr.
- Default Router configuration for IPv6.

12.3 Extensions

- No known extension parameters are defined in this version.

12.4 Limitations

- DHCPv6 is not currently supported.
- TcpIpV6Ctrl is not currently supported.
- The processing of Extension Headers at transmission is not supported except fragment header.
- Parsing and basic validations are performed over extension headers for received packets only, with the reassembly of fragmented packets supported if fragmented packets matches the reassembly configuration of the controller.
- Encapsulation headers are not supported and any IPv6 frames containing extension headers, except a fragmentation header are discarded. If any IPv6 frame with these 6 extension headers are received, it will not be processed as none of the supported extension headers can be duplicated except Destination Options header and fragment header. So in the case of an IPv6 frame with 6 valid extension headers will be discarded.
- Only one Router Prefix based IP address is supported per TcpIp Controller configured as IPv6.
- Source address selection for an IPv6 packet that is to be transmitted in response to a multicast packet received is selected based on the scope of the Source of the Remote Node. If a local IP matches the Prefix of the source node, then the local IP is considered to be an exact match. If not found, the local IP that has the same scope as that of the Source IP is selected. This limitation exists when multiple local IP addresses have the same Prefix configured.
- Selection of IP address when an IPv6 packet is to be transmitted in response to multicast ping or packet with incorrect destination IP is unpredictable when IP with exactly the same prefix is not found.

- No support for router detection address.
- Path MTU Discovery is not currently supported.

12.5 Exclusions

- There are no known exclusions in this version.

12.6 Deviations

- There are no known deviations in this version.

12.7 API Definition

Table 12.2.rba_EthIPv6 Supported API functions

rba_EthIPv6_Init	Initialises the module.
rba_EthIPv6_Transmit	Transmits an IPv6 packet.
rba_EthIPv6_ProvideTxBuffer	Provides a transmission buffer for an upper layer module.
rba_EthIPv6_CopyIPv6Addr	Utility function to be used in the whole Eth stack.
rba_EthIPv6_GetIPMulticastDestEthAddr	Calculates the corresponding MAC addr for a given IP address.
rba_EthIPv6_SetIPv6AddrTo0	Utility function to set the IPv6 addr to 0.
rba_EthIPv6_IsIPv6AddrTypeMulticast	Utility function to check that the IPv6 address is multicast.
rba_EthIPv6_ParseExtIPv6Headers	Helper function called by rba_EthIPv6_RxIndication to handle processing of all extension headers in the IPv6 Rx packet.
rba_EthIPv6_AddrIsIPv6LinkLocal	Helper function used for checking if the scope of an IPv6 address is link local.
rba_EthIPv6_FindLocalAddr	Helper function used to find a match for the IPv6 address from the incoming packet.
rba_EthIPv6_GetAndResetMeasurementData	Allows the reading and resetting of detailed measurement data for diagnostic purposes.
rba_EthIPv6_RxIndication	Callback function used by a lower layer module to signal reception of an IPv6 packet.
rba_EthIPv6_MainFunction	This function decrements the reassembly timer by value corresponding to TCPIP_MAIN_FUNCTION-PERIOD_MS.

12.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

12.8.1 General

Configuration of rba_EthIPv6 is determined by the IPv6 relevant parameters of the Tcplp configuration.

12.8.2 Multicast Frames

For multicast frames to be received on a switch, the multicast MAC address configuration parameter `EthSwPortPredefinedMacAddresses` needs to be manually updated on each of the switch ports on which multicast frame need to be received.

12.8.3 Sleeping Configurations

A forwarder script has been provided that checks if LinkLocal, SNMA, All-Node Multicast IPv6 addresses have been configured for the IPv6 Tcplp controller. Any that are not configured will be selectively forwarded to the Tcplp configuration before the Tcplp prepare action is executed.

These configurations are called Sleeping Tcplp configurations because they "wake up" in the configuration only when the respective IPv6 configuration is missing. The sleeping configurations should start with a ShortName of something like "zzz_" so that they get forwarded to the very end of LocalAddressId list and thus do not disturb any existing configurations.

12.8.4 Multiple IPv6 Addresses

More than one IPv6 address can be configured to each Tcplp controller that has been configured as IPv6. Each IPv6 controller may have one LinkLocal, One Global Unicast or Router Prefix based IP address, one All-Node Multicast and SNMA of static Unicast address for each IPv6-based Tcplp controller.

The SNMA of Unicast IPv6 addresses which have a `TcplpAssignmentMethod` of `TCPIP_IPV6_ROUTER` must not be configured.

12.8.5 Checksum Calculation

Hardware checksum calculation is supported from AR v4.2.2 via the `EthCtrlEnableOffloadChecksumIPv6` configuration parameter (in `Eth/EthGeneral/EthCtrlOffloading`). This parameter is set to `FALSE` by default, which means that checksum calculation is handled by rba_EthIPv6 rather than the Ethernet Driver. If Ethernet

Driver v4.0.3 is used, it does not support protocol checksum calculation, so `EthCtrlEnableOffloadChecksumIPv6` should be set as `FALSE`.

12.9 Integration Advice

12.9.1 General

The `rba_EthIPv6` module is fully dependent on the `Tcplp` module, since it does not have its own `EcucParamDef` file. `Tcplp_EcucParamDef.arxml` is required for `rba_EthIPv6` code generation.

12.9.2 Initialisation

The `rba_EthIPv6` module is initialised by the `Tcplp` module implicitly, when `TcpIp_Init` is called. Therefore `rba_EthIPv6` does not require separate initialisation.

12.9.3 Scheduled Functions

There are no scheduled functions.

12.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The `Rba_EthIPv6` module uses the following OS resources:

- `rba_EthIPv6_ExclusiveAreaMeasurementData`

12.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

13 rba_EthNdp

Module name	rba_EthNdp
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

13.1 Introduction

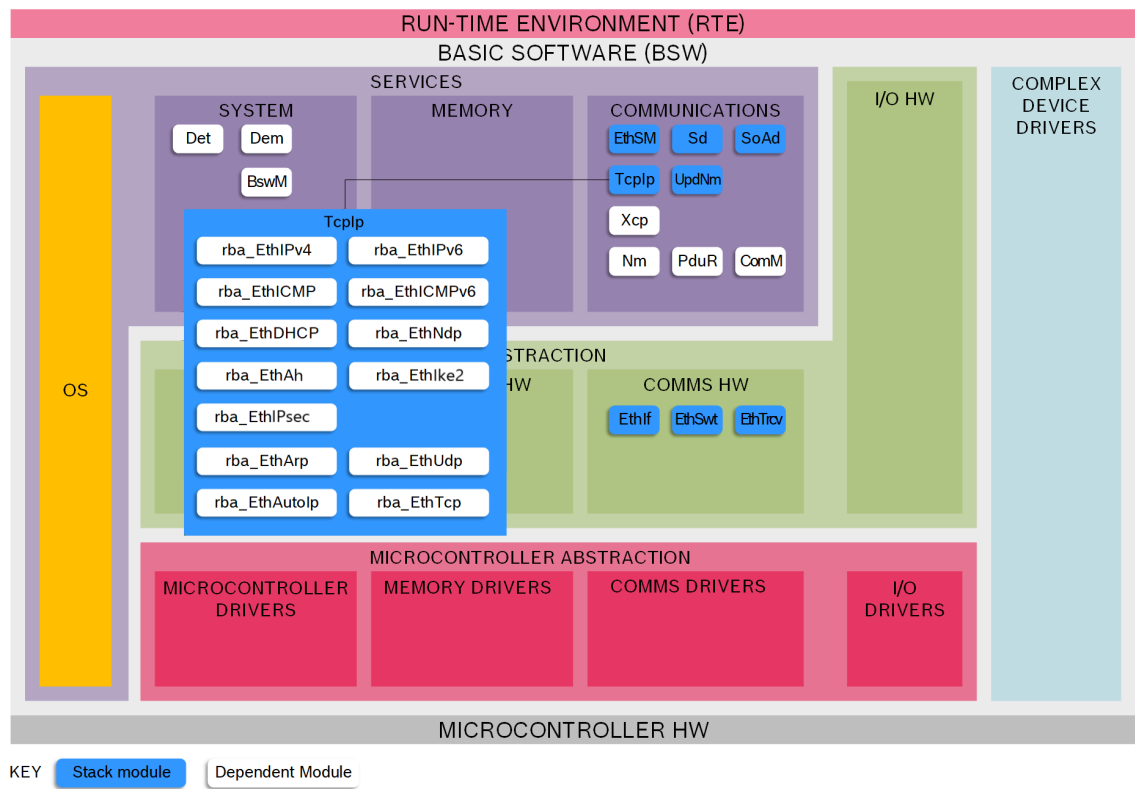


Figure 13.1.RTA-ETH Architecture

The IPv6 Neighbor Discovery Protocol (NDP) identifies the relationship between different neighbor devices in the IPv6 network, and it is used to resolve the Mac Address of an IPv6 Address.

IPv6 nodes on the same link use NDP to discover each other’s presence, to determine each other’s link-layer addresses, to find routers, and to maintain reachability information about the paths to active neighbors.

The rba_EthNdp module is the RTA-BSW implementation IPv6 NDP.

rba_EthNdp is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthNdp in the AUTOSAR layered architecture.

rba_EthNdp's GetPhysicalAddress API is called by rba_EthIPv6 during transmit operations in order to find the ETH MAC address associated with the remote IP address. rba_EthNdp also interacts with the upper layer rba_EthIcmpV6 module.

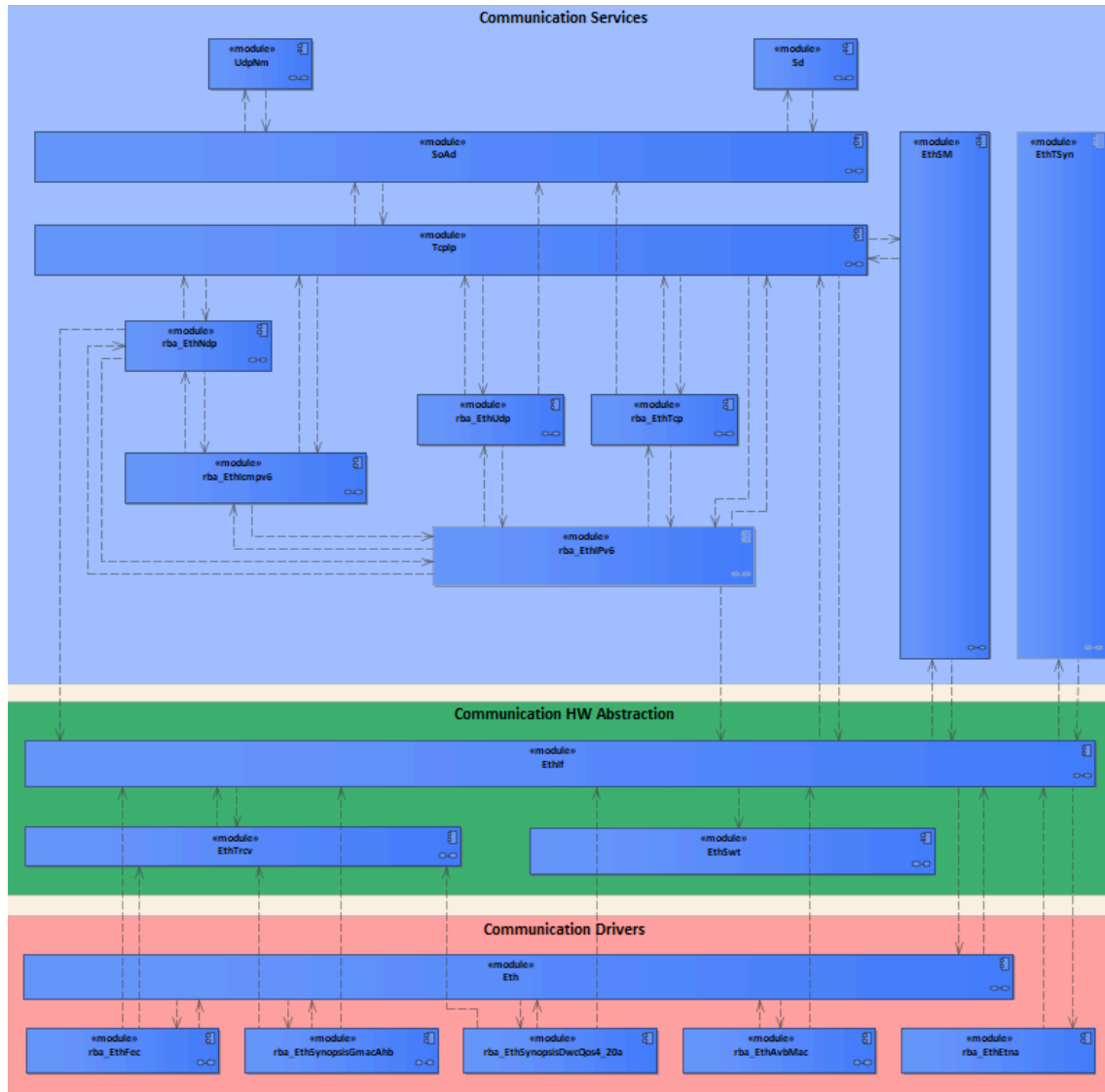


Figure 13.2. IPv6 Architecture

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

13.2 Supported Features

- On-link communication: rba_EthNdp detects neighbors on the local LAN and resolves the LAN local IP address to a MAC address.
- Off-link communication: See below.
- Neighbor Solicitation and Neighbor Advertisement message validations.

- Neighbor Unreachability Detection.
- Router Advertisement message validations.
- Router Discovery: See below.
- Default Router support to transmit packets to off-link remote nodes.
- Multiple address assignment for an IPv6 Local address.

13.2.1 Off-Link Communication

Off-link communication is supported via the configuration of a default router, which can be configured statically in Tcplp, or changed manually via Tcplp APIs (e.g. Tcplp_RequestIpAddrAssignment).

13.2.2 Router Discovery

The current NDP implementation supports Router Detection, but only for IP address assignment, and it has a limited off-link communication capability.

13.3 Extensions

- No known extension parameters are defined in this version.

13.3.1 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

13.3.1.1 rba_EthNdp_Start

Syntax	<code>rba_EthNdp_Start(TcpIp_LocalAddrIdType lLocalAddrId_tu8)</code>
Parameters	<code>lLocalAddrId_tu8</code> Local address ID which requests Router discovery.
Description	Starts Router Discovery process.

13.3.1.2 rba_EthNdp_Stop

Syntax	<code>rba_EthNdp_Stop(TcpIp_LocalAddrIdType lLocalAddrId_tu8)</code>
Parameters	<code>lLocalAddrId_tu8</code> Local address ID for which IP address has to be released.
Description	Stops Router Discovery process.

13.3.1.3 rba_EthNdp_UpdateNdpTable

Syntax	<code>rba_EthNdp_UpdateNdpTable(const TcpIp_IPv6StdHdr_tst* Std_Header_pcst, const rba_EthNdp_NeighborReachabilityState_ten TableEntryState_en, const rba_EthNdp_ArNudPrefRoutCacheConfig_to* ArNudPrefRoutCacheConfig_pcst, const uint8 * RmtPhysAddr_pcu8)</code>
--------	---

Parameters	Std_Header_pcst IPv6 header structure. TableEntryState_en Entry state. ArNudPrefRoutCacheConfig_pcst Pointer to NDP table. RmtPhysAddr_pcu8 Remote MAC address.
Description	Updates a NDP table for Reachability update from TP frames.

13.3.1.4 rba_EthNdp_FindMatchingTableForLocalAddress

Syntax	rba_EthNdp_Stop(TcpIp_LocalAddrIdType lLocalAddrId_tu8)
Parameters	Std_Header_pcst IPv6 header structure. EthIfCtrlIdx_u8 Controller index.
Description	Finds a NDP table for Reachability update from upper layers. The table is matched based on the scope of the IP address of the entry to be made.

13.3.1.5 rba_EthNdp_CleanNdpTable

Syntax	void rba_EthNdp_CleanNdpTable(uint8 LocalAddrId_u8)
Parameters In	LocalAddrId_u8 Local address Index.
Description	Called when an Ndp table belonging to a local address needs to be cleared.

13.3.1.6 rba_EthNdp_ReadNdpTable

Syntax	rba_EthNdp_ReadNdpTable(cuint8 EthIfCtrlIdx_u8, uint32 * const NumberEntries_cpu32, rba_EthNdp_NeighborCache_tst *NdpEntryList_pst, uint32 NdpTableEntryOffset_u32)
Parameters In	EthIfCtrlIdx_u8 Index of the EthIf controller for which Ndp table shall be read out. NdpTableEntryOffset_u32 Ndp table entry offset. NumberEntries_cpu32 The number of Ndp table entries to be read.
Parameters Out	NumberEntries_cpu32 The number of Ndp table entries read. NdpEntryList_pst Pointer to Ndp entries.
Description	Reads out the Ndp table for the EthIfCtrl.

13.3.1.7 rba_EthNdp_IpConflictDetection

Syntax	rba_EthNdp_IpConflictDetection(uint8 EthIfCtrlIdx_u8, const TcpIp_IPv6Addr_tst *IPv6Addr_pcu32, const uint8 *RmtPhysAddr_pcu8)
Parameters In	EthIfCtrlIdx_u8 Index of the EthIf controller. IPv6Addr_pcu32 IP address for which the conflict is to be detected. RmtPhysAddr_pcu8 Physical address of the Remote Node.
Description	Called when an IP conflict is detected in IPv6.

13.4 Limitations

- Only a limited version of the NDP State machine has been implemented in the current version of rba_EthNdp.
- Only the first prefix advertised with the auto address configuration flag set is processed.
- Multicast MAC filter must be updated manually for SNMA packets in switch supported hardware until this feature is fully supported.
- Router discovery is supported only for Prefix based IP address assignment; MTU discovery and Hop Limit is currently not supported.
- Redirect message is currently not supported.
- In cases where the advertised Reachable Time value will be the same in consecutive Router Advertisements, and a host's BaseReachableTime rarely changes, the current implementation does not ensure that a new random value gets re-computed at least once every few hours.
- If multiple options are advertised and any one of those options is invalid, then the entire packet is not discarded.
- Router Reachable time and Retransmit time are considered in multiples of a thousand.
- Router Advertisement from a router is processed only if Router Discovery has been triggered for a Local address.
- A manually configured Default Router has a higher priority compared to Routers that are discovered by Router Discovery. When a Default Router is manually configured, no attempt is made to transmit packets via discovered Routers. A manually configured Default Router is used as default router irrespective of whether the isRouter flag is set to TRUE or FALSE.
- Only Prefix with length 64 are accepted for IP address assignment when TcplpAssignmentMethod for a TcplpLocalAddr is TCPIP_IPV6_ROUTER.
- When a Router is configured for an TcplpLocalAddr with IPv6 address, or if TcplpNdpDefaultRouterListSize is configured, TcplpNdpMaxNeighbor-CacheSize must be set such that additional entries of Router's Physical Address can be stored in the neighbor cache.
- When routers discovered by Router Discovery are not reachable, one of the advertised routers is picked for address resolution. This selection does not follow any particular order.
- Source Link Layer option is required for processing of Neighbor Solicit packets.
- TcplpNdpDestinationCacheSize is currently unused.
- NDP does not create a reachable entry on reception of UNICAST TCP frame over IPv6.
- While transmitting a UNICAST frame using a local address, if the prefix of the remote node exactly matches the prefix of any of the assigned addresses

(of the same interface), the remote is considered on-link. Therefore, both the Prefix and the address assigned from the Prefix advertised by the router expires after Preferred Valid lifetime.

- NDP does not currently support DETs.
- Conflict detection is NO longer supported if NS is received with different Source Ip and Target Ip. Such frames may NOT be processed.

13.5 Exclusions

- There are no known exclusions in this version.

13.6 Deviations

- There are no known deviations in this version.

13.7 API Definition

Table 13.9.rba_EthNdp Supported API functions

rba_EthNdp_Init	Initialises the module.
rba_EthNdp_MainFunction	Schedules the NDP processes.
rba_EthNdp_GetRemotePhysAddr	Provides the MAC address of the provided IP.
rba_EthNdp_GetRouterPhyAddr	Provides the MAC address of the Router from Router Discovery.
rba_EthNdp_SendFrame	Internal function that sends an NDP frame to ICMP.
rba_EthNdp_RxIndication	Callback for the received NDP packets.

13.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

13.8.1 General

This module does not have a separate configuration parameter file. Its configuration is determined by rba_EthNdp specific parameters within the configuration of Tcplp, which are located in the TcplpV6General and TcplpV6Config containers. The following Tcplp parameters are relevant for the configuration of rba_EthNdp.

- TcplpNdpDefaultReachableTime: Configuration of the ReachableTime (in secs, default 30) as specified in RFC4861 6.3.2. Host Variables. "The time a

neighbor is considered reachable after receiving a reachability confirmation." If `TcplpNdpDynamicReachableTimeEnabled` is checked, this value may be reconfigured based on received Router Advertisements.

- `TcplpNdpDefaultRefreshTimeStaticRtr`: The refresh time (in secs) of the Default Router for all the Static Tcplp Local IPv6 Address configuration containers.
- `TcplpNdpDefaultRetransTimer`: Configures the default value (in secs, default 1) for the `RetransTimer` variable specified in RFC4861 6.3.2. Host Variables."The time between retransmissions of Neighbor Solicitation messages to a neighbor when resolving the address or when probing the reachability of a neighbor." If `TcplpNdpDynamicRetransTimeEnabled` is checked, this value may be reconfigured based on received Router Advertisements.
- `TcplpNdpMaxNeighborCacheSize`: Maximum number of entries in the neighbor cache, including ON-LINK and OFF-Link hosts. [RFC4861 5.1. Conceptual Data Structures]. If this value is too small, smaller than the maximum number of possible hosts, then the unicast communication to the last hosts added is not possible at all (since the IP cannot be solved).
- `TcplpNdpMaxRandomFactor`: Maximum random factor used for randomization.
- `TcplpNdpMinRandomFactor`: Minimum random factor used for randomization.
- `TcplpNdpNeighborUnreachabilityDetectionEnabled`: See Unsupported Parameters below.
- `TcplpNdpNumMulticastSolicitations`: Maximum number of multicast solicitations (default 3) that will be sent when performing address resolution. [RFC4861 7.2.2. Sending Neighbor Solicitations].
- `TcplpNdpNumUnicastSolicitations`: Maximum number of unicast solicitations (default 3) that will be sent when performing Neighbor Unreachability Detection. [RFC4861 7.3.3. Node Behavior].
- `TcplpNdpPacketQueueEnabled`: See Unsupported Parameters below.
- `TcplpNdpDefaultRouterListSize`: Maximum number of default router entries. [RFC4861 5.1. Conceptual Data Structures].
- `TcplpNdpDestinationCacheSize`: Maximum number of entries in the destination cache. [RFC4861 5.1. Conceptual Data Structures].
- `TcplpNdpDynamicHopLimitEnabled`: See Unsupported Parameters below.
- `TcplpNdpDynamicMtuEnabled`: See Unsupported Parameters below.
- `TcplpNdpMaxRtrSolicitationDelay`: Maximum delay before the first Router Solicitation will be sent after interface initialization.
- `TcplpNdpMaxRtrSolicitations`: Maximum number of Router Solicitations that will be sent before the first Router Advertisement has been received.
- `TcplpNdpPrefixListSize`: Maximum number of entries in the on-link prefix list. [RFC4861 5.1. Conceptual Data Structures].

- TcplpNdpRtrSolicitationInterval: Interval between consecutive Router Solicitations.
- TcplpRbNdpSlaacNeighborAdvertisementRetransTimer: Configures the default value(s) for the RetransTimer variable.
- TcplpNdpAddressResolutionUnreachabilityDetectionEnabled: Enables (TRUE) or disables (FALSE) support of Address Resolution and Neighbor Unreachability Detection via NDP.
- TcplpNdpPrefixAndRouterDiscoveryEnabled: Enables (TRUE) or disables (FALSE) support of Prefix and Router Discovery via NDP.

13.8.2 Unsupported Parameters

The following parameters from the above list are available for configuration, but they should be configured as FALSE as they are not supported in the current implementation of rba_EthNdp:

- TcplpNdpDynamicHopLimitEnabled
- TcplpNdpDynamicMtuEnabled
- TcplpNdpPacketQueueEnabled
- TcplpNdpNeighborUnreachabilityDetectionEnabled

13.8.3 Cache Stored Address

If the cache stored address is being cleared too quickly due to a timeout, you can adjust that timeout with the TcplpNdpDefaultReachableTime parameter.

13.9 Integration Advice

13.9.1 Init Functions

rba_EthNdp is initialised implicitly by the Tcplp module when Tcplp_Init is called, so rba_EthNdp does not require separate initialisation. rba_EthNdp does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthNdp code generation.

13.9.2 Delnit Functions

No specific handling necessary.

13.9.3 Scheduled Functions

There are no scheduled functions.

13.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in

the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthNdp module uses the following OS resources:

- rba_EthNdp_ExclusiveAreaRouterDiscoveryProcess
- rba_EthNdp_ExclusiveAreaNeighborCache
- rba_EthNdp_ExclusiveAreaDefaultReachableTime
- rba_EthNdp_ExclusiveAreaDefaultRetransmitTime
- rba_EthNdp_ExclusiveAreaStaticRouterRefreshTime
- rba_EthNdp_ExclusiveAreaUnsolicitNAInterval

13.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

14 rba_EthTcp

Module name	rba_EthTcp
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

14.1 Introduction

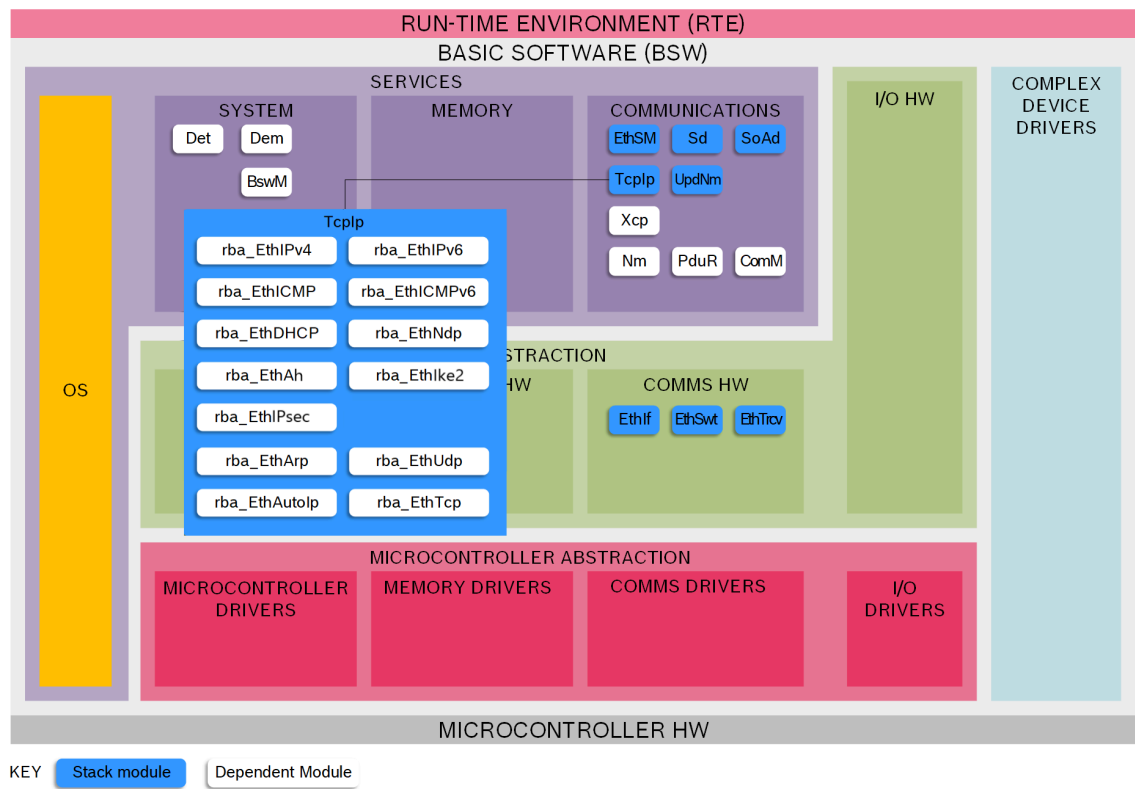


Figure 14.1.RTA-ETH Architecture

The rba_EthTcp module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthTcp in the AUTOSAR layered architecture.

The Tcplp and SoAd modules are designated as the upper layer module with respect to rba_EthTCP, while rba_EthIPv4/rbaEthIPv6 and EthIf are designated as the lower layer modules.

This RTA-BSW specific module provides TCP communication services which are used by the Tcplp module (acting as a wrapper) to provide intermediate level communication between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules. It is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data

transfer.

The specific functionality of rba_EthTcp is to process upper layer TCP communication requests by dividing the data into TCP segments before passing it as a series of IP datagrams to the lower layer.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

14.2 Supported Features

- Socket-based TCP data segmentation and transmission.
- Re-transmission.
- Fragmentation and reassembly.
- KeepAlive probes and timers.
- Sequence number generation
- Get and reset measurement data.
- Selective Acknowledgement mechanism (SACK).

14.3 Extensions

- No known extension parameters are defined in this version.

14.4 Limitations

The following features are not supported for version v12.7.0:

- Although some APIs and Parameters relating to TIs and IPSec are available, these features are not currently supported.

14.5 Exclusions

- There are no known exclusions in this version.

14.6 Deviations

- There are no known deviations in this version.

14.7 API Definition

List of supported API functions

Table 14.2.rba_EthTcp Supported API functions

rba_EthTcp_Init	Initialises the module.
rba_EthTcp_MainFunction	Calls the module to handle the TCP process and send TCP frame to the rba_EthIpv4 submodule.

rba_EthTcp_GetSocket	Requests creation of a new TCP socket connection.
rba_EthTcp_FindTcpSockId	Finds the socket ID for a connection based on the specified local and remote addresses.
rba_EthTcp_ChangeParameter	Requests a change to a specified parameter of the specified socket.
rba_EthTcp_Connect	Requests establishment of a TCP connection to a remote node using a specified socket/IP address and port.
rba_EthTcp_Listen	Requests a listen operation on the specified socket.
rba_EthTcp_Bind	Requests binding of a specified socket to a local resource.
rba_EthTcp_Transmit	Requests transmission of TCP data to a remote node.
rba_EthTcp_Received	Confirms reception of data to the module.
rba_EthTcp_AbortByLocalAddrId	Requests termination of a connection specified by the local address identifier.
rba_EthTcp_GetAndResetMeasurementData	Reads and resets measurement data for diagnostic purposes.
rba_EthTcp_RxIndication	Callback function to notify the module of a received frame and provide the received data.
rba_EthTcp_Close	Requests closure of a specified socket and release of all related resources.
rba_EthTcp_GetSocketInfo	Returns the local address Id and the local port mapped to the socket Id.

14.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

14.8.1 General

Configuration of rba_EthTcp relies on the configuration of the parent Tcplp module. The parameters of the Tcplp module relevant to the rba_EthTcp module

are as follows.

- TcplpRbTcpMaxQSizeForTxReq
- TcplpRbTcpReopenFromLastAck
- TcplpTcpCongestionAvoidanceEnabled
- TcplpTcpDlyAckEnabled
- TcplpTcpDynReTxTimerEnabled
- TcplpTcpDynReTxTimerUpBound
- TcplpTcpFastRecoveryEnabled
- TcplpTcpFastRetransmitEnabled
- TcplpTcpFinWait2Timeout
- TcplpTcpHalfDuplxCloseEnabled
- TcplpTcplsnGenerationMethod
- TcplpTcpKeepAliveEnabled
- TcplpTcpKeepAliveInterval
- TcplpTcpKeepAliveProbesMax
- TcplpTcpKeepAliveTime
- TcplpTcpMSSEnabled
- TcplpTcpMaxRtx
- TcplpTcpMaxTxBuffPerSock
- TcplpTcpMsl
- TcplpTcpNagleEnabled
- TcplpTcpReTxEnabled
- TcplpTcpReceiveWindowMax
- TcplpTcpRetransmissionTimeout
- TcplpTcpSlowStartEnabled
- TcplpTcpSynMaxRtx
- TcplpTcpSynReceivedTimeout
- TcplpTcpTtl
- TcplpTcpUserTimeout
- TcplpTcpUserTimeoutEnabled
- TcplpTcpZWPEnabled
- TcplpTcpZWPMAXRetries

Please refer to the Tcplp module section of this guide for more details on these parameters.

14.8.2 TCP Header Structure

The TCP module accepts data from an upper layer module, divides it into chunks and adds a TCP header to create a TCP segment. The segment is then encapsulated into an Internet Protocol (IP) datagram, and exchanged with a host ECU node. A segment consists of a segment header and a data payload section.

The TCP header contains 10 mandatory fields, and an optional extension field. The data section follows the header, and it contains the payload data carried from the upper layer.

The length of the data section is not specified in the TCP segment header. It can be calculated by subtracting the combined length of the TCP header and the encapsulating IP header from the total IP datagram length (specified in the IP header).

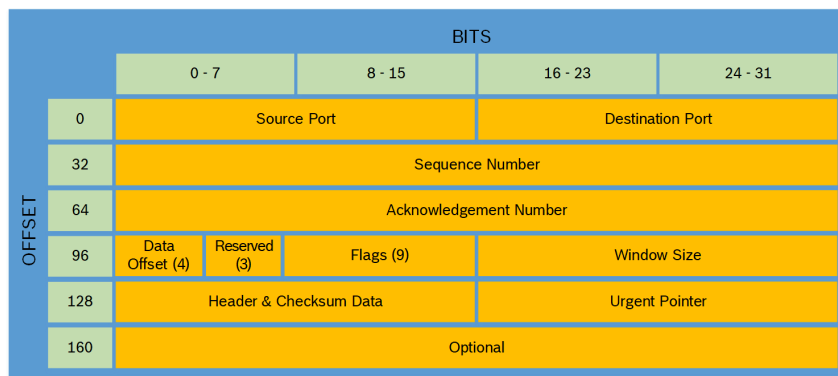


Figure 14.2.TCP Header Structure

14.8.3 Re-transmission

Re-transmission occurs when the source node retains a copy of the data it sent until the recipient acknowledges reception.

- If an acknowledgement is not received within the configured re-transmission timeout (parameter `TcplpTcpRetransmissionTimeout`), the frame is re-transmitted.
- The maximum number of retries is configured via parameter `TcplpTcpMaxRtx`. If no acknowledgement is received and the number of retries is reached, the socket connection is dropped.
- If the parameter `TcplpTcpDynReTxTimerEnabled` is set to `TRUE`, the re-transmission timeout (RTO) will be determined dynamically, otherwise it will be set to the value of `TcplpTcpRetransmissionTimeout`.
- The parameter `TcplpTcpDynReTxTimerUpBound` should be configured as the upper bound of the re-transmission timeout in seconds. If it is not configured, a default value of 10s is used. If `TcplpTcpDynReTxTimerEnabled` is set to `TRUE`, `TcplpTcpDynReTxTimerUpBound` cannot be smaller than `TcplpTcpRetransmissionTimeout` or `TcplpTcpSynReceivedTimeout`, and

must be less than 60.

14.8.4 User Timeout

If the parameter `TcplpTcpUserTimeout` is configured with a value greater than 0, the user timer option will be activated and set to the configured value, otherwise the user timer option will be deactivated.

14.8.5 KeepAlive Timers

The KeepAlive process uses a series of timers to maintain a socket connection for a configured period of time after the last transmission of a data packet. The following parameters control the behaviour of the KeepAlive probe operation.

- `TcplpTcpKeepAliveEnabled`
- `TcplpTcpKeepAliveInterval`
- `TcplpTcpKeepAliveProbesMax`
- `TcplpTcpKeepAliveTime`

14.8.6 Sequence Number Generation

An RTA-BSW specific parameter `TcplpTcplsnGenerationMethod` has been added in container `TcplpTcpConfig`, which allows the setting of the method used to generate initial sequence numbers. This can be set to the following values.

- `PSEUDO_RANDOM` (default) - Pseudo Random method.
- `CLOCK_DRIVEN` - Clock Driven method.
- `CLOCK_HASH_BASED` - Clock Driven + Hash Based method.

When considering the method to use, the `PSEUDO-RANDOM` and `CLOCK_DRIVEN` methods are more resource-efficient, but the `CLOCK_HASH_BASED` method is more secure as it uses an MD4 hashing algorithm when generating the initial sequence number.

14.8.7 Maximum Segment Size

The maximum segment size (MSS) is a parameter of the TCP protocol that specifies the largest amount of data, specified in octets, that a host node can receive in a single TCP segment.

If this option is enabled (`TcplpTcpMSSEnabled = TRUE`), then it communicates the maximum receive segment size to the node which transmits this segment. This field must only be sent in the initial connection request (i.e. in segments with the SYN flag set). It does not count the TCP header or the IP header.

The MSS value required to avoid fragmentation is equal to the largest datagram size that any host is required to be able to reassemble, minus the IP header size and TCP header size. Therefore IPv4 host nodes are required to be able to handle an MSS of 536 octets ($= 576 - 20 - 20$). A lower MSS will ensure that fragmentation will never occur along the path but will result in higher overheads. The MSS

value is exchanged by host nodes during the establishment of a connection. Each direction of data flow can use a different MSS. The relationship between the MSS value and the MTU (Maximum Transmission Unit) is $MSS = MTU - (TCP \text{ Header Size} + IP \text{ Header Size})$.

14.8.8 Checksum Calculation

Hardware checksum calculation is supported from AR v4.2.2 via the `EthCtrlEnableOffloadChecksumTCP` configuration parameter (in `Eth/EthGeneral/EthCtrlOf-floading`). This parameter is set to `FALSE` by default, which means that checksum calculation is handled by `rba_EthTcp` rather than the Ethernet Driver. If Ethernet Driver v4.0.3 is used, it does not support protocol checksum calculation, so `EthCtrlEnableOffloadChecksumTCP` should be set as `FALSE`.

14.9 Integration Advice

14.9.1 Init Functions

`rba_EthTcp` is initialised implicitly by the `Tcplp` module when `Tcplp_Init` is called, so `rba_EthTcp` does not require separate initialisation.

14.9.2 Delnit Functions

No `Delnit` is required.

14.9.3 General

`rba_EthTcp` does not have its own `EcucParamDef` file, and is fully dependent on the `Tcplp` module. `Tcplp_EcucParamDef.arxml` is required for `rba_EthTcp` code generation.

14.9.4 Scheduled Functions

`rba_EthTcp_MainFunction` requires scheduling for cyclic operation. This function loops through all open sockets, assembles any required TCP segments and forwards the resulting frames to the `rba_EthIPv4/rba_EthIPv6` module.

14.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The `Rba_EthTcp` module uses the following OS resources:

- `rba_EthTcp_TxBuf(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)`

- rba_EthTcp_TxOngoing(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)
- rba_EthTcp_RxOngoing(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)
- rba_EthTcp_MutualExclusiveClose(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)
- rba_EthTcp_SetNextSendFrame(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)
- rba_EthTcp_ExclusiveAreaLocalAddrInUse
- rba_EthTcp_RxSackOption(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)
- rba_EthTcp_TxSackOption(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)
- rba_EthTcp_Tmr(Tcplp_SocketIdTypeTcpDynSockTblIdx_uo)

14.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

15 rba_EthUdp

Module name	rba_EthUdp
Package	RTA-ETH
AUTOSAR Module ID	N/A
AUTOSAR Version	22-11

15.1 Introduction

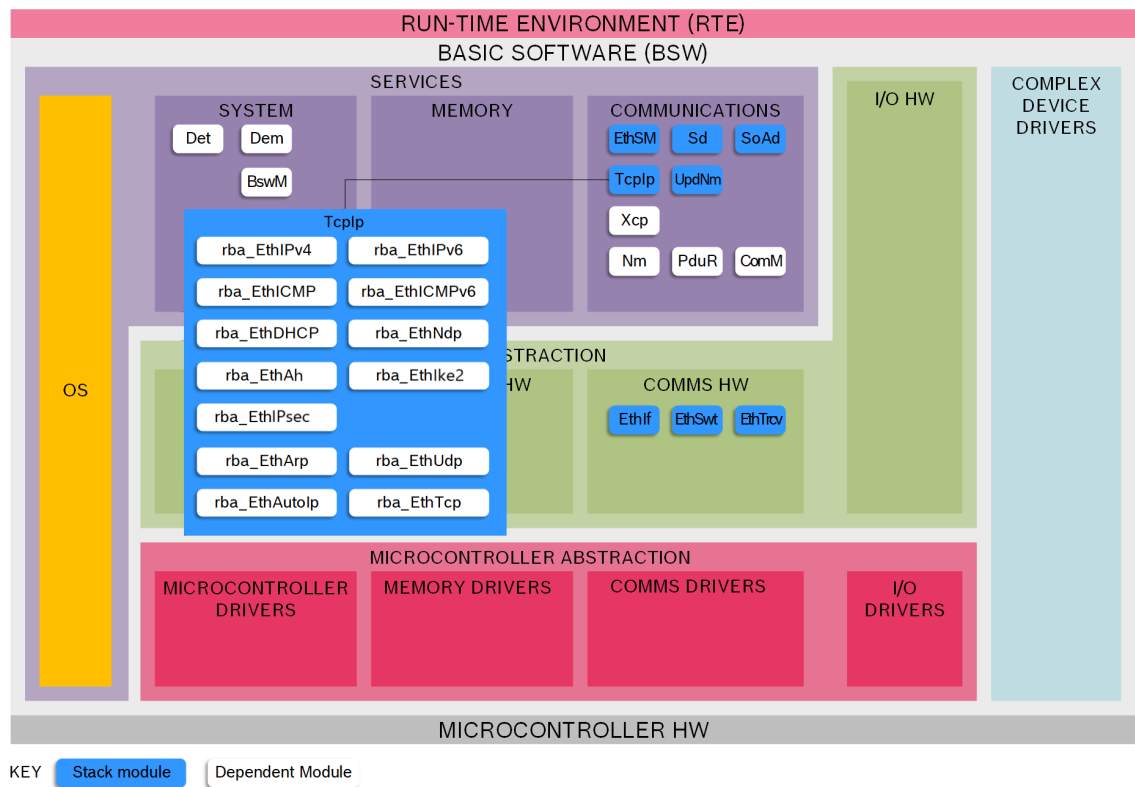


Figure 15.1.RTA-ETH Architecture

The rba_EthUdp module is located in the communication services abstraction layer, and is considered (in the RTA-ETH stack) as a sub-module of the Tcplp module. The diagram above shows the position of rba_EthUdp in the AUTOSAR layered architecture.

The modules rba_EthIPv4/rba_EthIPv6 and rba_EthICMP/rba_EthICMPv6 are designated as the lower layer modules with respect to rba_EthUdp, while Tcplp is designated as the upper layer module.

This RTA-BSW specific module provides UDP protocol services which are used by the Tcplp module to provide UDP data communication between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules.

rba_EthUdp is a member of the group of sub-modules encapsulated by Tcplp which together form the 'protocol stack' for TCP/IP messaging and data transfer.

The specific functionality of the rba_EthUdp module is to control transmission and reception of UDP datagrams between source and destination hosts. The UDP protocol is generally used for applications which require only a minimal protocol for network communication without the need for features such as error correction, guaranteed delivery or duplicate protection.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

15.2 Supported Features

- UDP datagram transmission/reception over IPv4/IPv6.
- UDP socket handling (allocate, bind, close).
- Read/reset measurement data.

15.2.1 Transmission (via the AR Interface)

For the transmission of UDP frames, rba_EthUdp implements the UDP protocol functionalities, with the Tcplp module acting as an interface to the upper layer.

Tcplp forwards the upper layer request for frame transmission to rba_EthUdp via rba_EthUdp_Transmit, with MetaDataPtr as valid pointer or NULL pointer.

The following actions are then performed:

- rba_EthUdp first checks the destination IP address for 0.0.0.0 and whether the socket is in a valid state of transmission (if it's not, the transmission is aborted).
- rba_EthUdp adds the UDP header length to the requested buffer length and forwards the request to the rba_EthIPv4 or rba_EthIPv6 module by calling rba_EthIPv4_ProvideTxBuffer or rba_EthIPv6_ProvideTxBuffer.
- If data is provided by the Upper layer as a pointer to a linear buffer, rba_EthUdp copies data into the allocated lower layer buffer by calling rba_BswSrv_MemCopy. Otherwise, data is retrieved from configurable upper layer via the configured callback Up_CopyTxData.
- The UDP header is then filled (Source and destination port, UDP frame length, checksum field). The UDP checksum is initially set to zero, as this will be filled by rba_EthIPv4/rba_EthIPv6 if the hardware checksum offload is disabled for the target EthIf controller or the frame to be transmitted undergoes fragmentation. If the hardware checksum offload is enabled, and the frame does not undergo fragmentation, the checksum calculation is performed in hardware.
- UDP software calculation by the upper layer can be disabled via Tcplp_ChangeParameter (TCPIP_PARAMID_UDP_CHECKSUM as STD_OFF).
- For the transmission of UDP frames with the IP address family as IPv4, the checksum field will be zero if the UDP software calculation is disabled as

above.

- Unlike IPv4, for the transmission of UDP frames with IP address family as IPv6, the UDP checksum is not optional. Therefore, calling Tcplp_ChangeParameter with param id TCPIP_PARAMID_UDP_CHECKSUM will have no influence for IPv6 sockets.
- After the UDP header data is filled, rba_EthIPv4_Transmit or rba_EthIPv6_Transmit is triggered to perform the transmission.

15.2.2 Transmission (via the ZeroCopy Interface)

rba_EthUdp performs the following actions for the successful transmission of an Ethernet frame using the ZeroCopy interface.

15.2.2.1 Header Filling

Tcplp forwards the filling of the ZeroCopy header buffer request from the upper layer to rba_EthUdp module via rba_EthUdp_ZeroCopyFillHeader.

Tcplp provides a ZeroCopy argument containing at least one header buffer of 36 bytes if the transmission is in Dynamic Mode or 46 bytes if the transmission is in Static Mode (Reuse header buffer).

- In Dynamic Mode, rba_EthUdp writes the static UDP header fields from index 32. The static UDP fields are those which do not change from one transmission to another for the same socket (Source Port and Destination Port).
- In Static Mode, rba_EthUdp writes the static UDP header fields from index 38. The static UDP fields are those which do not change from one transmission to another for the same socket (Source Port, Destination Port, Length and Checksum).

rba_EthUdp requests the Reuse header filling to the lower layer by calling rba_EthIPv4_ZeroCopyFillHeader. If the Reuse header is successfully filled, rba_EthUdp returns E_OK and the Index of the UDP socket owner to the caller.

15.2.2.2 ZeroCopy Transmission

Tcplp calls rba_EthUdp_ZeroCopyTransmit to request the transmission of a ZeroCopy Ethernet frame. Tcplp reuses the same ZeroCopy argument as the one provided previously in the call of EthIf_ZeroCopyFillHeader.

The upper layer additionally fills the ZeroCopy argument with the data buffer (containing the payload of the Ethernet frame) and the desired number of bytes to be transmitted. The TxConf and the TriggerTx flags are also set by Tcplp in the ZeroCopy argument. These flags will be used by the Eth module to know if TxConfirmation callback need to be invoked and if the transmission need to be placed on the bus directly.

The following actions are performed by rba_EthUdp:

- The destination IP address is checked for 0.0.0.0 and whether the socket is in a valid state of transmission. If not, the transmission is aborted.

- Only in Dynamic Mode are the dynamic parts of the UDP header filled in to the provided UDP OneTime header buffer in the ZeroCopy argument, which is 4 bytes long. The UDP frame length and Checksum field are filled.
- The UDP checksum is filled by the `rba_EthUdp_ZeroCopyChkSumCalculation` function if the hardware checksum offload is disabled or the frame to be transmitted undergoes fragmentation. If hardware checksum offload is enabled for the UDP and frame does not undergo fragmentation, hardware will do the checksum offload (in which case `rba_EthUdp` fills the checksum with 0).
- You should fill `rba_EthUdp_ZeroCopyChkSumCalculation` with a runtime efficient checksum computation algorithm.
- After the UDP header data is filled, `rba_EthIPv4_ZeroCopyTransmit` or `rba_EthIPv6_ZeroCopyTransmit` is triggered.

15.2.3 Reception

The `rba_EthIPv4_RxIndication/rba_EthIPv6_RxIndication` callback functions of `rba_EthIPv4/rba_EthIPv6` trigger `rba_EthUdp_RxIndication` if the received frame is a UDP frame.

On reception, the UDP datagram length is validated. The received length should be greater than or equal to the datagram length in the UDP header, and the datagram length should be greater than UDP header length.

The UDP datagram checksum is validated. If the checksum is not valid and the received frame is unicast, an ICMP Port Unreachable message is transmitted to the remote node that sent the UDP frame.

For the reception of UDP packets from an IPv4 node, UDP accepts datagrams containing a zero checksum (checksum was optionally disabled by the sender) only on sockets that have been configured with `TCPIP_PARAMID_UDP_CHECKSUM` set to `FALSE`. For the reception of UDP packets from an IPv6 node, the UDP checksum is not optional, so a UDP frame containing a checksum field equal to 0 is discarded for the IPv6 path.

Depending on the destination IP address and destination port in the received frame, the UDP socket table is checked for an open socket connection. If an open socket is found, the frame is forwarded to the upper layer owning the socket (`SoAd`, `rba_DHCP` or `CDD`).

If the received frame is broadcast or multicast, the frame is forwarded to all matching sockets. For broadcast frames, the `EthIf` controller index of the socket and the `EthIf` controller index on which the frame was received should match. If no open socket is found, an ICMP port unreachable message is transmitted to the remote node which sent the UDP frame.

15.2.4 Socket Handling

`rba_EthUdp` provides APIs to allocate, bind and close UDPs socket, as well as change the parameters of a socket.

Whenever a new UDP socket is required for communication, the upper layer

(SoAd or CDD) requests the Tcplp module to allocate a new socket.

Tcplp calls `rba_EthUdp_GetSocket`, which first checks for the availability of a free socket. If a free socket is available, it is reserved by setting the socket state to `RBA_ETHUDP SOCK_OPEN`, otherwise a return value `E_NOT_OK` is generated. The domain, the protocol type and the upper layer owning the socket is stored in a dynamic UDP socket table.

(Note: `rba_EthDHCP` uses UDP, but only via the Tcplp module. DHCP is treated specially though as `rba_EthUdp` does keep track of which sockets are UDP).

After allocation of a new socket, Tcplp or `rba_EthDHCP` requests the binding of the socket to a `LocalAddrId` mapped to an `EthIf` controller by calling `rba_EthUdp_Bind`. In the dynamic UDP socket table, the local port, local address ID and frame priority configured for the corresponding Tcplp controller are updated and the socket state set to `RBA_ETHUDP SOCK_BOUND`.

If a UDP socket has to be closed, the Tcplp module calls `rba_EthUdp_Close`, which resets the socket and releases all related resources. The upper layer module owning the socket is informed about the closure via the configured call-back function, and the state of the corresponding socket is set to `TCPIP_UDP_CLOSED`.

`rba_EthUdp` allows the closure of all UDP sockets (except those owned by `rba_EthDHCP`) through the `rba_EthUdp_TerminateSocUsingLocalAddr` function. If the input parameter is a valid local address ID, then all UDP sockets mapped to the local address ID are closed. If the input parameter is `TCPIP_LOCALADDRID_INVALID`, all UDP sockets are closed.

`rba_EthUdp` allows the changing of socket parameters during run time (e.g. Frame Priority and Flow Label). Tcplp can request changing the parameters by calling `rba_EthUdp_ChangeParameter``.



NOTE

The Socket Id used in the `rb_EthUdp` and Tcplp modules are different. The UDP socket Id range is from 0 to `TcplpUdpSocketMax`. Tcplp converts the socket Id provided by `rba_EthUdp` into a unique Tcplp socket Id by adding an offset, hence calls to API functions which address sockets in each module cannot be mixed.

15.2.5 Change Parameter

`rba_EthUdp` allows the changing of socket parameters during run time. Tcplp can request the changing of a parameter through `rba_EthUdp_ChangeParameter`.

If requested by the upper layer, `rba_EthUdp` can change the frame priority value, the flow label value, the DSCP value (if it is valid) and the socket checksum enabled value that controls whether the software checksum calculation/verification is skipped or calculated for a UDP frame.

15.2.6 Measurement Data

rba_EthUdp supports a configurable Measurement Data feature for diagnostic information, such as a count of dropped datagrams which cannot be mapped to a valid local IP/Port.

15.2.7 Security Events

rba_EthUdp supports calls to the ldsM module to report security events (e.g. dropped datagrams which cannot be mapped to a valid local IP/Port).

15.3 Extensions

- No known extension parameters are defined in this version.

15.3.1 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

15.3.1.1 rba_EthUdp_GetSocketInfo

Syntax	<code>rba_EthUdp_GetSocketInfo(TcpIp_SocketIdType SocketId_u16, TcpIp_LocalAddrIdType * LocalAddrId_pu8, uint16 * LocalPort_pu16)</code>
Parameters In	SocketId_u16 Socket identifier of the related local socket resource.
Parameters Out	LocalAddrId_pu8 Local Addr Id to which the socket is bound. LocalPort_pu16 Local port of the socket.
Description	Returns the Local Addr Id and local port mapped to the socket Id.

15.3.1.2 rba_EthUdp_CheckFragmentation

Syntax	<code>rba_EthUdp_CheckFragmentation(TcpIp_SocketIdType SocketId_u16, uint16 DataLength_u16, uint8 * NumberOfIpFragments_pu8)</code>
Parameters In	SocketId_u16 Socket id on which data is to be transmitted. LDataLength_u16 Length of data that will be request for transmission.
Parameters Out	NumberOfIpFragments_pu8 The number of IP fragments which will be formed by IP layer during transmission.
Description	Returns the number of IP fragments if fragmentation will occur.

15.3.1.3 rba_EthUdp_ZeroCopyFillHeader

Syntax	<code>rba_EthUdp_ZeroCopyFillHeader (TcpIp_SocketIdType SocketId_u16, const TcpIp_SockAddrType * RemoteAddr_pcst, const Eth_ZeroCopyArg_tst * ZeroCopyArg_pcst, uint8 * UL_SockOwnerIdx_pu8)</code>
--------	---

Parameters In	SocketId_u16 Socket id on which data is to be transmitted. RemoteAddr_pcst Pointer to a structure containing remote IP address and remote port. ZeroCopyArg_pcst ZeroCopy argument containing the allocated headers and the pointer to the UL data.
Parameters Out	UL_SockOwnerIdx_pu8 Index of the Udp socket owner.
Description	Prefills the reusable part of the Udp header.

15.3.1.4 rba_EthUdp_ZeroCopyTransmit

Syntax	<code>rba_EthUdp_ZeroCopyTransmit (TcpIp_SocketIdType SocketId_u16, const TcpIp_SockAddrType * RemoteAddr_pcst, const Eth_ZeroCopyArg_tst * ZeroCopyArg_pcst, uint32 * TxId_pu32)</code>
Parameters In	SocketId_u16 Socket id on which data is to be transmitted. RemoteAddr_pcst Pointer to a structure containing remote IP address and remote port. ZeroCopyArg_pcst ZeroCopy argument containing the allocated headers and the pointer to the UL data.
Parameters Out	TxId_pu32 Unique identifier of the ZeroCopy transmission, valid only if ZeroCopyArg_pst→TxConfFlag_b is TRUE.
Description	Executes the ZeroCopy transmission.

15.4 Limitations

- Changing TTL (time-to-live) parameter values per UDP socket using TcpIp_ChangeParameter is not supported.
- If the configuration used in the EthIf component implies that RxIndication path will be reentered, then the Udp Socket Owner must implement a reentrant RxIndication callback. The following cases are examples of EthIf configuration which implies reentrancy in RxIndication path:
 - Multiple Ingress Fifos are configured in one or more EthDrivers whose traffic will be destined to Tcplp/rba_EthUdp and the Fifos are polled by EthIf_MainFunctionRx_{PriorityProcessing ShortName} which are invoked from different tasks or from different cores.
 - Multiple EthDrivers configured in interrupt mode serving different interrupt vectors.
- TxConfirmation callbacks are always immediate in SoAd for the Udp path.
- 3rd party DTLS shall only use the first element, currently the usage of two buffers i.e. two calls to CopyTxData not supported by SoAd.
- EthIf does not support MacSec per hardware with Switch and MacSec per software.

15.5 Exclusions

- There are no known exclusions in this version.

15.6 Deviations

- There are no known deviations in this version.

15.7 API Definition

Table 15.6.rba_EthUdp Supported API functions

rba_EthUdp_Init	Initialises the module.
rba_EthUdp_MainFunction	rba_EthUdp Main Function.
rba_EthUdp_Transmit	Called from Tcplp module to request transmission of the payload data in the specified buffer to a specified socket and destination address.
rba_EthUdp_GetSocket	Requests allocation of a new UDP socket.
rba_EthUdp_Bind	Requests binding of a UDP socket to a local resource represented by the specified IP address.
rba_EthUdp_Close	Requests closure of a specified UDP socket.
rba_EthUdp_ChangeParameter	Requests a change of parameter for a specified socket.
rba_EthUdp_TerminateSocUsingLocalAddr	Closes all open UDP sockets or sockets mapped to the specified local address ID.
rba_EthUdp_RxIndication	Callback function used by lower layer modules to indicate reception of a UDP datagram.
rba_EthUdp_GetAndResetMeasurementData	Allows reading or resetting of measurement data for diagnostic purposes.

15.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

The module does not have a separate configuration parameter file. The Tcplp configuration parameters in containers TcplpGeneral and TcplpConfig/TcplpUdp-Config are used to configure the rba_EthUdp module. The following Tcplp parameters are relevant.

- TcplpUdpSocketMax - Maximum number of UDP sockets. Setting this to zero value implicitly disables the rba_EthUdp module.
- TcplpUdpTtl - Default time-to-live value of outgoing UDP packets.
- TcplpRbUdpMulticastAddrReuse - Enables/disables sockets to reuse the same local address and port in case of an IP address of type TCPIP_MULTICAST.

15.8.1 UDP Header Structure

A UDP datagram is carried in a single IP packet and is hence limited to a maximum payload of 65,507 bytes for IPv4. The UDP datagram header structure is formatted as follows.

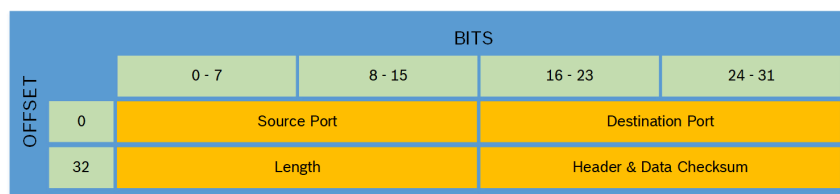


Figure 15.2.UDP Header Structure

15.8.2 Checksum Calculation

Hardware checksum calculation is supported from AR v4.2.2 via the EthCtrlEnableOffloadChecksumUDP configuration parameter (in Eth/EthGeneral/EthCtrlOffloading). This parameter is set to FALSE by default, which means that checksum calculation is handled by rba_EthUdp rather than the Ethernet Driver. If Ethernet Driver v4.0.3 is used, it does not support protocol checksum calculation, so EthCtrlEnableOffloadChecksumUDP should be set as FALSE.

15.8.3 Security Events

During reception, rba_EthUdp can report security events to the IdsM module if the corresponding event is configured in the configuration container TcplpSecurityEventRefs and if Tcplp/TcplpGeneral/TcplpEnableSecurityEventReporting is set to TRUE.

If the received frame is dropped and the related security event is configured, rba_EthUdp calls the API IdsM_SetSecurityEventWithContextData. In AUTOSAR v22-11, context data is not specified, so the contextData argument is set to NULL_PTR.

15.9 Integration Advice

15.9.1 Init Functions

rba_EthUdp is initialised implicitly by the Tcplp module when Tcplp_Init is called, so rba_EthUdp does not require separate initialisation.

15.9.2 Delnit Functions

No Delnit is required.

15.9.3 General

rba_EthUdp does not have its own EcucParamDef file, and is fully dependent on the Tcplp module. Tcplp_EcucParamDef.arxml is required for rba_EthUdp code generation.

15.9.4 Scheduled Functions

rba_EthUdp_MainFunction is triggered by Tcplp_MainFunction, so only the scheduling of Tcplp_Mainfunction is required, which should be scheduled cyclically.

15.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Rba_EthUdp module uses the following OS resources:

- rba_EthUdp_ExclusiveAreaSockState
- rba_EthUdp_ExclusiveAreaUsedSocCnt
- rba_EthUdp_ExclusiveAreaOngoingOperation
- rba_EthUdp_ExclusiveAreaLocalAddrInUse
- rba_EthUdp_ExclusiveAreaPortAny
- rba_EthUdp_ExclusiveAreaSockFramePrio
- rba_EthUdp_ExclusiveAreaMeasurementData

15.10 Hardware Requirements

Information relating to the hardware resources required by this module is not

currently available.

16 Service Discovery

Module name	Sd
Package	RTA-ETH
AUTOSAR Module ID	171
AUTOSAR Version	22-11

16.1 Introduction

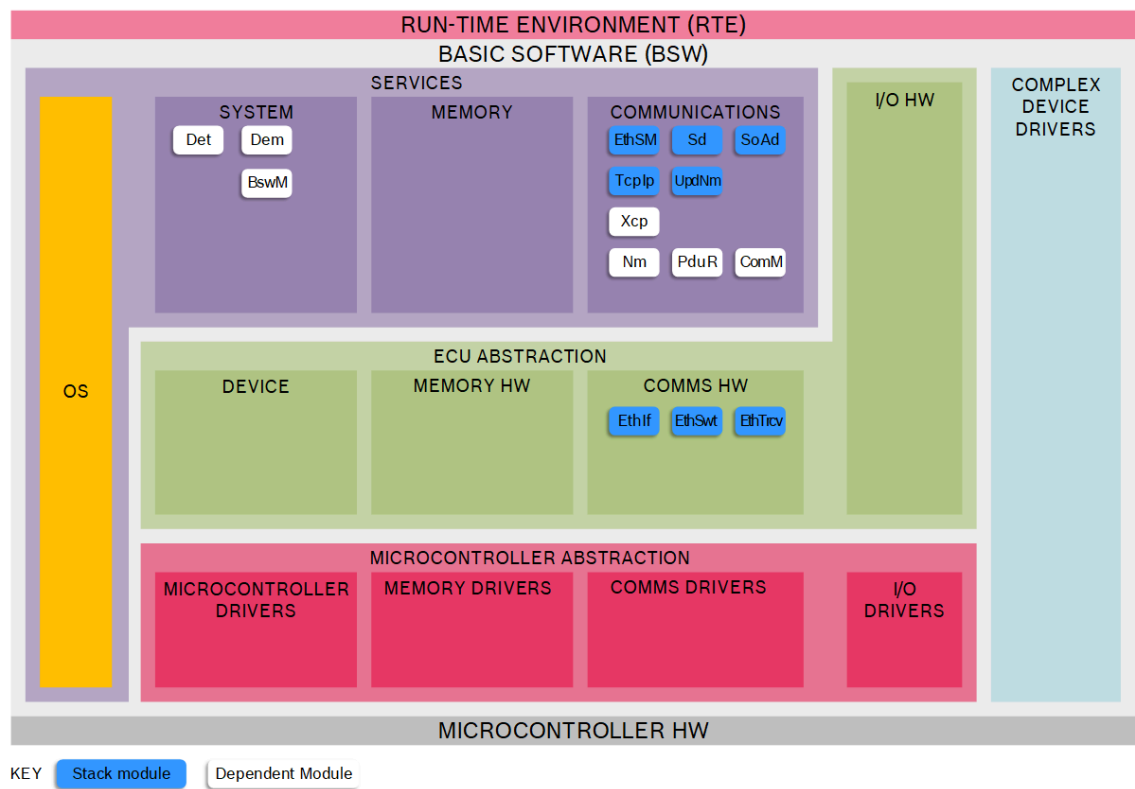


Figure 16.1. RTA-ETH Architecture

The Service Discovery (Sd) module is an upper layer module of the communication services layer. The diagram above shows the position of the module in the AUTOSAR layered architecture.

The Sd module provides a mechanism by which services within a vehicle network may be offered and/or detected. In this context, a “service” is defined as a functional entity provided by an ECU which contains an addressable interface.

The Sd module also processes events, which are defined as messages and which signal the availability and status of a service.

ECUs are classed as either server or client, depending on the context in which they make use of services. A service offered by an ECU is an instance of a server service with respect to the ECU hosting the service, whereas an ECU which consumes a service hosted by another ECU is considered to be a client of that

service instance. Therefore, an ECU must potentially deal with both client and server service instances, depending on which services it offers and/or consumes.

16.1.1 Service Discovery example

This diagram shows the initial “service discovery” phases that can be implemented by using the Sd module in multiple ECUs:

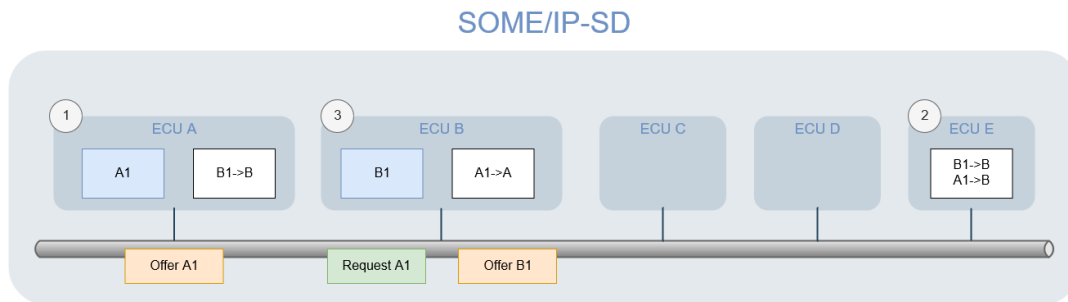


Figure 16.2. Service Discovery Example

ECU A starts (1) and sends offers for the service A1 in multicast. At this point, no ECU is listening to this Service Offer. ECU E then starts (2), and then ECU B starts (3) and sends offers for the service B1 in multicast. The following actions then occur:

- ECU A registers the B1→B mapping
- ECU E registers the B1→B mapping
- ECU B sends a request for the service A1 in multicast
- ECU A re-sends offers for the service A1 in multicast
- ECU B registers the A1→A mapping
- ECU E registers the A1→A mapping

In this example, ECU A is a Server, ECU B is acting as both a Client and a Server, and ECU E is just a Client.

16.1.2 Client/Server Communication example

The following diagram shows an example exchange of messages between a Client and Server using the SOME/IP/SD protocol. This example is not exhaustive as it is just based on a fire&forget RPC call and on an event dispatch. Note that the order of the messages is highly dependent on the timings configured in the Sd module, and the Stop messages are not reported in this example.

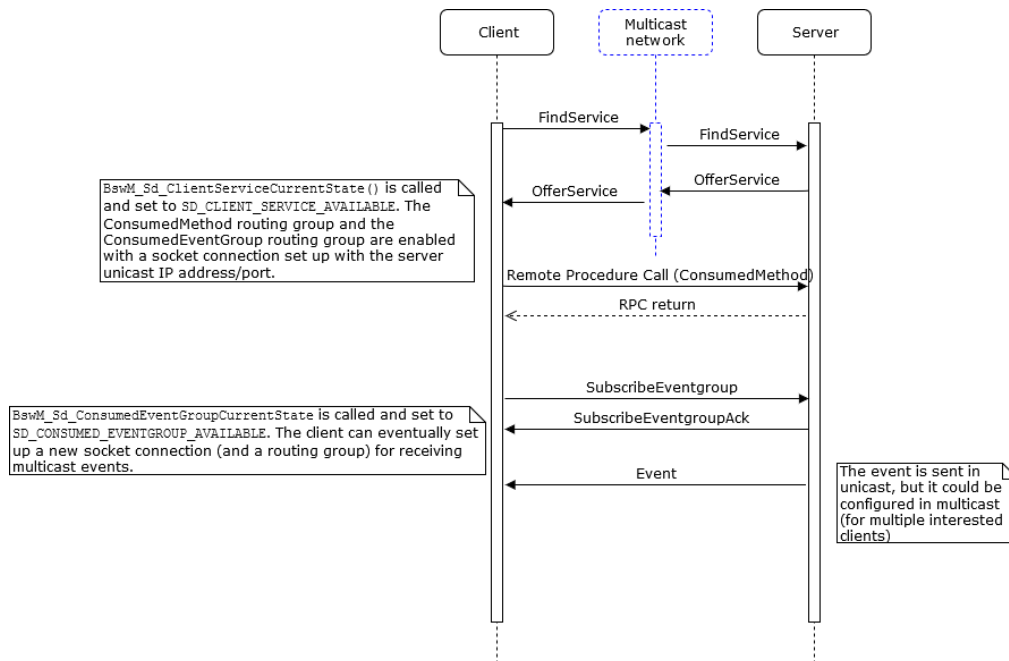


Figure 16.3.Sd Messages Example

1. The Client sends a **FindService** message requesting for a Service instance in multicast.
2. The Server receives the multicasted **FindService** message and replies with a corresponding **OfferService** message containing the service address and port for enabling unicast communication with the Client.
3. After the Client receives the **OfferService** message, the following routing groups specified in the Sd configuration are enabled with an open socket connection to the unicast address specified in the **OfferService** message:
 - `SdClientServiceActivationRef`: for performing Remote Procedure Calls (`ConsumedMethod`)
 - `SdConsumedEventGroupUdpActivationRef`: for receiving server Events (`ConsumedEventGroup`)
4. The Client then performs a Remote Procedure Call on the Server Provided-Method (using unicast, on the socket connection opened in step 3). The RPC could be fire & forget, or it could have a return value.
5. The Client requests the subscription to a specific Eventgroup with the **SubscribeEventgroup** message.
6. The Server acknowledges the subscription with the **SubscribeEventgroupAck** message.
7. The Server sends an **Event** to the subscribed Client (this could be in unicast or multicast, depending on the configuration).

16.1.3 Registration/Deregistration example (with multiple clients)

The following diagram shows the registration/deregistration behaviour of multiple Clients to one Server. This is only a publisher/subscriber use case (i.e. without the RPC calls shown in the previous example).

(Note: This diagram can be found in the Autosar document AUTOSAR_TR_-SomelpExample_4.2.1.pdf)

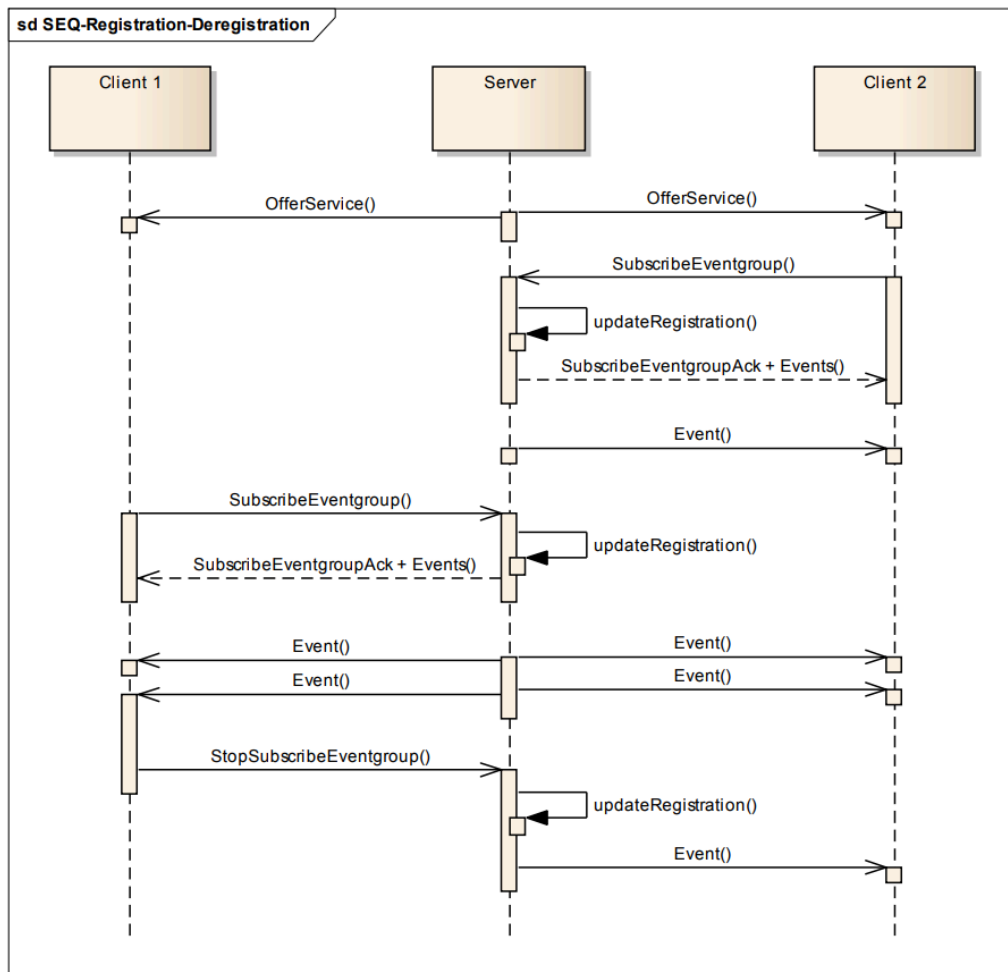


Figure 16.4.Registration/Deregistration Example

16.1.4 Interaction with other Modules

The Sd module primarily interacts with the BSW Mode Manager (BswM) module, which provides the link between generic mode requests and service requests, and the Socket Adapter (SoAd) module, which provides communication between the Sd module and network devices which use the RTA-ETH stack. The messages processed by the Sd module rely on the IP protocol, and also the SOME/IP-SD message protocol.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

16.2 Supported Features

- Server and client service instances.
- Message types Find, Offer, Stop Offer, Subscribe, Stop Subscribe and Event.
- Event groups.
- IPv4 and IPv6 addressing.
- TCP and UDP protocols.
- Retry Subscriptions.
- Management of Publish/Subscribe relationships.

16.3 Extensions

16.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

16.3.1.1 Sd/SdConfig/SdAclCheckBlockDescriptorRef

Short Name	SdAclCheckBlockDescriptorRef
Description	Reference to the Nvm block description in the Nvm module configuration in which the Acl will be stored. Tags: atp.Status=draft
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

16.3.1.2 Sd/SdConfig/SdInstance/SdClientService/SdClientServiceAllowedProvider

Short Name	SdClientServiceAllowedProvider
Description	The container defines the allowed providers for this ClientService. Tags: atp.Status=draft
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

16.3.1.3 Sd/SdConfig/SdInstance/SdClientService/SdClientServiceAllowedProvider/SdIpAddress

Short Name	SdIpAddress
Description	This parameter defines the IP Address of the remote communication partner. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

16.3.1.4 Sd/SdConfig/SdInstance/SdClientService/SdClientServiceAllowedProvider/SdIpAddressType

Short Name	SdIpAddressType
Description	This parameter defines the IP version that is used for communication with the remote communication partner. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	-
Enumeration Literals	SD_AF_INET, SD_AF_INET6
Type	ECUC-ENUMERATION-PARAM-DEF

16.3.1.5 Sd/SdConfig/SdInstance/SdClientService/SdDisableBackgroundServiceDiscoveryTraffic

Short Name	SdDisableBackgroundServiceDiscoveryTraffic
Description	ETAS (Vendor11) specific parameter: If configured to True Sd module will cease the transmission and reception of Sd messages during non-learning phase for this service.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.6 Sd/SdConfig/SdInstance/SdClientService/SdMaxNumOfIpAddressesInAcl

Short Name	SdMaxNumOfIpAddressesInAcl
Description	The maximum number of IP addresses to be saved in the ACL.
Multiplicity	0..1
Value Range	1..255

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.7 Sd/SdConfig/SdInstance/SdInstanceMulticastRxPdu/SdRxSoConRef

Short Name	SdRxSoConRef
Description	ETAS (Vendor11) specific parameter: Reference to the Multicast Rx Socket-Connection.
Introduction	Note: This parameter is required for SD over PduR design.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

16.3.1.8 Sd/SdConfig/SdInstance/SdInstanceTxPdu/SdTxSoConRef

Short Name	SdTxSoConRef
Description	ETAS (Vendor11) specific parameter: Reference to the Tx SocketConnection.
Introduction	Note: This parameter is required for SD over PduR design.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

16.3.1.9 Sd/SdConfig/SdInstance/SdInstanceUnicastRxPdu/SdRxSoConRef

Short Name	SdRxSoConRef
Description	ETAS (Vendor11) specific parameter: Reference to the Unicast Rx Socket-Connection.
Introduction	Note: This parameter is required for SD over PduR design.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

16.3.1.10 Sd/SdConfig/SdInstance/SdServerService/SdDisableBackgroundServiceDiscoveryTraffic

Short Name	SdDisableBackgroundServiceDiscoveryTraffic
Description	ETAS (Vendor11) specific parameter: If configured to True Sd module will cease the transmission and reception of Sd messages during non-learning phase for this service.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.11 Sd/SdConfig/SdInstance/SdServerService/SdMaxNumOfIpAddressesInAcl

Short Name	SdMaxNumOfIpAddressesInAcl
Description	The maximum number of IP addresses to be saved in the ACL.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.12 Sd/SdConfig/SdInstance/SdServerService/SdServerServiceAllowedConsumers

Short Name	SdServerServiceAllowedConsumers
Description	This container defines a list of consumers that are allowed to access this SdServerService. Tags: atp.Status=draft
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

16.3.1.13 Sd/SdConfig/SdInstance/SdServerService/SdServerServiceAllowedConsumers/SdIpAddress

Short Name	SdIpAddress
Description	This parameter defines the IP Address of the remote communication partner. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

16.3.1.14 Sd/SdConfig/SdInstance/SdServerService/SdServerServiceAllowedConsumers/SdIpAddressType

Short Name	SdIpAddressType
------------	-----------------

Description	This parameter defines the IP version that is used for communication with the remote communication partner. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	-
Enumeration Literals	SD_AF_INET, SD_AF_INET6
Type	ECUC-ENUMERATION-PARAM-DEF

16.3.1.15 Sd/SdGeneral/SdAcICheckEnableApi

Short Name	SdAcICheckEnableApi
Description	ETAS (Vendor11) specific parameter: Enables (TRUE) or Disables (FALSE) the Sd_AcICheckEnable Api.
Introduction	This parameter is to enable or disable Sd_AcICheck-Enable Api, When both SdEnableAcIPolicyCheck and SdAcICheckEnableApi parameters are configured to False ACL functionality will be disabled completely to reduce the memory usage and runtime.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.16 Sd/SdGeneral/SdAdditionalEntries

Short Name	SdAdditionalEntries
Description	ETAS (Vendor11) specific parameter: This parameter indicates the number of SubscribeEventGroupNack entries that can be transmitted in one main function cycle when the Server Services are Down or Services/EventGroups are unconfigured. Note: This parameter indicates the extra memory that will be reserved for these irregular entries.
Introduction	If configured to zero, SubscribeEventGroupNack will be not be transmitted for unconfigured eventgroup or client services which are not running. Integrator should estimate, how many such "invalid" subscriptions could be received during one Sd main function cycle and configure it accordingly. Too high value would increase memory consumption of Sd module.
Multiplicity	0..1
Value Range	0..255
Default Value	0

Type	ECUC-INTEGER-PARAM-DEF
------	------------------------

16.3.1.17 Sd/SdGeneral/SdAdditionalOfferServicesTobeLearned

Short Name	SdAdditionalOfferServicesTobeLearned
Description	ETAS (Vendor11) specific parameter: This parameter indicates the number of additional offer services to be learned by the client when the calibration parameter is set to SdLearnAllServices.
Introduction	if configured to zero, unconfigured OfferServices will not be learnt even though SdLearnAllServices is set to TRUE. Configure the value based on possible unconfigured OfferServices during learning session.
Multiplicity	0..1
Value Range	0..255
Default Value	10
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.18 Sd/SdGeneral/SdEnableAcIPolicyCheck

Short Name	SdEnableAcIPolicyCheck
Description	Switches the Sd AcIPolicy check:
Introduction	* true: feature is enabled. * false: feature is disabled. Tags: atp.Status=draft
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.19 Sd/SdGeneral/SdEnableClient

Short Name	SdEnableClient
Description	ETAS (Vendor11) specific parameter: Enables (TRUE) or Disables (FALSE) the Client functionality.
Introduction	By default Sd client will be enabled. This parameter is only for debugging/ testing purpose to disable client quickly and test only server.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.20 Sd/SdGeneral/SdEnableExtendedLearningTableFormat

Short Name	SdEnableExtendedLearningTableFormat
------------	-------------------------------------

Description	ETAS (Vendor11) specific parameter: Enables and disables the learning table extended format. If True learning table includes Service ID, Service Instance ID and MajorVersion, If False learning table includes only Service ID.
Introduction	The configured value of this parameter will be passed as initial value for calibration parameter SdEnableExtendedLearningTableFormat.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.21 Sd/SdGeneral/SdEnableFastStartUp

Short Name	SdEnableFastStartUp
Description	ETAS (Vendor11) specific parameter: Enables and disables the Fast start up of services.
Introduction	Enables or disables the feature "starting the services in the init based on NvM data". Default value is False. Configure it to True to enable the feature.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.22 Sd/SdGeneral/SdEnableLearning

Short Name	SdEnableLearning
Description	ETAS (Vendor11) specific parameter: Enables and disables the Learning of services.
Introduction	Enables or disables Learning Sd services and storing them to NvM feature. Default value is False. Configure it to True to enable the feature. To start the services in init based on NvM data configure the parameter SdEnableFastStartUp to True.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.23 Sd/SdGeneral/SdEnableO1ConfigOption

Short Name	SdEnableO1ConfigOption
Description	ETAS (Vendor11) specific parameter: Enables (TRUE) or Disables (FALSE) the OEM specific config option.

Introduction	Enables or disables OEM specific config option consisting of ServiceId and MessageType addition for Tx Sd messages and processing for Rx Sd messages.If set to true, it will increase Sd memory consumption and will increase Sd run time.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.24 Sd/SdGeneral/SdEnableSecureConnection

Short Name	SdEnableSecureConnection
Description	ETAS (Vendor11) specific parameter: Enables (TRUE) or Disables (FALSE) if the SD will be allowed to check if secure connection is established or not.
Introduction	Enables/Disables the feature Sd checking for secure connection established incase of network security protocols in use. This parameter shall be True, if security protocol like TLS is used for Somelp communication. Otherwise it can be set to false to skip checking for secure connection established.If set to false, Sd run time will improve.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.25 Sd/SdGeneral/SdEnableSecurityEventReporting

Short Name	SdEnableSecurityEventReporting
Description	Switches the reporting of security events to the IdsM:
Introduction	* true: reporting is enabled. * false: reporting is disabled.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.26 Sd/SdGeneral/SdEnableServer

Short Name	SdEnableServer
Description	ETAS (Vendor11) specific parameter: Enables (TRUE) or Disables (FALSE) the Server functionality.
Introduction	By default Sd server will be enabled. This parameter is only for debugging/testing purpose to disable server quickly and test only client.

Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.27 Sd/SdGeneral/SdIgnoreRetCodeMask

Short Name	SdIgnoreRetCodeMask
Description	ETAS (Vendor11) specific parameter: Ignore Return Code Mask, if this parameter is configured then the value configured describes which bits of the Return Code to be ignored. The bits which are set to 0 will be ignored. For example, if the value is 0x00 then all the bits of return code are ignored. If it is 0xFF, then none of the bits are ignored. If it is 0x03, then MSB 6 bits are ignored.
Introduction	This is OEM specific requirement to make return code configurable other than 0. This can be configured to 0.
Multiplicity	0..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.28 Sd/SdGeneral/SdLearnAllServices

Short Name	SdLearnAllServices
Description	ETAS (Vendor11) specific parameter: Enables and disables the Learn all services feature, If True allows the ECU to learn and persist all services it receives their offer messages regardless of whether the service is part of the ECU's Sd configuration or not.
Introduction	The configured value of this parameter will be passed as initial value for calibration parameter SdLearnAllServices.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.29 Sd/SdGeneral/SdLearningCompleteTimeout

Short Name	SdLearningCompleteTimeout
Description	ETAS (Vendor11) specific parameter: This parameter indicates the learning complete timeout timer in seconds.

Introduction	The configured value of this parameter will be passed as initial value for calibration parameter SdLearningCompleteTimeout.
Multiplicity	0..1
Value Range	0..10
Default Value	5
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.30 Sd/SdGeneral/SdLrngDemEventParameterRefs

Short Name	SdLrngDemEventParameterRefs
Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId symbolic value.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

16.3.1.31 Sd/SdGeneral/SdLrngDemEventParameterRefs/SD_E_LRNG_INCOMPLETE

Short Name	SD_E_LRNG_INCOMPLETE
Description	Reference to the DemEventParameter which shall be issued when the Services learning is incomplete.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

16.3.1.32 Sd/SdGeneral/SdLrngDemEventParameterRefs/SD_E_LRNG_NVM_FAILED

Short Name	SD_E_LRNG_NVM_FAILED
Description	Reference to the DemEventParameter which shall be issued when the NvMblock is corrupted or NvM write failed .
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

16.3.1.33 Sd/SdGeneral/SdMaxRxOptions

Short Name	SdMaxRxOptions
------------	----------------

Description	ETAS (Vendor11) specific parameter: This parameter is to configure the Maximum number of Endpoint Options Module can process when Sd_RxIndication API is called from Lower Layer.
Introduction	If more options received than configured value, then the extra options will be ignored. Integrator should observe traffic on system level to select appropriate value or get this input from OEM. 2 is typical number of endpoint options in Sd frame. Too high value would increase memory consumption of Sd module.
Multiplicity	0..1
Value Range	1..255
Default Value	2
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.34 Sd/SdGeneral/SdMessageQueueSize

Short Name	SdMessageQueueSize
Description	ETAS (Vendor11) specific parameter: Parameter to set the Queue Size to process SD Entries.
Introduction	If this parameter is not configured Sd code will generate the queue size by considering all socket connections. If integrator thinks that generated queue size (SD_INFO_ENTRY_COUNT) is too high or too less than what project will need then integrator can configure this parameter to meet the project needs. This could be the case, if not all Sd services are active at same time. Too high value would increase memory consumption of Sd module.
Multiplicity	0..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

16.3.1.35 Sd/SdGeneral/SdMulticastResponseToFind

Short Name	SdMulticastResponseToFind
Description	ETAS (Vendor11) specific parameter: Enables (TRUE) or Disables (FALSE) if the SD will respond to the Find-Service Entry with OfferService Entry over Unicast or Multicast IP depending upon the Time Difference between Cyclic OfferService and FindService.

Introduction	When this parameter is configured to True: 1. OfferService will be transmitted over Unicast IP as a response of FindService if FindService is received less than 50% of SdServerTimerOfferCyclicDelay. 2. OfferService will be transmitted over Multicast IP as an response of FindService if FindService is received greater than 50% of SdServerTimerOfferCyclicDelay. When this parameter is configured to FALSE then OfferService will always be transmitted over Unicast IP as an response of FindService Req: PRS_SOMEIPSD_00423 Value selection is mainly based on the customer requirement, as specification allows both options.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

16.3.1.36 Sd/SdGeneral/SdNvMWriteTimeout

Short Name	SdNvMWriteTimeout
Description	ETAS (Vendor11) specific parameter: This parameter defines the maximum allowed time to successfully complete the learning table's NvM block writing in seconds.
Introduction	The configured value of this parameter will be passed as initial value for calibration parameter SdNvMWriteTimeout.
Multiplicity	0..1
Value Range	0..5
Default Value	0.5
Type	ECUC-FLOAT-PARAM-DEF

16.3.1.37 Sd/SdGeneral/SdRemoteNodeSafetyMargin

Short Name	SdRemoteNodeSafetyMargin
Description	ETAS (Vendor11) specific parameter: Parameter that is used to configure Safety Margin.

Introduction	In a network with active DHCP Server, ECUs may get different IP Addresses for each DHCP renew cycle and we need additional memory for such scenarios. This parameter is the factor with which number of static remote nodes are multiplied to get the maximum size of the Remote Node table. (This table stores information regarding each remote node). Each change of remote node IP address will cause a new entry in the Sd table. Currently existing entries will not be deleted during driving cycle. Thus integrator needs to estimate based on system configuration possible number of different remote IP addresses during driving cycle and configure this parameter accordingly.
Multiplicity	0..1
Value Range	1..65535
Default Value	1
Type	ECUC-FLOAT-PARAM-DEF

16.3.1.38 Sd/SdGeneral/SdSecurityEventRefs

Short Name	SdSecurityEventRefs
Description	Container for the references to IdsMEvent elements representing the security events that the SD module shall report to the IdsM in case the corresponding security related event occurs (and if SdEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

16.3.1.39 Sd/SdGeneral/SdSecurityEventRefs/SD_SEV_DROP_ENTRY_AUTH_FAILED

Short Name	SD_SEV_DROP_ENTRY_AUTH_FAILED
Description	ETAS (Vendor11) specific parameter: An Sd Entry is dropped because combination of Service id and Message type in Sd entry are not present in OEM specific config option.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

16.3.1.40 Sd/SdGeneral/SdSecurityEventRefs/SEV_DUPLICATE_OFFER

Short Name	SEV_DUPLICATE_OFFER
Description	SD rejected Offer for a ServiceInstance which is already offered by a different endpoint and TTL still valid.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

16.3.1.41 Sd/SdGeneral/SdTriggerSelfHealingAction_Cbk

Short Name	SdTriggerSelfHealingAction_Cbk
Description	ETAS (Vendor11) specific parameter: This parameter configures the name of the callback to indicate the corruption of the NvM block .
Introduction	Sd modules invokes this user callback, If NvM block was written before and no learning session is undergoing when the NvM corruption is detected.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

16.3.1.42 Sd/SdGeneral/SdUserCbkHeaderFileName

Short Name	SdUserCbkHeaderFileName
Description	ETAS (Vendor11) specific parameter: This parameter specifies the name of the header file containing the definition of the SdTriggerSelfHealingAction_Cbk Api.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

16.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

- There are no additional API defined in this version.

16.4 Limitations

- Exclusive Areas and Critical Sections are currently not supported.
- Measurement variables are not currently supported.

- Dual stack is not supported (i.e. only IPv4 Endpoint option or IPv6 Endpoint option in all SD instances, not a mixture).
- If SdClientServiceMulticastRef and SdClientServiceTCPRef are configured, then the client will only send one endpoint either TCP or Multicast endpoint based on EventGroup ActivationRef.
- The timing for SdServerTimerInitialOfferDelayMax, SdServerTimerInitialOfferDelayMin, SdServerTimerInitialOfferRepetitionBaseDelay, SdServerTimerOfferCyclicDelay, SdServerTimerRequestResponseMaxDelay, SdServerTimerRequestResponseMinDelay, SdClientTimerInitialFindDelayMax, SdClientTimerInitialFindDelayMin, SdClientTimerInitialFindRepetitionsBaseDelay, SdClientTimerRequestResponseMaxDelay and SdClientTimerRequestResponseMinDelay should be configured as a Minimum 1ms and Maximum 49 days.
- If the SoAd API SoAd_GetLocalAddr fails (when called in the context of Sd_LocalIpAddrAssignmentChg) then SD Frames will be transmitted with the EndPoint Options IP as 0.0.0.0 and any frame received by SD will not be acknowledged.
- If the server receives SubscribeEventGroup with both endpoints i.e a Multicast endpoint and a TCP endpoint option, then the server will send SubscribeEventGroupNack.
- StopSubscribeEventGroup will be ignored if it is received with the Multicast endpoint option.

16.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- Sd_AclUpdate
- Sd_AclPolicyCheck

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- Sd/SdConfig/SdCapabilityRecordMatchCallout
- Sd/SdConfig/SdCapabilityRecordMatchCallout/SdCapabilityRecordMatchCalloutName
- Sd/SdConfig/SdInstance/SdClientService/SdClientCapabilityRecord
- Sd/SdConfig/SdInstance/SdClientService/SdClientCapabilityRecord/SdClientServiceCapabilityRecordKey
- Sd/SdConfig/SdInstance/SdClientService/SdClientCapabilityRecord/SdClientServiceCapabilityRecordValue
- Sd/SdConfig/SdInstance/SdClientService/SdClientCapabilityRecordMatchCalloutRef

- Sd/SdConfig/SdInstance/SdInstanceHostname
- Sd/SdConfig/SdInstance/SdServerService/SdServerCapabilityRecord
- Sd/SdConfig/SdInstance/SdServerService/SdServerCapabilityRecord/
SdServerCapabilityRecordKey
- Sd/SdConfig/SdInstance/SdServerService/SdServerCapabilityRecord/
SdServerCapabilityRecordValue
- Sd/SdConfig/SdInstance/SdServerService/SdServerCapabilityRecordMatch-
CalloutRef
- Sd/SdConfig/SdInstance/SdServerService/SdServerServiceLoadBalancing-
Priority
- Sd/SdConfig/SdInstance/SdServerService/SdServerServiceLoadBalancing-
Weight

16.6 Deviations

16.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

16.6.1.1 Sd/SdConfig/SdInstance

[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

16.6.1.2 Sd/SdConfig/SdInstance/SdClientService/SdClientServiceMulticastRef

[RTA-BSW] Description	Reference to the SoAdSocketConnection representing the data path (UDP) for communication with the server.
[AUTOSAR] Description	Reference to the SoAdSocketConnection representing the data path (UDP) for communication with the server. This element is also used to set the remote address of the server.
[RTA-BSW] Introduction	This element is also used to set the remote address of the server.
[AUTOSAR] Introduction	This is used, if a ClientService subscribes with a Consumed Eventgroup multicast endpoint. This is an alternative to subscribe with a Consumed Eventgroup unicast endpoint (see SdClientServiceUdpRef). Please note: usage of this reference is mutually exclusive to SdClientServiceUdpRef.

16.6.1.3 Sd/SdConfig/SdInstance/SdClientService/SdClientServiceUdpRef

[RTA-BSW] Introduction	This element is also used to set the remote address of the server.
---------------------------	--

[AUTOSAR] Introduction	This element is also used to set the remote address of the server. This is used, if a ClientService subscribes with a Consumed Eventgroup unicast endpoint. This is an alternative to subscribe with a Consumed Eventgroup multicast endpoint. (see SdClientServiceMulticastRef). Please note: usage of this reference is mutually exclusive to SdClientServiceMulticastRef.
---------------------------	--

16.6.1.4 Sd/SdConfig/SdInstance/SdClientService/SdConsumedEventGroup/SdConsumedEventGroupMulticastActivationRef

[RTA-BSW] Description	The reference of a Routing Group in order to activate and setup the Socket Connection for Multicast Events of this EventGroup. The multicast address from the received Multicast option is setup by SoAd_RequestIppAddrAssignment.
[AUTOSAR] Description	The reference of a Routing Group in order to activate and setup the Socket Connection for Multicast Events of this EventGroup. The Multicast address from the received Multicast Option is setup by SoAd_RequestIppAddrAssignment.

16.6.1.5 Sd/SdConfig/SdInstance/SdClientService/SdConsumedEventGroup/SdConsumedEventGroupUdpActivationRef

[RTA-BSW] Introduction	This element is also being used for getting the IP address and port number for building the UDP endpoint option for the Subscribe EventGroup entry. If no UDP methods are used in the service, this element is also being used for setting the remote address (UDP Endpoint option referenced by the Offer Service entry). If multiple EventGroups of the same Service Instance are subscribed the UDP Socket Connection will be shared and must be set only once.
[AUTOSAR] Introduction	This element is also being used for getting the IP address and port number for building the UDP Endpoint option or Consumed Multicast option for the Subscribe EventGroup entry. If no UDP methods are used in the service, this element is also being used for setting the remote address (UDP Endpoint option referenced by the Offer Service entry). If multiple EventGroups of the same Service Instance are subscribed the UDP Socket Connection will be shared and must be set only once.

16.6.1.6 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerInitialFindDelayMax

[ADDED] Default Value	0
-----------------------	---

16.6.1.7 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerInitialFindDelayMin

[ADDED] Default Value	0
-----------------------	---

16.6.1.8 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerInitialFind-RepetitionsBaseDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.9 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerInitialFind-RepetitionsMax

[ADDED] Default Value	0
-----------------------	---

16.6.1.10 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerRequestResponseMaxDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.11 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerRequestResponseMinDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.12 Sd/SdConfig/SdInstance/SdClientTimer/SdClientTimerTTL

[RTA-BSW] Description	Time to live for find and subscribe messages.
[AUTOSAR] Description	Time to live for find and subscribe messages. Note! The TTL value for find messages shall be ignored by the server service and the configuration is only kept for backward compatibility
[ADDED] Default Value	1

16.6.1.13 Sd/SdConfig/SdInstance/SdInstanceLocalAddressCheckLength

[ADDED] Default Value	0
-----------------------	---

16.6.1.14 Sd/SdConfig/SdInstance/SdServerService/SdEventHandler/SdEventHandlerMulticast/SdMulticastEventSoConRef

[RTA-BSW] Description	Reference to the SoAdSocketConnection representing the multicast data path (UDP).
[AUTOSAR] Description	Reference to the SoAdSocketConnection representing the EventHandler Multicast data path (UDP).

16.6.1.15 Sd/SdConfig/SdInstance/SdServerService/SdEventHandler/SdEventHandlerMulticastThreshold

[RTA-BSW] Description	Specifies the number of subscribed clients that trigger the Server to change the transmission of events to Multicast.
[AUTOSAR] Description	Specifies the number of subscribed clients with different endpoint information (see SWS_SD_00754) that triggers the Server to change the transmission of events via the EventHandler Multicast connection.
[RTA-BSW] Introduction	If configured to 0 only unicast will be used. If configured to 1 the first client will be already served by multicast. If configured to 2 the first client will be served with unicast and as soon as the second client arrives both will be served by multicast. This does not influence the handling of initial events, which are served using unicast only.
[AUTOSAR] Introduction	If configured to 0 only Consumed Evengroup unicast connections and Consumed Eventgroup multicast connections will be used. If configured to 1 the first client and all further subscribed clients will be served via the EventHandler Multicast connection as configured in SdMulticastEventSoConRef. If configured to n up to n-1 clients with different endpoint information will be served via Consumed Evengroup unicast connections and Consumed Eventgroup multicast connections. As soon as the number of subscribed clients with different endpoint information reaches n, then all subscribed clients are served via the EventHandler Multicast connection as configured in SdMulticastEventSoConRef. This does not influence the handling of initial events.

16.6.1.16 Sd/SdConfig/SdInstance/SdServerService/SdEventHandler/SdEventHandlerTcp/SdEventActivationRef

[RTA-BSW] Description	Reference to a SoAdRoutingGroup for activation of the data path for a subscribed client (start sending events after subscribe). This is usually equal to the SdEventActivationRef referenced by SdEventHandlerTcp
[AUTOSAR] Description	Reference to a SoAdRoutingGroup for activation of the data path for a subscribed client (start sending events after subscribe). This is usually equal to the SdEventActivationRef referenced by SdEventHandlerUdp

16.6.1.17 Sd/SdConfig/SdInstance/SdServerService/SdEventHandler/SdEventHandlerUdp

[RTA-BSW] Introduction	The activation ref (or triggering ref if no activation ref exists) is also being used for identification of the related socket connections in order to set the remote address of the client or find the related client by iterating the SdEventHandlerUdp elements (remote address statically configured or automatically set by method call before subscription).
[AUTOSAR] Introduction	The activation ref (or triggering ref if no activation ref exists) is also being used for identification of the related socket connections in order to set the remote address (either unicast address or multicast address) of the client or find the related client by iterating the SdEventHandlerUdp elements (remote address statically configured or automatically set by method call before subscription).

16.6.1.18 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerInitialOfferDelayMax

[ADDED] Default Value	0
-----------------------	---

16.6.1.19 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerInitialOfferDelayMin

[ADDED] Default Value	0
-----------------------	---

16.6.1.20 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerInitialOfferRepetitionBaseDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.21 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerInitialOfferRepetitionsMax

[ADDED] Default Value	0
-----------------------	---

16.6.1.22 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerOfferCyclicDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.23 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerRequestResponseMaxDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.24 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerRequestResponseMinDelay

[ADDED] Default Value	0
-----------------------	---

16.6.1.25 Sd/SdConfig/SdInstance/SdServerTimer/SdServerTimerTTL

[ADDED] Default Value	1
-----------------------	---

16.6.1.26 Sd/SdGeneral/SdSetRemAddrOfClientRxMulticastSoCon

[RTA-BSW] Introduction	Note: setting SdSetRemAddrOfClientRxMulticastSoCon to FALSE supports the re-use of a multicast socket connection for multiple ClientServices which are located on the same ECU and subscribed to ServerServices which are located on different ECUs. The configuration of the ECU where the ClientServices are located, could be simplified by only configuring one socket connection within the multicast socket connection group.
[AUTOSAR] Introduction	Note: setting SdSetRemAddrOfClientRxMulticastSoCon to FALSE supports the re-use of a multicast socket connection for multiple ClientServices which are located on the same ECU and subscribed to ServerServices which are located on different ECUs. The configuration of the ECU where the ClientServices are located, could be simplified by only configuring one socket connection within the multicast socket connection group.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

16.7 API Definition

Table 16.70.Sd Supported API functions

Sd_Init	Initialises the module.
Sd_MainFunction	Schedules the module.
Sd_GetVersionInfo	Returns version information about the module.
Sd_ServerServiceSetState	Sets the state of the specified server service instance.
Sd_ClientServiceSetState	Sets the state of the specified client service instance.
Sd_ConsumedEventGroupSetState	Sets the state of the specified consumed event group.
Sd_LocalIpAddressAssignmentChg	Callback function indicates a change to the IP address assignment for a specified socket.

Sd_SoConModeChg	Callback function indicates a change to the connection mode for a specified socket connection.
Sd_ServiceGroupStart	Used by BswM to start a preconfigured SdServiceGroup.
Sd_ServiceGroupStop	Used by BswM to stop a preconfigured SdServiceGroup.
Sd_RxIndication	Callback function indicating reception of a PDU from a lower layer communication module.

For Further information please refer to AUTOSAR_SWS_ServiceDiscovery.pdf

16.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

16.8.1 Delay Times

The minimum configured time for SdClientTimerInitialFindDelayMin and SdServerTimerInitialOfferDelayMin should be twice the value of SdMainFunctionCycleTime. All other delay parameters should be set so that they occur within the range of 1ms to 49 days accordingly.

16.8.2 Validation of Timers

Validation will prohibit the use of the same timer for different services if SdClientServiceAutoRequire is set to FALSE. For this reason, new Sd Timers might need to be configured and used.

16.8.3 Client Service Endpoints

- If a client service has both data path types (UDP and TCP) configured, it will expect to receive offer messages containing both endpoint options. The offer message will be dropped if only one endpoint option is contained within the message.
- SdMaxRxOptions is set to 2 by default and must be configured if the expected number of endpoints is not 2.

16.8.4 Return Codes

If the parameter SdIgnoreRetCodeMask is configured, the value configured describes which bits of the ReturnCode are to be ignored (those set to 0). For

example, if the value is 0x00 then all the bits of return code are ignored. If it is 0xFF, then none of the bits are ignored. If it is 0x03, then the most significant 6 bits are ignored.

16.8.5 Retry Subscription

To use the Retry Subscription mechanism, `SdSubscribeEventgroupRetryEnable` should be set to `TRUE` and `SdSubscribeEventgroupRetryMax` and `SdSubscribeEventgroupRetryDelay` should be configured to a non-zero value in `SdClientTimer` for the Client Services that require Retry Subscriptions.

16.8.6 Configuring Sd and SoAd

See the Integration Advice for information specific to configuring Sd to work with SoAd.

16.8.7 Sd-SoAd communication

Sd-SoAd communication must be routed through PduR (this is a deviation from the AUTOSAR specification). Therefore, PduR must be configured when the Sd and SoAd modules are used in a project.

16.8.8 DEM module

To report the status of events to the DEM module it is necessary to configure the `SdInstanceDemEventParameterRefs` that refer to the `DemEventParameter` in the DEM module.

16.9 Integration Advice

16.9.1 Init Functions

`Sd_Init` must be called on first startup of the target ECU.

16.9.2 Delnit Functions

No `Delnit` is required.

16.9.3 Scheduled Functions

`Sd_MainFunction` must be scheduled for cyclic operation. It can be configured from a minimum of 1ms, and the maximum will depend upon the module's timer configuration.

16.9.4 Integrating Sd with SoAd

When integrating the Sd and SoAd modules, the following points should also be noted.

- You should have different `RoutingGroups` and `PdUs` for handling Sd (control) messages and actual `Methods/Events`.
- You should have different `SoAdSocketConnectionGroup` for handling Sd (control) messages (via `SdTxPduRef`) and actual `Methods/Events` (via `SdClientService`).

- When configuring the SoAd module to work with Sd, only one SoAdPduRouteDest should be configured for transmission of unicast events. In SoAdPduRouteDest and SoAdSocketRoute, SoAdRxSocketConnOrSocketConnBundleRef should be configured with a reference to SoAdSocketConnection (not SoAdSocketConnectionGroup).
- You should create the new Socket Connection Group to receive Multicast Events in ClientServices with local port configured statically.
- Set the SoAdSocketIpAddrAssignmentChgNotification to true for the SoAdSocketConnectionGroup so that the Sd module can set the remote IP address at runtime depending on the received Service Finds/Offers.
- For TCP socket connection groups controlled by Sd, the 'keep alive' feature in SoAd should be configured, which includes the parameter SoAdSocketTcpKeepAlive, and also the optional parameters SoAdSocketTcpKeepAliveInterval, SoAdSocketTcpKeepAliveProbesMax and SoAdSocketTcpKeepAliveTime.
- Set the SoAdSocketConnections to the wildcard IP address and port because those are set by the Sd module.
- Set up the SoAdSocketConnection callbacks to Sd_LocalIpAddrAssignmentChg and Sd_SoConModeChg.
- You should configure the SoAdSocketSoConModeChgNotifUpperLayerRef parameters as 'Sd' of SoAd module for the functioning of SOME/IP datapath controlled by Sd.
- You should configure a PDU for receiving multicast SD messages (but not necessarily another one for receiving multicast Events).
- To transmit the EventGroup over a multicast datapath you should configure the multicast IP address in SoAdSocketRemotepAddress and SoAdSocketRemotePort.
- For the Project which required to get the response of FindService with OfferService over Unicast/Multicast IP depending upon the time difference between Reception of FindService and SdServerTimerOfferCyclicDelay the configuration parameter SdMulticastResponseToFind should be set to TRUE.
- As per SWS_SD_00460, the SoAdHeaderId should be enabled, and the header IDs of Sd control messages should be set to the value of 4294934784.

16.9.4.1 Sd Configuration Example

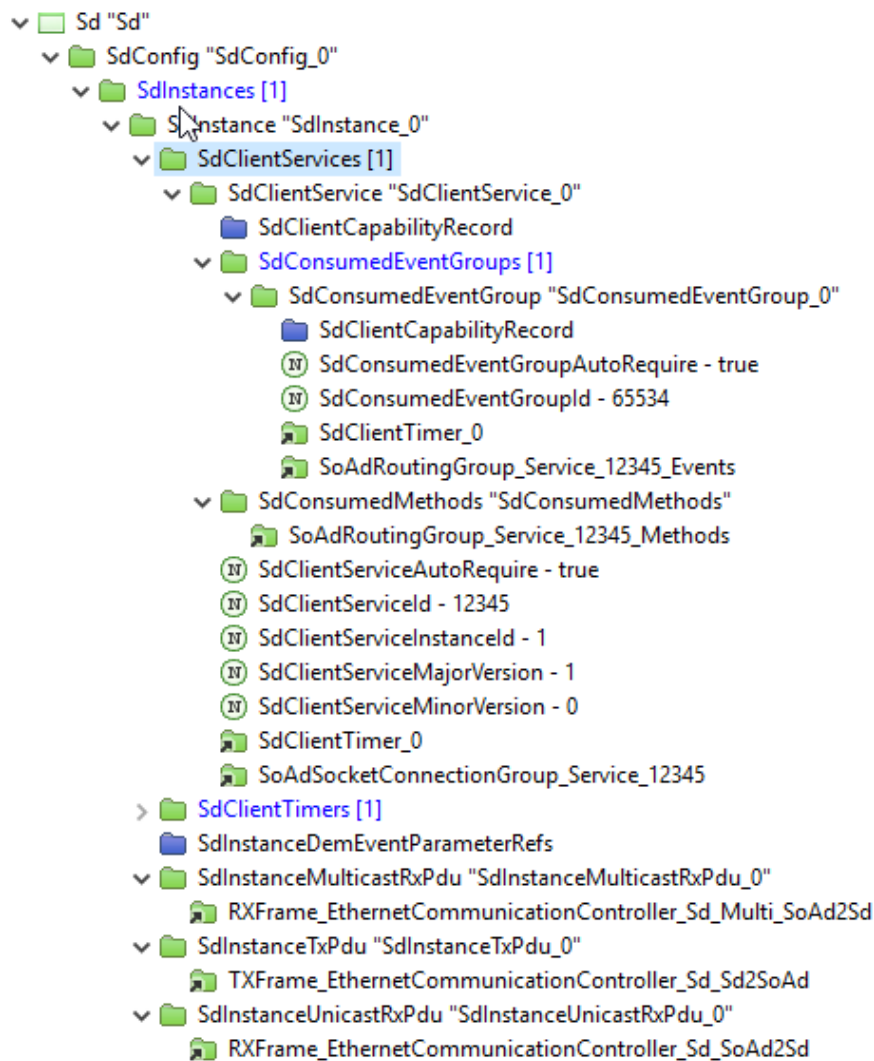


Figure 16.5.Sd Configuration Example

16.9.4.2 SoAd Configuration Example

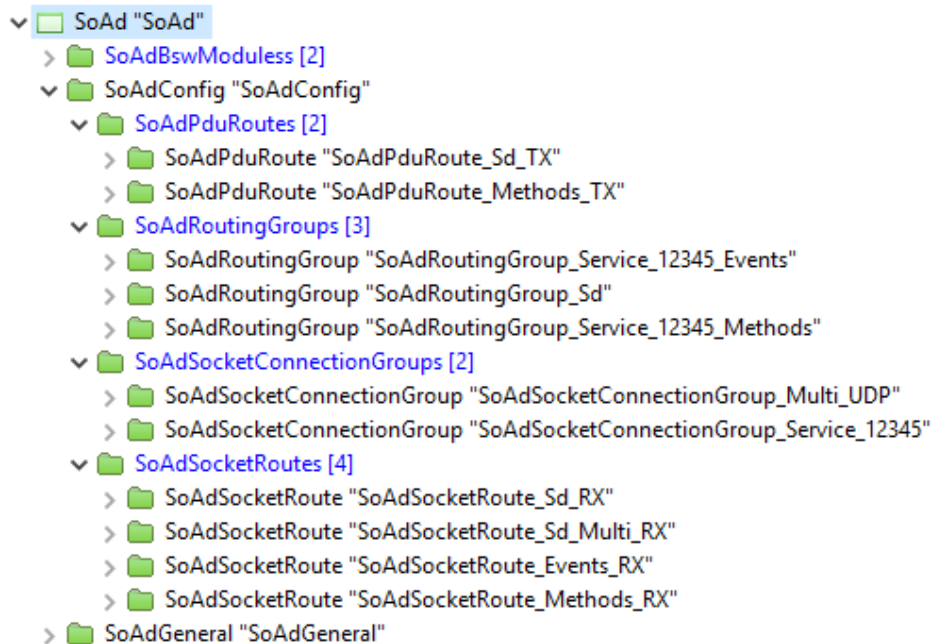


Figure 16.6. SoAd Configuration Example

16.9.5 General

- If the Socket Adapter function `SoAd_GetLocalAddr` fails when called by the function `Sd_LocalIpAddressAssignmentChg`, then the Sd frames will be transmitted with an endpoint IP address of 0.0.0.0, and any frame received by Sd will not be acknowledged.
- In the parameter definition file (`Sd_EcucParamDef.arxml`), the upper multiplicity of containers `SdEventHandlerUdp` and `SdEventHandlerTcp` have been changed from '*' to 1. Due to this, redundant containers should be deleted in order for code generation to be successful.

16.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Sd module uses the following OS resources:

- Sd
- Sd_LocalAddrNetMask
- SdSd_IpAddrProcSt

16.10 Hardware Requirements

Table 16.71.Sd Hardware Requirements

Support	Usage
FLASH	23010 bytes
RAM	2795 bytes

Usage calculated based on a sample configuration.

17 Socket Adaptor

Module name	SoAd
Package	RTA-ETH
AUTOSAR Module ID	56
AUTOSAR Version	22-11

17.1 Introduction

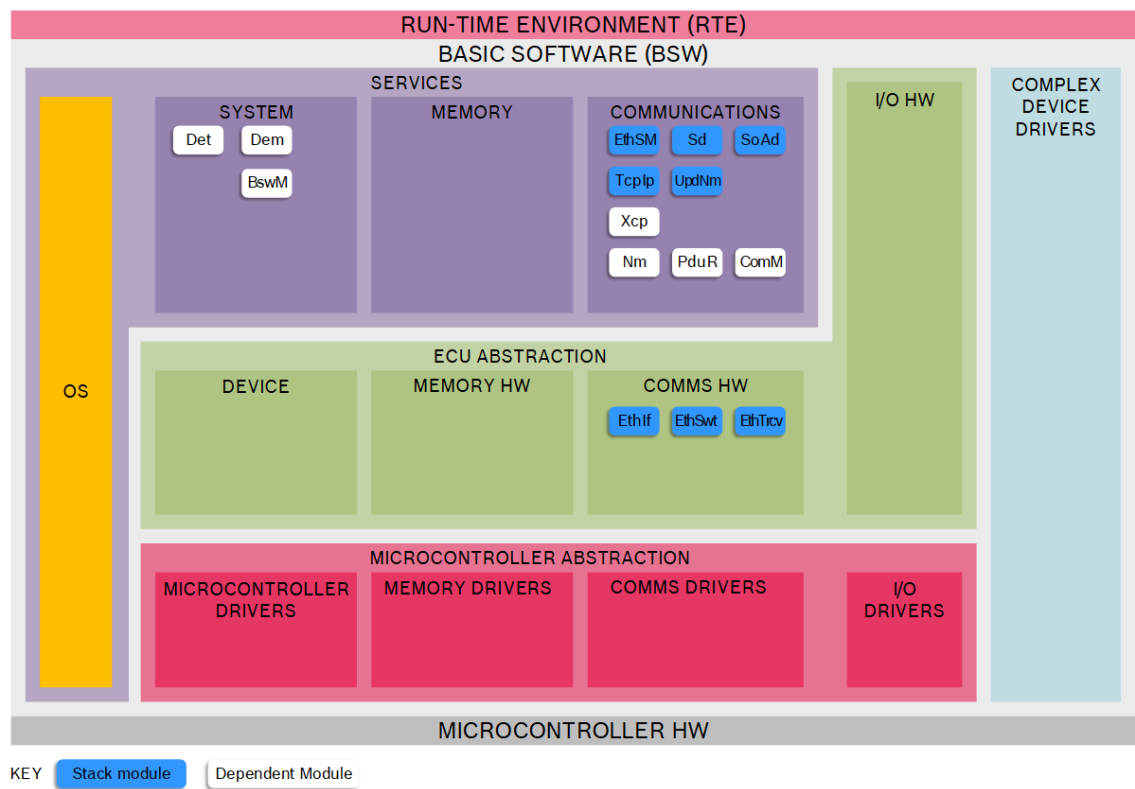


Figure 17.1.RTA-ETH Architecture

The Socket Adapter (SoAd) module is an upper layer module of the communication services layer. The diagram above shows the position of SoAd in the AUTOSAR layered architecture.

SoAd enables PDU-based communications on a TCP/IP network by allowing the configuration and maintenance of socket connections. In this context, a socket is the endpoint of a communication link identified by an IP address and port number. The SoAd module is required by the TcpIp module, as the protocols supported by the latter rely on socket-based connections. SoAd-configured socket connections specify local and remote sockets for all connections required by the calling module, as well as connection-specific parameters such as the transport protocol, buffer requirements and PDU header configuration. Socket connections are mapped to I-PDUs (and vice versa) through configuration parameter settings

within SoAd.

The SoAd module is part of the RTA-ETH stack, functioning as an interface between communication service modules which use PDU-based communications (e.g. PduR, UdpNm, Xcp, Sd, DoIP) and the TCP/IP transport protocols configured in the Tcplp module.

SoAd provides both a communication interface (IF) API and a transport protocol (TP) API for PDU transmission. The IF-API is used by modules such as UDP Network Management (UdpNm), XCP on Ethernet (Xcp) and Service Discovery (Sd), while the TP-API is used by transport protocol modules. Some modules such as the PDU Router (PduR) use both IF-API and TP-API calls.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

17.2 Supported Features

- Synchronous and asynchronous transmission of messages.
- Triggered transmission of multiple I-PDUs simultaneously.
- Separate transmission and reception indication for both IF-PDU and TP-PDU.
- Zero Copy transmission (for IF-PDU only).
- Start of reception notification (for TP-PDU only).
- Copy transmit data function (for TP-PDU only).
- Socket connection mode change notification.
- Local IP Address change notification.

17.3 Extensions

17.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

17.3.1.1 SoAd/SoAdBswModules/SoAdCddHeaderFileName

Short Name	SoAdCddHeaderFileName
Description	RB (Vendor06) specific parameter: If upper layer of SoAd is an AUTOSAR module, then the header files are statically specified and automatically included in SoAd. In case of non-AUTOSAR Upper Layer components, the name of the CDD header would be configured with this parameter and the same would be included.
Multiplicity	0..1

String Length Range	1..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.2 SoAd/SoAdBswModules/SoAdGetZeroCopyIfUdpTransmit

Short Name	SoAdGetZeroCopyIfUdpTransmit
Description	RB (Vendor06) specific parameter: Enables or disables zero copy IF Udp transmission in SoAd. TRUE: Enables SoAdGetZeroCopyIfUdpTransmit feature. FALSE: Disables SoAdGetZeroCopyIfUdpTransmit feature.
Introduction	Zero copy transmission shall be accordingly configured in Tcplp. It may increase the throughput.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.3 SoAd/SoAdConfig/SoAdPduRoute/SoAdTxUser

Short Name	SoAdTxUser
Description	RB (Vendor06) specific parameter: Specifies the upper layer module which owns this Pdu.
Introduction	This is mandatory parameter which should be configured in SoAdPduRoute
Multiplicity	1..1
Default Value	-
Enumeration Literals	PduR, Sd, Cdd, UdpNm, Xcp, DoIP, EthDiagMM
Type	ECUC-ENUMERATION-PARAM-DEF

17.3.1.4 SoAd/SoAdConfig/SoAdPduRoute/SoAdUpperLayerTxPduld

Short Name	SoAdUpperLayerTxPduld
Long Name	SoAdUpperLayerTxPduld (automatically Generated)
Description	RB (Vendor06) specific parameter: Tx PDU ID of the upper layer module would be configured with this parameter if Upper Layer is CDD.
Introduction	This parameter is automatically generated.
Multiplicity	0..1
Value Range	0..65535

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

17.3.1.5 SoAd/SoAdConfig/SoAdPduRoute/UL_SoAdIfTxConfirmation

Short Name	UL_SoAdIfTxConfirmation
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is used to notify upper layers about IfTxConfirmation.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type is IF.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.6 SoAd/SoAdConfig/SoAdPduRoute/UL_SoAdTpCopyTxData

Short Name	UL_SoAdTpCopyTxData
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback function that is called to retrieve the data from upper layer.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type in TP. It is mandatory for TP UL.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.7 SoAd/SoAdConfig/SoAdPduRoute/UL_SoAdTpTxConfirmation

Short Name	UL_SoAdTpTxConfirmation
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is used to notify the upper layers about the TpTxConfirmation.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type is TP.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.8 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdBypassMsgAcceptanceFilterPolicy

Short Name	SoAdBypassMsgAcceptanceFilterPolicy
------------	-------------------------------------

Description	RB (Vendor06) specific parameter: Activates the bypass for the message acceptance filter policy (remote address is always overwritten and socket goes ONLINE when remote address contained wild-card). If this parameter is set to TRUE, the UL should check the acceptance policy.
Introduction	TRUE: Bypass allows SoAd to overwrite remote address and make the socket ONLINE during reception of UDP message. FALSE: Bypass is disabled and normal processing of SoAdSocketMsgAcceptance-Filter disabled takes place. If bypass is enabled, may increase the throughput, as SoAd will perform less checks.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.9 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocket-Connection/UL_LocallpAddrAssignmentChg

Short Name	UL_LocallpAddrAssignmentChg
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback function that is used to notify the upper layers about the Local Ip address assignment change. This callback function is required if Upper Layer is CCD and SoAdSocketIpAddrAssignmentChgNotification is TRUE.
Introduction	It can also be configured for other non-CDD Upper Layer and will overrule automatic generated name. This could be useful, if same socket connection group is used for both data and control path by different upper layers. In this case it is possible to configure, which upper layer shall receive callback.
Multiplicity	0..*
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.10 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocket-Connection/UL_SoConModeChg

Short Name	UL_SoConModeChg
------------	-----------------

Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is used to notify upper layers about SocketConnection Mode change information. This parameter needs only to be configured if SoAdSocketSoConModeChgNotification is set to TRUE for this group.
Multiplicity	0..*
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.11 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketConnectionGroupId

Short Name	SoAdSocketConnectionGroupId
Long Name	SoAdSocketConnectionGroupId (automatically Generated)
Description	RB (Vendor06) specific parameter: By this parameter the Identifier per SoAdConnectionGroup is configured. This is needed for other BSW modules to refer this configuration container. The configured ID must be unique across the configured SoAdConnectionGroups. This Socket Connection Group Id will be generated automatically in sequential incremental order which starts from '0'
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

17.3.1.12 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketRbTcpTimestampEnable

Short Name	SoAdSocketRbTcpTimestampEnable
Description	RB (Vendor06) specific parameter: Specifies to use the Timestamp option for this connection. It will not be defined for UDP sockets.
Introduction	TRUE: This TCP connection will support Timestamp option feature. FALSE: This TCP connection will not support Timestamp option feature. Note: This parameter must not be set to TRUE if TcpIpRbTcp-TimestampEnabled is set to FALSE.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

17.3.1.13 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTcpOutOfOrderEnable

Short Name	SoAdSocketTcpOutOfOrderEnable
Description	RB (Vendor06) specific parameter: Specifies whether this socket connection is able to receive Tcp OutOfOrder frames or not.
Introduction	TRUE: TCP OutOfOrder is configured. FALSE: TCP OutOfOrder is not configured
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.14 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTcpSackOptionEnable

Short Name	SoAdSocketTcpSackOptionEnable
Description	RB (Vendor06) Specifies if SACK option can be used by the socket to convey extended acknowledgment information from the receiver to the sender over an established TCP connection according to RFC2018
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.15 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTcpSackPermittedOptionEnable

Short Name	SoAdSocketTcpSackPermittedOptionEnable
Description	RB (Vendor06) Specifies if SACK-Permitted option is allowed to be sent in SYN segment to permit remote to send SACK option according to RFC2018 (Vendor specific parameter)
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.16 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketUdp/SoAdSocketPduUdpTpTxBuffer

Short Name	SoAdSocketPduUdpTpTxBuffer
------------	----------------------------

Description	RB (Vendor06) specific parameter: Specifies the amount of data in bytes (PDU data provided by the upper layer and PDU Header if used) the SoAd shall buffer the data's under the relevant socket connection if TP PDU wanted to be buffered for UDP transmission. This parameter is used for the nPdu feature and is configured per socket connection. Same value will be used for each socket that is part of the same socket group. Too high value would increase memory consumption.
Introduction	Note: in case of a UDP socket and an upper layer with TP API is configured, the required buffer size can be determined automatically. This optional parameter is only relevant if a nPduUdpTxBuffer is used.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

17.3.1.17 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketUdp/SoAdSocketUdpImmediateTxConfirmation

Short Name	SoAdSocketUdpImmediateTxConfirmation
Description	RB (Vendor06) specific parameter: If set to FALSE, SoAd notifies the IF upper layer via transmit confirmation in the next main function. If set to TRUE, SoAd notifies the IF upper layer via transmit confirmation after transmit accepted by TcpIp. May increase the throughput.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.18 SoAd/SoAdConfig/SoAdSocketRoute/SoAdSocketRouteDest/SoAdRxUser

Short Name	SoAdRxUser
Description	RB (Vendor06) specific parameter: Specifies the upper layer module, owner of this RxPdu.
Multiplicity	1..1
Default Value	-

Enumeration Literals	PduR, Sd, Cdd, UdpNm, Xcp, DoIP, EthDiagMM
Type	ECUC-ENUMERATION-PARAM-DEF

17.3.1.19 SoAd/SoAdConfig/SoAdSocketRoute/SoAdSocketRouteDest/SoAdUpperLayerRxPduld

Short Name	SoAdUpperLayerRxPduld
Long Name	SoAdUpperLayerRxPduld (automatically Generated)
Description	RB (Vendor06) specific parameter: Rx PDU ID of the upper layer module would be configured with this parameter if Upper Layer is CDD.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

17.3.1.20 SoAd/SoAdConfig/SoAdSocketRoute/SoAdSocketRouteDest/UL_-SoAdIfRxIndication

Short Name	UL_SoAdIfRxIndication
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is used to notify upper layers about IfRxIndication.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type is IF.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.21 SoAd/SoAdConfig/SoAdSocketRoute/SoAdSocketRouteDest/UL_-SoAdTpCopyRxData

Short Name	UL_SoAdTpCopyRxData
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is called to copy received data to upper layer.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type is TP.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.22 SoAd/SoAdConfig/SoAdSocketRoute/SoAdSocketRouteDest/UL_-SoAdTpRxIndication

Short Name	UL_SoAdTpRxIndication
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is used to notify upper layers about TpRxIndication.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type is TP.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.23 SoAd/SoAdConfig/SoAdSocketRoute/SoAdSocketRouteDest/UL_-SoAdTpStartOfReception

Short Name	UL_SoAdTpStartOfReception
Description	RB (Vendor06) specific parameter: This parameter configures the name of the callback that is used to notify upper layers about the TpStartOfReception.
Introduction	This callback function is required if SoAdTxUser is CCD and TxUpperLayer-Type is TP.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

17.3.1.24 SoAd/SoAdGeneral/SoAdBestMatchRxHeaderIdCheck

Short Name	SoAdBestMatchRxHeaderIdCheck
Description	RB (Vendor06) specific parameter: This parameter would be used to enable the Rx Header Id check for Best Match Algorithm. TRUE: Enables the Rx Header Id check for Best Match Algorithm. FALSE: Disables the Rx Header Id check for Best Match Algorithm.
Introduction	Enabling SoAdBestMatchRxHeaderIdCheck, SoAd will check within the Best Match Algorithm if the Header Id of the received frame matches the Header Id of the mapped Pdu. Additional check of Header Id would allow to find better matching socket and prevent potential blocking a socket over which data can not be received later due to mismatch of Pdu Header Id.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

17.3.1.25 SoAd/SoAdGeneral/SoAdChangeParameterEnabled

Short Name	SoAdChangeParameterEnabled
Description	RB (Vendor06) specific parameter: Allows Upper Layer to change the Parameter.
Introduction	TRUE: Allows Upper Layer to change the Parameter. FALSE: Does not allow Upper Layer to change the Parameter. Throughput may improve if change parameter is called to update/enable parameters of the Transport layer (e.g. If SACK option is enabled for the TCP, throughput may improve)
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.26 SoAd/SoAdGeneral/SoAdDHCPFQDNEnabled

Short Name	SoAdDHCPFQDNEnabled
Description	RB (Vendor06) specific parameter: Allows the Upper Layer to read/write the DHCP FQDN options. TRUE: Allows the Upper Layer to read/write the DHCP FQDN options. FALSE: Does not allow the Upper Layer to read/write the DHCP FQDN options.
Introduction	This parameter enables SoAd_ReadDhcpHostNameOption() and SoAd_WriteDhcpHostNameOption() APIs. If disabled, corresponding APIs will always return E_NOT_OK.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.27 SoAd/SoAdGeneral/SoAdIPAddrAssignmentEnabled

Short Name	SoAdIPAddrAssignmentEnabled
Description	RB (Vendor06) specific parameter: Allows Upper Layer to release/request the IP address assignment. TRUE: Allows Upper Layer to release/request the IP address assignment. FALSE: Does not allow Upper Layer to release/request the IP address assignment.

Introduction	This parameter enables SoAd_RequestIpAddrAssignment() and SoAd_ReleaseIpAddrAssignment() APIs. If disabled, corresponding APIs will always return E_NOT_OK. If enabled, Upper layer will be able to control manually IP address assignment process.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.28 SoAd/SoAdGeneral/SoAdRbTcpTlsEnabled

Short Name	SoAdRbTcpTlsEnabled
Description	RB (Vendor06) specific parameter. TRUE: TLS feature enabled. FALSE: TLS feature disabled.
Introduction	More memory will be consumed if enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.29 SoAd/SoAdGeneral/SoAdRbTcpTxQueueSize

Short Name	SoAdRbTcpTxQueueSize
Description	RB (Vendor06) specific parameter: This parameter defines number of IF/TP Tcp transmissions possible per socket before receiving ACK from the remote node. Any further transmission will be discarded. This parameter is relevant only if a socket is mapped to multiple IF Pdus and any one of the Pdu has configured uL_SoAdIfTxConfirmation or if a socket is mapped to both IF and TP 'pdu(s)' and any of the Pdu has configured UL Txconfirmation (either uL_SoAdIfTxConfirmation or uL_SoAdTpTxConfirmation). Higher the TcpTx Queue size, higher will be memory consumption.
Introduction	Note: Each entry costs $4 * (\text{Number of Tcp sockets with multiple IF Pdus and with at least one PDU has configured uL_SoAdIfTxConfirmation})$ bytes of RAM.
Multiplicity	0..1
Value Range	0..255
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

17.3.1.30 SoAd/SoAdGeneral/SoAdRbWildcardPort

Short Name	SoAdRbWildcardPort
Description	RB (Vendor06) specific parameter: This Parameter would be used to choose the wildcard port.
Introduction	ZERO: SoAd treats 0 as wildcard port (0xFFFF is a valid port), FFFF: SoAd treats both 0xFFFF and 0 as wild card port.
Multiplicity	0..1
Default Value	ZERO
Enumeration Literals	ZERO, FFFF
Type	ECUC-ENUMERATION-PARAM-DEF

17.3.1.31 SoAd/SoAdGeneral/SoAdRestUdpMsgDiscardByInvalidPduHeaderId

Short Name	SoAdRestUdpMsgDiscardByInvalidPduHeaderId
Description	RB (Vendor06) specific parameter: This parameter would be used to enable the feature, Discarding of the rest of the message if no socket route can be found.
Introduction	TRUE: Discards rest of the message(which might include another pdu too) if no socket route can be found for the current pdu which is being processed. FALSE: discard only unknown Udp Pdu and process the rest of the message(which might include another pdu too).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.32 SoAd/SoAdGeneral/SoAdTpCancelComEnabled

Short Name	SoAdTpCancelComEnabled
Description	RB (Vendor06) specific parameter: Allows the Upper Layer to request the cancelation of the active ongoing TP transmission/reception
Introduction	TRUE: Upper Layer can cancel/terminate the active ongoing packets FALSE: Upper Layer cannot cancel/terminate the active ongoing packets
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.1.33 SoAd/SoAdGeneral/SoAdTpTcpTransmitRetryCount

Short Name	SoAdTpTcpTransmitRetryCount
Description	RB (Vendor06) specific parameter: Determines the number of times SoAd shall retry Tp Tcp transmission through Tcplp_TcpTransmit when Tcp buffer is unavailable.
Introduction	If parameter is not configured or configured with 65535 then retry would happen infinitely. Note: Only used if upper layer is TP.
Multiplicity	0..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

17.3.1.34 SoAd/SoAdGeneral/SoAdUseLocalPortFromConfig

Short Name	SoAdUseLocalPortFromConfig
Description	RB (Vendor06) specific parameter: This parameter would be used only if UL wants to get the valid local port address from the configuration if any, when the API SoAd_GetLocalAddr() is called. Otherwise it will take the local port from the dynamic socket configurations.
Introduction	TRUE: Upper layer can get preconfigured Local Port, before socket is bound. If configured port is wildcard, API will return dynamically selected port after bind. FALSE: Upper layer can only get Local Port for bound sockets. Before socket is bound, API will return wildcard as port value.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

17.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

17.3.2.1 SoAd_ChangeParameter

Syntax	SoAd_ChangeParameter(SoConId,ParameterId,ParameterValuePtr)
Parameters	(IN) SoConId The socket connection index specifying the socket connection to be changed.
	(IN) ParameterId Parameter which is requested to be changed.

	(IN) ParameterValuePtr Pointer to an address which contains the value for the corresponding ParameterId.
Return Value	Returns E_OK if the request is accepted, E_NOT_OK if not accepted.
Description	Changes a connection parameter for the given socket.

17.3.2.2 SoAd_GetAndResetMeasurementData

Syntax	SoAd_GetAndResetMeasurementData(SoAd_MeasurementIdxType MeasurementIdx, boolean MeasurementResetNeeded, uint32 * MeasurementDataPtr)
Parameters In	MeasurementIdx Data index of measurement data. MeasurementResetNeeded Flag to trigger a reset of the measurement data.
Parameters Out	MeasurementDataPtr Reference to data, where to copy measurement data.
Description	Allows the reading and resetting of detailed measurement data for diagnostic purposes.

17.3.2.3 SoAd_ZeroCopyIfTransmit

Syntax	SoAd_ZeroCopyIfTransmit(PduIdType SoAdSrcPduId, const Eth_ZeroCopyTxDataInfo_tst * ZeroCopyData_pcst, boolean TxConfFlag_b, uint32 * TxId_pu32)
Parameters In	SoAdSrcPduId This parameter contains a unique identifier referencing to the PDU Routing Table and thereby specifying the socket to be used for transmission of the data. SoAdSrcPduInfoPtr A pointer to a structure with socket related data: data length and pointer to a data buffer. TxConfFlag_b TxId_pu32 Variable to store index of TxPdu config table
Parameters Out	TcpIpSocketId_u16 TcpIp Socket connection.
Description	Requests for Zero Copy transmission of the L-PDU specified by the SoAd-SrcPduId.

17.3.2.4 SoAd_ZeroCopyTxConfirmation

Syntax	SoAd_ZeroCopyTxConfirmation (TcpIp_SocketIdType SocketId_u16, uint32 TxId_u32)
Parameters In	SocketId_u16 Socket identifier of the related local socket resource. TxId_u32 Variable to store index of TxPdu config table.
Description	The TCP/IP stack calls this function after the data has been acknowledged by the peer for TCP.

17.3.2.5 SoAd_IsRemoteAddrUnLock

Syntax	Std_ReturnType SoAd_IsRemoteAddrUnLock(SoAd_SoConIdType SoConId_uo, SoAd_SoConIdType IdxDynSocket_uo)
Parameters In	SoConId_uo Socket connection index. IdxDynSocket_uo Dynamic socket connection index for the remote address to be checked.
Description	Called from SoAd_ReleaseRemoteAddr to check whether remote address is locked.

17.3.2.6 SoAd_CopyTxDataMetaData

Syntax	SoAd_CopyTxDataMetaData(TcpIp_SocketIdType SocketId , uint8 * BufPtr, uint16 BufLength, const void * CopyTxMetaData_cpst)
Parameters In	TcpIp_SocketIdType Socket identifier of the related local socket resource. uint8* Pointer to buffer for transmission data. uint16 Length of provided data buffer.
Description	Triggered by Tcplp_Transmit, requests to copy data for transmission to the buffer indicated.

17.4 Limitations

17.4.1 General

The following features are not currently supported in SoAd:

- POST-BUILD selectable and loadable.
- Link Time.
- Traffic Class in IPv6 Header.
- TCP Option Filter parameter to turn on/off segments with unknown options per socket connection.
- An API to read the ARP table.
- Support for DEM.
- Path MTU Discovery set for individual sockets.
- Extension of the SoAd Up_SoConModeChg callback to support all Upper Layer modules.
- Target Ids for Xcp in SoAd RxPdu table are not generated using symbolic names.

The following AUTOSAR-defined APIs, Mandatory Interfaces and Parameters are not currently supported in SoAd:

- The mandatory interface Tcplp_GetRemotePhysAddr to get the physical address related to the IP address specified by IpAddrPtr.

- The mandatory interface `Tcplp_GetCtrlIdx` to get the index of the controller.
- New parameter types in `Tcplp_ChangeParameter`.
- `Up_[SoAd][If]TxConfirmation` to be called after successful/ failure transmission.

SoAd also has the following AUTOSAR Variations:

- Locking the remote address during ongoing transmissions and receptions (AR4.3).
- `SoAd_TpChangeParameter` allowing TP upper layer to change some `Tcplp` parameters (as per AR4.3)
- The socket opening/closing sequence correction (Autosar spec section 7.1.3)

17.4.2 Multiple Instances

The current version only supports a single configured instance of the module, therefore `SOAD_INSTANCE_ID` will always be set to 0.

17.4.3 Socket Control

In the current implementation, a socket may be controlled only by a single upper layer module. Support for multiple upper layers will be added in a future version.

17.4.4 Transmission

The module does not currently support retry operations for upper layer modules. Therefore, the retry parameter in functions such as `SoAd_CopyTxData` is always set to `NULL_PTR`.

17.4.5 Multicore Constraint

SoAd will postpone processing of a close request from upper layer in main function during ongoing reception or IF transmission.

17.4.6 Multicore Race Condition

`SoAd_IfTransmit` increments a counter by `Pdulength` to keep track of the number of bytes transmitted, but the same counter gets decremented in `SoAd_TxConfirmation`. When the counter reaches zero, `UL_TxConfirmation` is called by `SoAd_TxConfirmation`.

17.4.7 Tls Support

- Although some Tls-related APIs and Parameters are available, the Tls feature is not currently supported.

17.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

17.5.1 API Functions

The following API functions are not supported:

- SoAd_TpChangeParameter

17.5.2 Configuration Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketPathMTUEnable
- SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTCPOptionFilterRef

17.6 Deviations

17.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

17.6.1.1 SoAd/SoAdBswModules/SoAdLocalIpAddrAssignmentChg

[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

17.6.1.2 SoAd/SoAdBswModules/SoAdSoConModeChg

[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

17.6.1.3 SoAd/SoAdConfig/SoAdPduRoute/SoAdPduRouteDest/SoAdTxPduHeaderId

[ADDED] Introduction	Note: For SomeIP messages, this ID is composed of a Service ID (16 bits) occupying the most significant 16 bits, and a Method/Event ID (16 bits), which is a unique identifier for either a method or an event, occupying the least significant 16 bits. To differentiate between methods and events, bit 15 of the Method/Event ID is reserved as a flag: 0 represents a method, and 1 represents an event.
----------------------	--

17.6.1.4 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTcpAutoConnectTimeout

[RTA-BSW] Description	Specifies the time in seconds how long TCP connect attempts are repeated to reach SOAD_SOCON_ONLINE. This parameter is restricted to socket connection groups which are initiating a TCP connection and are under control of SoAd.
[AUTOSAR] Description	Specifies the time in seconds how long TCP connect attempts are repeated to reach SOAD_SOCON_ONLINE. This parameter is restricted to socket connection groups which are initiating a TCP connection and are under control of SoAd.

17.6.1.5 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTcpKeepAlive

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

17.6.1.6 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketTcp/SoAdSocketTcpRetransmissionTimeout

[RTA-BSW] Description	Timeout in [s] before an unacknowledged TCP segment is sent again. If the timeout is set to INF, no TCP segments shall be retransmitted.
[AUTOSAR] Description	Timeout in [s] before an unacknowledged TCP segment is sent again.
[REMOVED] Introduction	If the timeout is set to INF, no TCP segments shall be retransmitted.

17.6.1.7 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketProtocol/SoAdSocketUdp/SoAdSocketUdpStrictHeaderLenCheckEnabled

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

17.6.1.8 SoAd/SoAdConfig/SoAdSocketConnectionGroup/SoAdSocketSoConModeChgBswMNotification

[RTA-BSW] Description	Specifies if the BswM_SoAd_SoConModeChg notification shall be called in case of SoCon mode change.
[AUTOSAR] Description	Specifies if the BswM_SoAd_SoConModeChg notification shall be called in case of SoCon mode change
[ADDED] Default Value	false

17.6.1.9 SoAd/SoAdGeneral/SoAdEnableSecurityEventReporting

[RTA-BSW] Description	Switches the reporting of security events to the IdsM:
[AUTOSAR] Description	Switches the reporting of security events to the IdsM: Tags: atp.Status=draft

17.6.1.10 SoAd/SoAdGeneral/SoAdGetAndResetMeasurementDataApi

[ADDED] Introduction	TRUE: Enables the SoAd_GetAndResetMeasurementData feature. FALSE: Disables the SoAd_GetAndResetMeasurementData feature.
----------------------	--

17.6.1.11 SoAd/SoAdGeneral/SoAdSecurityEventRefs

[RTA-BSW] Description	Container for the references to IdsMEvent elements representing the security events that the SoAd module shall report to the IdsM in case the corresponding security related event occurs (and if SoAdEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events.
[AUTOSAR] Description	Container for the references to IdsMEvent elements representing the security events that the SoAd module shall report to the IdsM in case the corresponding security related event occurs (and if SoAdEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events. Tags: atp.Status=draft

17.6.1.12 SoAd/SoAdGeneral/SoAdSecurityEventRefs/SOAd_SEV_-DROP_MSG_RX_UDP_LENGTH

[RTA-BSW] Description	SoAd dropped a message. The message contains at least one PDU which violates stack configuration and was received via a UDP socket. The violation relates to the length of the PDUs compared to the overall length of the message.
[AUTOSAR] Description	SoAd dropped a message. The message contains at least one PDU which violates stack configuration and was received via a UDP socket. Tags: atp.Status=draft
[REMOVED] Introduction	The violation relates to the length of the PDUs compared to the overall length of the message.

17.6.1.13 SoAd/SoAdGeneral/SoAdSecurityEventRefs/SOAd_SEV_-DROP_MSG_RX_UDP_SOCKET

[RTA-BSW] Description	SoAd received a UDP message which violates stack configuration and was dropped. No suitable socket connection matching to configuration was found.
-----------------------	--

[AUTOSAR] Description	SoAd received a UDP message which violates stack configuration and was dropped. Tags: atp.Status=draft
[REMOVED] Introduction	No suitable socket connection matching to configuration was found.

17.6.1.14 SoAd/SoAdGeneral/SoAdSecurityEventRefs/SOAd_SEV_DROP_PDU_RX_TCP

[RTA-BSW] Description	SoAd dropped a PDU. The PDU violates stack configuration and was received via a TCP socket.
[AUTOSAR] Description	SoAd dropped a PDU. The PDU violates stack configuration and was received via a TCP socket. Tags: atp.Status=draft

17.6.1.15 SoAd/SoAdGeneral/SoAdSecurityEventRefs/SOAd_SEV_DROP_PDU_RX_UDP

[RTA-BSW] Description	SoAd dropped a PDU. The PDU violates stack configuration and was received via a UDP socket.
[AUTOSAR] Description	SoAd dropped a PDU. The PDU violates stack configuration and was received via a UDP socket. Tags: atp.Status=draft

17.6.1.16 SoAd/SoAdGeneral/SoAdSecurityEventRefs/SOAd_SEV_REJECTED_TCP_CONNECTION

[RTA-BSW] Description	SoAd rejected a TCP connection. The connection request violates stack configuration.
[AUTOSAR] Description	SoAd rejected a TCP connection. The connection request violates stack configuration. Tags: atp.Status=draft

17.6.1.17 SoAd/SoAdGeneral/SoAdSoConMax

[RTA-BSW] Introduction	Furthermore it defines the platform type used for SoAd_SoConIdType. If SoAdSoConMax is not greater than 256, a uint8 is used, otherwise uint16. DataType SoAd_SoConIdType depends on this number. Bigger number (> 256) would consume more memory as uint16 will be used.
[AUTOSAR] Introduction	Furthermore it defines the platform type used for SoAd_SoConIdType. If SoAdSoConMax is not greater than 256, a uint8 is used, otherwise uint16.

17.7 API Definition

List of supported API functions

Table 17.59. SoAd Supported API functions

SoAd_Init	Initialises all global variables of a Socket Adaptor instance and puts it into the idle state (SOAD_SOCON_OFFLINE).
SoAd_MainFunction	SoAd Main Function.
SoAd_GetVersionInfo	Return version information for the module.
SoAd_IfTransmit	Initiates a request for transmission of the IF-PDU specified by the SoAdSrcPduId.
SoAd_IfRoutingGroupTransmit	Triggers the transmission of all transmit IF-PDUs identified by the parameter id after requesting the data from the related upper layer.
SoAd_IfSpecificRoutingGroupTransmit	Triggers the transmission of all transmit IF-PDUs identified by the parameter id for a specific socket after requesting the data from the related upper layer.
SoAd_TpTransmit	Requests transmission of a TP-PDU.
SoAd_TpCancelTransmit	Requests cancellation of the transmission of a specific TP-PDU.
SoAd_TpCancelReceive	Requests cancellation of the reception of a specific TP-PDU.
SoAd_GetSoConId	Returns the socket connection index related to a specified transmit PduId.
SoAd_OpenSoCon	Opens the socket connection specified by SoConId.
SoAd_CloseSoCon	Closes the socket connection specified by SoConId.
SoAd_RequestIpAddrAssignment	Requests a local IP address assignment for the specified socket.
SoAd_ReleaseIpAddrAssignment	Releases the local IP address assignment for the specified socket.
SoAd_GetLocalAddr	Obtains the local address (IP address and port) actually used for the SoAd socket connection specified by SoConIdx.

SoAd_GetPhysAddr	Obtains the physical source address of the EthIf controller used by the SoAd socket connection specified by SoConIdx.
SoAd_GetRemoteAddr	Retrieves the remote address (IP address and port) actually used for the SoAd socket connection specified by SoConId.
SoAd_EnableRouting	Enables routing of a group of PDUs related to the RoutingGroup specified by parameter id.
SoAd_EnableSpecificRouting	Enables routing of a group of PDUs related to the RoutingGroup specified by parameter id (only for the socket connection identified by SoConId).
SoAd_DisableRouting	Disables routing of a group of PDUs related to the RoutingGroup specified by parameter id.
SoAd_DisableSpecificRouting	Disables routing of a group of PDUs related to the RoutingGroup specified by parameter id (only for the socket connection identified by SoConId).
SoAd_SetRemoteAddr	Sets the remote IP address and port of the specified socket connection.
SoAd_SetUniqueRemoteAddr	Returns the socket connection index related to the specified connection group or assigns the remote address to an unused socket connection in the group.
SoAd_ReleaseRemoteAddr	Releases remote address (IP address and port) of specified socket connection returning to the configured remote address setting.
SoAd_ReadDhcpHostNameOption	Reads the currently configured DHCP hostname.
SoAd_WriteDhcpHostNameOption	Sets the DHCP hostname.
SoAd_RxIndication	Callback function (called by the Tcplp module) after reception of data on a specified socket.
SoAd_CopyTxData	Callback function which copies the requested PDU data for transmission into the specified buffer (called by the transmit function of the Tcplp module).

SoAd_TxConfirmation	Callback function (called by the Tcplp module) indicating acknowledgement of a received transmission request.
SoAd_TcpAccepted	Callback function (called by the Tcplp module) after a socket was set to the listen state and a TCP connection was requested.
SoAd_TcpConnected	Callback function (called by the Tcplp module) after a socket and TCP connection was requested.
SoAd_LocalIpAddrAssignmentChg	Called by the Tcplp module if an IP address assignment changes.
SoAd_TcpIpEvent	Callback function (called by the Tcplp module) if a condition described in the event parameters occurs.
SoAd_GetSoConMode	Returns current state of the socket connection specified by SoConId.
SoAd_IsConnectionReady	Checks if communication over socket is possible for a dedicated remote address.

For Further information please refer to AUTOSAR_SWS_SocketAdapter.pdf

17.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

17.8.1 General

- If SoAdBestMatchRxHeaderIdCheck = TRUE, SoAd checks the Rx Header Id to identify the best match socket using a 'Best Match' algorithm.
- If SoAdRestUdpMsgDiscardByInvalidPduHeaderId = TRUE, SoAd discards the rest of the message if the received PDU cannot be mapped to a configured Rx-PDU.
- If FAN-OUT is configured, and if any of the PduRouteDests are linked BundleRef, then at least one socket connection for each PduRouteDest should be opened before calling the SoAd_IfTransmit, otherwise E_NOT_OK will be returned.

17.8.2 Routing Groups

- If multiple I-PDUs are mapped to a single Routing Group Id, then the upper layer triggers the transmission via the Routing Group Id. If a triggered transmission is activated, then SoAd will make a transmit request for each I-PDU mapped to the Routing Group Id.
- If a Routing Group is disabled for a transmitted I-PDU, the transmission request from the upper layer will be rejected. Similarly, if it is disabled for received I-PDUs, then received packets will be discarded in the SoAd_RxIndication callback function.

17.8.3 New Parameters

The following new parameters have been introduced in the SoAd configuration:

- SoAdChangeParameterEnabled
- SoAdDHCPNameEnabled
- SoAdIPAddrAssignmentEnabled
- SoAdTpCancelComEnabled

These parameters must be configured once, after which their configured values will be reflected in project-specific EcuC values, which will be retained until you change the values for the parameters. If you don't configure these new parameters after their introduction, CodeGen will fail.

17.8.4 Additional Advice and Examples

Multiple RxPdu's under different socketRoute mapped to different socket connection/group should not have same header Id

See the Integration Advice for Sd for some additional information on configuring and integrating the SoAd and Sd modules.

17.9 Integration Advice

17.9.1 Init Functions

SoAd_Init must be called during the start-up phase of the target ECU. It can also be used to re-initialise the module.

17.9.2 Delnit Functions

No Delnit is required.

17.9.3 Scheduled Functions

SoAd_MainFunction is responsible for setting the socket states of OFFLINE, RECONNECT and ONLINE. It also processes TP transmissions and triggered transmit actions. It performs all cyclic actions, including timeout checking.

If the values of SoAdTxUdpTriggerTimeout, SoAdSocketUdpTriggerTimeout or

SoAdSocketUdpAliveSupervisionTimeout are not divisible exactly by the value of SoAdMainFunctionPeriod, then SoAd_MainFunction (period = SoAdMainFunctionPeriod) and SoAd_MainFunction_1ms (period = SoAdMainFunctionPeriod_1ms) are both generated. Otherwise, only SoAd_MainFunction is generated.

17.9.4 General

Successful integration of the SoAd module depends on the correct configuration of upper and lower layer modules. The Tcplp module is always configured for use with SoAd, as the callback services and TP-PDU processing functions are specifically tied to Tcplp. For the upper layer, SoAd implements a generic interface, so any suitable upper layer module may be configured for using the IF-PDU services provided by SoAd. At least one upper layer module must be configured for SoAd, and will typically be one of the following.

- PDU Router (PduR); IF-PDU and TP-PDU interfaces.
- UDP Network Management (UdpNm); IF-PDU interface.
- XCP on Ethernet (Xcp); IF-PDU interface.
- Service Discovery (Sd); IF-PDU interface.

Note that the PduR module is a special case because it uses both IF-PDU and TP-PDU API services. However, for the purposes of integration, it is considered as an upper layer module.

The SoAd module now supports the previously unsupported PDU route path of SoAd=>PduR=>DCM.

17.9.5 SoAd_CopyTxDataMetaData

The SoAd_CopyTxDataMetaData extension API is called by the Tcplp layer if NULL_PTR is passed in place of a data pointer when Tcplp_UdpTransmit or Tcplp_TcpTransmit is called. This happens when the Header is enabled for IF-PDU and for TP irrespective of enabled header. As Tx buffers are not available in SoAd, it cannot store the data, so it depends on the upper layer copyTxData function. See the Tcplp Reference Guide for more information.

17.9.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The SoAd module uses the following OS resources:

- SoAd_UdpSupervisionTimer
- SoAd_nPduUdpTx
- SoAd_nPduUdpTxTimer
- SoAd_RoutingGroupStatus
- SoAd_ReserveFreeDynSoc
- SoAd_ULIfUdpTxConfirm
- SoAd_MeasurementData
- SoAd_RxBufferAccess
- SoAd_IfRxBuf
- SoAd_TpRxBuf
- SoAd_OpenCloseReq
- SoAd_SocConModeProperties
- SoAd_RemoteAddrUpdate
- SoAd_TxRxReq
- SoAd_IfPduInUse
- SoAd_TpPduInUse
- SoAd_IfTcpTxQueueFreeCnt
- SoAd_IfTcpTx
- SoAd_IfRgTx

17.10 Hardware Requirements

Table 17.60. SoAd Hardware Requirements

Support	Usage
FLASH	32455 bytes
RAM	7399 bytes

Usage calculated based on a sample configuration.

18 SOME/IP Transport Protocol

Module name	SomelpTp
Package	RTA-ETH
Module ID	177
AUTOSAR Version	22-11

18.1 Introduction

The Scalable service-Oriented MiddlewarE over IP (SOME/IP) Transport Protocol module, SomelpTp, is located next to PduR and is used to segment and re-assemble SOME/IP packets between LdCom and SoAd which do not fit into a single UDP packet.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

18.2 Supported Features

18.2.1 AUTOSAR Features

- Segmentation of SOME/IP messages (TX Path)
- Assembly of received SOME/IP messages (RX path)

18.2.2 Non-AUTOSAR Features

In addition to the above AUTOSAR-specified features, the RTA-BSW implementation of SomelpTp also supports the following:

- Transmission of multiple segments in one Tx MainFunction (see “Fast Transmission Mode” in the Configuration Advice for more details).
- The above feature also includes the ability to disable the TxConfirmation of the lower layers (to speed up data transmission).

18.3 Extensions

18.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

18.3.1.1 SomelpTp/SomelpTpChannel/SomelpTpRbTxBusytimeout

Short Name	SomelpTpRbTxBusytimeout
------------	-------------------------

Description	This timer starts when PduR_SomelpTpCopyTx-Data() returns BUFRE_E_BUSY during the transmissions of N-PDUs. And this is stopped when PduR_SomelpTpCopyTxData() returns BUFRE_E_OK/BUFRE_E_NOT_OK for the same N-PDU. If the data is still not available from upper layer even after the SomelpTpRbTxBusytimeout is elapsed then the transmission is aborted. Hint: the sum of SomelpTpRbTxBusytimeout and SomelpTpNPduSeparationTime should be less than SomelpTp receiver SomelpTpRxTimeoutTime for proper communication. [(SomelpTpRbTxBusytimeout + SomelpTpNPduSeparationTime) less than SomelpTpRxTimeoutTime]
Multiplicity	0..1
Value Range	0..Inf
Default Value	0
Type	ECUC-FLOAT-PARAM-DEF

18.4 Limitations

The following features are not supported for version v12.7.0

- There are no known limitations in this version.

18.5 Exclusions

There are no unsupported API functions in this release.

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- SomelpTp/SomelpTpGeneral/SomelpTpVersionInfoApi

18.6 Deviations

18.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

18.6.1.1 SomelpTp/SomelpTpChannel/SomelpTpNPduSeparationTime

[RTA-BSW] Maximum Value	Inf
[AUTOSAR] Maximum Value	INF

18.6.1.2 SomelpTp/SomelpTpChannel/SomelpTpRxTimeoutTime

[RTA-BSW] Maximum Value	Inf
[AUTOSAR] Maximum Value	INF

18.6.1.3 SomelpTp/SomelpTpGeneral/SomelpTpRxMainFunctionPeriod

[RTA-BSW] Maximum Value	Inf
[AUTOSAR] Maximum Value	INF

18.6.1.4 SomelpTp/SomelpTpGeneral/SomelpTpTxMainFunctionPeriod

[RTA-BSW] Maximum Value	Inf
[AUTOSAR] Maximum Value	INF

18.7 API Definition

List of supported API functions

Table 18.7.SomelpTp Supported API functions

SomeIpTp_GetVersionInfo	Returns the version information of this module.
SomeIpTp_MainFunctionTx	Performs the processing of SomelpTp's transmission activities.
SomeIpTp_MainFunctionRx	Performs the processing of SomelpTp's reception activities.
SomeIpTp_Init	Initializes the SomelpTp module.
SomeIpTp_Transmit	Requests transmission of a PDU.
SomeIpTp_TriggerTransmit	Used by the called module to check whether the available data fits into the buffer size reported by PduInfoPtr→SduLength.
SomeIpTp_RxIndication	Indication of a received PDU from a lower layer communication interface module.
SomeIpTp_TxConfirmation	Indication of a transmitted PDU from a lower layer communication interface module.

For further information on these functions, please refer to AUTOSAR_SWS_SOMEIPTransportProtocol.pdf

18.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

18.8.1 AUTOSAR Version

The RTA-BSW implementation of SomelPtp is based on R21-11, with an R21-11 Paramdef.

18.8.2 Fast Transmission Mode

The RTA-BSW implementation of SomelPtp allows for the transmission of multiple segments in one Tx MainFunction. This feature is controlled by the following configuration parameters:

- **SomelPtpRbFastTransmission:** Set this to TRUE to enable Fast Transmission Mode.
- **SomelPtpRbFastTransNumberOfSegments:** Defines the number of segments that can be sent in FastTransmission mode.

18.8.3 SomelPtpChannel

As shown in the following example, you must define each routing path from SomelPtp to/from PduR. For the Tx path it would be as follows:

- **SomelPtpTxNSduRef:** Pdu from PduR to SomelPtp
- **SomelPtpTxNPduRef:** Pdu sent from SomelPtp to PduR

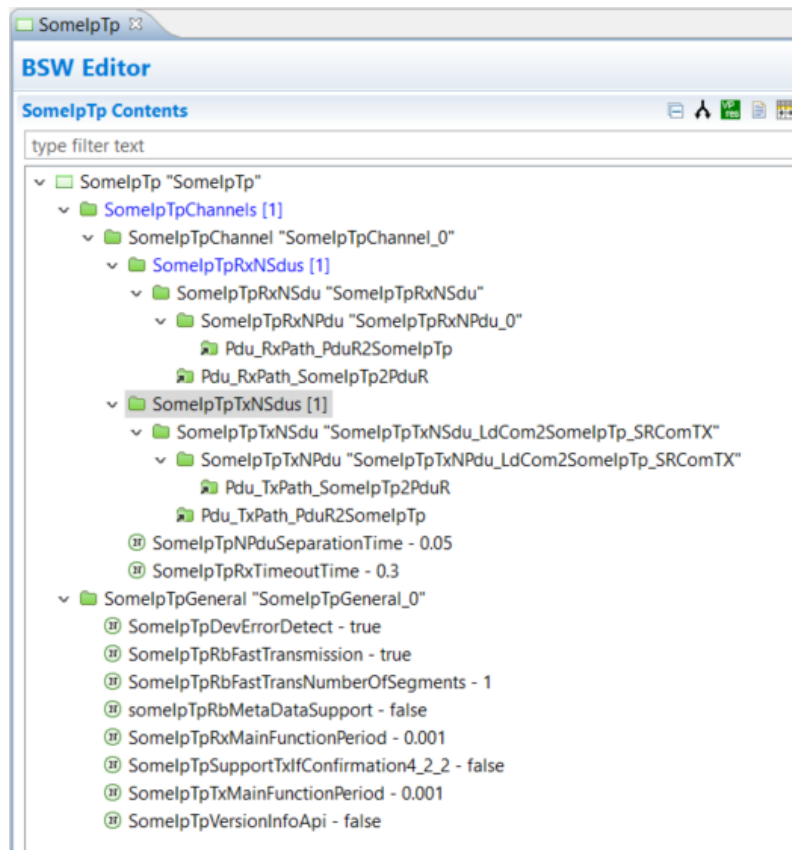


Figure 18.1.Example Configuration

18.9 Integration Advice

18.9.1 Init Functions

SomelpTp is initialized by calling SomelpTp_Init.

18.9.2 Delnit Functions

No Delnit is required.

18.9.3 Scheduled Functions

SomelpTp provides the following scheduled interfaces:

- SomelpTp_MainfunctionRx
- SomelpTp_MianfunctionTx

Both of these interfaces have to be scheduled cyclically. The calling period has to be the same as that which is configured in the SomelpTp EcuC-values file.

18.9.4 Pdu Size - EcuC

As illustrated in the example below, Pdu size will be calculated from the Data

Structure Size (35bytes) + SomelpXf header (8bytes), equalling 43 bytes in all.

- Rte_Rips_SRCom $\leftrightarrow$ LdCom $\leftrightarrow$ PduR $\leftrightarrow$ SomelpTp $\leftrightarrow$ SomelpXf

The Pdu from the lower layer will decide the UDP Data size to be sent in one Ethernet Frame. When SOME/IP data is segmented it requires 12 bytes for the SOME/IP TP Headers. The offset field will be calculated as 16 bytes for each unit, so a minimum UDP Data size that will support all fields is 28bytes. In our example below, the UDP Data Size is configured as 32 bytes.

- SomelpTp $\leftrightarrow$ PduR $\leftrightarrow$ SoAd $\leftrightarrow$ Tcplp



Figure 18.2.Example configuration for Pdu size

18.9.5 rba_SysElem Configuration

As shown in the example below, although Xfrm is used for transformation, it is `rba_SysElem` that should be defined, and the data structure should be transformed as per this example which contains a structure with one `unit8`, one `unit16`, and 8 `unit32` elements to align with the `SomelpTp` configuration.

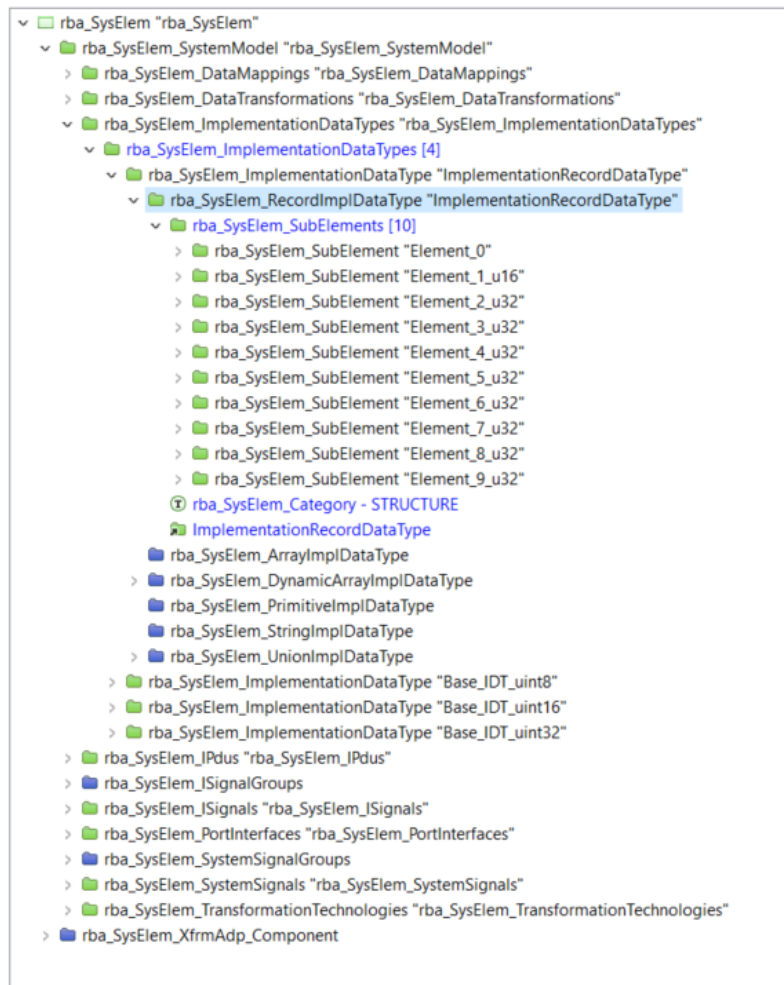


Figure 18.3. Example configuration for `rba_SysElem`

18.9.6 API Type

The following API Type configuration is required:

- Interface from upper layer to `LdCom`: TP
- Interface from `LdCom` to `PduR`: TP
- Interface from `PduR` to `SomelpTp`: TP
- Interface from `PduR` to `SoAd`: IF

18.9.7 PduR Configuration

Because SomelpTp is at the same level as PduR, the following configuration of PduR related parameters is required:

- PduRCommunicationInterface: TRUE
- PduRLowerModule: TRUE
- PduRTransportProtocol: TRUE
- PduRTriggertransmit: TRUE
- PduRUpperModule: TRUE

18.9.8 Rba_DiagLib

If your project uses SomelpTp, the Rba_DiagLib module must also be included.

18.9.9 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The SomelpTp module uses the following OS resources:

- SomelpTp
- SomelpTpSomelpTp_Reception

18.10 Hardware Requirements

Table 18.8.SomelpTp Hardware Requirements

Support	Usage
FLASH	2112 bytes
RAM	632 bytes

Usage calculated based on a sample configuration.

19 SOME/IP Transformer

Module name	SomelpXf
Package	RTA-ETH
Module ID	174
AUTOSAR Version	22-11

19.1 Introduction

The Scalable service-Oriented MiddlewarE over IP (SOME/IP) Transformer, SomelpXf, serializes and de-serializes data according to the SOME/IP on-the-wire format and specifies an automotive/embedded mechanism for Client/Server communication.

Beyond this format conversion, SomelpXf is also responsible for handling the SOME/IP protocol specific header information, and it builds a portion of the SOME/IP header for data messages (the Message ID and Length are added by SoAd).

Although SomelpXf is part of the RTA-ETH stack, it also depends on the rba_SysElem and rba_Xfrm modules, which are part of the RTA-COM stack.

SomelpXf reads the Com module configuration, which contains the Signal name used to build the TransformerId, and it reads the LdCom module configuration, which contains the PDU name used to build the TransformerId.

The diagram below shows the position of SomelpXf and its dependent modules in the AUTOSAR layered architecture.

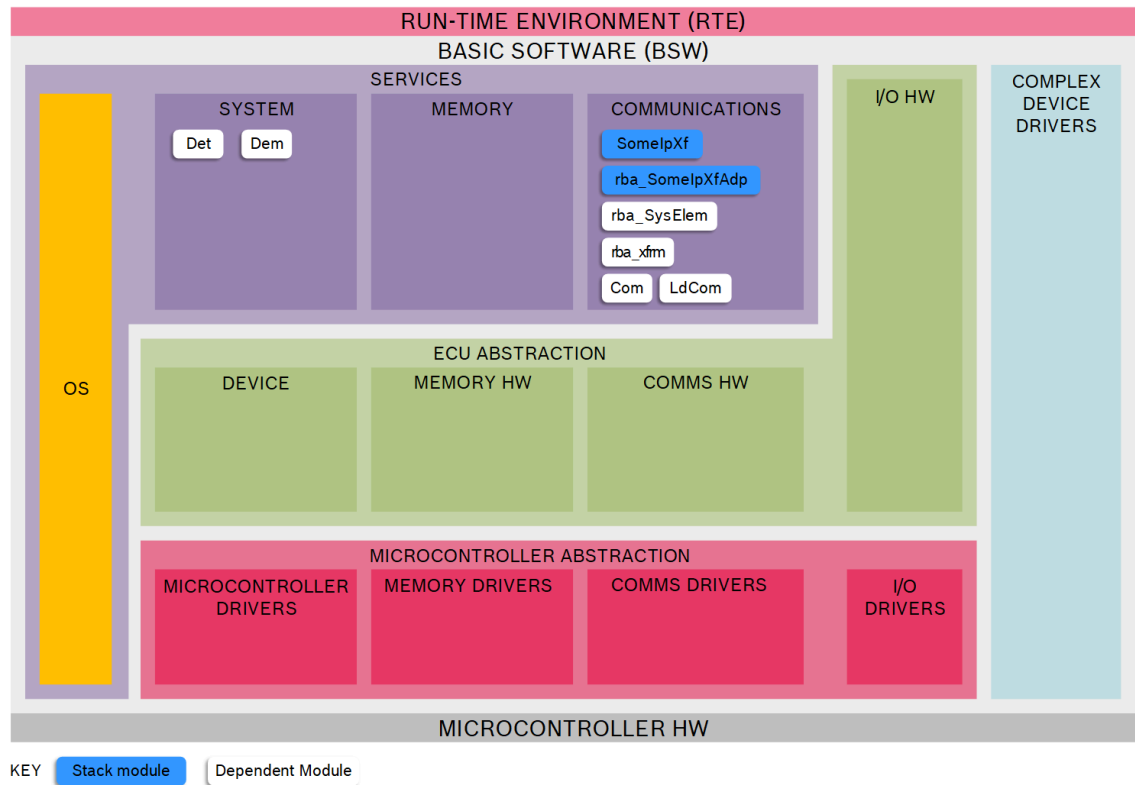


Figure 19.1.SomelpXf Architecture

According to the AUTOSAR standard, the transformer functionality is intended to manipulate system signal data before handing it over to the Com module.

A SWC initiates an inter-ECU communication, which is configured to be transformed, and then hands the data over to the RTE. The RTE executes the configured transformer chain which contains (if the configuration demands this) SomelpXf.

The Output of SomelpXf is handled by the RTE as an array of bytes and it is passed to the subsequent transformers in the chain (if any) and then to Com or LdCom.

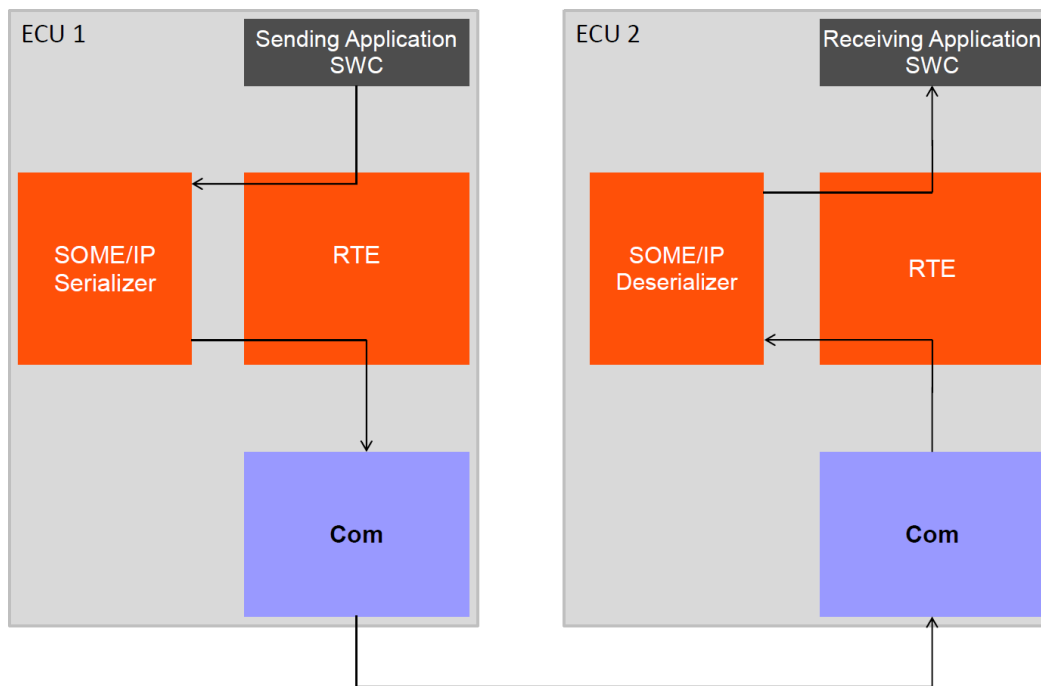


Figure 19.2.SomelpXf Architecture

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

19.2 Supported Features

19.2.1 Using SomelpXf on other networks

The SOME/IP Transformer can be used on networks other than Ethernet (e.g. CAN, Flexray, etc.) and on both UDP and TCP, with the following caveats:

- Tunnelling of SOME/IP messages through CAN and Flexray leads to SOME/IP messages without parts of the header inserted by the Socket Adaptor.
- SomelpXf currently achieves "maybe" reliability when using the UDP binding and "exactly once" reliability when using the TCP binding by a suitable configuration of the Ethernet modules.

19.2.2 Protobuf

The SOME/IP Transformer has an extension allowing for the transfer of data payloads in the Protobuf format.

Therefore, besides offering serialization/de-serialization of data in the SOME/IP format, SomelpXf also supports either C/S or S/R communication via Protobuf, which is Google's language-neutral, platform-neutral, extensible mechanism for serialization/de-serialization of structured data.

19.3 Extensions

- No known extension parameters are defined in this version.

19.4 Limitations

The following features are not supported for version v12.7.0

- No support for Nested Union and Dynamic Array within the Union.
- No support for TLV all Unions and its Union members.
- A Union shall not be detected if the structure does not contain exactly two elements a "memberSelector" and "payload", and a validation error shall occur.
- No Support of Trigger Interfaces
- No Support of dynamic UTF-16 String Datatype
- No support of Init value for Primitive Datatype

The following limitations apply when Protobuf is enabled:

- If a Protobuf message is of the nested structure type, and the definition of the nested element types are present in other (.proto) protofiles, all the protofile names must be configured (comma separated) in the rba_SysElem_I_Signal under the parameter rba_SysElem_ProtobufFileNames.
- Bufferlength of type uint32 is supported up to the value of 2^{18} due to tool constraints.

19.5 Exclusions

List of excluded API functions

Table 19.2.SomelpXf Excluded API functions

SomeIpXf_Init	Initializes the Transformer.
SomeIpXf_DeInit	Deinitializes the transformer.

For further information on these functions, please refer to AUTOSAR_SWS_SOMEIPTransformer.pdf

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

19.6 Deviations

19.6.1 Session Counter Size

For each XfrmMapping that refers to a transformer technology that has session handling enabled and the communication direction is "OUT", SomelpXf will main-

tain a session counter and use it as the session Id.

The session counter size specified by AUTOSAR is 4 bytes, but the ISO norm specifies 2 bytes. To ensure that there are no incompatibilities to non-AUTOSAR implementations of SOME/IP, SomeIpXf implements a 2 byte (uint16) session counter.

If the received bufferLength is less than 4, Then the Client ID and Sequence Counter in the Transaction Handle (out parameter of SomeIpXf_Inv_transformerId API) would be updated with 0 during deserialization.

19.6.2 Configuration Deviations

- There are no known deviations in this version.

19.7 API Definition

List of supported API functions

Table 19.3. SomeIpXf Supported API functions

SomeIpXf_<transformerId>	Transforms a Sender/Receiver communication using the serialization of SOME/IP.
SomeIpXf_Inv_<transformerId>	Deserializes a Sender/Receiver communication using the deserialization of SOME/IP.
SomeIpXf_ExtractProtocolHeaderFields	Extracts SOME/IP protocol header fields and type of message result.

For further information on these functions, please refer to AUTOSAR_SWS_SOMEIPTransformer.pdf

19.8 Configuration Advice

 **NOTE**

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

SomeIpXf has no module-specific EcuC because its whole configuration is based on the SOMEIPTransformationDescription and SOMEIPTransformationSignalProps.

19.8.1 ISOLAR-A/B

ISOLAR-A/B does not currently support mappings of complex data structures to System Signals (required for the SOME/IP serialisation). The mapping should be performed manually in the ARXML.

19.8.2 Configuration of PDU Objects

The PDU objects flowing through the Com stack should be correctly configured with the proper PduLength (based on the SOME/IP Transformer header + payload) in the EcuC module configuration.

In this example, 8 bytes (SOME/IP Transformer header) + 7 bytes (data structure to be serialised) = 15 bytes.

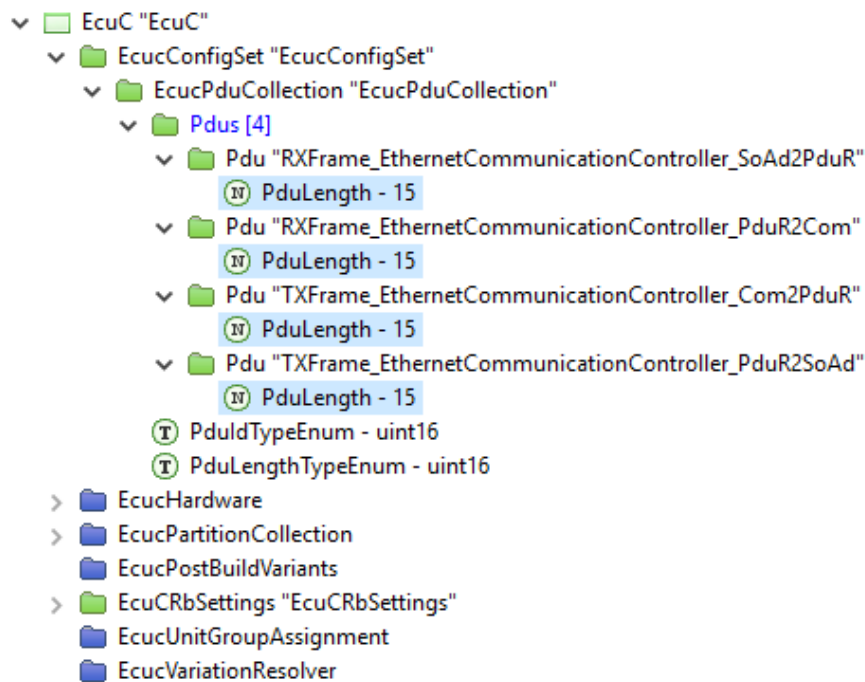


Figure 19.3.PduLength Configuration

Note that the header bytes should only be taken into account when configuring the PduLength, but not when configuring the size of the ISignal (which should include only the serialised data size).

19.8.3 Configuration of Protobuf

If the `rba_SysElem_ProtobufFileNames` parameter is configured in the `rba_SysElem_ISignal` container, and the `rba_SysElem_ProtobufMessageName` parameter is configured in the container `VariableDataPrototype/argumentDataPrototype` (for either S/R or C/S communication), then `SomelpXf` will serialize/de-serialize data using Protobuf. Otherwise, data will be serialized/de-serialized according to the (default) SOME/IP protocol.

One of the fields configured in the proto file must be mapped to the payload subelement of `UnionImplDataType` in the arxml. The payload subelement should be part `RecordImplDataType` having exactly two subelements member selector and payload.

19.8.4 Configuration of SomelpXf Init Value

- If the overall size of the init values configured are equal to payload plus the header size, then the init values for header should be considered. Any extra bytes more than the payload plus the header size shall be ignored.
- If the Init values configured is less than Payload plus the header, then consider the init values for the payload only. Any extra bytes more than the payload size shall be ignored plus the header init value shall be added.
- If Init values size is equal to Payload size, then header init value shall be added.
- If Init values size is less than payload size, then an error will occur.
- TLV IDs are not considered for UnionImplementationDatatype, therefore this will be ignored if configured.

19.9 Integration Advice

To integrate the SOME/IP Transformer:

1. Import the systemExtract with the rba_SysElem importer in order to generate the required configuration for the SOME/IP generation.
2. Import the systemExtract with the Com importer in order to generate the required configuration for the generation of the ComStack code.
3. Generate the SOME/IP Transformer.

To integrate the SOME/IP Transformer with Protobuf:

1. Import the systemExtract with the rba_SysElem importer in order to generate the required configuration for the SOME/IP generation.
2. Import the systemExtract with the Com importer in order to generate the required configuration for the generation of the ComStack code.
3. Place the required (.proto) Protobufs in a folder and configure the path of the root folder in the parameter rba_XfrmSomelpXfProtofilePath in (SomelpXf's) XfrmGeneral container.
4. Generate the SOME/IP Transformer.

19.9.1 Init Functions

There are no specific initialization actions required for the SomelpXf module.

19.9.2 Delnit Functions

No Delnit is required.

19.9.3 Scheduled Functions

SomelpXf does not have a scheduled function.

19.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The SomelpXf module does not use any exclusive OS resources.

19.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

20 TCP/IP Stack

Module name	Tcplp
Package	RTA-ETH
AUTOSAR Module ID	170
AUTOSAR Version	22-11

20.1 Introduction

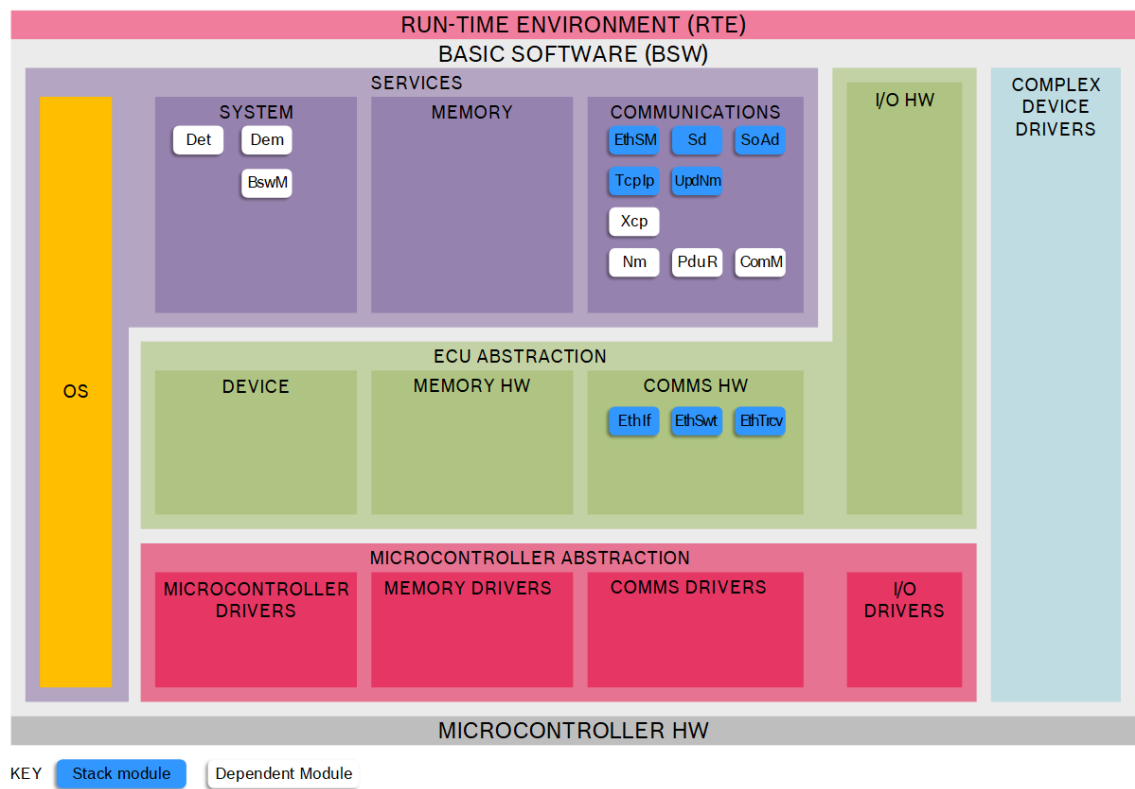


Figure 20.1.RTA-ETH Architecture

The TCP/IP Stack module (Tcplp) is located in the communication services abstraction layer, and sits between the Socket Adapter (SoAd) and Ethernet Interface (EthIf) modules. The diagram above shows the position of Tcplp in the AUTOSAR layered architecture.

The Ethernet State Manager (EthSM) controls the communication mode of the Tcplp module by requesting communication modes from it. In return, Tcplp notifies EthSM about communication mode changes.

The Tcplp module consists of an abstracted, protocol-independent wrapper, within which a number of sub-modules are encapsulated. Each of these sub-modules provide a specific communication protocol, which may be used to send and receive Internet Protocol (IP) data for the purpose of in-vehicle and diagnostic communication.

The functionality of Tcplp (with respect to the supported protocols) can be summarised as follows:

- Message Transmission.
- Message Reception.
- State Handling.
- Socket Handling.
- Generic operations (e.g. IP address assignment, initialisation, callback notification).

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

20.2 Supported Features

20.2.1 Message Transmission (using AR Interface)

For transmission of UDP messages, the Tcplp module provides the Tcplp_UdpTransmit and Tcplp_UdpTransmitMetaData APIs.

For transmission of TCP messages, the Tcplp module provides the Tcplp_TcpTransmit and Tcplp_TcpTransmitMetaData APIs.

Tcplp_UdpTransmitMetaData and Tcplp_TcpTransmitMetaData can be used by the Upper Layer for parallel transmission (reentrancy) on the same socket with DataPtr set to NULL_PTR of UDP/TCP messages, otherwise Tcplp_UdpTransmit and Tcplp_TcpTransmit can be used to transmit the message.

Parallel transmission on the same socket requires extra information, which is achieved by passing MetaDataPtr in Tcplp_UdpTransmitMetaData and Tcplp_TcpTransmitMetaData.

- For UDP messages, transmission with DataPtr set to NULL_PTR and with valid MetaData, Tcplp_UdpTransmitMetaData in turn calls UL_CopyTxDataMetaData with the same MetaDataPtr, which allows the Upper Layer to identify the data to be copied into the transmit buffer.
- The same applies for TCP messages, where Tcplp_TcpTransmitMetaData in turn calls UL_CopyTxDataMetaData.

When there is reentrancy on the same socket, the Autosar callback UL_CopyTxData cannot be used because it is lacking context information (the argumentSocketId is not sufficient during reentrancy to identify the data to be copied into transmit buffer).

The Tcplp module transmits the messages of three different protocol types via the AR defined interface:

- TCP (Transmission Control Protocol)

- UDP (User Datagram Protocol)
- ICMP (Internet Control Message Protocol)

Upper layer requests for transmission of TCP and ICMP messages are forwarded to the respective sub-modules (rba_EthTcp and rba_EthIcmp/rba_EthIcmpv6).

For UDP data transmission, Tcplp requests for access to a transmit buffer via API rba_EthUdp_ProvideTxBuffer (sub-module rba_EthUdp). On successful allocation of the buffer, rba_EthUdp_Transmit is called to transmit the message.

20.2.2 Message Transmission (using ZeroCopy Interface)

The Tcplp module transmits messages of the following protocol types via the ZeroCopy interface (Tcplp_ZeroCopyUdpTransmit):

- UDP (User Datagram Protocol)

ZeroCopy message transmission is allowed either in Static Mode or in Dynamic Mode.

20.2.2.1 Static mode

In Static Mode, support of IP fragmentation is not available, and the Checksum must be computed in HW.

Only Reuse headers are allocated with a predefined length. Due to availability of headers with only predefined length, Static mode is be more efficient for transmission of frames with the same sizes, or a limited set of sizes.

The upper layer can give multiple data pointers (multiple Ethernet frames will be constructed) a part of a single Tcplp_ZeroCopyUdpTransmit.

20.2.2.2 Dynamic mode

In Dynamic Mode, IP fragmentation is available, and the checksum calculation is computed in SW.

As well as Reuse Headers, IpOneTimeHeader and UdpOneTimeHeader are also available.

For data packets with variable lengths, Dynamic Mode is more than Static Mode because OneTime header is reallocated for the different length, which prevents the reallocation and the rewriting of a completed Reuse header.

Multiple data pointers, as part of a single Tcplp_ZeroCopyUdpTransmit, is not allowed with Dynamic mode.

20.2.2.3 ZeroCopy Transmission

For ZeroCopy data transmission, Tcplp allocates buffers to store the header fields.

In Dynamic Mode, the size of the Reuse header buffer is 36 bytes. In Static Mode, the size of the Reuse header buffer is 46 bytes, and it is used to store the static header field from the Udp header, the IPv4 header and the Ethernet header. These static header fields are those which do not change from one transmission

to another for the same socket.

The Reuse header is newly allocated only for the first transmission of the socket or when the upper layer sets the ConnChanged flag to TRUE in Tcplp_ZeroCopy-UdpTransmit. Otherwise, in Dynamic Mode, no new allocation is performed and the last Reuse header is reused, and in Static Mode, if the length linked to the Reuse header is the same as the Data length, no new allocation is performed and the last Reuse header is used.

For IP fragmentation (Dynamic Mode only), the IP OneTime header is 6 bytes. Multiple IP OneTime headers are allocated. The maximum number of IP OneTime header buffers that are part of one transmission is equal to the TCPIP_ZERO-COPY_MAX_IP_FRAGMENT_PER_TX Macro. If there's no IP fragmentation, only one IP OneTime header is required. These buffers store the dynamic IPv4 header fields, which change from one transmission to another.

In the case of Udp, one UDP header buffer (of 4 bytes) is present. This buffer stores the dynamic UDP header fields, which are different from one transmission to another. On successful allocation of the buffer:

- If the Reuse header buffer was newly allocated, rba_EthUdp_ZeroCopyFill-Header is called to request the filling of the buffer, then rba_EthUdp_ZeroCopyTransmit is called to transmit the message.
- If TxConf set is to TRUE in Tcplp_ZeroCopyUdpTransmit, a unique TxId is returned, and successful transmission is confirmed by the driver layer.

This confirmation is provided in the following way:

- After a transmission marked with TxConf set to TRUE is completed, the Eth module MainFunction triggers EthIf_ZeroCopyTxConfirmation and then Tcplp_ZeroCopyTxConfirmation callback with the same TxId.
- Tcplp releases all the allocated header buffers used before and during the transmission marked with TxConf set to TRUE. Only the Reuse header buffer remains allocated for further transmission requests.
- The ZeroCopyTxConfirmation call is forwarded to the configured upper layer callback function. ZeroCopyTxConfirmation must be configured so that the upper layer of Tcplp will know when to release the allocated data buffers.

20.2.3 Message Reception

Message reception notification is triggered by the lower layer EthIf. Tcplp_RxIndication is triggered by EthIf_RxIndication on reception of an Ethernet frame.

Depending on whether an IP frame or an ARP frame is received, the respective callback is triggered by Tcplp. Based on the type of the received frame, the following is triggered:

rba_EthIPv4_RxIndication (IPv4 frame) rba_EthIPv6_RxIndication (IPv6 frame)
rba_EthArp_RxIndication (ARP frame)

20.2.4 Tcplp State Handling

The Tcplp module maintains a separate state for each configured Ethlf controller. The following states are supported.

- TCPIP_STATE_OFFLINE
- TCPIP_STATE_STARTUP
- TCPIP_STATE_ONLINE
- TCPIP_STATE_ONHOLD
- TCPIP_STATE_SHUTDOWN

The TCPIP_STATE_STARTUP and TCPIP_STATE_SHUTDOWN states are used internally by the Tcplp module, and they cannot be requested by an upper layer module.

20.2.5 IP Address Assignment

The Tcplp module currently supports the following address assignment methods.

- Static assignment (IPv4 and IPv6 addresses).
- DHCP address assignment (IPv4 addresses).
- AUTO-IP address assignment (IPv4 addresses).
- Assignment of unicast and multicast local IP addresses.

Address assignment can be either AUTOMATIC or MANUAL, based on the configuration (see Configuration Advice section).

20.2.6 Scalability Class & Features

The Tcplp module (together with associated sub-modules) supports Scalability Class 1 features, as defined in the AUTOSAR specification. This includes the following protocol:

- IPv4
- IPv6
- ARP
- ICMPv4
- ICMPv6
- DCHPv4
- Auto-IP
- NDP
- UDP
- TCP

20.2.7 Measurement Data

The Tcplp module supports a configurable Measurement Data feature for diagnostic information purposes, such as extracting a count of dropped Rx frames which have invalid an Ip address (IPv4/IPv6) or which cannot be mapped to a valid local IP/Port (TCP/UDP).

20.2.8 TCP SACK Option

20.3 Introduction

The TCP Selective Acknowledgment (SACK) option when enabled corrects multiple data packet losses from a window of data which can have a negative effect on TCP throughput. Without the (SACK) option enabled TCP uses a cumulative acknowledgment scheme, and when the received segments that are not at the left edge of the receive window are not acknowledged. The sender then either has to wait for a roundtrip time to find out about each lost packet, or to unnecessarily retransmit segments which have been correctly received.

With the cumulative acknowledgment scheme multiple dropped segments generally cause TCP to lose its ACK-based clock, which reduces the overall throughput. With the selective acknowledgment, the data receiver can inform the sender about all segments that have arrived successfully, and the sender only needing to retransmit the segments that have actually been lost.

20.4 Advantages of using SACK

Enabling SACK can improve network performance by reducing the number of retransmissions and the amount of wasted bandwidth. This can result in faster data delivery, lower latency, and higher throughput. SACK can also enhance the reliability of TCP connections, as it can recover from multiple packet losses in a single round-trip time. SACK can be especially beneficial for applications that use large or variable-sized segments, such as video streaming or file transfer.

20.5 Disadvantages of using SACK

There are also potential disadvantages when using SACK and these include, increasing the overhead of the TCP header, as more bytes are added to each segment. This can reduce the effective payload size and increase the fragmentation risk. Another limitation is that SACK can introduce security vulnerabilities, as it can be exploited by attackers to launch denial-of-service (DoS) attacks or to infer the state of the TCP connection.

To enable TCP SACK refer to the Configuration Advice section.

20.6 Extensions

20.6.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

20.6.1.1 Tcplp/TcplpConfig/TcplpCtrl/TcplpIpVXCtrl/TcplpIpV4Ctrl/TcplpArpTblAgeingDisabled

Short Name	TcplpArpTblAgeingDisabled
Description	ETAS (vendor 11) specific parameter: Disables (TRUE) or enables (FALSE) ageing of ARP Table Entries per TcplpCtrl.
Introduction	When enabled(FALSE), the ARP table entries will be discarded after ARP table entry timeout occurs. This parameter does not enable ageing of Arp entries of IP addresses within the same subnet as that of the local address(In - vehicle ECUs). If TcplpArpTableEntryTimeout is set to INF, then irrespective of TcplpArpTblAgeingDisabled value, Arp entries will not be timed out. Memory Impact: NONE. Throughput Impact: Increases slightly on TRUE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.2 Tcplp/TcplpConfig/TcplpCtrl/TcplpIpVXCtrl/TcplpIpV4Ctrl/TcplpIcmpEchoBroadcastEnabled

Short Name	TcplpIcmpEchoBroadcastEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) reply to ICMP Broadcast echo requests per TcplpCtrl.
Introduction	If the parameter is configured to true, then echo reply will be sent on receiving a broadcast echo request. Broadcast echo request has a broadcast destination IP address. The echo reply sent out will not be a broadcast frame. It is suggested to disable this feature to prevent security issues like denial of service attack by flooding ECU with broadcast Echo requests. Memory Impact: NONE. Throughput Impact: Decrease in throughput when parameter is enabled. Throughput decreases in direct proportion to the number of broadcast echo requests received.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.3 Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV4Ctrl/TcplpIpTtlConfigurationEnabled

Short Name	TcplpIpTtlConfigurationEnabled
Description	ETAS (vendor 11) Enables (TRUE) or disables (FALSE) usage of configured TTL value per EthIfCtrl (VLAN). If it is disabled, default value 1 shall be used.
Introduction	For IPv4 default TTL value is 1 to prevent frames to be unintentionally sent out of the local network, so to prevent forwarding of vehicle internal traffic to outside of the vehicle as suggested by RFC 3927. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.4 Tcplp/TcplpConfig/TcplpCtrl/TcplpZeroCopyModeType

Short Name	TcplpZeroCopyModeType
Description	ETAS (vendor 11) Selects the Mode in which ZeroCopy transmission takes place (TCPIP_ZEROCOPY_DYNAMIC or TCPIP_ZEROCOPY_STATIC Mode). In TCPIP_ZEROCOPY_STATIC mode, checksum offload need to be ON and IP fragmentation OFF. Note: This container is mandatory only when Tcplp ZeroCopy is required.
Introduction	Note: In TCPIP_ZEROCOPY_STATIC Mode, only Reuse header are allocated with a predefined length, so this mode will be more efficient for transmission of frames with same sizes or a limited set of sizes and transmission of multiple Data buffers in a single transmit API. In TCPIP_ZEROCOPY_DYNAMIC Mode, all the headers (Reuse Header, IpOneTimeHeader and UdpOneTimeHeader) are included and this mode will be more efficient for transmission of frames with variable length but transmission of multiple Data pointers is not allowed in a single transmit API. Zero-copy transmission currently supported only for Udp/IPv4 frames sent by applications on top of SoAd and requires non-AUTOSAR Eth driver extension Memory Impact: More memory will be consumed if both TCPIP_ZEROCOPY_STATIC and TCPIP_ZEROCOPY_DYNAMIC are configured in TcplpCtrls. Throughput Impact: If used, it would allow faster Tx of big amounts of data.
Multiplicity	0..1

Default Value	-
Enumeration Literals	TCPIP_ZEROCOPY_STATIC, TCPIP_ZEROCOPY_DYNAMIC
Type	ECUC-ENUMERATION-PARAM-DEF

20.6.1.5 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpArpMaxRequests

Short Name	TcplpArpMaxRequests
Description	ETAS (vendor 11) specific parameter: Maximum number of ARP requests that can be sent for a particular destination IP address. Suggested Default Value (1)
Introduction	Note: Maximum number of ARP requests that are sent to resolve a particular destination IP. If ARP reply is not received after max number of requests are sent, the ARP entry is freed. It is possible to restart address resolution after entry has been freed. User may configure only one parameter out of TcplpArpRequestTxTimeout and TcplpArpMaxRequests as these features are mutually exclusive
Multiplicity	0..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.6 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpArpReplyBroadcastEnabled

Short Name	TcplpArpReplyBroadcastEnabled
Description	ETAS (vendor 11) Enables (TRUE) or disables (FALSE) If The ARP reply shall be sent as broadcast for a IPv4 Link- Local address range.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.7 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpArpRequestTxTimeout

Short Name	TcplpArpRequestTxTimeout
Long Name	TcplpArpRequestTxTimeout (TcplpArpTrialMax)

Description	ETAS (vendor 11) specific parameter: Timeout in seconds after which an ECU stops sending ARP request.
Introduction	Note: After configured timeout is elapsed ECU stops sending ARP request for the particular destination IP, if no reply is received, address resolution sequence will not be repeated for this remote IP afterwards. I.e. further transmit requests to this destination will be rejected. The ARP entry for this IP would be permanently occupied and never freed. User may configure only one parameter out of TcplpArpRequestTxTimeout and TcplpArpMaxRequests as these features are mutually exclusive
Multiplicity	0..1
Value Range	0.0001..4294967.295
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.8 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpIpConflictDetectionArpFrames

Short Name	TcplpIpConflictDetectionArpFrames
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) IP conflict detection based on ARP frames. This parameter shall be enabled if Dhcp or Autolp is configured, and may be disabled if only static IP addresses are used for in-vehicle communication.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.9 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpIpConflictDetectionIPv4Frames

Short Name	TcplpIpConflictDetectionIPv4Frames
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) IP conflict detection based on IPv4 frames. This parameter needs to be enabled only if IP conflict detection needs to be done on reception every IPv4 frame. This might be required to identify IP conflicts after some node has assigned same IP as the current node, but for some reason probing frame has not reached the current node (eg. link was down for the current node).

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.10 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpRbArpEntryDoNotUpdateOnIpFrameReceive

Short Name	TcplpRbArpEntryDoNotUpdateOnIpFrameReceive
Description	ETAS (vendor 11) specific parameter: Disables (TRUE) or enables (FALSE) the feature of updating ARP entry on receiving IPv4 frames.
Introduction	By default, ARP entry gets updated on receiving ARP frames and IPv4 frames. This feature is essential for responding to multicast frames by learning an ARP entry so that ARP resolution is NOT required for transmitting a reply from ECU. This feature can be disabled so that ARP entries are not updated by received IP frames and ARP table is NOT compromised. Currently, all arpconfigs take same value.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.11 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpRbArpGratuitousARPCycleTime

Short Name	TcplpRbArpGratuitousARPCycleTime
Description	ETAS (vendor 11) specific parameter: Specifies a cycle time in seconds for transmission of ARP gratuitous messages.
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.12 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpRbArpProtocolDisabledByDefault

Short Name	TcplpRbArpProtocolDisabledByDefault
Description	ETAS (vendor 11) specific parameter: Disables (TRUE) or enables (FALSE) the support of ARP protocol by default.

Introduction	Disabling ARP protocol means that there will be no ARP requests and no ARP replies sent from ECU, ARP table will not be updated neither on reception of ARP messages nor IP messages and no IP conflict detection will be executed. When configuring this parameter, the API <code>Tcplp_ArpEnable()</code> will be available to enable or disable ARP protocol during runtime. When ARP protocol is disabled, it is still possible to add static entries in the ARP table using API <code>Tcplp_SetRemotePhysAddr()</code> .
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.13 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpIcmpConfig/TcplpRblcmpProtocolDisabledByDefault

Short Name	TcplpRblcmpProtocolDisabledByDefault
Description	ETAS (vendor 11) specific parameter: Disables (TRUE) or enables (FALSE) the support of ICMP protocol by default.
Introduction	Disabling ICMP protocol means that all ICMP message types will be deactivated. When configuring this parameter, the API <code>Tcplp_IcmpEnable()</code> will be available to enable or disable ICMP protocol during runtime. Memory Impact: NONE. Throughput Impact: If Icmp protocol is disabled, throughput increases as there will be no processing of Icmp frames.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.14 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpIpFragmentationConfig/TcplpIpReassemblyBufferCount

Short Name	TcplpIpReassemblyBufferCount
Description	ETAS (vendor 11) specific parameter: Number of buffers that can be used for fragment reassembly. A value of 0 disables fragment reassembly.

Introduction	For all the fragments of a particular quadruple (source IP, destination IP, frame identifier and protocol) only one buffer will be allocated during reassembly. And hence value of the parameter specifies the maximum number of parallel reassembly operations possible for different quadruples. Memory Impact: RAM memory increases as more number of reassembly buffers are configured. Throughput Impact: Increase in throughput as more number of parallel reassembly of fragmented frames are possible by configuring more buffers.
Multiplicity	1..1
Value Range	0..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.15 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpIpFragmentationConfig/TcplpIpReassemblyBufferSize

Short Name	TcplpIpReassemblyBufferSize
Description	ETAS (vendor 11) specific parameter: The parameter specifies size of the buffer in bytes required to reassemble all the fragments of an original frame. Expected value here corresponds to only IPv4 payload size before frame undergoing fragmentation.
Introduction	Note: This parameter is only relevant if TcplpIpReassemblyBufferCount is configured to a value greater than zero. It shall be configured to a value greater than or equal to the EthIfCtrlMtu. The node must be able to reassemble frames that is atleast equal to the size of the EthIfCtrlMtu according to RFC 1122. Memory Impact: RAM memory increases as the size of the parameter is increased. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	576..65535
Default Value	576
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.16 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpIpFragmentationConfig/TcplpIpTxFragmentBufferSize

Short Name	TcplpIpTxFragmentBufferSize
------------	-----------------------------

Description	ETAS (vendor 11) specific parameter: Specifies the fragmentation buffer size in terms of bytes for the controller. If the fragmentation needs to be disabled for the controller the parameter shall not be configured.
Introduction	Note: It shall be configured to a value greater than EthIfCtrlMtu. A frame gets fragmented only when the frame length is greater than EthIfCtrlMtu. Hence IPv4 is always expected to buffer frame sizes greater than EthIfCtrlMtu during fragmentation. Memory Impact: RAM memory increases as the size of the parameter is increased. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.17 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpArNudConfig/TcplpRbNdpProbeOnReceivedMessage

Short Name	TcplpRbNdpProbeOnReceivedMessage
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) the support of probing when a unicast or multicast IPv6 frame is received (with Udp or Icmpv6 protocol).
Introduction	When ENABLED : If an IP frame is received from a remote node for which neighbor entry does not exist, an entry will be created in PROBE state. The entry will become reachable on receiving NA from remote node else it will be deleted after PROBE sequence is complete. When DISABLED: On receiving IP frame, neighbor entry is created in REACHABLE state.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.18 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpSlaacConfig/TcplpRbNdpSlaacNeighborAdvertisementRetransTimer

Short Name	TcplpRbNdpSlaacNeighborAdvertisementRetransTimer
Description	Configures the default value (s) for the RetransTimer variable specified in [RFC4861 7.2.6. Sending Unsolicited Neighbor Advertisements].

Introduction	The time between retransmissions of Neighbor Advertisement messages when IP conflict is detected. IP conflict is detected when source IP address and destination IP address of an incoming IPv6 packet are equal or when target IP address of Neighbor Advertisement is assigned to the TcplpCtrl. When IP conflict is detected, Unsolicited Neighbor Advertisement is sent to All Node Multicast address as a way of defending the IP address. IP address is defended only for TCPIP_LINKLOCAL_DOIP, TCPIP_LINKLOCAL, TCPIP_IPV6_ROUTER and TCPIP_STATIC IPv6 addresses. For TCPIP_DHCP addresses, new address is requested from DHCPv6 server when conflict is detected and no Neighbor Advertisement is sent in return. When the parameter is not configured or configured to 0 seconds, Ndp disables transmission of Unsolicited Neighbor Advertisements for defend mechanism purpose.
Multiplicity	0..1
Value Range	0..60
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.19 Tcplp/TcplpConfig/TcplpRbDtIsConfig

Short Name	TcplpRbDtIsConfig
Long Name	TcplpRbDtIsConfig (currently unused)
Description	Specifies the configuration parameters of the DTLS (Datagram Transport Layer Security) sub module.
Multiplicity	0..0
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

20.6.1.20 Tcplp/TcplpConfig/TcplpSocketOwnerConfig/TcplpSocketOwner/TcplpSocketOwnerCopyTxDataMetaDataName

Short Name	TcplpSocketOwnerCopyTxDataMetaDataName
Description	ETAS (vendor 11) specific parameter: This parameter defines the name of the <Up_CopyTxDataMetaData> function of the TcplpSocketOwner module. The function name shall only be configurable if TcplpSocketOwnerUpperLayerType is set to CDD.

Introduction	For standard AR ULs of Tcplp name of this callback will be automatically generated by Tcplp. Configuration of Up_CopyTxDataMetaData is required when parallel transmissions need to be executed on the same socket (reentrancy). In that case, CDD can use Tcplp_UdpTransmitMetaData or Tcplp_TcpTransmitMetaData and provide context information in MetaDataPtr_pv argument that will be given back during the Up_Copy-TxDataMetaData callback. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

20.6.1.21 Tcplp/TcplpConfig/TcplpSocketOwnerConfig/TcplpSocketOwner/TcplpSocketOwnerDtIsEventName

Short Name	TcplpSocketOwnerDtIsEventName
Description	ETAS (vendor 11) specific parameter: This parameter defines the name of the <Up_DtIsEvent_pfct> function of the TcplpSocketOwner module. The function name shall only be configurable if TcplpSocketOwnerUpperLayerType is set to CDD.
Introduction	Up_DtIsEvent callback provides information about the ongoing DTLS connections (for example when handshake is completed or when DTLS connection is closed). This parameter is relevant only when DTLS protocol is configured. Currently, only 3rd party DTLS can be configured by setting the parameter TcplpRbExternalDtIsSupport to TRUE. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

20.6.1.22 Tcplp/TcplpConfig/TcplpSocketOwnerConfig/TcplpSocketOwner/TcplpSocketOwnerZeroCopyTxConfirmationName

Short Name	TcplpSocketOwnerZeroCopyTxConfirmationName
Description	ETAS (vendor 11) specific parameter: This parameter defines the name of the <Up_ZeroCopyTxConfirmation> function of the TcplpSocketOwner module. The function name shall only be configurable if TcplpSocketOwnerUpperLayerType is set to CDD.

Introduction	Up_ZeroCopyTxConfirmation is mandatory for Zero Copy transmission (enabled by configuring TcplpZeroCopy-General container). When transmissions are completed at driver layer, the Up_ZeroCopyTxConfirmation callback will be provided to CDD so that allocated buffers for these Tx can be released. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

20.6.1.23 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpMaxNumberOfSocketsSupportingSack

Short Name	TcplpRbTcpMaxNumberOfSocketsSupportingSack
Description	ETAS (vendor 11) specific parameter: This parameter specifies the maximum number of sockets that can support the transmission of the SACK option when out of order segments are queued. Value 0 indicates SACK is disabled.
Introduction	Note: This parameter is dependent on TcplpRbTcpReorderBufferCount, it's value shall be always less than or equal to TcplpRbTcpReorderBufferCount. If the socket doesn't have Reorder buffer then SACK will not be supported on that socket. If enabled, Selective Acknowledgement (SACK) will allow to retransmit only actually missing segments (so less frames) in case of out of order reception. Memory Impact: Too high value would increase memory consumption. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	0..65535
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.24 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpMaxQSizeForTxReq

Short Name	TcplpRbTcpMaxQSizeForTxReq
Description	ETAS (vendor 11) specific parameter: Maximum queue size to store tx requests for TLS data.

Introduction	Note: It specifies upto how many Transmit request information can be stored before Tx confirmation to UL for each request. Default: default of 5 queues will be used to store transmit requests. Dependency: TcplpTcpMaxTxBuffPerSock is configured. TcplpTlsConfig container is configured or TcplpRbExternalTlsSupport is set to TRUE. Memory Impact: Too high value would increase memory consumption. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	1..255
Default Value	5
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.25 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpMaxTlsConnections

Short Name	TcplpRbTcpMaxTlsConnections
Description	ETAS (vendor 11) specific parameter: Defines the maximum number of TLS connections that can be handled at the same time within Tcp.
Introduction	Note: When communication need to be secured via TLS, the Tcp component need to store extra information about ongoing Tcp communication for each TLS connection. When TcplpTlsConfig container is configured, this parameter should be set to the size of TcplpTlsConfig.TcplpTlsMaxConnections. Dependency: TcplpTlsConfig container is configured or TcplpRbExternalTlsSupport is set to TRUE. Memory Impact: Too high value would increase memory consumption. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.26 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpNagleOverrideTimeout

Short Name	TcplpRbTcpNagleOverrideTimeout
------------	--------------------------------

Description	ETAS (vendor 11) specific parameter: This timer (in seconds) is used to override the sender's Silly Window Syndrome (SWS) avoidance algorithm (i.e Nagle Algorithm) to force transmission of data when certain conditions are met. For further details, see also IETF RFC 1122 section 4.2.3.4, Rule 4
Introduction	Note: Default: 1s is used when parameter is disabled. Dependency: TcplpTcpNagleEnabled
Multiplicity	0..1
Value Range	0.1..1
Default Value	1
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.27 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpReopenFromLastAck

Short Name	TcplpRbTcpReopenFromLastAck
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) reopening of the socket from state LAST_ACK upon reception of a SYN frame.
Introduction	Note: If reopening of the socket from state LAST_ACK is considered as a security threat this parameter needs to be disabled. Enabling TcplpRbTcpReopenFromLastAck, Tcp will reopen the socket immediately after entering into the LAST_ACK state, no waiting till the LAST_ACK timeout expired. This would allow to reuse previously closed socket faster. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.28 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpReorderBufferCount

Short Name	TcplpRbTcpReorderBufferCount
Description	ETAS (vendor 11) specific parameter: Number of reorder buffer indicates number of TCP sockets capable of handling out of order reception in parallel. Each buffer can store multiple TCP segments belonging to the same TCP socket. Value 0 indicates out of order handling is disabled.

Introduction	Note: A received frame shall be considered as out of order when received sequence number is inside the receive window but higher than the next expected sequence number. If the reordering is enabled and received out of order TCP segment fits in the available reorder buffer, the TCP segment will be stored in the reorder buffer and processed later when missing segments arrive. Out of order reception may occur e.g. if in the network there are different paths that frame may traverse. If in the network there is low probability to have out of order reception, it should be rather disabled to reduce resource consumption. Memory Impact: Too high value would increase memory consumption. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	0..65535
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.29 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpReorderBufferSize

Short Name	TcplpRbTcpReorderBufferSize
Description	ETAS (vendor 11) specific parameter: Size of each reorder buffer and indicates memory available for individual TCP socket in bytes for out of order reception. Each buffer can store multiple TCP segments belonging to the same TCP socket.
Introduction	Note: This parameter is only relevant if the parameter TcplpRbTcpReorderBufferCount is configured to a non-zero value. Number of TCP segments stored in a buffer depends on the value of the parameter, size of incoming TCP segment and availability of enough continuous memory for the incoming out of order segment. For each TCP segment in addition to TCP segment length (TCP header + Payload length) 4 more bytes are consumed to store meta data to handle buffer management. If the out of order segment cannot be stored because of unavailability of the memory the segment will be dropped. Minimum value for the parameter must be 24 = TCP header length (20) + Meta data for buffer management (4). Memory Impact: Too high value would increase memory consumption. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	24..65535

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.30 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpSocAbortClose-DuringShutDown

Short Name	TcplpRbTcpSocAbortCloseDuringShutDown
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) support of abort closing of TCP sockets during EthSM ntwrk transition from FULL_COMM to NO_COMM.
Introduction	Note: As per Tcplp 24-11 specification, graceful closing of sockets during the shutdown sequence of an EthSM network is the expected default behaviour. If this parameter is not configured or configured to false, this newly introduced behaviour will be activated. In case the project needs to have the old behaviour where the sockets where abort closed, then this parameter shall be configured to true. This parameter is a temporary measure to maintain backward compatibility and will be removed after one year of its introduction. Memory Impact: NONE. Throughput Impact: NONE. Transition from FULL_COMM to NO_COMM sequence is faster if parameter is configured to true.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.31 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpRbTcpTimeStampEnabled

Short Name	TcplpRbTcpTimeStampEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) support of Timestamp feature.
Introduction	Note: If the parameter is enabled Timestamp option is negotiated during 3-way handshaking. If successfully negotiated, Timestamp option is included in every segment for the duration of connection. Timestamp option is disabled by default. It can be enabled, e.g. if remote node expects time stamp to be present in the Tcp frame. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

20.6.1.32 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpDelayedAckTimeout

Short Name	TcplpTcpDelayedAckTimeout
Description	The maximal time an acknowledgment is delayed for transmission in seconds. For further details, see also IETF RFC 1122 section 4.2.3.2
Introduction	Note: Default: 0.5 is used when parameter is disabled. Dependency: TcplpTcpDlyAckEnabled
Multiplicity	0..1
Value Range	0..0.5
Default Value	0.5
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.33 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpDlyAckEnabled

Short Name	TcplpTcpDlyAckEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) support of Delayed Ack feature.
Introduction	Note: If the parameter is enabled the acknowledgments sent to the remote shall be delayed by 200ms or ACK responses shall be sent after two MSS. Enabling this feature would allow to reduce number of ACK on the bus. E.g. in case there are many small Tcp frames and delayed ACK is disabled, each small received Tcp frame will be acknowledged individually. Memory Impact: NONE. Throughput Impact: Delayed ACK can reduce throughput.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.34 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpDynReTxTimerEnabled

Short Name	TcplpTcpDynReTxTimerEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) use of TCP Dynamic ReTx Timer.

Introduction	Note: If the parameter is enabled the Retransmission Timeout shall be calculated dynamically according to the time the remote acknowledge data segments(RTT). TcplpTcpDynReTxTimerEnabled is only effective, if TcplpTcpReTxEnabled is enabled. Even if TcplpTcpDynReTxTimerEnabled is enabled, TcplpTcpRetransmissionTimeout shall be still configured. If TcplpTcpDynReTxTimerEnabled is enabled then TCP calculates retransmission timeout dynamically based on Round trip time (RTT): Time between sent frame and ACK received for it. TcplpTcpRetransmissionTimeout is used, when Retransmission timeout is not yet calculated dynamically. This usually happens, when very first frame is sent, but there is no ACK for it from remote. In such scenario, TCP retransmits frame (SYN in case of client and SYN-ACK in case of server) again and uses TcplpTcpRetransmissionTimeout as timeout. Memory Impact: NONE. Throughput Impact: May increase the throughput.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.35 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpDynReTxTimerLwr-Bound

Short Name	TcplpTcpDynReTxTimerLwrBound
Description	ETAS (vendor 11) specific parameter: Lower bound of Timeout in [s] when Retransmission TimeOut is calculated dynamically.
Introduction	Note: This parameter specifies the minimum limit that shall be used during computing of dynamic retransmission timeout. Default: 1 second is used when parameter is disabled (RFC2988, p3, section 2.4) Dependency: TcplpTcpDynReTxTimerEnabled. Memory Impact: NONE. Throughput Impact: May increase the throughput.
Multiplicity	0..1
Value Range	0..INF
Default Value	1
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.36 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpDynReTx-TimerUpBound

Short Name	TcplpTcpDynReTxTimerUpBound
Description	ETAS (vendor 11) specific parameter: Upper bound of Timeout in [s] when Retransmission TimeOut is calculated dynamically.
Introduction	Note: This parameter specifies the maximum limit that shall be used during computing of dynamic retransmission timeout (from RTT or exponential backoff). Default: INF is used when parameter is disabled Dependency: TcplpTcpDynReTxTimerEnabled. Memory Impact: NONE. Throughput Impact: May increase the throughput.
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.37 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpHalfDuplexCloseEnabled

Short Name	TcplpTcpHalfDuplexCloseEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) Implementation of Half Duplex Close as per RFC 1122
Introduction	Note: If the parameter is enabled the Half Duplex Close feature is activated which shall send a reset segment if Close call was requested and new data is received. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.38 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcplsnGeneration-Method

Short Name	TcplpTcplsnGenerationMethod
Description	ETAS (vendor 11) specific parameter: Specify how the Initial Sequence Number should be generated.

Introduction	Note: PSEUDO_RANDOM : The Initial Sequence Number is generated pseudo randomly (resource efficient, this method is not secure and does not respect RFC793 and RFC1122 assumption) CLOCK_DRIVEN : The Initial Sequence Number is chosen based on clock driven selection (resource efficient, this method is not secure but respect RFC793 and RFC1122 assumption) CLOCK_HASH_BASED : The Initial Sequence Number is chosen using clock and MD4 hash as per RFC6528 (higher resource consumption, this method is more secure) Default: PSEUDO_RANDOM is used when parameter is disabled. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	PSEUDO_RANDOM
Enumeration Literals	PSEUDO_RANDOM, CLOCK_DRIVEN, CLOCK_HASH_BASED
Type	ECUC-ENUMERATION-PARAM-DEF

20.6.1.39 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpMSSEnabled

Short Name	TcplpTcpMSSEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) support of TCP Maximum Segment Size option.
Introduction	Note: If the parameter is enabled the Maximum Segment Size permitted on reception shall be advertised to the remote within SYN or SYN-ACK segments (MSS option) . If disabled, remote node will use default MSS value of 536, like specified in the RFC 6691. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.40 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpMaxTxBuffPerSock

Short Name	TcplpTcpMaxTxBuffPerSock
Description	ETAS (vendor 11) specific parameter: Maximum size of TCP Tx window buffer allocated per TCP socket.

Introduction	Note: This parameter specifies the size of the Tcp Tx Buffer per socket that shall be used to store the data before transmission. Memory Impact: Too high value would increase memory consumption. Throughput Impact: bigger value increase the throughput.
Multiplicity	0..1
Value Range	0..65535
Default Value	4096
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.41 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpReTxEnabled

Short Name	TcplpTcpReTxEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) support of Retransmission feature.
Introduction	Note: If the parameter is enabled the retransmissions of Tcp segments (SYN, DATA, FIN-ACK (1st packet of close sequence)) shall be possible. It is not recommended to disable retransmission, as it is key feature of Tcp protocol. If retransmission will be disabled in TCP, it may significantly impact reliability of the protocol. If retransmission is disabled, Tcp will wait endless for ACK from remote node and any lost of the Tcp packet may significantly impact Tcp communication. In the worst case communication may get stuck and recover only after closing Tcp connection after TcplpTcpUserTimeout expires (if user timeout is configured). Memory Impact: NONE. Throughput Impact: May increase the throughput.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.42 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpSackEnabled

Short Name	TcplpTcpSackEnabled
Description	Defines if the SACK (selective acknowledgement) mechanism shall be supported according to IETF RFC 2018.

Introduction	If the parameter is enabled, SACK Permitted option will be sent during connection establishment (in SYN segment) and the received SACK option will be processed. This will allow a selective retransmission of lost segments.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.43 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpUserTimeout

Short Name	TcplpTcpUserTimeout
Description	ETAS (vendor 11) specific parameter: Maximum amount of time in [s] that transmitted data may remain unacknowledged before TCP forcefully closes the connection.
Introduction	Note: This parameter specifies the User Timeout which shall be used to forcefully close a Tcp connection when remote does not respond to sent data within selected time. Dependency: TcplpTcpUserTimeoutEnabled. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.44 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpUserTimeoutEnabled

Short Name	TcplpTcpUserTimeoutEnabled
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) support of User Timeout feature.

Introduction	Note: If the parameter is enabled the TcplpTcpUser-Timeout shall be advertised to the remote within SYN or SYN-ACK segments (UTO option). If the remote does not respond to sent data within the TcplpTcpUserTimeout the connection shall be forcefully closed. E.g. this parameter should be enabled, if Tcp retransmission is disabled to be able to break endless waiting for remote ACK. User Time out can be also enabled, when retransmission is enabled, but in this case User time out should be greater than retransmission timeout. Else connection will be closed before retransmission can be triggered. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.45 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpWindowScale

Short Name	TcplpTcpWindowScale
Long Name	TcplpTcpWindowScale (currently unused)
Description	Defines the TCP window scale.
Multiplicity	0..0
Value Range	0..14
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.46 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpWindowScaleOptionEnabled

Short Name	TcplpTcpWindowScaleOptionEnabled
Long Name	TcplpTcpWindowScaleOptionEnabled (currently unused)
Description	Defines if the TCP window scale option and mechanism shall be supported according to IETF RFC 7323, chapter 2.
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.47 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpZWPEEnabled

Short Name	TcplpTcpZWPEEnabled
------------	---------------------

Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) TCP Zero Window Probes according to IETF RFC 1122 chapter 4.2.2.17
Introduction	Note: If the parameter is enabled Zero Window Probes shall be transmitted to probe the remote window when remote advertise a window of 0. TCP Zero Window Probes can be only enabled, when TcplpTcpReTxEnabled is also enabled. Memory Impact: NONE. Throughput Impact: May increase the throughput.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.48 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpZWPMMaxRetries

Short Name	TcplpTcpZWPMMaxRetries
Description	ETAS (vendor 11) specific parameter: Maximum number of times that a Zero Window Probe segment transmitted.
Introduction	Note: This parameter specifies the maximum number of times that a Zero Window Probe segment is retransmitted without any response from the remote before the TCP connection is closed. Dependency: TcplpTcpZWPEEnabled. Memory Impact: NONE. Throughput Impact: May increase the throughput.
Multiplicity	0..1
Value Range	0..255
Default Value	255
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.49 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsCertificateAttributesMaxLength

Short Name	TcplpRbTlsCertificateAttributesMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum length of a certificate attributes.
Introduction	Note: This parameter is used to hold the maximum length of certificate attributes such as extension, version number, issuer name, subject name etc which could be retrieved via KeyM_CertElementGet() API. If this parameter is not configured, then it can be configured with default value 32.

Multiplicity	0..1
Value Range	1..255
Default Value	32
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.50 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsClientCertificate-TypesMaxCount

Short Name	TcplpRbTlsClientCertificateTypesMaxCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies the maximum number of certificate types could be supported in TLS Client.
Introduction	Note: This parameter shall be configured only for TLS Client and Client authentication is enabled. This parameter specifies the number of certificate types like RSA, ECC, RSA fixed ECDH, ECDSA fixed ECDH that could be processed in the received Certificate Request message from TLS Server. If this parameter is not configured, then it can be configured with default value 1.
Multiplicity	0..1
Value Range	1..4
Default Value	2
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.51 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsDistinguished-NameMaxLength

Short Name	TcplpRbTlsDistinguishedNameMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum length of a server or client certificate attribute - Distinguish name.
Introduction	Note: This parameter is configured only when client authentication is enabled and TcplpRbTlsRequest CertificateAuthority is enabled. This parameter will specify the maximum length of Distinguish name that can be sent in certificate request handshake message.
Multiplicity	0..1
Value Range	1..65535
Default Value	512
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.52 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsDistinguished-NamesMaxCount

Short Name	TcplpRbTlsDistinguishedNamesMaxCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies the maximum number of distinguished names supported by TLS Client.
Introduction	Note: This parameter shall be configured only for TLS Client and Client authentication is enabled. This parameter specifies the number of distinguished names that could be processed in the received Certificate Request message from TLS Server. If this parameter is not configured, then it can be configured with default value 1.
Multiplicity	0..1
Value Range	1..4
Default Value	2
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.53 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsECSupportedGrpsMaxCount

Short Name	TcplpRbTlsECSupportedGrpsMaxCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum number of Elliptic curve groups that can be supported if Elliptic curve related ciphersuite is selected in the configuration.
Introduction	Note: If this parameter is not configured, then it can be configured with default value 10.
Multiplicity	0..1
Value Range	1..16
Default Value	10
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.54 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsECSupported-PointsMaxCount

Short Name	TcplpRbTlsECSupportedPointsMaxCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum number of Elliptic curve point formats that can be supported if Elliptic curve related ciphersuite is selected in the configuration.
Introduction	Note: If this parameter is not configured, then it can be configured with default value 1.

Multiplicity	0..1
Value Range	1..5
Default Value	2
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.55 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsExtensionData-MaxLength

Short Name	TcplpRbTlsExtensionDataMaxLength
Description	ETAS (vendor 11) specific parameter: This parameter specifies maximum length of a TLS extension data that could be allowed per extension. If this parameter is not configured then by default configure it with 128bytes.
Multiplicity	0..1
Value Range	0..256
Default Value	128
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.56 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsHandshakeRe-assemblyBufferSizePerConnection

Short Name	TcplpRbTlsHandshakeReassemblyBufferSizePerConnection
Description	ETAS (vendor 11) specific parameter: The parameter specifies the size of the Handshake Reassembly buffer per TLS connection.
Introduction	Note: According to TLS Rfc 5246, single TLS handshake message could be fragmented into multiple TLS records and it shall be reassembled until complete TLS handshake message is received. This parameter shall be used to reassemble the received fragmented handshake message in rx path and it shall be processed after complete TLS handshake message is received. This parameter shall be configured at max of individual handshake message could be received from peer. If this parameter is not configured then by default configure it with 4KB by assuming that 3 level certificates could be received.
Multiplicity	0..1
Value Range	0..16777215
Default Value	4096

Type	ECUC-INTEGER-PARAM-DEF
------	------------------------

20.6.1.57 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMainFunctionPeriod

Short Name	TcplpRbTlsMainFunctionPeriod
Description	ETAS (vendor 11) specific parameter: Period of rba_EthTls_MainFunction in [s]. This main function handles the functionality of open and close tls connection. In other words, it exchanges messages such as Handshake, Change Cipher Spec, Alert.
Multiplicity	1..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.58 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxBufferSizeForCompleteHandshakeMsg

Short Name	TcplpRbTlsMaxBufferSizeForCompleteHandshakeMsg
Description	ETAS (vendor 11) specific parameter: The parameter specifies the size of the buffer to hold TLS handshake messages from client hello to client finished.
Introduction	Note: If this buffer is configured with zero, then two TcplpTlsCsmHashVerifyJobRef should be configured. Because two jobs are mandatory to calculate the hash for server and client finished messages separately. In the configured two TcplpTlsCsmHashVerifyJobRefs either of them should be mapped to rba_CryptoCCL. If this parameter is not configured then by default configure it with 4KB.
Multiplicity	0..1
Value Range	0..16777215
Default Value	4096
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.59 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxBufferSizeForHashFailedHandshakeMsg

Short Name	TcplpRbTlsMaxBufferSizeForHashFailedHandshakeMsg
------------	--

Description	ETAS (vendor 11) specific parameter: The parameter specifies the size of the buffer to hold TLS handshake messages for hash calculation in case hash job is failed/ not performed. This buffer is used only when TcplpRbTlsMaxBufferSizeForCompleteHandshakeMsg is configured as zero.
Introduction	Note: This buffer is used to accumulate the hash failed or not performed TLS messages to the existing buffer messages. So that hash calculation can be retried during next iteration for all buffer messages. If this parameter is not configured then by default configure it with 4KB.
Multiplicity	0..1
Value Range	0..16777215
Default Value	4096
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.60 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxCertificate-Length

Short Name	TcplpRbTlsMaxCertificateLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum size for a certificate which is used during server certificate or client certificate handshakes. The size is measured in bytes
Multiplicity	0..1
Value Range	512..16777216
Default Value	1024
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.61 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxPseudoHash-OperationCount

Short Name	TcplpRbTlsMaxPseudoHashOperationCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum number of Pseudo Hash operation that could be performed in same main function.
Multiplicity	0..1
Value Range	1..1024
Default Value	5
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.62 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxResumption- sPerSession

Short Name	TcplpRbTlsMaxResumptionsPerSession
Description	ETAS (vendor 11) specific parameter: The parameter specifies the number of times a given session can be resumed by Client by providing the same Session Ticket given by the Server.
Introduction	Note: This parameter should be enabled only if TcplpRbTlsSessionResumptionEnabled is enabled.
Multiplicity	0..1
Value Range	0..65535
Default Value	5
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.63 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxRxBuffer- SizePerConnection

Short Name	TcplpRbTlsMaxRxBufferSizePerConnection
Description	ETAS (vendor 11) specific parameter: The parameter specifies the size of the Rx buffer per Tls connection.
Introduction	Note: The Tls payload present in Tcp buffer is copied to Rx buffer present in Tls component if Tls msg is fragmented. If this parameter is not configured then by default configure it with 4KB.
Multiplicity	0..1
Value Range	256..4294967295
Default Value	4096
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.64 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMaxTxBuffer- SizePerConnection

Short Name	TcplpRbTlsMaxTxBufferSizePerConnection
Description	ETAS (vendor 11) specific parameter: The parameter specifies the size of the Tx buffer per Tls connection.
Introduction	Note: The Tls messages are framed and stored in this buffer and it is copied to Tcp buffer. If this parameter is not configured then by default configure it with 4KB.
Multiplicity	0..1
Value Range	256..4294967295
Default Value	4096

Type	ECUC-INTEGER-PARAM-DEF
------	------------------------

20.6.1.65 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsMultipleOfHMacOutputPerIteration

Short Name	TcplpRbTlsMultipleOfHMacOutputPerIteration
Description	ETAS (vendor 11) specific parameter: The parameter specifies how many HMAC operation needs to be performed based on the output size of algorithm selected for Tls handshake.
Multiplicity	0..1
Value Range	1..8
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.66 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsPerformFinishedMsgOperationConsecutively

Short Name	TcplpRbTlsPerformFinishedMsgOperationConsecutively
Description	ETAS (vendor 11) specific parameter: This parameter specifies if preparation/processing of finished message has to be performed consecutively.
Introduction	Note: If this parameter is TRUE then Tls server/client finished messages will be prepared and processed in same main function. Finished Msg contains the following operations, Hash Calculation, PRF Calculation and then Encryption/decryption which happen in parallel for processing client finished and preparing server finished during Tls Server vice versa for Tls Client.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.67 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsPerformKeyExchangeHMacOperationConsecutively

Short Name	TcplpRbTlsPerformKeyExchangeHMacOperationConsecutively
Description	ETAS (vendor 11) specific parameter: This parameter specifies if HMAC operation for key exchange process has to be performed in same mainfunction if requested output size is more than output which shall be produced per HMAC operation. HMAC output will be varied based on the ciphersuite selection.

Introduction	Note: Consecutively means that all HMAC operations needed for key exchange are done in same main function.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.68 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsPresharedKeyIdentityHintMaxLength

Short Name	TcplpRbTlsPresharedKeyIdentityHintMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum length of the PresharedKeyIdentityHint.
Introduction	Dependency: This parameter is used only when Pre-shared key related ciphersuites are selected. If this parameter is not configured, then it can be configured with default value 32.
Multiplicity	0..1
Value Range	1..128
Default Value	32
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.69 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsPresharedKeyIdentityMaxLength

Short Name	TcplpRbTlsPresharedKeyIdentityMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum length of the PresharedKeyIdentity.
Introduction	Dependency: This parameter is used only when Pre-shared key related ciphersuites are selected. If this parameter is not configured, then it can be configured with default value 32.
Multiplicity	0..1
Value Range	1..128
Default Value	32
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.70 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsPresharedKeyMaxLength

Short Name	TcplpRbTlsPresharedKeyMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies length of the TcplpTlsPresharedKey.

Introduction	Note: Dependency: This parameter is used only when Pre-shared key related ciphersuites are selected. If this parameter is not configured, then it can be configured with default value 32.
Multiplicity	0..1
Value Range	1..128
Default Value	32
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.71 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsProcessMultiple-HandshakeMsgConsecutively

Short Name	TcplpRbTlsProcessMultipleHandshakeMsgConsecutively
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether multiple handshake messages can be processed consecutively in same main function context if it is set to true else handshake messages shall be processed in different main functions. This is required to reduce runtime of tls main function.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.72 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsRemoteCertificateChainMaxCount

Short Name	TcplpRbTlsRemoteCertificateChainMaxCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum number of certificates that can be in chain of certificate sent by remote node. The count is including the user certificate.
Multiplicity	0..1
Value Range	1..255
Default Value	3
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.73 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsRxMainFunctionPeriod

Short Name	TcplpRbTlsRxMainFunctionPeriod
Description	ETAS (vendor 11) specific parameter: Period of rba_EthTls_MainFunctionRx in [s]. This main function handles the functionality of reception for application data.
Multiplicity	1..1

Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.74 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsServerNameIdentificationMaxLength

Short Name	TcplpRbTlsServerNameIdentificationMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies the maximum length of the TcplpTlsServerNameIdentification received from remote client. This parameter is applicable for server connection only.
Introduction	Note: If this parameter is not configured, then it can be configured with default value 32.
Multiplicity	0..1
Value Range	1..128
Default Value	32
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.75 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsSessionRenegotiationInitiateTimeout

Short Name	TcplpRbTlsSessionRenegotiationInitiateTimeout
Description	ETAS (vendor 11) specific parameter: The parameter specifies the time in seconds for which ECU waits before initiating the renegotiation process after renegotiation request is received from Upper layer.
Introduction	Note: This parameters helps to be in synchronization with remote ecu. For example: There are chances to receive the application data message from remote after renegotiation is initiated at ECU. During this time interval, ECU will ignore the application data and will wait until handshake messages are received.
Multiplicity	0..1
Value Range	0..INF
Default Value	1
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.76 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsSessionRenegotiationTimeout

Short Name	TcplpRbTlsSessionRenegotiationTimeout
------------	---------------------------------------

Description	ETAS (vendor 11) specific parameter: The parameter specifies the time duration of the session in seconds.
Introduction	Note: ECU will initiate the renegotiation of the session if this time expires.
Multiplicity	0..1
Value Range	2..INF
Default Value	240
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.77 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsSessionResumptionTimeout

Short Name	TcplpRbTlsSessionResumptionTimeout
Description	ETAS (vendor 11) specific parameter: The parameter specifies the time in seconds for which established Session's session identifier or session ticket is valid.
Introduction	Note: In Stateful Session Resumption, a) If connection type is Client, then Server's Session Id will be used to perform the session resumption until timer is expired. b) If connection type is Server, then Server accepts the Client's Session Id based resumption request until timer is expired. In Stateless Session Resumption, a) If connection type is Client, then Server's Session Ticket will be used to perform the session resumption until timer is expired. b) If connection type is Server, then Server accepts the Client's Session Ticket based resumption request until timer is expired. This parameter should be used only if TcplpRbTlsSessionResumptionEnabled is enabled.
Multiplicity	0..1
Value Range	0..INF
Default Value	120
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.78 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsSessionTicketMaxSize

Short Name	TcplpRbTlsSessionTicketMaxSize
Description	ETAS (vendor 11) specific parameter: The parameter specifies the size of Session Ticket which will be included in New Session Ticket handshake message.

Introduction	Note: Minimum size of this parameter is set to 128 bytes by considering all mandatory fields (key-name(16) + IV(16) + encrpted_state(64) + mac(32)) of the Session Ticket according to Rfc 5077. Default size of this parameter is set to 1024 bytes by considering minimum of one client certificate in encryption state if client authentication is enabled. This parameter should be enabled only if TcplpRbTlsSessionResumptionEnabled is enabled.
Multiplicity	0..1
Value Range	128..65535
Default Value	4096
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.79 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsSignatureAlgorithmsMaxCount

Short Name	TcplpRbTlsSignatureAlgorithmsMaxCount
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum number of signature algorithms supported.
Introduction	Note: If this parameter is not configured, then it can be configured with default value 12.
Multiplicity	0..1
Value Range	2..32
Default Value	12
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.80 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsSignatureValueMaxLength

Short Name	TcplpRbTlsSignatureValueMaxLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies maximum number of signature values that could be supported.
Introduction	Note: If this parameter is not configured, then it can be configured with default value 128.
Multiplicity	0..1
Value Range	1..512
Default Value	128
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.81 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsTxMainFunctionPeriod

Short Name	TcplpRbTlsTxMainFunctionPeriod
Description	ETAS (vendor 11) specific parameter: Period of rba_EthTls_MainFunctionTx in [s]. This main function handles the functionality of transmission for application data.
Multiplicity	1..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.82 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsVersionNotifHeaderFile

Short Name	TcplpRbTlsVersionNotifHeaderFile
Description	Defines header file for version callback functions which shall be included in case TcplpRbTlsVersionNotificationCallback parameter is configured.
Multiplicity	0..1
String Length Range	1..32
Default Value	-
Type	ECUC-STRING-PARAM-DEF

20.6.1.83 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsVersionNotificationCallback

Short Name	TcplpRbTlsVersionNotificationCallback
Description	This parameter defines the name of the user callback function name, which needs to be called with negotiated TLS version after successful handshake.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

20.6.1.84 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpRbTlsWaitForPeerTimeout

Short Name	TcplpRbTlsWaitForPeerTimeout
------------	------------------------------

Description	ETAS (vendor 11) specific parameter: The parameter specifies the timeout for which ECU will wait until it receives next TLS handshake message from remote.
Introduction	Note: This parameter is used to determine the timeout for TLS message reception. Timeout in seconds after which an ECU stops waiting for the reception of TLS messages and ECU notifies to remote node by sending Alert message with Alert description code 0x50 (internal error).
Multiplicity	0..1
Value Range	0..INF
Default Value	0.3
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.85 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket

Short Name	TcplpRbTlsSessionTicket
Description	This container provides Session Ticket information. This container should be enabled only if TcplpRbTlsSessionResumptionEnabled is enabled.
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

20.6.1.86 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmDecryptJobRef

Short Name	TcplpRbTlsSessionTicketCsmDecryptJobRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM job used by Server to perform decryption of Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.87 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmDecryptKeyRef

Short Name	TcplpRbTlsSessionTicketCsmDecryptKeyRef
------------	---

Description	ETAS (vendor 11) specific parameter: Reference to a CSM key associated to the CSM job used by Server to perform decryption of Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.88 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmEncryptJobRef

Short Name	TcplpRbTlsSessionTicketCsmEncryptJobRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM job used by Server to perform encryption of Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.89 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmEncryptKeyRef

Short Name	TcplpRbTlsSessionTicketCsmEncryptKeyRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM key associated to the CSM job used by Server to perform encryption of Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.90 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmMacGenerateJobRef

Short Name	TcplpRbTlsSessionTicketCsmMacGenerateJobRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM job used by Server to calculate the mac on fields present in Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.91 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmMacGenerateKeyRef

Short Name	TcplpRbTlsSessionTicketCsmMacGenerateKeyRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM key associated to the CSM job used by Server to calculate the mac on fields present in Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.92 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmMacVerifyJobRef

Short Name	TcplpRbTlsSessionTicketCsmMacVerifyJobRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM job used by Server to verify the mac on fields present in Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.93 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpRbTlsSessionTicket/TcplpRbTlsSessionTicketCsmMacVerifyKeyRef

Short Name	TcplpRbTlsSessionTicketCsmMacVerifyKeyRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM key associated to the CSM job used by Server to verify the mac on fields present in Session Ticket.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.94 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsCiphersuiteDefinition/TcplpRbTlsUseSecurityExtensionSupportedGroups

Short Name	TcplpRbTlsUseSecurityExtensionSupportedGroups
Description	ETAS (vendor 11) specific parameter: This parameter if configured to TRUE then supported groups extension can be included in the extensions list.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.95 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsCiphersuiteDefinition/TcplpRbTlsUseSecurityExtension-SupportedPointFormats

Short Name	TcplpRbTlsUseSecurityExtensionSupportedPointFormats
Description	ETAS (vendor 11) specific parameter: This parameter if configured to TRUE then supported points extension can be included in the extensions list.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.96 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpRbTlsCsmKeyElementSetJobRef

Short Name	TcplpRbTlsCsmKeyElementSetJobRef
Description	ETAS (vendor 11) specific parameter: Reference to a CSM job to load the private key to generate the Signature for Edward curve.
Introduction	This parameter shall be configured only when Edward curve is selected for any cryptographic operations in handshake container. Because, time taken to load the key for Edward curve exceeds main function period and hence system will crash. So the key loading operation for Edward curve shall be performed asynchronously via this CSM job
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.97 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpRbTlsCsmKeyExchangeKeyPairKeyRef

Short Name	TcplpRbTlsCsmKeyExchangeKeyPairKeyRef
Description	Reference to a CSM key used for Diffie Hellman (DH) key exchange operation with static key pair.

Introduction	Note: This parameter shall be configured for cipher suite which requires ECDH key exchange operation. In this key exchange operation, configured static key pair shall be used instead of Csm_JobCalcPubVal operation. Also, static key pair could be loaded via Csm_KeyElementSet() API dynamically.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

20.6.1.98 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsContinueHandshakeIfEmptyClientCertificateRcvd

Short Name	TcplpRbTlsContinueHandshakeIfEmptyClientCertificateRcvd
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether handshake can be continued or terminated when empty client certificate message is received from the client during client authentication.
Introduction	Note: During Client authentication feature, if Client sends empty client certificate message to Server then this parameter shall decide whether server needs to continue the handshake or it has to terminate the handshake. In case of termination, Server sends Certificate required Alert Message to the Client. This parameter shall be configured if Client Authentication parameter is set to TRUE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.99 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsContinueHandshakeIfSessionRenegotiationDisabled

Short Name	TcplpRbTlsContinueHandshakeIfSessionRenegotiationDisabled
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether handshake can be continued or terminated when session renegotiation is disabled.
Introduction	Note: This parameter should decide whether to continue or terminate the handshake when session renegotiation request is not received from the peer. If this parameter is set as false and we do not receive session renegotiation request from the peer, handshake will be terminated by sending handshake failure alert.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.100 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsContinueHandshakelfUntrustedClientCertificateRcvd

Short Name	TcplpRbTlsContinueHandshakelfUntrustedClientCertificateRcvd
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether handshake can be continued or terminated when untrusted client certificate message is received from the client during client authentication.
Introduction	Note: During Client authentication feature, if Client sends untrusted client certificate message to Server then this parameter shall decide whether server needs to continue the handshake or it has to terminate the handshake. In case of termination, Server sends Handshake Failure Alert Message to the Client. This parameter shall be configured if Client Authentication parameter is set to TRUE.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.101 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsContinueHandshakelfWarningAlertRcvd

Short Name	TcplpRbTlsContinueHandshakelfWarningAlertRcvd
Description	ETAS (vendor 11) specific parameter: This parameter indicates whether handshake can be continued or terminate the connection if alert message type warning is received.
Introduction	Note: If this parameter is set to TRUE, then TLS Server/Client can continue the handshake even after receiving alert message type as warning, else connection shall be terminated.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.102 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsDisableSessionResumptionOnSessionRenegotiation

Short Name	TcplpRbTlsDisableSessionResumptionOnSessionRenegotiation
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether Session Resumption is allowed during Session Renegotiation.
Introduction	Note: If this parameter is enabled, Session Resumption wont be allowed during Session Renegotiation even if remote node tries to perform the resumption handshake.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.103 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsRecordSizeLimit

Short Name	TcplpRbTlsRecordSizeLimit
Description	ETAS (vendor 11) specific parameter: The parameter specifies length of record in the Record size limit extension.
Introduction	Note: The value of RecordSizeLimit is the maximum size of record in octets that the endpoint is willing to receive. This value is used to limit the size of records that are created when encoding application data and the protected handshake message into records. The limit is $2^{14}+1$ octets. If this parameter is not configured, then it can be configured with default value 4096 bytes.
Multiplicity	0..1
Value Range	1..16384
Default Value	4096
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.104 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsRequestCertificateAuthority

Short Name	TcplpRbTlsRequestCertificateAuthority
Description	ETAS (vendor 11) specific parameter: This parameter will be configured if Client Authentication parameter is set to TRUE and TcplpTlsConnection-Type is TLS_SERVER.
Introduction	Note: This parameter is used to decide whether issuer of certificates configured per connection will be sent in Certificate Request message in distinguishedName certificate_authorities field or not.

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.105 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsSessionIdLength

Short Name	TcplpRbTlsSessionIdLength
Description	ETAS (vendor 11) specific parameter: The parameter specifies length of session id for current TLS connection.
Introduction	Note: If this parameter is not configured, then it can be configured with default value 0.
Multiplicity	0..1
Value Range	0..32
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.106 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsSessionRenegotiationCertComparisonEnabled

Short Name	TcplpRbTlsSessionRenegotiationCertComparisonEnabled
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether the renegotiated handshake's certificate(s) or certificate SNI(Server name indication) will be compared with the initial handshake's certificate(s) or SNI(Server name indication) during renegotiation.
Introduction	Note: If this parameter is enabled, handshake failure alert shall be sent if renegotiated handshake's certificate(s) or certificate SNI(Server name indication) is different from the initial handshake's certificate(s) or SNI(Server name indication).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.107 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsSessionRenegotiationEnabled

Short Name	TcplpRbTlsSessionRenegotiationEnabled
Description	ETAS (vendor 11) specific parameter: This parameter will be configured when session renegotiation is supported.

Introduction	Note: If Session Renegotiation parameter is configured to true then TLS Server/Client supports renegotiation feature. If Session Renegotiation parameter is true for server, then server can allow the client hello msg when session is ongoing. Similarly, If Session Renegotiation parameter is enabled for client, client can allow the hello request msg when session is complete. If Session Renegotiation parameter is enabled and remote does not send the renegotiation info extension then ECU will terminate the current session by sending the alert msg with no renegotiation or continue the handshake depending on whether TcplpRbTlsContinueHandshakelfRenegotiationDisabled is configured as false or true respectively.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.108 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpRbTlsSessionRenegotiationSCSV CiphersuiteEnabled

Short Name	TcplpRbTlsSessionRenegotiationSCSV CiphersuiteEnabled
Description	ETAS (vendor 11) specific parameter: This parameter specifies renegotiation SCSV (Special Signalling Cipher-suite Value) ciphersuite shall be enabled or not.
Introduction	Note: This parameter specifies whether TLS_EMPTY_RENEGOTIATION_INFO_SCSV ciphersuite will be sent in the initial client hello message during session renegotiation. It shall be enabled to communicate with legacy server when ECU is configured as Client. If ECU Client will be communicating with latest Server always, then this parameter shall be disabled and same semantics could be achieved through renegotiation_info extension. Also, It is not a true ciphersuite and cannot be negotiated as like as other ciphersuites
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.109 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpRbTlsSessionResumptionEnabled

Short Name	TcplpRbTlsSessionResumptionEnabled
------------	------------------------------------

Description	ETAS (vendor 11) specific parameter: This parameter specifies whether Session Resumption is supported.
Introduction	Note: If Session Resumption is enabled: Server can allow the Client to resume a previous session by presenting the Session Ticket instead of establishing a full handshake. Client can resume a previous session by presenting a Session Ticket obtained in the previous handshake.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.110 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsSessionResumptionMethod

Short Name	TcplpRbTlsSessionResumptionMethod
Description	Type of method which is used for session resumption. This parameter should be enabled only if TcplpRbTlsSessionResumptionEnabled is enabled.
Multiplicity	0..1
Default Value	SESSION_ID_AND_TICKET_BASED
Enumeration Literals	SESSION_ID_BASED, SESSION_TICKET_BASED, SESSION_ID_AND_TICKET_BASED
Type	ECUC-ENUMERATION-PARAM-DEF

20.6.1.111 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsUseRootCertInRequestCertificateAuthority

Short Name	TcplpRbTlsUseRootCertInRequestCertificateAuthority
Description	ETAS (vendor 11) specific parameter: This parameter specifies whether certificate authority identifier(Name) has to be fetched only for root certificate or for complete chain of certificates configured for that particular connection. This parameter should be TRUE only when TcplpTlsUseClientAuthenticationRequest parameter is TRUE, TcplpRbTlsRequestCertificateAuthority parameter is TRUE and and TcplpTlsConnectionType is TLS_SERVER.

Introduction	Note: For example if 3 TcplpTlsCertificateIdentityRefs are configured to the connection, and TcplpRbTlsUseRootCertInRequestCertificateAuthority is configured to TRUE, then certificate authority identifier will be fetched only for root certificates of all 3 configured TcplpTlsCertificateIdentityRefs. If TcplpRbTlsUseRootCertInRequestCertificateAuthority is configured to FALSE, then certificate authority identifier should be fetched for complete chain of certificates configured in 3 TcplpTlsCertificateIdentityRefs. Certificate authority identifier attribute is required to frame the certificate request handshake message which is to be sent during client authentication feature (TcplpTlsUseClientAuthenticationRequest) enabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.112 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsUseSecurityExtensionMaxFragmentLength

Short Name	TcplpRbTlsUseSecurityExtensionMaxFragmentLength
Description	ETAS (vendor 11) specific parameter: Enables Maximum fragment length Extension.
Introduction	Note: This parameter used only when maximum fragment length extension parameter is required. This parameter is enabled when maximum fragment size extension is used.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.113 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsUseSecurityExtensionMaxFragmentLengthOfRemote

Short Name	TcplpRbTlsUseSecurityExtensionMaxFragmentLengthOfRemote
Description	ETAS (vendor 11) specific parameter: This parameter specifies if Maximum fragment length extension from remote has to be considered for handshake or not.
Introduction	Note: This parameter used only when maximum fragment length extension parameter is required. This parameter is enabled when maximum fragment size extension is used.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

20.6.1.114 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsUseSecurityExtensionRecordSizeLimitOfRemote

Short Name	TcplpRbTlsUseSecurityExtensionRecordSizeLimitOfRemote
Description	Defines if the security extension for record size limit if received from remote has to be considered for handshake or not.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.115 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/ TcplpRbTlsUseSecurityExtensionSignatureAlgorithms

Short Name	TcplpRbTlsUseSecurityExtensionSignatureAlgorithms
Description	ETAS (vendor 11) specific parameter: If this parameter is configured to TRUE then signature algorithm extension can be included in the extension list.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.116 Tcplp/TcplpConfig/TcplpUdpConfig/TcplpRbUdpMulticastAddr-Reuse

Short Name	TcplpRbUdpMulticastAddrReuse
Description	ETAS (vendor 11) specific parameter: Enables (TRUE) or disables (FALSE) sockets to reuse same local address and port in case of IP address of type TCPIP_MULTICAST.
Introduction	If the parameter is disabled, binding multiple sockets on the same local address and port will be forbidden for Multicast addresses. Therefore, Udp will allow runtime optimized search in RxIndication where the breaking of the socket searching loop can be done. If multiple Upper Layers are configured above Tcplp, and if more than one UL want to receive the same multicast messages, then this parameter shall be set to true. Memory Impact: NONE. Throughput Impact: When parameter is set to FALSE, higher Rx throughput is possible for Multicast messages.
Multiplicity	0..1
Default Value	true

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

20.6.1.117 Tcplp/TcplpGeneral/TcplpV4General/TcplpDHCPGeneral

Short Name	TcplpDHCPGeneral
Description	ETAS (vendor 11) specific container: DHCP general configuration parameters
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

20.6.1.118 Tcplp/TcplpGeneral/TcplpV4General/TcplpDHCPGeneral/TcplpDHCPClientGeneral

Short Name	TcplpDHCPClientGeneral
Description	ETAS (vendor 11) specific container: General configuration parameters used for the DHCP Client process.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

20.6.1.119 Tcplp/TcplpGeneral/TcplpV4General/TcplpDHCPGeneral/TcplpDHCPClientGeneral/TcplpDHCPTimeToWaitBeforeReTx

Short Name	TcplpDHCPTimeToWaitBeforeReTx
Description	ETAS (vendor 11) specific parameter: Parameter for DHCP Client. Time to wait before the retransmission if no response received from a Server. (Time in Seconds).
Introduction	According to RFC 2131 "The delay between retransmissions should be chosen to allow sufficient time for replies from the server to be delivered based on the characteristics of the internetwork between the client and the server. " For a 10Mb/s Ethernet network, RFC2131 recommends a 4 second initial delay plus a randomized value between -1 and +1 seconds. Therefore, the minimum value is 1 second. Memory Impact: NONE. Throughput Impact: NONE.
Multiplicity	1..1
Value Range	1.0..4.0
Default Value	4
Type	ECUC-FLOAT-PARAM-DEF

20.6.1.120 Tcplp/TcplpGeneral/TcplpV4General/TcplpIcmpMessageFrag-

mentationAllowedTcplpIcmpMessageFragmentationAllowed

Short Name	TcplpIcmpMessageFragmentationAllowed
Description	ETAS (vendor 11) specific parameter: Parameter to allow fragmentation of ICMP messages exceeding MTU size. TRUE allows fragmentation of ICMP messages (messages with length greater than MTU size), FALSE truncates the ICMP message length to MTU size.
Introduction	Memory Impact: NONE. Throughput Impact: Throughput decreases if message fragmentation is enabled and Icmp messages with length greater than MTU size is transmitted.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.121 Tcplp/TcplpGeneral/TcplpIPv4General/TcplpRbIPv4DiscardDirectedBrdcastFrames

Short Name	TcplpRbIPv4DiscardDirectedBrdcastFrames
Description	ETAS (vendor 11) specific parameter: Parameter to discard directed broadcast IPv4 frames. TRUE discards IPv4 directed broadcast frames, FALSE allows all IPv4 directed broadcast frames to be processed.
Introduction	Note: An IPv4 frame is a directed broadcast frame if its destination IP address is having a directed broadcast IP address(subnet specific broadcast address). If this parameter is configured to true then all the received directed broadcast frames will be not processed further and silently discarded. Memory Impact: NONE. Throughput Impact: Throughput decreases if directed broadcast frames are allowed as the frames will be processed and forwarded to upper layer.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.122 Tcplp/TcplpGeneral/TcplpRbExternalDtIsSupport

Short Name	TcplpRbExternalDtIsSupport
------------	----------------------------

Description	ETAS (vendor 11) specific parameter: Enables/disables the support of third party DTLS component in Tcplp.
Introduction	If this parameter is enabled, the DTLS functionality of Tcplp and Udp sub module will be enabled. rba_EthDtls interfaces which are called by rba_EthUdp module need to be implemented by integrator in order to interact with the third party DTLS component. In this configuration, the TcplpDtlsConfig container can be completely removed. This parameter shall be configured only when UDP protocol is enabled. Header file named rba_EthDtls.h shall be created to contain the DTLS interfaces called by UDP module. Memory Impact: NONE. Throughput Impact: When 3rd party DTLS is integrated, throughput for Tx and Rx will be decreased as each Tx/Rx will be handed over to DTLS for encryption/decryption before UDP can process the message.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

20.6.1.123 Tcplp/TcplpGeneral/TcplpRbExternalTlsSupport

Short Name	TcplpRbExternalTlsSupport
Description	ETAS (vendor 11) specific parameter: Enables/disables the support of third party TLS component in Tcplp.
Introduction	If this parameter is enabled, the TLS functionality of Tcplp and Tcp sub module will be enabled. rba_EthTls interfaces which are called by rba_EthTcp module need to be implemented by integrator in order to interact with the third party TLS component. In this configuration, the TcplpTlsConfig container can be completely removed. This parameter shall be configured only when TCP protocol is enabled. Header file named rba_EthTls.h shall be created to contain the TLS interfaces called by TCP module. Memory Impact: NONE. Throughput Impact: When 3rd party TLS is integrated, throughput for Tx and Rx will be decreased as each Tx/Rx will be handed over to TLS for encryption/decryption before TCP can process the packet.
Multiplicity	0..1
Default Value	false

Type	ECUC-BOOLEAN-PARAM-DEF
------	------------------------

20.6.1.124 Tcplp/TcplpGeneral/TcplpSecurityEventRefs/SEV_ARP_IP_ADDR_CONFLICT

Short Name	SEV_ARP_IP_ADDR_CONFLICT
Description	Received local IP address in ARP reply for different MAC.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

20.6.1.125 Tcplp/TcplpGeneral/TcplpSecurityEventRefs/SEV_IPV4_DROP_INV_ADDR

Short Name	SEV_IPV4_DROP_INV_ADDR
Description	Dropped datagram because of invalid IPV4 address.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

20.6.1.126 Tcplp/TcplpGeneral/TcplpSecurityEventRefs/SEV_IPV6_DROP_INV_ADDR

Short Name	SEV_IPV6_DROP_INV_ADDR
Description	Dropped datagram because of invalid IPV6 address.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

20.6.1.127 Tcplp/TcplpGeneral/TcplpSecurityEventRefs/SEV_TCP_DROP_INV_PORT

Short Name	SEV_TCP_DROP_INV_PORT
Description	Dropped TCP packet because of invalid destination TCP-Port.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

20.6.1.128 Tcplp/TcplpGeneral/TcplpSecurityEventRefs/SEV_UDP_DROP_INV_PORT

Short Name	SEV_UDP_DROP_INV_PORT
------------	-----------------------

Description	Dropped UDP packet because of invalid destination UDP-Port.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

20.6.1.129 Tcplp/TcplpGeneral/TcplpZeroCopyGeneral

Short Name	TcplpZeroCopyGeneral
Description	ETAS (vendor 11) specific container: This is a vendor specific container introduced in addition to AUTOSAR which specifies the general configuration parameters for ZeroCopy transmission.
Introduction	This container is mandatory only when Tcplp ZeroCopy is required. When container is disabled, ZeroCopy API will be disabled.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

20.6.1.130 Tcplp/TcplpGeneral/TcplpZeroCopyGeneral/TcplpZeroCopyDescriptorCount

Short Name	TcplpZeroCopyDescriptorCount
Description	ETAS (vendor 11) specific parameter: Maximum number of ZeroCopy descriptors. When a ZeroCopy transmission is executed on a socket, a new descriptor will be allocated. The parameter TcplpZeroCopyDescriptorCount therefore determines the number of sockets that can execute Zero Copy Tx.
Introduction	The descriptor contains information about the ongoing Zero Copy transmissions and about the allocated header buffers. Each time a Zero Copy transmission is executed a new buffer is used to store the protocol headers. Memory Impact: More memory is consumed if number of Zero Copy descriptors is increased. Throughput Impact: When high number of descriptors is configured, the first ZeroCopyTx might take more time because the first free descriptor can be longer to find.
Multiplicity	1..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.131 Tcplp/TcplpGeneral/TcplpZeroCopyGeneral/TcplpZeroCopyMaxIpFragmentPerTx

Short Name	TcplpZeroCopyMaxIpFragmentPerTx
Description	ETAS (vendor 11) specific parameter: Maximum number of IP fragments that can be part of a single ZeroCopy transmission. Relevant only if fragmentation is enabled.
Introduction	This parameter is only used when TcplpCtrls are configured with TCPIP_ZEROCOPY_DYNAMIC. The configured value will be used to allocate IPv4 header portions related to fragmentation. Memory Impact: More memory is consumed if number of fragments is increased. Throughput Impact: In general IP fragmentation reduces the Tx throughput and should be avoided in possible.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.132 Tcplp/TcplpGeneral/TcplpZeroCopyGeneral/TcplpZeroCopyMaxMonitoredTxConf

Short Name	TcplpZeroCopyMaxMonitoredTxConf
Description	ETAS (vendor 11) specific parameter: Maximum number of ZeroCopy TxConfirmations that can be monitored by Tcplp. Each time Tcplp_ZeroCopyTransmit() API is called with TxConfFlag_b set to TRUE, new TxConf monitoring entry will be used until ZeroCopy TxConfirmation callback is invoked.
Introduction	Memory Impact: More memory is consumed if number of TxConfirmations to be monitored is higher. If less TxConfirmations are monitored, then more header buffers will be required to be allocated during the transmission process because the allocated buffers can be released only when TxConfirmation is received. Throughput Impact: The more often Tcplp_ZeroCopyTransmit() is called with TxConfFlag_b set to TRUE, the more runtime will be consumed to handle the TxConfirmations and provide callback to UL.
Multiplicity	1..1
Value Range	1..65535

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.1.133 Tcplp/TcplpGeneral/TcplpZeroCopyGeneral/TcplpZeroCopyMax-TxBeforeConfirmation

Short Name	TcplpZeroCopyMaxTxBeforeConfirmation
Description	ETAS (vendor 11) specific parameter: Maximum number of ZeroCopy transmissions that can be executed on a socket connection until ZeroCopyTx-Confirmation is received. This parameter influences the number of available ZeroCopy header buffers.
Introduction	Memory Impact: More memory is consumed if parameter is configured too high (number of allocated header buffers will be higher). The number of buffers shall be configured appropriately based on the throughput that needs to be achieved. Throughput Impact: If more header buffers are configured, it can take more time to search for a free header buffer during each ZeroCopy Tx.
Multiplicity	1..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

20.6.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

20.6.2.1 Tcplp_AllocBaseIpAddr

Syntax	TcpIp_AllocBaseIpAddr(LocalAddrId_u8, IpAddrAsgnMethType_en, LocalIpAddr_pst)
Parameters	(IN) LocalAddrId_u8 Local address id (reference to IPv4 address config and EthIf controller index).
	(IN) IpAddrAsgnMethType_en Address assignment method (Static/DHCP/Autop) used by the LocalAddrId.
	(OUT) LocalIpAddr_pst Pointer holding the base IP address to be used.
Return Values	E_OK - Request processed successfully.
	E_NOT_OK - Request NOT processed successfully.
Description	Enables the interface between SoAd and the rba_EthAutop module to provide the base IP address stored in the NVM.

20.6.2.2 Tcplp_ReadArpTable

Syntax	TcpIp_ReadArpTable(LocalAddrId_u8, ArpTable_cpu8, NumberEntries_cpu16)
Parameters	(IN) LocalAddrId_u8 IP address identifier of the EthIf controller for which the ARP table will be returned.
	(INOUT) NumberEntries_cpu16 Number of ARP table entries. As input, specifies the number of ARP table entries to be read.
	As output, contains the number of the returned ARP table entries.
	(OUT) ArpTable_cpu8 Pointer to first element of array, which contains the requested ARP table.
Return Values	E_OK - Request processed successfully.
	E_NOT_OK - Request NOT processed successfully.
Description	Reads the ARP table for the EthIf controller which is indirectly specified by the LocalAddrId.

20.6.2.3 Tcplp_ArpEnable

Syntax	TcpIp_ArpEnable(CtrlIdx_u8, Enable_b)
Parameters	(IN) CtrlIdx_u8 EthIf controller index to identify the related ARP table.
	(IN) Enable_b Boolean indicating whether the ARP protocol need to be enabled or disabled.
Return Values	E_OK - Request processed successfully.
	E_NOT_OK - Request NOT processed successfully.
Description	Used by the upper layer to enable or disable ARP protocol for a given TcplpCtrl.

20.6.2.4 Tcplp_LocalIpAddrAssignmentChg

Syntax	TcpIp_LocalIpAddrAssignmentChg(LocalAddrId_u8, State_en, IPAddrParam_pcun, IPAddrAssignMethod_en, IpDomainType_cu32)
Parameters	(IN) LocalAddrId_u8 IP address index specifying the IP address to be assigned.
	(IN) State_en IP address state (assigned or unassigned).
	(IN) IPAddrParam_pcun Pointer to a structure containing the assigned IP address, subnet mask and default router IP address.
	(IN) IPAddrAssignMethod_en IP address assignment method - DHCP or Autolp.
	(IN) IpDomainType_cu32 IP address domain type - TCPIP_AF_INET or TCPIP_AF_INET6.
Return Values	NONE

Description	Callback function used after a new IP address has been created. The new IP address is assigned to the corresponding LocalAddrId.
-------------	--

20.6.2.5 TcpIp_IcmpEnable

Syntax	TcpIp_ArpEnable(Enable_b)
Parameters	(IN) Enable_b Boolean indicating whether the ARP protocol need to be enabled or disabled.
Return Values	E_OK - Request processed successfully.
	E_NOT_OK - Request NOT processed successfully.
Description	This API is used by the upper layer to enable or disable ICMP protocol.

20.6.2.6 TcpIp_ZeroCopyUdpTransmit

Syntax	TcpIp_ZeroCopyUdpTransmit(TcpIp_SocketIdType SocketId_u16, const Eth_ZeroCopyTxDataInfo_tst * ZeroCopyData_pcst, const TcpIp_SockAddrType * RemoteAddr_pcst, boolean TxConfFlag_b, boolean ConnChanged_b, uint32 * TxId_pu32)
Parameters In	SocketId_u16 Socket identifier of the related local socket resource. ZeroCopyData_pcst Structure containing the ZeroCopy UL data to be transmitted. RemoteAddr_pcst IP address and port of the remote host to transmit to. TxConfFlag_b Flag indicating if TxConf need to be monitored (if TRUE, TxId will be returned in out argument). ConnChanged_b Flag indicating if the connection parameters have changed (new Reuse header will be allocated).
Parameters Out	TxId_pu32 Out argument storing the TxId (valid only if TxConfFlag_b is set to TRUE).
Returns	TCPIP_OK: UDP message has been forwarded to EthIf for transmission. TCPIP_E_NOT_OK: UDP message could not be sent because of a permanent error, e.g. message is too long. TCPIP_E_ARP_CACHE_MISS: UDP message could not be sent because of an ARP cache miss, ARP request has been sent and upper layer may retry transmission by calling this function later again.
Description	Transmits data via UDP ZeroCopy API to a remote node.

20.6.2.7 TcpIp_ZeroCopyReset

Syntax	TcpIp_ZeroCopyReset(TcpIp_SocketIdType SocketId_u16)
Parameters	SocketId_u16 Socket id for which the ZeroCopy structures need to be cleared..
Description	Resets all ZeroCopy structures for a particular socket.

20.6.2.8 Tcplp_ZeroCopyTxConfirmation

Syntax	TcpIp_ZeroCopyTxConfirmation(uint8 EthIfCtrlIdx_u8, uint32 TxId_u32)
Parameters	EthIfCtrlIdx_u8 Index of the EthIf controller. TxId_u32 TxId which identify the transmission which is now completed.
Description	Callback function invoked when a transmission marked with TxId is completed.

20.6.2.9 Tcplp_UdpTransmitMetaData

Syntax	TcpIp_UdpTransmitMetaData(TcpIp_SocketIdType SocketId_u16, const uint8 * DataPtr_pcu8, const TcpIp_SockAddrType * RemoteAddrPtr_pcst, uint16 TotalLength_u16, const void * MetaDataPtr_pcv)
Parameters In	SocketId_u16 Socket identifier of the related local socket resource. DataPtr_pcu8 Pointer to a linear buffer of TotalLength bytes containing the data to be transmitted. RemoteAddr_pcst IP address and port of the remote host to transmit to. TotalLength_u16 Indicates the payload size of the UDP datagram. MetaDataPtr_pcv MetaData provided by UpperLayer which will be given back within Up_CopyTxDataMetaData.
Returns	TCPIP_OK: UDP message has been forwarded to EthIf for transmission. TCPIP_E_NOT_OK: UDP message could not be sent because of a permanent error, e.g. message is too long. TCPIP_E_ARP_CACHE_MISS: UDP message could not be sent because of an ARP cache miss, ARP request has been sent and upper layer may retry transmission by calling this function later again.
Description	Transmits data via UDP to a remote node with the possibility to provide meta-data.

20.6.2.10 Tcplp_TcpTransmitMetaData

Syntax	TcpIp_TcpTransmitMetaData(TcpIp_SocketIdType SocketId_u16, const uint8 * DataPtr_pcu8, uint32 AvailableLength_u32, boolean ForceRetrieve_b, void * MetaDataPtr_pv)
Parameters In	SocketId_u16 Tcplp internal handle for sockets used in the internal dynamic TCP socket table. DataPtr_pcu8 Pointer to a linear buffer of AvailableLength bytes containing the data to be transmitted. AvailableLength_u32 Available data for transmission in bytes. ForceRetrieve_b Only valid if DataPtr is a NULL_PTR, and indicates how the TCP/IP stack retrieves data from SoAd. MetaDataPtr_pcv MetaData provided by UpperLayer which will be given back within Up_CopyTxDataMetaData.
Description	Transmits data via TCP to a remote node with the possibility to provide metadata.

20.6.2.11 Tcplp_DuplicateAddressIndication

Syntax	<code>TcpIp_DuplicateAddressIndication(TcpIp_LocalAddrIdType LocalAddrId_u8, TcpIp_DadStateType_ten DadState_en, const TcpIp_IPAddrParamType_tun * IPAddrParam_pcur, const uint8 * RemotePhysAddrPtr_pcu8)</code>
Parameters In	LocalAddrId_u8 IP address identifier representing the local IP address. DadState_en IP address is Duplicate or Unique. IPAddrParam_pcur Pointer to a memory where the conflicted IP address is stored. RemotePhysAddrPtr_pcu8 Pointer to memory where remote physical address (MAC address) related to the specified IP address is stored in network byte order.
Description	Callback function called from Ipv6 module when it has to inform Tcplp that the assigned IP address is Duplicate or Unique.

20.6.2.12 Tcplp_SetRemotePhysAddr

Syntax	<code>TcpIp_SetRemotePhysAddr(uint8 CtrlIdx_u8, const TcpIp_SockAddrType* IpAddrPtr_pcst, const uint8* PhysAddrPtr_pcu8)</code>
Parameters In	CtrlIdx_u8 EthIf controller index to identify the related ARP table. IpAddrPtr_pcst Specifies the IP address for which the physical address shall be written in ARP table. PhysAddrPtr_pcu8 Pointer to the memory where the physical source address (MAC address) related to the specified IP address is stored in network byte order.
Returns	TcpIp_ReturnType TCPIP_OK: The request was successful, IP and MAC addresses have been successfully added into ARP table as static entry. TCPIP_E_NOT_OK: The request was not successful.
Description	Used by the upper layer to initialize ARP table with static IP/MAC combinations.

20.7 Limitations

20.7.1 IPv6 Support

IPv6 is not fully supported, currently only the basic functionality of IPv6 (and NDP and ICMPv6) is supported.

20.7.2 DHCPv6 Support

DHCPv6 is currently supported with limitations.

20.7.3 Dual Stack Support

Tcplp does not support dual stack configuration (both IPv4 and IPv6) on the same VLAN. Currently IPv4 and IPv6 communication need to be configured in a separate TcplpCtrl and, therefore, on a separate VLAN.

20.7.4 DHCP Server Support

The server part of DHCP is not implemented in the stack. The stack supports only DHCP client part.

20.7.5 Socket Handling

Changing parameters of a socket via Tcplp_ChangeParameter is not supported for TTL.

20.7.6 IP Addressing

- Wildcard specifiers for local Address ID and Local IP address are not supported.
- Automatic selection of the Local IP address and EthIf Controller are not supported.
- TcplpCtrl must always have maximum one unicast IPv4 address assigned at the same time. Multiple unicast IPv4 addresses cannot be assigned simultaneously to the same TcplpCtrl

20.7.7 ZeroCopy

- Tcplp ZeroCopy feature does not support reentrancy and multicore / multi-task distribution. It need to be ensured that the UL using ZeroCopy APIs, the Tcplp_MainFunction() and the Eth_MainFunction() are executed from the same execution context if the feature is used.
- Tcplp ZeroCopy feature does not support the modification of parameters (like FramePrio parameter) via Tcplp_ChangeParameter() API after the first ZeroCopy transmission is executed. The parameters have to be changed after the socket is open and before the first ZeroCopyTx
- Tcplp ZeroCopy feature is not supported for IPv6 and IPSec

20.7.8 IPSec

- Security for the IP layer is not currently supported, so to avoid warning messages and error reports you should not configure TcplpSecConfigSet.

20.7.9 Other Limitations

- Not all Runtime error notifications via Det_ReportRunTimeError are currently supported.

- Transmission of a UDP frame using a socket that is not bound, or is bound to a multicast IP address is not supported.
- Discovery of the maximum transmission unit (MTU) for a path is not supported.
- For IPv6 communication, the EthIfCtrlMtu should not be configured larger than 1280 octets (see RFC2460, chapter 5. Packet Size Issues). If only in-vehicle communication is performed on the related EthIfCtrl and if the network architecture is known and fixed, then the EthIfCtrlMtu can be manually chosen greater than 1280 (the value chosen needs to be lower than or equal to the MTU of the intermediate and destination links over the path).
- The TcpIp_GetRemotePhysAddr API, which queries the IP/physical address translation table (specified by CtrlIdx) and returns the physical address related to the IP address (specified by IpAddrPtris) is not supported for INET domain.

20.7.10 TcpIpArpRequestTimeout

- It is not permitted to set the value of this parameter to 0 as this is not supported in RTA-BSW.

20.8 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

20.8.1 API Functions

The following API functions are not supported in this release.

- TcpIp_ResetIpAssignment
- TcpIp_DhcpV6ReadOption
- TcpIp_DhcpV6WriteOption
- TcpIp_GetArpCacheEntries
- TcpIp_GetNdpCacheEntries

20.8.2 Configuration Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- TcpIp/TcpIpConfig/TcpIpCtrl/TcpIpDhcpServerConfigRef
- TcpIp/TcpIpConfig/TcpIpCtrl/TcpIpIpvXCtrl/TcpIpIpv4Ctrl/TcpIpAutoIpConfigRef
- TcpIp/TcpIpConfig/TcpIpCtrl/TcpIpIpvXCtrl/TcpIpIpv4Ctrl/TcpIpDhcpConfigRef
- TcpIp/TcpIpConfig/TcpIpCtrl/TcpIpIpvXCtrl/TcpIpIpv4Ctrl/TcpIpIpv4MtuConfig

- Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV4Ctrl/TcplpV4Mtu-Config/TcplpV4PathMtuEnabled
- Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV4Ctrl/TcplpV4Mtu-Config/TcplpV4PathMtuTimeout
- Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV6Ctrl/TcplpV6Mtu-Config
- Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV6Ctrl/TcplpV6Mtu-Config/TcplpV6PathMtuEnabled
- Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV6Ctrl/TcplpV6Mtu-Config/TcplpV6PathMtuTimeout
- Tcplp/TcplpConfig/TcplpCtrl/TcplpVXCtrl/TcplpV6Ctrl/TcplpV6Ndp-ConfigRef
- Tcplp/TcplpConfig/TcplpDhcpServerConfig
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpAddressAssignment
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpAddressAssignment/TcplpDhcpAddressLowerBound
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpAddressAssignment/TcplpDhcpAddressUpperBound
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpAddressAssignment/TcplpDhcpSwitchPortRef
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpDefaultRouter
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpEthIfSwitchRef
- Tcplp/TcplpConfig/TcplpDhcpServerConfig/TcplpDhcpNetmask
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpArpConfig/TcplpArpDefensiveProcessing
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpArpConfig/TcplpArpPacketQueueEnabled
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpDhcpConfig
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpIpFragmentation-Config/TcplpIpFragmentationRxEnabled
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpIpFragmentation-Config/TcplpIpNumFragments
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpIpFragmentation-Config/TcplpIpNumReassDgrams
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV6Config/TcplpDhcpV6Config
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV6Config/TcplpV6Fragmentation-Config/TcplpV6ReassemblySegmentCount
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV6Config/TcplpNdpConfig/TcplpNdpArNudConfig/TcplpNdpDefensiveProcessing
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpV6Config/TcplpNdpConfig/Tcplp-

- NdpPrefixRouterDiscoveryConfig/TcplpNdpPrefixList
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpPrefixRouterDiscoveryConfig/TcplpNdpPrefixList/TcplpNdpPrefixListEntry
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpPrefixRouterDiscoveryConfig/TcplpNdpPrefixList/TcplpNdpPrefixListEntry/TcplpNdpPrefixListEntryPrefixAddress
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpPrefixRouterDiscoveryConfig/TcplpNdpPrefixList/TcplpNdpPrefixListEntry/TcplpNdpPrefixListEntryPrefixLength
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpSlaacConfig/TcplpNdpSlaacDadNumberOfTransmissions
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpSlaacConfig/TcplpNdpSlaacDadRetransmissionDelay
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpSlaacConfig/TcplpNdpSlaacDelayEnabled
- Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpSlaacConfig/TcplpNdpSlaacOptimisticDadEnabled
- Tcplp/TcplpConfig/TcplpIpSecConfigSet/TcplpEncryptionAlgorithm
- Tcplp/TcplpConfig/TcplpIpSecConfigSet/TcplpEncryptionAlgorithm/TcplpEncryptionTransformJobPair
- Tcplp/TcplpConfig/TcplpIpSecConfigSet/TcplpIntegrityAlgorithm
- Tcplp/TcplpConfig/TcplpIpSecConfigSet/TcplpIntegrityAlgorithm/TcplpIntegrityTransformJobPair
- Tcplp/TcplpConfig/TcplpLocalAddr/TcplpAddrAssignment/TcplpAssignmentLifetime
- Tcplp/TcplpConfig/TcplpNvmBlock
- Tcplp/TcplpConfig/TcplpNvmBlock/TcplpNvmBlockDescriptorRef
- Tcplp/TcplpConfig/TcplpPhysAddrConfig
- Tcplp/TcplpConfig/TcplpPhysAddrConfig/TcplpPhysAddrChgHandler
- Tcplp/TcplpConfig/TcplpPhysAddrConfig/TcplpPhysAddrChgHandler/TcplpPhysAddrChgHandlerName
- Tcplp/TcplpConfig/TcplpTcpConfig/TcplpDelayedAckTimeout
- Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpConfigOptionFilter
- Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpConfigOptionFilter/TcplpTcpConfigOptionFilterEntry
- Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpConfigOptionFilter/TcplpTcpConfigOptionFilterId
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsCipherSuiteDefinition/TcplpTlsUseAEADCipher

- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsCipher-suiteWorker/TcplpTlsCipherAEADCipherKeyLength
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsCipher-suiteWorker/TcplpTlsCipherEncryptKeyLength
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsCipher-suiteWorker/TcplpTlsCipherMacKeyLength
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHand-shake/TcplpTlsCsmMasterSecretKeyRef
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpTlsConnec-tionGroupRef
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpTlsPortAs-signment
- Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnectionGroup
- Tcplp/TcplpConfig/TcplpUdpConfig/TcplpUdpTtl
- Tcplp/TcplpGeneral/TcplpBufferMemory
- Tcplp/TcplpGeneral/TcplpDhcpServerEnabled
- Tcplp/TcplpGeneral/TcplpIpv4General/TcplpArpEnabled
- Tcplp/TcplpGeneral/TcplpIpv4General/TcplpAutoIpEnabled
- Tcplp/TcplpGeneral/TcplpIpv4General/TcplpDhcpClientEnabled
- Tcplp/TcplpGeneral/TcplpIpv4General/TcplpIpv4Enabled
- Tcplp/TcplpGeneral/TcplpIpv4General/TcplpLocalAddrIpv4EntriesMax
- Tcplp/TcplpGeneral/TcplpIpv4General/TcplpPathMtuDiscoveryEnabled
- Tcplp/TcplpGeneral/TcplpIpv6General/TcplpDhcpV6ClientEnabled
- Tcplp/TcplpGeneral/TcplpIpv6General/TcplpIpv6Enabled
- Tcplp/TcplpGeneral/TcplpIpv6General/TcplpIpv6PathMtuDiscoveryEnabled
- Tcplp/TcplpGeneral/TcplpIpv6General/TcplpLocalAddrIpv6EntriesMax
- Tcplp/TcplpGeneral/TcplpResetIpAssignmentApi
- Tcplp/TcplpGeneral/TcplpSecurityEventRefs/TCPIP_SEV_ARP_IP_ADDR_-CONFLICT
- Tcplp/TcplpGeneral/TcplpSecurityEventRefs/TCPIP_SEV_DROP_INV-_V_IPV4_ADDR
- Tcplp/TcplpGeneral/TcplpSecurityEventRefs/TCPIP_SEV_DROP_INV-_V_IPV6_ADDR
- Tcplp/TcplpGeneral/TcplpSecurityEventRefs/TCPIP_SEV_DROP_INV_-PORT_TCP
- Tcplp/TcplpGeneral/TcplpSecurityEventRefs/TCPIP_SEV_DROP_INV_-PORT_UDP
- Tcplp/TcplpGeneral/TcplpTcpEnabled

- Tcplp/TcplpGeneral/TcplpUdpEnabled

20.9 Deviations

20.9.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

20.9.1.1 Tcplp/TcplpConfig/TcplpCtrl/TcplpEthIfCtrlRef

[ADDED] Introduction	Separately for INET and INET6 domains, each TcplpCtrl shall refer to one EthIfCtrl uniquely. It is possible to have one EthIfCtrl referenced by two TcplpCtrl only if the domains are different (dual stack functioning on the same VLAN)
----------------------	---

20.9.1.2 Tcplp/TcplpConfig/TcplpCtrl/TcplpIpVXCtrl

[RTA-BSW] Description	Specifies whether this controller is an Internet Protocol version 4 (IPv4) or Internet Protocol version 6 (IPv6) instance. Note: one TcplpCtrl can not support both IPv4 and IPv6 at the same time.
[AUTOSAR] Description	Specifies whether this controller is an Internet Protocol version 4 (IPv4) or Internet Protocol version 6 (IPv6) instance.

20.9.1.3 Tcplp/TcplpConfig/TcplpCtrl/TcplpIpVXCtrl/TcplpIpV4Ctrl

[RTA-BSW] Description	Specifies an Internet Protocol version 4 (IPv4) instance. Container must be configured to specify domain of TcplpCtrl.
[AUTOSAR] Description	Specifies an Internet Protocol version 4 (IPv4) instance.

20.9.1.4 Tcplp/TcplpConfig/TcplpCtrl/TcplpIpVXCtrl/TcplpIpV6Ctrl

[RTA-BSW] Description	Specifies an Internet Protocol version 6 (IPv6) instance. Container must be configured to specify domain of TcplpCtrl.
[AUTOSAR] Description	Specifies an Internet Protocol version 6 (IPv6) instance.

20.9.1.5 Tcplp/TcplpConfig/TcplpDuplicateAddressDetectionConfig/TcplpDuplicateAddressDetectionCalloutName

[ADDED] Introduction	Note:It specifies the name of callout function which will be called by Tcplp for informing the upper layer whether the IP address is duplicate on the network.
----------------------	--

20.9.1.6 Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpArpConfig/TcplpArpNumGratuitousARPOnStartup

[RTA-BSW] Description	Specifies the number of gratuitous ARP replies which shall be sent after Tcplp_ArpEnable() has been called.
[AUTOSAR] Description	Specifies the number of gratuitous ARP replies which shall be sent on assignment of a new IP address.
[ADDED] Introduction	Note: If this parameter is not configured and if TcplpRbArpGratuitousARPCycleTime is configured, then the gratuitous ARP replies will be sent indefinitely.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

20.9.1.7 Tcplp/TcplpConfig/TcplpIpConfig/TcplpV4Config/TcplpArpConfig/TcplpArpRequestTimeout

[RTA-BSW] Description	Specifies a timeout in seconds for the validity of ARP requests. After the transmission of an ARP request the Tcplp shall skip the transmission of any further ARP requests to the same destination within a duration of TcplpArpRequestTimeout seconds. (IETF RFC 1122, section 2.3.2.1) The value for this parameter shall be an integral multiple of TcplpMainFunction-Period.
[AUTOSAR] Description	Specifies a timeout in seconds for the validity of ARP requests. After the transmission of an ARP request the Tcplp shall skip the transmission of any further ARP requests to the same destination within a duration of TcplpArpRequestTimeout seconds. (IETF RFC 1122, section 2.3.2.1) The value for this parameter shall be an integral multiple of TcplpMainFunction-Period or 0. If this parameter set to 0 this features is disabled and no delay between ARP requests is enforced.
[ADDED] Introduction	The Timeout in seconds after which ARP resends an ARP Request if ARP Reply was not received. The maximum number of ARP requests to be send before quitting the address resolution is governed by TcplpArpMaxRequests or TcplpArpRequestTxTimeout parameters.

20.9.1.8 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpArpConfig/TcplpArpTableSizeMax

[RTA-BSW] Minimum Value	1
[AUTOSAR] Minimum Value	0

20.9.1.9 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpIcmpConfig/TcplpIcmpTtl

[ADDED] Introduction	This Time-To-Live value decides how long packet stays alive on network when destination is not reachable
----------------------	--

20.9.1.10 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV4Config/TcplpIpFragmentationConfig/TcplpIpReassTimeout

[RTA-BSW] Description	Specifies the timeout in [s] after which an incomplete datagram gets discarded. RFC 1122 recommends the value to be in between 60 seconds and 120 seconds.
[AUTOSAR] Description	Specifies the timeout in [s] after which an incomplete datagram gets discarded.
[RTA-BSW] Introduction	Note: This parameter is only relevant if TcplpIpReassemblyBufferCount is configured to a value greater than zero.
[AUTOSAR] Introduction	Note: this parameter is only relevant if TcplpIpFragmentationRxEnabled is TRUE.
[RTA-BSW] Value Range	0.001..120
[AUTOSAR] Value Range	0..INF

20.9.1.11 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config

[RTA-BSW] Description	Specifies the configuration parameters of the IPv6 (Internet Protocol version 6) sub-module. This is not supported in present implementation and hence not to be configured.
[AUTOSAR] Description	Specifies the configuration parameters of the IPv6 (Internet Protocol version 6) sub-module.

20.9.1.12 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpIcmpV6Config/TcplpIcmpV6MsgDestinationUnreachableEnabled

[RTA-BSW] Description	Dis/Enables transmission of Destination Unreachable Messages.
[AUTOSAR] Description	Dis/Enables transmission of Destination Unreachable Messages

20.9.1.13 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpIpV6ConfigExtHeaderFilter

[ADDED] Introduction	Next Header values for UL protocols are not considered as Extension Headers, therefore UL values should not be configured as Filter entries. It is possible to configure an empty white list, such a configuration will not allow any IPv6 frame with an Extension Header to be received.
----------------------	---

20.9.1.14 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpIpV6ConfigExtHeaderFilter/TcplpIpV6ConfigExtHeaderFilterEntry

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

20.9.1.15 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpIpV6FragmentationConfig/TcplpIpV6TxFragmentBufferCount

[RTA-BSW] Minimum Value	0
[AUTOSAR] Minimum Value	1

20.9.1.16 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig

[RTA-BSW] Description	Specifies the configuration parameters of the Neighbor Discovery Protocol for IPv6.
[AUTOSAR] Description	Specifies the configuration parameters of the Neighbor Discovery Protocol for IPv6

20.9.1.17 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpArNudConfig/TcplpNdpDelayFirstProbeTime

[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

20.9.1.18 Tcplp/TcplpConfig/TcplpIpConfig/TcplpIpV6Config/TcplpNdpConfig/TcplpNdpArNudConfig/TcplpNdpNumUnicastSolicitations

[RTA-BSW] Description	Maximum number of unicast solicitations that will be sent when performing Neighbor Unreachability Detection.
[AUTOSAR] Description	Maximum number of unicast solicitations that will be sent when performing Neighbor Unreachability Detection.

20.9.1.19 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpAddrAssignment/Tcplp-

pAssignmentMethodTcplpAssignmentMethod

[ADDED] Introduction	TCPIP_DHCP is dynamic ip address assignment based on Dhcp protocol, TCPIP_LINKLOCAL and TCPIP_LINKLOCAL_DOIP are based on address autoconfiguration (Autolp which dynamically assigns IP address in range of 169.254.1.0 to 169.254.254.255 for IPv4, SLAAC LinkLocal for IPv6) and TCPIP_STATIC is for static IP address assignment where address is decided at configuration time if trigger is TCPIP_AUTOMATIC and if trigger is TCPIP_MANUAL then Tcplp_RequestIpAddrAssignment api does the assignment. For IPv6 address assignment, there is no difference between TCPIP_LINKLOCAL and TCPIP_LINKLOCAL_DOIP assignment methods.
----------------------	---

20.9.1.20 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpAddrAssignment/TcplpAssignmentTrigger

[ADDED] Introduction	In TCPIP_AUTOMATIC trigger method assignment shall be initiated automatically by TCP/IP stack. TCPIP_MANUAL assignment shall be initiated manually via Tcplp_RequestIpAddrAssignment() api.
----------------------	---

20.9.1.21 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpAddrId

[RTA-BSW] Maximum Value	254
[AUTOSAR] Maximum Value	65535

20.9.1.22 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpAddressType

[ADDED] Introduction	Address type indentifies whether IP address is unicast or multicast address. Valid range for multicast IP is 224.0.0.0 to 239.255.255.255. No Unicast IP should be configured in that range. With Multicast, TcplpAssignmentMethod shall be TCPIP_STATIC
----------------------	--

20.9.1.23 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpDomainType

[ADDED] Introduction	When address family is TCPIP_AF_INET, TcplpIPv4Ctrl container for the referenced TcplpCtrl shall be configured and for TCPIP_AF_INET_6, TcplpIPv6Ctrl container for the referenced TcplpCtrl shall be configured
----------------------	--

20.9.1.24 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpStaticIpAddressConfig/TcplpDefaultRouter

[ADDED] Introduction	DefaultRouter shall not be configured for multicast address.
----------------------	--

20.9.1.25 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpStaticIpAddressConfig/TcplpNetmask

[ADDED] Introduction	Netmask is mandatory if TcplpAssignmentMethod is configured as TCPIP_STATIC, TcplpAssignmentTrigger is configured as TCPIP_AUTOMATIC, and TcplpAddressType is configured as TCPIP_UNICAST. Netmask shall not be configured for multicast address.
----------------------	---

20.9.1.26 Tcplp/TcplpConfig/TcplpLocalAddr/TcplpStaticIpAddressConfig/TcplpStaticIpAddress

[RTA-BSW] Introduction	To specify any IP address for a certain EthIfCtrl, "ANY" has to be set as wildcard. See Tcplp_Bind() for more details. Static IP address is mandatory if TcplpAssignmentMethod is configured as TCPIP_STATIC, TcplpAssignmentTrigger is configured as TCPIP_AUTOMATIC and TcplpAddressType is configured as TCPIP_UNICAST.
[AUTOSAR] Introduction	To specify any IP address for a certain EthIfCtrl, "ANY" has to be set as wildcard. See Tcplp_Bind() for more details.

20.9.1.27 Tcplp/TcplpConfig/TcplpSocketOwnerConfig/TcplpSocketOwner/TcplpSocketOwnerUpperLayerType

[ADDED] Introduction	SoAd can not be configured as upper layer for more than 1 time. All the configuration parameters specifying the name of the callback function should not be configured for SoAd
----------------------	---

20.9.1.28 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpCongestionAvoidanceEnabled

[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

20.9.1.29 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpFastRecoveryEnabled

[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

20.9.1.30 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpKeepAliveInterval

[ADDED] Introduction	Dependency: TcplpTcpKeepAliveEnabled
----------------------	--------------------------------------

20.9.1.31 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpKeepAliveProbesMax

[ADDED] Introduction	Dependency: TcplpTcpKeepAliveEnabled
----------------------	--------------------------------------

20.9.1.32 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpKeepAliveTime

[RTA-BSW] Description	Specifies the time in [s] between the last data packet sent (simple ACKs are not considered data) and the first keepalive probe.
[AUTOSAR] Description	Specifies the time in [s] between the last data packet sent (simple ACKs are not considered data) and the first keepalive probe. Note: Setting this configuration parameter to a value smaller or equal to the value of TcplpMainFunctionPeriod results in the transmission of keep alive probes within every MainFunction cycle.
[ADDED] Introduction	Note: Setting this configuration parameter to a value smaller or equal to the value of TcplpMainFunctionPeriod results in the transmission of keep alive probes within every MainFunction cycle. Dependency: TcplpTcpKeepAliveEnabled

20.9.1.33 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpMaxRtx

[RTA-BSW] Intro- duction	Note: This parameter also applies for FIN retransmissions. Default: 255 is used when parameter is disabled Dependency: TcplpTcpReTxEnabled
[AUTOSAR] Intro- duction	Note: This parameter also applies for FIN retransmissions.

20.9.1.34 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpRetransmission-Timeout

[RTA-BSW] Introduction	Note: If the timeout is disabled or set to INF, no TCP segments shall be retransmitted. Value can be overwritten by Tcplp_ChangeParameter() API for a particular connection. Default: INF is used when parameter is disabled Dependency: TcplpTcpReTxEnabled
[AUTOSAR] Introduction	If the timeout is disabled or set to INF, no TCP segments shall be retransmitted. Value can be overwritten by Tcplp_ChangeParameter() API for a particular connection.

20.9.1.35 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpSlowStartEnabled

[RTA-BSW] Multiplicity	0..0
------------------------	------

[AUTOSAR] Multiplicity	1..1
------------------------	------

20.9.1.36 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpSynMaxRtx

[RTA-BSW] Introduction	Note: SYN will be retried after TcplpTcpRetransmissionTimeout. The connection will be dropped if no matching connection request has been received after the last TCP SYN has been sent and TcplpTcpRetransmissionTimeout has been expired. Dependency: TcplpTcpReTxEnabled
[AUTOSAR] Introduction	Note: SYN will be retried after TcplpTcpRetransmissionTimeout. The connection will be dropped if no matching connection request has been received after the last TCP SYN has been sent and TcplpTcpRetransmissionTimeout has been expired.

20.9.1.37 Tcplp/TcplpConfig/TcplpTcpConfig/TcplpTcpTtl

[ADDED] Introduction	This Time-To-Live value decides how long TCP packet stays alive on network when destination is unreachable.
[RTA-BSW] Minimum Value	0
[AUTOSAR] Minimum Value	1

20.9.1.38 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpTlsCsmHashVerifyJobRef

[RTA-BSW] Upper Multiplicity	2
[AUTOSAR] Upper Multiplicity	1

20.9.1.39 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpTlsCsmKeyExchangeDecryptJobRef

[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

20.9.1.40 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpTlsCsmKeyExchangeDecryptKeyRef

[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

20.9.1.41 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpTlsCsmKeyExchangeEncryptJobRef

[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

20.9.1.42 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsCiphersuites/TcplpTlsHandshake/TcplpTlsCsmKeyExchangeEncryptKeyRef

[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

20.9.1.43 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpTlsConnectionGetTimeFunc

[RTA-BSW] Upper Multiplicity	0
[AUTOSAR] Upper Multiplicity	1

20.9.1.44 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpTlsMaxFragmentLength

[ADDED] Default Value	4096
-----------------------	------

20.9.1.45 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsConnection/TcplpTlsUseSecurityExtensionRecordSizeLimit

[RTA-BSW] Description	Defines if the security extension for max_fragment_length shall be supported as defined in IETF RFC 8449.
[AUTOSAR] Description	Defines if the security extension for max_fragment_length shall be supported as defined in IETF RFC 8449, chapter 4.1.

20.9.1.46 Tcplp/TcplpConfig/TcplpTlsConfig/TcplpTlsMaxConnections

[ADDED] Introduction	This parameter will refer to the number of dynamic connections. Note: The parameter specifies number of remote connections used simultaneously. For example if one TLS server wants to communicate with two TLS clients at same time then value for this parameter will be 2.
----------------------	---

20.9.1.47 Tcplp/TcplpGeneral/TcplpDevErrorDetect

[RTA-BSW] Description	Switches the Default Error Tracer (Det) notification ON or OFF.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[RTA-BSW] Introduction	* true: detection and notification is enabled. * false: notification is disabled (detection remains active).
[AUTOSAR] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.

20.9.1.48 Tcplp/TcplpGeneral/TcplpEnableSecurityEventReporting

[RTA-BSW] Description	Switches the reporting of security events to the IdsM:
[AUTOSAR] Description	Switches the reporting of security events to the IdsM: Tags: atp.Status=draft

20.9.1.49 Tcplp/TcplpGeneral/TcplpIPv4General

[RTA-BSW] Description	This container is a subcontainer of Tcplp and specifies the general configuration parameters of the TCP/IP stack for IPv4. This container is mandatory only when TcplpScalabilityClass is configured as SC1 or SC3. TcplpIPv4General must not be configured for Scalability Class 2.
[AUTOSAR] Description	This container is a subcontainer of Tcplp and specifies the general configuration parameters of the TCP/IP stack for IPv4
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

20.9.1.50 Tcplp/TcplpGeneral/TcplpIPv6General

[ADDED] Introduction	This container is mandatory only when TcplpScalabilityClass is configured as SC2 or SC3. TcplpIPv6General must not be configured for Scalability Class 1.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

20.9.1.51 Tcplp/TcplpGeneral/TcplpIPv6General/TcplpNdpAddressResolutionUnreachabilityDetectionEnabled

[RTA-BSW] Description	Enables (TRUE) or disables (FALSE) support of Address Resolution and Neighbor Unreachability Detection via NDP.
[AUTOSAR] Description	Enables (TRUE) or disables (FALSE) support of Address Resolution and Neighbor Unreachability Detection via NDP.

20.9.1.52 Tcplp/TcplpGeneral/TcplpScalabilityClass

[RTA-BSW] Description	In order to customize the Tcplp Stack to the specific needs of the user it can be scaled according to the scalability classes. There are 3 possible scalability classes: SC1 (IPv4 - In-Vehicle and Diagnostic Communication), SC2 (IPv6 - In-Vehicle and Diagnostic Communication) and SC3 (IPv4 and IPv6 (Dual Stack) - In-Vehicle and Diagnostic).
[AUTOSAR] Description	In order to customize the Tcplp Stack to the specific needs of the user it can be scaled according to the scalability classes.
[ADDED] Introduction	If SC1: TcplpV4General shall be configured, all TcplpCtrl shall have TcplpV4Ctrl configured and all TcplpLocalAddr shall have domain type TCP_IP_AF_INET If SC2: TcplpV6General shall be configured, all TcplpCtrl shall have TcplpV6Ctrl configured and all TcplpLocalAddr shall have domain type TCP_IP_AF_INET_6 If SC3: TcplpV4General and TcplpV6General shall be configured

20.9.1.53 Tcplp/TcplpGeneral/TcplpSecurityEventRefs

[RTA-BSW] Description	Container for the references to IdsMEvent elements representing the security events that the Tcplp module shall report to the IdsM in case the corresponding security related event occurs (and if TcplpEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events.
[AUTOSAR] Description	Container for the references to IdsMEvent elements representing the security events that the Tcplp module shall report to the IdsM in case the corresponding security related event occurs (and if TcplpEnableSecurityEventReporting is set to "true"). The standardized security events in this container can be extended by vendor-specific security events. Tags: atp.Status=draft

20.9.1.54 Tcplp/TcplpGeneral/TcplpTcpSocketMax

[RTA-BSW] Description	Maximum number of TCP sockets. Important: If this parameter is set to 0, then all TCP protocol specific configuration will be disabled.
[AUTOSAR] Description	Maximum number of TCP sockets

20.9.1.55 Tcplp/TcplpGeneral/TcplpUdpSocketMax

[RTA-BSW] Description	Maximum number of UDP sockets. Important: If this parameter is set to 0, then all UDP protocol specific configuration will be disabled.
[AUTOSAR] Description	Maximum number of UDP sockets.

[ADDED] Introduction	If Udp sockets are configured to 0, then DHCP assignment shall not be configured (DHCP uses Udp) and there shall be no TcpIpLocalAddr configured with TcpIpAssignmentMethod set as TCPIP_DHCP.
----------------------	--

20.10 API Definition

List of supported API functions

Table 20.202.Tcplp Supported API functions

TcpIp_Init	Initialises the module.
TcpIp_GetVersionInfo	Returns version information for the module.
TcpIp_Close	Requests closure of a socket and the release of all related resources.
TcpIp_Bind	Requests binding of a UDP or TCP socket to a local resource.
TcpIp_TcpConnect	Requests a TCP connection to the configured node.
TcpIp_Tcplisten	Requests a listen operation for the specified socket.
TcpIp_TcpReceived	Confirms reception of socket data for the specified socket.
TcpIp_RequestComMode	Changes the communication mode for the network identified by the EthIf controller index.
TcpIp_RequestIpAddrAssignment	Requests an IP address assignment for the specified LocalAddrId.
TcpIp_ReleaseIpAddrAssignment	Releases the IP address assignment for the specified LocalAddrId.
TcpIp_IcmpTransmit	Sends an ICMP message according to the specified parameters.
TcpIp_DhcpReadOption	Retrieves the DHCP option data identified by LocalIpAddrId.
TcpIp_DhcpWriteOption	Writes the DHCP option data identified by LocalIpAddrId.
TcpIp_ChangeParameter	Requests a change to the parameter of a socket.
TcpIp_GetIpAddr	Obtains the local IP address/subnet mask/default router used by LocalIpAddrId.
TcpIp_GetPhysAddr	Obtains the physical address used by the EthIf controller specified by LocalAddrId.

TcpIp_GetRemotePhysAddr	Queries the IP/physical address translation table specified by CtrlIdx and returns the physical address related to the IP address specified by IpAddrPtr.
TcpIp_GetCtrlIdx	Returns the index of the controller identified by LocalAddrId.
TcpIp_GetAndResetMeasurementData	Allows the TCP/IP stack to read/reset the diagnostic measurement data.
TcpIp_UdpTransmit	Transmits data via UDP to a remote node via EthIf.
TcpIp_TcpTransmit	Requests transmission of data to remote node using specified socket.
TcpIp_RxIndication	Callback function which confirms a received frame and returns the frame payload data.
TcpIp_IcmpV6Transmit	The TCP/IP stack sends an ICMPv6 message according to the specified parameters.
TcpIp_IsConnectionReady	Checks if a communication over this socket is possible for a dedicated remote address.

For Further information please refer to AUTOSAR_SWS_Tcplp.pdf

20.11 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

20.11.1 IP Address Assignment

Address assignment can be either AUTOMATIC or MANUAL, the latter only being possible if TcplpAssignmentTrigger is set as TCPIP_MANUAL.

If assignment is AUTOMATIC, the IP address can be assigned only after online communication mode is requested by EthSM (TCPIP_STATE_ONLINE). If assignment is MANUAL, the IP address assignment is triggered by the upper layer (SoAd or CDD) via Tcplp_RequestIpAddrAssignment.

A Static IP address will be assigned/released immediately, but the assignment of a Dynamic IP address will be triggered, which might take some time until the IP address is obtained by the relevant sub-module (i.e. rba_EthAutoIp, rba_EthD-

hcp).rba_EthDhcp).

If a new address from an assignment method with a higher priority is available, it overwrites the IP address previously assigned by an assignment method with a lower priority.

Tcplp_RequestIpAddrAssignment returns E_OK if the requested IP assignment type is already assigned to the local address ID. If the requested static IP address is a different IP address (for unicast different IP, default router address or subnet mask) to that currently assigned, then Tcplp_RequestIpAddrAssignment releases the current IP address parameters and re-assigns with the requested IP address parameters.

For Static MANUAL manual IP address assignment, the local IP address along with netmask and default router IP address can either be passed as input parameters of Tcplp_RequestIpAddrAssignment or configured in the TcplpStaticIPAddressConfig container. The Local IP address passed as input parameter via Tcplp_RequestIpAddrAssignment will always overrule that of the configured value in TcplpStaticIPAddressConfig, in case both are present.

AutoIP address range can be configured in the TcplpStaticIPAddressConfig container for both AUTOMATIC and MANUAL Static assignment methods.

Tcplp allows all the configured manual IP address assignment methods to be started for a LocalAddrId by passing the parameter type as TCPIP_IPADDR_ASSIGNMENT_ALL to Tcplp_RequestIpAddrAssignment.

20.11.2 Allocation of Sockets

Whenever a new socket is required for communication, the upper layer (SoAd or CDD) requests Tcplp for a new socket.

For each configured TcplpSocketOwner, Tcplp provides a separate Tcplp_<Up>GetSocket API by replacing the tag <Up> with "SoAd" (if TcplpSocketOwnerUpperLayerType is set to SOAD), or with the short name of the TcplpSocketOwner container (if TcplpSocketOwnerUpperLayerType is set to CDD).

The Tcplp_<Up>GetSocket is invoked by the upper layer for allocation of new sockets. Tcplp checks the protocol, and then forwards a request to the corresponding module (i.e. rba_EthTcp or rba_EthUdp).

TCP and UDP allocate a socket (if a free socket is available) and returns a zero based socket ID in the range [0; TcplpTcpSocketMax] for TCP and [0; TcplpUdpSocketMax] for UDP.

Tcplp converts the socket ID provided by UDP into a unique Tcplp socket ID by adding an offset.

20.11.2.1 Configurable Upper Layer

The upper layer of Tcplp is configurable via the TcplpSocketOwnerConfig container (in Tcplp/TcplpConfig/) as follows:

- If the upper layer is set to SoAd, no further configuration required. All necessary API names and header file inclusion is generated implicitly by Tcplp.
- If the upper layer is set to CDD, all necessary API names and header file to be

included must be configured. Note: if some APIs available for configuration are not required for the particular CDD module, they can be left unconfigured.

20.11.3 Fragmentation and Reassembly

Fragmentation and/or reassembly can be enabled separately for a specified Tcplp controller by configuring the optional parameter TcplpFragmentationConfigRef. If no reference is configured, both fragmentation and reassembly are considered disabled.

The optional parameter TcplpTxFragmentBufferSize specifies the size of the static buffer per controller. In order to optimise memory usage, a single buffer is used to serve the multiple simultaneous fragmentation requests. Do not configure this container if fragmentation needs to be disabled for this controller.

Reassembly can be enabled by setting the TcplpReassemblyBufferCount parameter to a value greater than zero. One buffer is used to reassemble all fragments of one datagram, so TcplpReassemblyBufferCount configures the number of datagrams that can be reassembled in parallel. A value of zero disables reassembly. If it cannot be ensured that a fragmented frame will never be received by a node, reassembly should be enabled, otherwise any received fragmented frames will be dropped.

Multiple controllers can have a reference to the same TcplpFragmentationConfig. The following diagram shows some example configurations of IPv4 fragmentation.

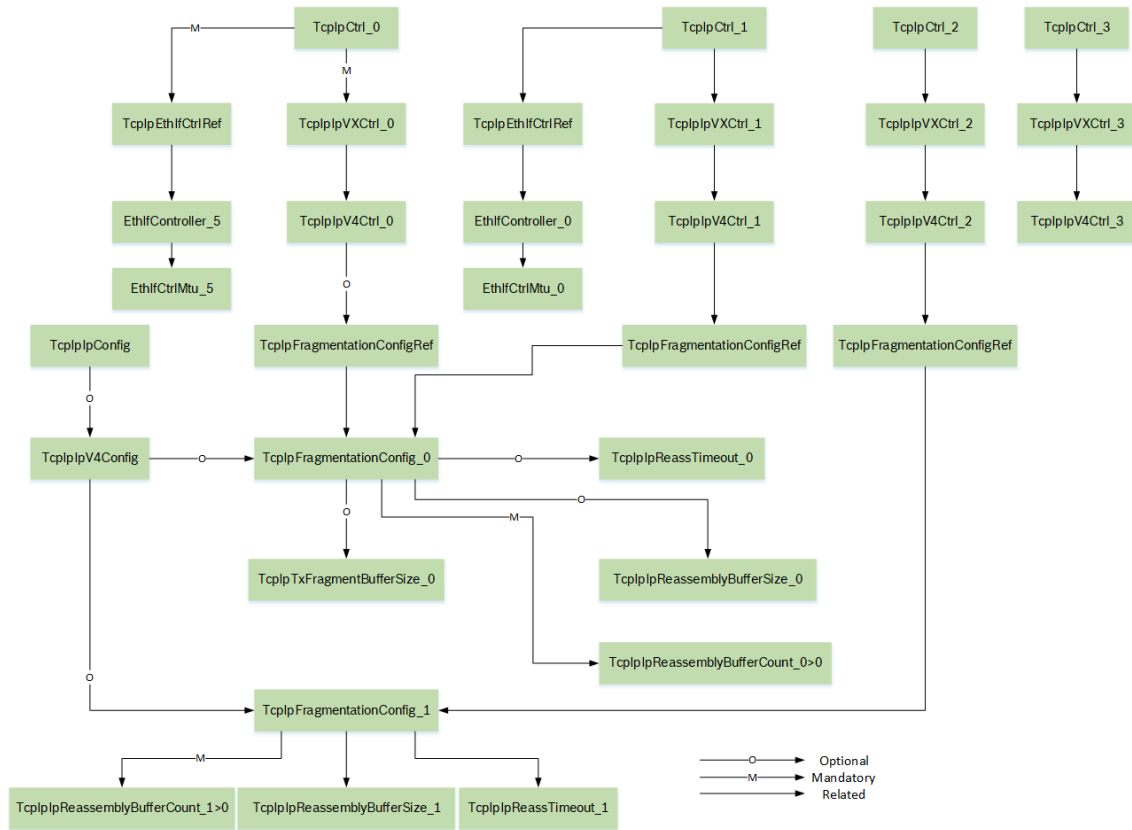


Figure 20.2.Examples of IPv4 Fragmentation Configuration

In the above example, the following configurations are shown:

- TcplpCtrl_0 and TcplpCtrl_1 have fragmentation enabled. Each controller instance has its own fragmentation buffer size (set in TcplpTxFragmentBufferSize_0). This value must be greater than the value of EthIfCtrlMtu_5 and EthIfCtrlMtu_0. Each controller instance also has a configured reassembly buffer of size TcplpReassemblyBufferSize_0.
- TcplpCtrl_2 has fragmentation disabled (TcplpTxFragmentBufferSize is not configured). However, reassembly is enabled.
- TcplpCtrl_3 has both fragmentation and reassembly disabled.

20.11.4 Maximum Transmission Unit (MTU)

For IPv6 communication, the EthIfCtrlMtu should be configured larger than 1280 octets. EthIfCtrlMtu must always be higher than or equal to the MTU of the intermediate and destination links over the path.

20.11.5 Enabling TCP SACK

The Selective Acknowledgement (SACK) mechanism shall be supported according to IETF RFC 2018. When TcpIpTcpSackEnabled is set to TRUE, the SACK option will then be enabled and will be sent in the TCP handshake.

20.12 Integration Advice

20.12.1 Init Functions

Tcplp_Init performs initialisation of the Tcplp module and all its associated sub-modules. Therefore the following sub-modules do not require a separate initialisation.

- rba_EthArp
- rba_EthAutolp
- rba_EthICMP
- rba_EthICMPv6
- rba_EthIPv4
- rba_EthIPv6
- rba_EthNdp
- rba_EthTcp
- rba_EthUdp
- rba_EthDHCP
- rba_EthDHCPv6

Tcplp_Init should be called during the startup phase of the ECU, and whenever the Tcplp module requires re-initialisation.

20.12.2 Delnit Functions

No Delnit is required.

20.12.3 Scheduled Functions

Tcplp_MainFunction should be scheduled cyclically with a fixed cycle time, and after EthSM_MainFunction.

20.12.4 Parameter Definition

The following Tcplp sub-modules do not have their own EcucParamDef files, so Tcplp_EcucParamDef.arxml has to be used for their code generation.

- rba_EthArp
- rba_EthAutolp
- rba_EthDHCP
- rba_EthDHCPv6
- rba_EthICMP
- rba_EthICMPv6
- rba_EthIPv4
- rba_EthIPv6

- rba_EthNdp
- rba_EthTcp
- rba_EthUdp

20.12.5 Implicit Disabling of Protocols

If a protocol is not used, it will be implicitly disabled. For example, if the DHCP address assignment method is not used for any TcplpLocalAddr, DHCP will be disabled.

20.12.6 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Tcplp module uses the following OS resources:

- Tcplp_ExclusiveAreaCurrAsgnedAddrInProcess
- Tcplp_ExclusiveAreaRandNumGenerating
- Tcplp_ExclusiveAreaSockOperations
- Tcplp_ExclusiveAreaReqRelAddrAssign
- Tcplp_ExclusiveAreaRequestedAssignPrioAccess
- Tcplp_ExclusiveAreaLocalIPChgParamAccess
- Tcplp_ExclusiveAreaManualStaticIPAddrAccess
- Tcplp_ExclusiveAreaArpProtocolEnable

20.13 Hardware Requirements

Table 20.203.Tcplp Hardware Requirements

Support	Usage
FLASH	11744 bytes
RAM	1761 bytes

Usage calculated based on a sample configuration.

21 UDP Network Management

Module name	UdpNm
Package	RTA-ETH
AUTOSAR Module ID	33
AUTOSAR Version	22-11

21.1 Introduction

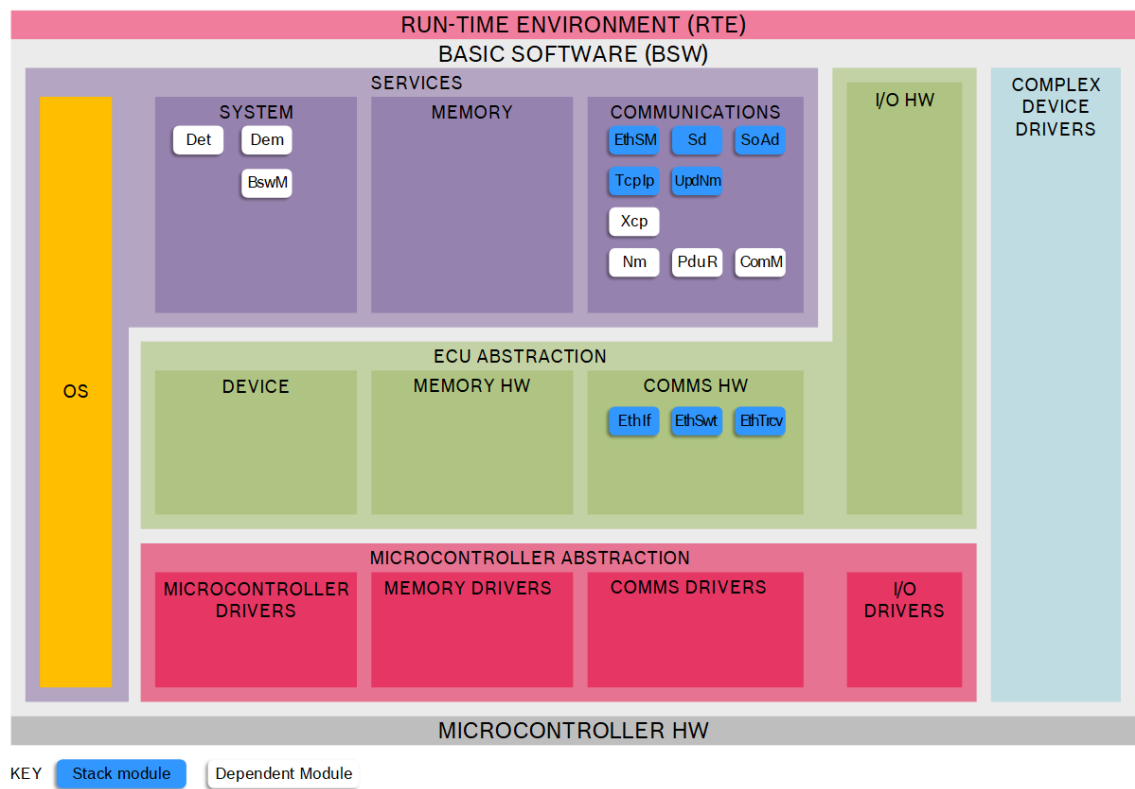


Figure 21.1.RTA-ETH Architecture

The UDP Network Management module (UdpNm) is located in the communication services abstraction layer. The diagram above shows the position of UdpNm in the AUTOSAR layered architecture.

The Network Management Interface module (Nm) is designated as the upper layer with respect to UdpNm, while the Socket Adapter (SoAd) is designated as the lower layer module.

The UdpNm module provides a hardware-independent protocol for UDP networks. Its main purpose is to co-ordinate the transition between bus-sleep and normal communication modes for nodes on a designated network cluster.

UdpNm also provides additional services which can be used (for example) to detect nodes which require bus communication and those which are ready to transition to sleep mode. Bus sleep is used to reduce power consumption on

nodes which no longer require reception or transmission of network management messages.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

21.2 Supported Features

- Node identification.
- Get/set user data.
- PDU transmission control.
- Get/set network mode.
- Partial networking.

21.3 Extensions

21.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

21.3.1.1 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmMsg-TimeoutTime

Short Name	UdpNmMsgTimeoutTime
Description	ETAS (Vendor 11) Transmission Timeout of NM-message. If there is no transmission confirmation by the UDP Interface within this timeout, the UDPNM module shall give an error notification.
Introduction	This parameter is only valid if UDPNM_PASSIVE_MODE_ENABLED is disabled. UDPNM_MSG_TIMEOUT_TIME should be a multiple of UDPNM_MSG_CYCLE_TIME.
Multiplicity	1..1
Value Range	0.001..65.535
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

21.3.1.2 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmPnEraCalcEnabled

Short Name	UdpNmPnEraCalcEnabled
Description	Specifies if UdpNm calculates the PN request information for external requests. (ERA)
Introduction	false: PN request are not calculated true: PN request are calculated. dependency: only available if UdpNmPnEnabled == true

Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

21.3.1.3 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmPnEraRxNSduRef

Short Name	UdpNmPnEraRxNSduRef
Description	Reference to a Pdu in the COM-Stack. The SduRef is required for every UdpNm Channel, because ERA is reported per channel.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

21.3.1.4 UdpNm/UdpNmGlobalConfig/UdpNmPnEiraCalcEnabled

Short Name	UdpNmPnEiraCalcEnabled
Description	Specifies if UdpNm calculates the PN request information for internal and external requests. (EIRA) true: PN request are calculated
Introduction	false: PN request are not calculated dependency: only available if UdpNmPnEnabled == true for at least one UdpNm Channel
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

21.3.1.5 UdpNm/UdpNmGlobalConfig/UdpNmPnEiraRxNSduRef

Short Name	UdpNmPnEiraRxNSduRef
Description	Reference to a Pdu in the COM-Stack. Only one SduRef is required for UdpNm because the EIRA is the aggregation over all Ethernet Channels.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

21.3.1.6 UdpNm/UdpNmGlobalConfig/UdpNmPnInfo

Short Name	UdpNmPnInfo
Description	PN information configuration
Multiplicity	0..1

Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

21.3.1.7 UdpNm/UdpNmGlobalConfig/UdpNmPnInfo/UdpNmPnFilter-MaskByte

Short Name	UdpNmPnFilterMaskByte
Description	PN information configuration
Multiplicity	0..7
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

21.3.1.8 UdpNm/UdpNmGlobalConfig/UdpNmPnInfo/UdpNmPnFilter-MaskByte/UdpNmPnFilterMaskByteIndex

Short Name	UdpNmPnFilterMaskByteIndex
Description	Index of the filter mask byte. Specifies the position within the filter mask byte array.
Multiplicity	1..1
Value Range	0..6
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

21.3.1.9 UdpNm/UdpNmGlobalConfig/UdpNmPnInfo/UdpNmPnFilter-MaskByte/UdpNmPnFilterMaskByteValue

Short Name	UdpNmPnFilterMaskByteValue
Description	Parameter to configure the filter mask byte.
Multiplicity	1..1
Value Range	0..255
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

21.3.1.10 UdpNm/UdpNmGlobalConfig/UdpNmPnInfo/UdpNmPnInfoLength

Short Name	UdpNmPnInfoLength
Description	Specifies the length of the PN request information in the NM message.
Multiplicity	1..1
Value Range	1..7
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

21.3.1.11 UdpNm/UdpNmGlobalConfig/UdpNmPnInfo/UdpNmPnInfoOffset

Short Name	UdpNmPnInfoOffset
Description	Specifies the offset of the PN request information in the NM message.
Multiplicity	1..1
Value Range	1..7
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

21.3.1.12 UdpNm/UdpNmGlobalConfig/UdpNmPnResetTime

Short Name	UdpNmPnResetTime
Description	Specifies the runtime of the reset timer in seconds. This reset time is valid for the reset of PN requests in the EIRA and in the ERA. The value shall be the same for every channel. Thus it is a global config parameter.
Multiplicity	0..1
Value Range	0.001..65.535
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

21.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

21.3.2.1 UdpNm_RequestSynchronizedPncShutdown

Syntax	UdpNm_RequestSynchronizedPncShutdown(NetworkHandleType nmChannelHandle , PNCHandleType pncId)
Parameters In	nmChannelHandle Identifier of the NM-Channel where the given PNC (pncId) is assigned to. pncId Identifier of the PNC which is requested for a synchronized shutdown across the PN topology.
Description	Called by NM to request a synchronized PNC shutdown.

21.4 Limitations

- One instance of UdpNm is associated with only one network management cluster in one network.
- One network management cluster can have only one instance of UdpNm in one node.
- One instance of UdpNm is associated with only one network within the same

- ECU.
- UdpNm is only applicable for TCP/IP based systems.
- Measurement variables via BSWMD files is not supported.
- PostBuild configurations are not supported.
- NM co-ordinator feature is not supported.
- Not all requirments under PNC shutdown Part 1 functionality are implemented. Shutdown request does not check if there is a pending txconfirmation from an earlier transmission.

21.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.
The following API functions are not supported.

- UdpNm_SetSleepReadyBit
- UdpNm_SoAdIfTriggerTransmit
- UdpNm_RepeatMessageIndication

21.5.1 Configuration Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmCarWakeUpRx-Enabled
- UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmDynamicPnc-ToChannelMappingEnabled
- UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmStayInPbsEnabled
- UdpNm/UdpNmGlobalConfig/UdpNmCoordinatorSyncSupport
- UdpNm/UdpNmGlobalConfig/UdpNmDynamicPncToChannelMappingSupport

21.6 Deviations

21.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

21.6.1.1 UdpNm

[ADDED]	Description	UdpNm
---------	-------------	-------

21.6.1.2 UdpNm/UdpNmGlobalConfig

[RTA-BSW] Description	This container contains all global configuration parameters of UDP NM configured from the CanTrcv Module perspective.
[AUTOSAR] Description	This container contains all global configuration parameters of UDP NM. The parameters and the parameters of the sub containers shall be mapped to the C data type UdpNm_ConfigType (for parameters where it is possible) which is passed to the UdpNm_Init function.

21.6.1.3 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmActiveWakeupBitEnabled

[ADDED] Introduction	dependency: This parameter is only valid if UdpNmPassiveModeEnabled is False.
----------------------	---

21.6.1.4 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmCarWakeupBitPosition

[ADDED] Default Value	false
-----------------------	-------

21.6.1.5 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmCarWakeupBytePosition

[ADDED] Default Value	false
-----------------------	-------

21.6.1.6 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmCarWakeupFilterNodeId

[ADDED] Introduction	dependency: only available if UdpNmCarWakeupFilterEnabled == TRUE
[ADDED] Default Value	false

21.6.1.7 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmImmediateNmCycleTime

[ADDED] Introduction	dependency: This parameter is only valid if UdpNmImmediateNmTransmissions is greater one.
----------------------	---

21.6.1.8 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmImmediateNmTransmissions

[ADDED] Introduction	dependency: If UdpNmImmediateRestartEnabled = true then UdpNmImmediateNmTransmissions = 0 If UdpNmPnHandleMultipleNetworkRequests == True then UdpNmImmediateNmTransmissions
----------------------	--

21.6.1.9 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmMsgCycleOffset

[RTA-BSW] Introduction	less than UDPNM_MSG_CYCLE_TIME This parameter is only valid if UdpNmPassiveModeEnabled is disabled.
[AUTOSAR] Introduction	< UdpNmMsgCycleTime This parameter is only valid if UdpNmPassiveModeEnabled is disabled.

21.6.1.10 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmNodeDetectionEnabled

[RTA-BSW] Introduction	This parameter shall be derived from NmNodeDetectionEnabled. This parameter shall only be enabled if UdpNmNodeIdEnabled == true. If (UdpNmPduCbvPosition != UDPNM_PDU_OFF) then Equal(NmNodeDetectionEnabled) else Equal(False). dependency: Not available if UdpNmPassiveModeEnabled
[AUTOSAR] Introduction	This parameter shall be derived from NmNodeDetectionEnabled. This parameter shall only be enabled if UdpNmNodeIdEnabled == true. If (UdpNmPduCbvPosition != UDPNM_PDU_OFF) then Equal(NmNodeDetectionEnabled) else Equal(False).

21.6.1.11 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmNodeId

[ADDED] Introduction	This parameter is only valid if UdpNmPassiveModeEnabled is set to OFF and UdpNmNodeDetectionEnabled is set to ON.
----------------------	---

21.6.1.12 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmPnEnabled

[RTA-BSW] Introduction	false: Partial networking Range not supported true: Partial networking supported dependency: only available if UdpNmPnEnabled == true
[AUTOSAR] Introduction	false: Partial networking Range not supported true: Partial networking supported

21.6.1.13 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmPnHandleMultipleNetworkRequests

[RTA-BSW] Introduction	true: UdpNm_NetworkRequest triggers a change from NO to RM. dependency: only available if UdpNmPnEnabled == true
[AUTOSAR] Introduction	true: UdpNm_NetworkRequest triggers a change from NO to RM.

21.6.1.14 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmRe-

repeatMessageTimeUdpNmRepeatMessageTime

[RTA-BSW] Description	Timeout for Repeat Message State. It defines the time in seconds how long the NM shall stay in the Repeat Message State.
[AUTOSAR] Description	Timeout for Repeat Message State.
[RTA-BSW] Introduction	dependency: $\text{UdpNmRepeatMessageTime} = n * \text{UdpNmMsgCycleTime}$; $\text{UdpNmRepeatMessageTime} > \text{UdpNmImmediateNmTransmissions} * \text{UdpNmImmediateNmCycleTime}$ Typically it should be equal to: $n * \text{UdpNmMsgCycleTime}$, where n denotes the number of NM PDUs that are normally sent in the Repeat Message State. The value of n decremented by one determines the amount of lost NM PDUs that can be tolerated by the node detection procedure. The value 0 denotes that no Repeat Message State is configured. It means that Repeat Message State is transient what implicates that it is left immediately after entrance and in result no start-up stability is guaranteed and no node detection procedure is possible.
[AUTOSAR] Introduction	It defines the time in seconds how long the NM shall stay in the Repeat Message State.

21.6.1.15 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmRepeatMsgIndEnabled

[ADDED] Introduction	dependency: $\text{calculationFormula} = \text{If}(\text{UdpNmPassiveModeEnabled} == \text{False}) \text{ then } \text{Equal}(\text{NmRepeatMsgIndEnabled}) \text{ else } \text{Equal}(\text{False})$
----------------------	---

21.6.1.16 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmRetryFirstMessageRequest

[ADDED] Introduction	dependency: $\text{UdpNmRetryFirstMessageRequest} = \text{false}$ if $\text{UdpNmPassiveModeEnabled} == \text{true}$
----------------------	--

21.6.1.17 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmRxPdu/UdpNmRxPduld

[RTA-BSW] Maximum Value	4294967295
[AUTOSAR] Maximum Value	4294967296

21.6.1.18 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmSynchronizedPncShutdownEnabled

[RTA-BSW] Description	ETAS (Vendor 11) Specifies if UdpNm handle PN shutdown messages to support a synchronized PNC shutdown across a PN topology. This is only used for ECUs in the role of a top-level PNC coordinator or intermediate PNC coordinator. Thus, the PNC gateway functionality is enabled and therefore ERA calculation is used. false: synchronized PNC shutdown is disabled true: synchronized PNC shutdown is enabled. .
[AUTOSAR] Description	Specifies if UdpNm handle PN shutdown messages to support a synchronized PNC shutdown across a PN topology. This is only used for ECUs in the role of a top-level PNC coordinator or intermediate PNC coordinator. Thus, the PNC gateway functionality is enabled and therefore ERA calculation is used.
[REMOVED] Introduction	FALSE: synchronized PNC shutdown is disabled TRUE: synchronized PNC shutdown is enabled
[REMOVED] Default Value	false

21.6.1.19 UdpNm/UdpNmGlobalConfig/UdpNmChannelConfig/UdpNmTxPdu/UdpNmTxConfirmationPduld

[RTA-BSW] Maximum Value	4294967295
[AUTOSAR] Maximum Value	4294967296

21.6.1.20 UdpNm/UdpNmGlobalConfig/UdpNmComControlEnabled

[ADDED] Introduction	dependency: calculationFormula = If (UdpNmPassiveModeEnabled == False) then Equal(NmComControlEnabled) else Equal(False)
----------------------	--

21.6.1.21 UdpNm/UdpNmGlobalConfig/UdpNmComUserDataSupport

[ADDED] Introduction	dependency: If UdpNmPassiveModeEnabled == True then UdpNmComUserDataSupport = False
----------------------	---

21.7 API Definition

List of supported API functions

Table 21.36.UdpNm Supported API functions

UdpNm_Init	Initialises the module.
UdpNm_GetVersionInfo	Returns version information for the module.
UdpNm_PassiveStartUp	Triggers passive startup of the module.

UdpNm_NetworkRequest	Changes the network state to requested for the specified channel.
UdpNm_NetworkRelease	Changes the network state to released for the specified channel.
UdpNm_DisableCommunication	Disables PDU transmission on the specified channel.
UdpNm_EnableCommunication	Enables PDU transmission on the specified channel.
UdpNm_SetUserData	Sets the user data to be included in the next transmitted message for a specified channel.
UdpNm_GetUserData	Returns the user data from the last received message on the specified channel.
UdpNm_GetNodeIdentifier	Returns the node identifier from the last received message on the specified channel.
UdpNm_GetLocalNodeIdentifier	Returns the node identifier for the local node on the specified channel.
UdpNm_RepeatMessageRequest	Sets the Repeat Message Request bit for all transmitted messages on the specified channel.
UdpNm_GetPduData	Returns the PDU data from the last received message on the specified channel.
UdpNm_GetState	Returns the current network state for the specified channel.
UdpNm_RequestBusSynchronization	Triggers non-periodic PDU transmission on the specified channel.
UdpNm_CheckRemoteSleepIndication	Checks whether or not remote sleep indication has taken place on the specified channel.
UdpNm_SoAdIfTxConfirmation	Callback function which signals message transmission by the SoAd module.
UdpNm_SoAdIfRxIndication	Callback function which signals message reception by the SoAd module.
UdpNm_Transmit	Dummy function required for compatibility with PduR module.

For Further information please refer to AUTOSAR_SWS_UDPNetworkManagement.pdf

21.8 Configuration Advice



NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

21.8.1 General

The Message Cycle timer and NM-Timeout timer are not needed if UdpNmPassiveModeEnabled is set to TRUE because no NM-PDUs are transmitted by such nodes.

21.9 Integration Advice

21.9.1 Init Functions

UdpNm is initialized by UdpNm_Init, which must only be called after Tcplp has been initialised. The behaviour of UdpNm during initialisation is as follows.

- The network management state is set to NM_STATE_BUS_SLEEP.
- The network state is set to 'released'.
- Each byte of the user data is set to 0xFF.
- The Control Bit Vector is set to 0x00.
- All transmission of NM-PDUs is stopped.
- The Message Cycle timer and NM-Timeout timer are stopped.

UdpNm should be initialised after SoAd and before any Nm service is called.

Calling UdpNm_PassiveStartUp causes a passive startup of the UdpNm module. It triggers the transition from Bus-Sleep Mode (or Prepare Bus-Sleep Mode) to the Network Mode in Repeat Message State.

21.9.2 Delnit Functions

No Delnit is required.

21.9.3 Scheduled Functions

UdpNm_MainFunction should be scheduled for cyclic operation, and before SoAd_MainFunction.

21.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's respon-

sibility to ensure that the ENTER and EXIT resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The UdpNm module uses the following OS resources:

- UdpNm_MainFunctionNoNest

21.10 Hardware Requirements

Table 21.37. UdpNm Hardware Requirements

Support	Usage
FLASH	3742 bytes
RAM	140 bytes

Usage calculated based on a sample configuration.

22 Glossary

Activation Line

A routing path for diagnostic messages

API Application Programming Interface

ARP

Address Resolution Protocol. A communication protocol used for discovering link layer (MAC) addresses

BswM

AUTOSAR BSW Mode Manager, supports those aspects of Vehicle Mode Management and Application Mode Management that are applicable for BSW

CBV

Control Bit Vector. An array of Boolean values which signal the control properties of an NM-PDU

CDD

Complex Device Driver. A software entity not standardised by AUTOSAR, but which can access (or be accessed by) AUTOSAR interfaces and/or RTA-BSW module APIs

Channel

A logical bus connection to a particular node or ECU

Cluster

A communication system made up of a number of nodes linked by a physical transmission medium

ComM

AUTOSAR Communication Manager, provides the control of the underlying communication services

CRC

Cyclic Redundancy Checksum

DHCP

Dynamic Host Configuration Protocol. A network management protocol used in UDP/IP networks

DoIP

AUTOSAR Diagnostic Over IP module

EthIf

AUTOSAR Ethernet Interface module

EthSM

AUTOSAR Ethernet State Manager module

EthTSyn

AUTOSAR Time Sync Over Ethernet module

Gratuitous ARP

An unrequested ARP reply, normally used to announce the presence of a new network device

ICMP

Internet Control Message Protocol. A supporting protocol used in the Internet protocol suite to send error messages and operational information

IP Internet Protocol. The principal communications protocol of the Internet protocol suite

IP Address

A numerical label assigned to each device on an IP network. In IPv4, the IP address is a 32-bit value

IPv4

First major version of the Internet Protocol

IPv6

Latest major version of the Internet Protocol

I-PDU

PDU of the interaction layer

Link-local

A type of connection which links devices located on the same physical or logical network

MAC Address

Media Access Control address. A unique ID number which identifies a device on a network and is usually assigned at the hardware level

MSS

Maximum Segment Size. The largest amount of data that can be received in a single TCP segment, specified in octets

MTU

Maximum Transmission Unit

NDP

Neighbor Discovery Protocol

Nm AUTOSAR Network Management Interface, responsible for coordinated ECU network access, in particular, the transition to sleep mode

NM-PDU

Network Management PDU. A PDU containing the payload of a network management message, consisting of the user data, CBV and source node identifier

Node

An ECU which is connected to a communications network

N-SDU

PDU of the network transport layer

OFS

Offset Synchronization

PDU

Protocol Data Unit

PduR

AUTOSAR PDU Router, responsible for routing PDUs between communication modules and/or transport protocol modules

Port

A physical or logical endpoint on a computer network

PTP

Precision Time Protocol

rba_EthArp

RTA-BSW module, provides address resolution services which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the SoAd and EthIf modules

rba_EthAutoIp

RTA-BSW module, provides auto-configuration of local IP addresses which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the SoAd and EthIf modules

rba_EthDHCP

RTA-BSW module, provides DHCP host configuration services which are used by the Tcplp module to assign host-specific configuration parameters and network addresses to host network devices

rba_EthICMP

RTA-BSW module, provides ICMP services which are used by the Tcplp module to provide error message and query communications between the SoAd and EthIf modules

rba_EthICMPv6

RTA-BSW implementation of AUTOSAR ICMPv6

rba_EthIPv4

RTA-BSW module, provides IPv4 protocol services which are used by the Tcplp module (acting as a wrapper) to provide IP functionality for communication between the SoAd and EthIf modules

rba_EthIPv6

RTA-BSW implementation of AUTOSAR IPv6

rba_EthNdp

RTA-BSW implementation of AUTOSAR IPv6 NDP

rba_EthTcp

RTA-BSW module, provides TCP communication services used by the Tcplp module (acting as a wrapper) to provide intermediate level communication between the SoAd and EthIf modules

rba_EthUdp

RTA-BSW module, provides UDP protocol services are used by the Tcplp module to provide UDP data communication between the SoAd and EthIf modules

rba_SomelpXfAdp

RTA-BSW module, responsible for allocating the memory needed for data transformation and calling the transformer functions

RTO

Retransmission Timeout

Sd AUTOSAR Service Discovery module

SDU

Service Data Unit

SNMA

Solicited-Node Multicast Address, used in Neighbor Discovery Protocol (NDP) for obtaining link-layer addresses

SoAd

AUTOSAR Socket Adapter module

Socket

An internal endpoint for sending or receiving data within a node on a communications network

SOME/IP

Scalable Service-Oriented Middleware Over IP. An automotive middleware solution which can be used to send and receive control messages on a vehicle network

SomelpTp

AUTOSAR Scalable service-Oriented MiddlewarE over IP (SOME/IP) Transport Protocol module

SomelpXf

AUTOSAR SOME/IP Transformer module

SOME/IP-SD

The service discovery component of SOME/IP

StbM

AUTOSAR Synchronized Time-Base Manager module

Switch

A networking device which routes information via data packet switching

TCP

Transmission Control Protocol. One of the main communication protocols of the Internet protocol suite

TcpIp

AUTOSAR TCP/IP module

TLV

Time, length and value coding

TP-PDU

A PDU transmitted or received by a lower layer transport protocol module

Transformer

A transformer employs a data transformation mechanism to linearize and transform data

UDP

User Datagram Protocol. One of the main communication protocols of the Internet protocol suite

UdpNm

AUTOSAR UDP Network Management module

VIN Vehicle Identification Number

VLAN

Virtual Local Area Network